NextPlay

Pantallas y funciones

28 pantallas · 734 funciones de Python en 84 archivos y 45 de JavaScript en 14 bloques de las plantillas · generado el 04/11/2026 19:25

Cómo leer este documento

La primera parte es el mapa de pantallas: qué es cada pantalla de la web, en qué dirección está, qué vista y qué plantillas la pintan y qué funciones la afectan, archivo por archivo y con la parte de la pantalla entre paréntesis. La segunda parte son todas las funciones del proyecto, archivo por archivo, con lo que hace cada una y sus pantallas, y la tercera, el JavaScript de las plantillas.

Las pantallas de una función son las pantallas en las que se nota lo que hace: donde se ve lo que calcula o pinta, desde donde se lanza (sus botones y formularios) y donde sale lo que guarda. Si la cambias, son las pantallas que tienes que mirar. «todas» quiere decir que sale en cualquier pantalla (la barra de arriba, la foto y el nombre de cada jugador); «ninguna», que no se ve en la web (el arranque de Django, las opciones de un comando...).

En el código cada función lo dice al final de su docstring, en un párrafo «Pantallas: …» con los nombres de docs/pantallas.json, y cada bloque de JavaScript, en una línea «// Pantallas: …» de su comentario. Las que no lo dicen, o nombran una pantalla que no está en el catálogo, salen en rojo.

Parte 1 · Mapa de pantallas

Pantalla	Dirección	Funciones
Inicio	/	167
Catálogo	/juegos/	43
Ficha del juego	/juegos/<slug:slug>/	300
Listas del juego	/juegos/<slug:slug>/listas/	17
Perfil	/u/<str:username>/	301
Seguidores y seguidos	/u/<str:username>/seguidores/	14
Publicar	/publicar/	10
Página de la publicación	/u/<str:username>/publicaciones/<int:post_id>/	36
Escribir reseña	/biblioteca/<int:game_id>/resenas/nueva/	59
Página de la reseña	/u/<str:username>/resenas/<int:review_id>/	50
Lista de deseos	/biblioteca/lista-de-deseos/	91
Repasar la biblioteca	/biblioteca/repasar/	14
Página de la lista	/u/<str:username>/listas/<int:list_id>/	143
Nueva lista	/listas/nueva/	12
Nueva tier list	/tier-lists/nueva/	38
Editar lista	/listas/<int:list_id>/editar/	54
Tier lists	/tier-lists/	10
Retos	/retos/	78
Página del reto	/retos/<int:challenge_id>/	106
Nuevo reto	/retos/nuevo/	13
Editar reto	/retos/<int:challenge_id>/editar/	9
Avisos	/avisos/	55
Editar perfil	/cuenta/perfil/	166
Juegos importados	/cuenta/juegos-importados/	67
Registro	/cuenta/registro/	3
Iniciar sesión	/cuenta/login/	0
Recuperar contraseña	/cuenta/password_reset/	0
Panel de administración	/admin/	74
En todas las pantallas		28
En ninguna pantalla		11

Inicio

La portada (/). Con sesión: tu resumen y tu menú a la izquierda, la caja para publicar y el feed (pestañas «Siguiendo» y «Descubrir», con «Cargar más») en el centro y los juegos en tendencia y los géneros a la derecha. Sin sesión: la bienvenida, los juegos populares y lo último de la comunidad.

Dirección: / (home → games.views.home)
/actividad/ (social:feed → social.views.feed)

Plantillas: games/home.html, social/_activities.html
Trozos que incluye: accounts/_avatar.html, _icon.html, social/_composer.html, games/_card.html, social/_person.html, games/_trending.html, social/_activity.html, library/_stars.html, library/_platform_mark.html, social/_follow_button.html, social/_post_body.html, social/_post_like.html, lists/_like.html, library/_review_images.html, library/_like.html, lists/_mosaic.html, social/_follower_count.html, social/_post_images.html

Funciones que la afectan (167)

accounts/images.py: process (imágenes de la caja para publicar)

accounts/management/commands/nightly.py: Command.handle (feed: la importación y «consiguió el reto»)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.review (feed) · Command.activity (feed)

accounts/management/commands/sync_libraries.py: Command.handle (feed: la importación)

accounts/matching.py: clean_name (feed: la importación) · normalize (feed: la importación) · name_key (feed: la importación) · _name_keys (feed: la importación) · _catalog_games (feed: la importación) · _title_parts (feed: la importación) · add_title (feed: la importación) · _platform_names (feed: la importación) · _fits (feed: la importación) · _share_platform (feed: la importación) · _seen_games (feed: la importación) · _score (feed: la importación) · match_by_name (feed: la importación) · save_titles (feed: la importación) · update_library (feed: la importación)

accounts/models.py: Profile.get_absolute_url · Profile.visible_filter (feed y sugerencias)

accounts/psn.py: access_token (feed: la importación) · _get (feed: la importación) · _account_id_from_token (feed: la importación) · played_games (feed: la importación) · duration_minutes (feed: la importación) · import_library (feed: la importación)

accounts/steam.py: _api (feed: la importación) · owned_games (feed: la importación) · _match_games (feed: la importación) · _timestamp (feed: la importación) · _played_at (feed: la importación) · import_library (feed: la importación)

accounts/views/cuentas_vinculadas.py: _import_steam (feed: la importación) · steam_callback (feed: la importación) · steam_import (feed: la importación) · psn_sync (feed: la importación) · xbox_callback (feed: la importación)

accounts/xbox.py: _post (feed: la importación) · xbox_tokens (feed: la importación) · _get (feed: la importación) · played_titles (feed: la importación) · minutes_played (feed: la importación) · import_library (feed: la importación)

games/igdb.py: IGDBClient.top_games · IGDBClient.games_by_steam_ids (feed: la importación) · IGDBClient.games_by_ids · IGDBClient.games_by_names (feed: la importación) · IGDBClient._with_editions · upsert_games

games/management/commands/import_igdb.py: Command.handle · Command._import_top

games/management/commands/seed_demo.py: Command.handle

games/models.py: spanish_genre_name (géneros) · igdb_image_url · Game.get_absolute_url · Game.cover_url · Game.release_year

games/spanish.py: europe_cover (la portada europea) · apply_igdb (la portada europea)

games/views.py: trending_games (juegos en tendencia) · home

library/backlog.py: answer (feed: «ordenó su biblioteca»)

library/forms.py: stars_label

library/markup.py: _inline · render · plain

library/models.py: LibraryEntry.imported_status (feed: la importación) · LibraryEntry.update_import (feed: la importación) · LibraryEntry.playtime_minutes · LibraryEntry.playtime_hours · LibraryEntry.hours_by_platform · ReviewQuerySet.published (feed) · ReviewQuerySet.visible_to (feed) · Review.get_absolute_url (feed) · ReviewComment.get_absolute_url (feed) · attached_image_upload_to (dónde se guardan las imágenes subidas a reseñas y publicaciones) · AttachedImage.upload_folder (carpeta de las imágenes subidas) ·

AttachedImage.is_official · AttachedImage.url (el visor a pantalla completa de las imágenes) · AttachedImage.thumb_url

library/templatetags/reviews.py: review_html · review_plain · stars_label

library/views/escribir_resena.py: _review_editor (feed)

library/views/ficha_del_juego.py: mark_liked (feed: tus «me gusta» marcados) · set_status (feed y tu resumen) · remove (feed y tu resumen) · rate (feed)

library/views/pagina_de_la_resena.py: _readable_review (404 al pulsar algo de una reseña que no se puede leer) · review_like

library/views/repasar_la_biblioteca.py: tidy_answer (feed: «ordenó su biblioteca»)

social/activity.py: _recent (feed: junta lo que haces seguido en una sola actividad) · _game_activity (feed: lo que haces con un juego) · status_changed (feed) · rated (feed) · review_published (feed) · game_removed (feed) · post_published (caja para publicar y feed) · games_imported (feed) · _list_activity (feed: lo que haces con una lista) · list_created (feed) · list_games_added (feed) · list_game_removed (feed) · _challenge_activity (feed: lo que haces con un reto) · challenge_joined (feed) · challenge_won (feed: «consiguió el reto») · library_tidied (feed: «ordenó su biblioteca») · challenge_left (feed)

social/forms.py: PostForm.clean_text (caja para publicar) · PostForm.clean (caja para publicar) · PostForm.save (caja para publicar y feed)

social/models.py: FollowManager.follow (botón «Seguir» de las sugerencias y feed «Siguiendo») · FollowManager.unfollow (botón «Seguir» y feed «Siguiendo») · Post.get_absolute_url (feed) · PostImage.upload_folder (caja para publicar) · Activity.is_empty (feed: se borran las actividades que se quedan vacías) · Activity.has_content (feed: lo que se enseña de cada actividad) · Activity.list_created (feed: actividades de listas) · Activity.list_added (feed: actividades de listas) · Activity.challenge_verb (feed: actividades de retos) · Activity.verb (feed: qué hizo con el juego) · Activity.shown_rating (feed: la nota que se enseña) · Activity.platform_name (feed: actividades de importación) · Activity.import_count (feed: actividades de importación)

social/signals.py: forget_empty_activity (feed: al borrar una reseña)

social/views.py: follow_context (sugerencias de gente a la que seguir) · follow (botón «Seguir») · _batch (feed: cada tanda, también la de «Cargar más») · feed_context (feed: pestañas «Siguiendo» y «Descubrir») · suggested_people (gente a la que podrías seguir) · _cursor (feed: «Cargar más») · _parse_cursor (feed: «Cargar más») · feed (feed: «Cargar más») · _posts (feed) · mark_liked_posts · composer_context (caja para publicar) · post_new (caja para publicar) · _readable_post · post_delete (feed) · post_like

lists/forms.py: GameListForm.save (feed)

lists/models.py: GameListQuerySet.visible_to (feed) · GameListQuerySet.with_preview (feed) · GameList.get_absolute_url (feed) · GameList.is_ranked (los puestos de las listas numeradas) · GameList.kind_note (el tipo de cada lista en su tarjeta) · GameList.noun · GameList.public_parent (feed: «hizo su versión de…») · GameList.make_version (feed) · GameList.mosaic_gaps · GameList.touch (feed)

lists/views/nueva_lista.py: _create (feed)

lists/views/pagina_de_la_lista.py: _readable_list (404 al pulsar algo de una lista que no se puede ver) · make_version (feed: «hizo su versión de…») · like (botón «me gusta» de las listas)

lists/views/tarjetas.py: mark_progress (feed: cuántos juegos de cada lista has jugado) · mark_liked_lists (tus «me gusta» marcados)

challenges/closing.py: close_all (feed: «consiguió el reto»)

challenges/management/commands/close_challenges.py: Command.handle (feed: «consiguió el reto»)

challenges/progress.py: mark_completed (feed: «consiguió el reto») · _award (feed)

challenges/views.py: new (feed: «creó el reto») · join (feed: «se unió al reto») · leave (feed)

JavaScript: social/_composer.html (línea 37) (caja para publicar)

Catálogo

«Juegos» en el menú y el buscador de la barra de arriba: todos los juegos en tarjetas, con filtros por nombre, género, plataforma e idioma, el estado de cada uno en tu biblioteca y, si buscas algo que no está, el botón para traerlo de IGDB.

Dirección: /juegos/ (games:catalog → games.views.catalog)

Plantillas: games/catalog.html
Trozos que incluye: games/_card.html, _icon.html, library/_stars.html, library/_platform_mark.html

Funciones que la afectan (43)

accounts/management/commands/nightly.py: Command.handle (búsqueda por nombre en España y filtro de idioma)

games/igdb.py: is_configured (botón para traer de IGDB lo que buscas) · IGDBClient.__init__ (traer de IGDB) · IGDBClient._token (traer de IGDB) · IGDBClient.query (traer de IGDB) · IGDBClient.top_games (los juegos populares que trae el comando import_igdb) · IGDBClient.search (botón para traer de IGDB lo que buscas) · IGDBClient.games_by_ids · IGDBClient.spanish_data (filtro de idioma) · IGDBClient._with_editions · _upsert_ref (filtros de género y plataforma) · upsert_games (botón para traer de IGDB)

games/management/commands/import_igdb.py: Command.handle · Command._import_top (los juegos más populares de IGDB)

games/management/commands/localize_games.py: Command.handle (filtro de idioma y portadas europeas) · Command._from_igdb (filtro de idioma y portadas europeas)

games/management/commands/seed_demo.py: Command.handle (los 12 juegos de prueba)

games/models.py: spanish_genre_name (filtro de género) · igdb_image_url · language_filter (filtro de idioma) · search_filter (búsqueda por nombre, también desde la barra de arriba) · Genre.__str__ (filtro de género) · Platform.__str__ (filtro de plataforma) · Game.get_absolute_url · Game.cover_url · Game.merged_languages (búsqueda por nombre en España y filtro de idioma) · Game.refresh_spanish (búsqueda por nombre en España y filtro de idioma) · Game.release_year

games/spanish.py: igdb_languages (búsqueda por nombre en España y filtro de idioma) · spanish_names (búsqueda por nombre en España y filtro de idioma) · europe_cover · apply_igdb (búsqueda por nombre en España y filtro de idioma) · steam_languages (filtro de idioma) · localize_games (búsqueda por nombre en España y filtro de idioma)

games/views.py: catalog · import_search (botón para traer de IGDB lo que buscas)

library/forms.py: stars_label

library/models.py: LibraryEntry.playtime_minutes · LibraryEntry.playtime_hours · LibraryEntry.hours_by_platform

library/templatetags/reviews.py: stars_label

library/views/ficha_del_juego.py: set_status (estado de cada tarjeta) · remove (estado de cada tarjeta)

Ficha del juego

La página de cada juego: cabecera con su arte, datos y notas (crítica y comunidad), tu biblioteca (estado, nota, reseña, lista de deseos), favoritos y «Añadir a una lista», tus horas y tus logros, el carrusel de imágenes y vídeos, «Dónde comprarlo» con su historial de precios, las reseñas y las listas en las que sale.

Dirección: /juegos/<slug:slug>/ (games:detail → games.views.detail)
/juegos/<slug:slug>/precios/ (games:prices → games.views.game_prices)
/juegos/<slug:slug>/historial-de-precios/ (games:price_history → games.views.price_history)
/juegos/<slug:slug>/imagenes-y-videos/ (games:media → games.views.game_media)
/cuenta/logros/<int:game_id>/ (game_achievements → accounts.views.ficha_del_juego.game_achievements)

Plantillas: games/detail.html, games/_prices.html, games/_price_history.html, games/_media.html, accounts/_achievements.html
Trozos que incluye: games/_hero.html, games/_community.html, accounts/_achievement_card.html, lists/_game_box.html, library/_widget.html, accounts/_favorite.html, lists/_picker.html, library/_reviews.html, library/_wish.html, games/_price.html, _icon.html, accounts/_achievement.html, games/_spanish.html, lists/_mosaic.html, library/_platform_mark.html, library/_stars.html, library/_review.html, accounts/_avatar.html, library/_review_body.html, library/_like.html, library/_report.html, library/_review_images.html

Funciones que la afectan (300)

accounts/achievements.py: progress_by_game (tus logros) · save_info (tus logros) · save_unlocked (tus logros) · _by_rarity (tus logros, del más raro al más común) · detail («Tus logros», uno a uno)

accounts/admin.py: ExternalTitleAdmin.save_model (tus horas)

accounts/forms.py: ProfileForm._save_favorites (botón de favoritos)

accounts/management/commands/nightly.py: Command.handle (tus horas; tus logros; botón «Lo quiero»; Dónde comprarlo; nombre en España, sinopsis en español e idiomas)

accounts/management/commands/rematch_titles.py: Command.handle (tus horas) · Command.rematch (tus horas)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.achievements (tus logros) · Command.demo_achievements (tus logros de ejemplo) · Command.review (reseñas) · Command.lists (listas en las que sale)

accounts/management/commands/sync_libraries.py: Command.handle (tus horas; tus logros; botón «Lo quiero») · Command.sync_wishlist (botón «Lo quiero»)

accounts/matching.py: clean_name (tus horas) · normalize (tus horas) · name_key (tus horas) · _name_keys (tus horas) · _catalog_games (tus horas) · _title_parts (tus horas) · add_title (tus horas) · _platform_names (tus horas) · _fits (tus horas) · _share_platform (tus horas) · _seen_games (tus horas) · _score (tus horas) · match_by_name (tus horas) · save_titles (tus horas) · update_library (tus horas) · reassign (tus horas)

accounts/models.py: Profile.get_absolute_url · Profile.visible_filter (reseñas y listas) · ImportedTitle.set_achievements (tus logros) · Achievement.rarity (tus logros: su rareza)

accounts/psn.py: access_token (tus horas; tus logros) · _get (tus horas; tus logros) · _account_id_from_token (tus horas) · played_games (tus horas) · trophy_titles (tus logros) · trophies_by_title_id (tus logros) · trophy_key (tus logros) · trophy_count (tus logros) · save_trophies (tus logros) · duration_minutes (tus horas) · import_library (tus horas; tus logros)

accounts/steam.py: _api (tus horas; botón «Lo quiero») · owned_games (tus horas) · wishlist (botón «Lo quiero») · _match_games (tus horas; botón «Lo quiero») · _timestamp (tus horas; tus logros) · _played_at (tus horas) · import_library (tus horas; tus logros) · player_achievements (tus logros) · _icon (tus logros) · achievement_schema (tus logros) · global_percentages (tus logros) · achievement_info (tus logros) · _needs_info (tus logros) · import_achievements (tus logros) · _first (botón «Lo quiero») · _wishlist_games (botón «Lo quiero») · import_wishlist (botón «Lo quiero»)

accounts/views/cuentas_vinculadas.py: _fetch_library_prices (Dónde comprarlo de lo recién importado) · _import_steam (tus horas; tus logros; botón «Lo quiero») · steam_callback (tus horas; tus logros) · steam_import (tus horas; tus logros) · steam_wishlist (botón «Lo quiero») · psn_sync (tus horas; tus logros) · xbox_callback (tus horas; tus logros)

accounts/views/editar_perfil.py: shops_update (Dónde comprarlo: primero tus tiendas)

accounts/views/ficha_del_juego.py: favorite_context (botón «Añadir a mis favoritos») · favorite_toggle (botón «Añadir a mis favoritos») · game_achievements («Tus logros», al desplegarlos y al cambiar el orden) · banner_from_game (botón del visor del carrusel para ponerla de cabecera)

accounts/views/juegos_importados.py: imported_title_assign (tus horas)

accounts/xbox.py: _post (tus horas) · xbox_tokens (tus horas) · _get (tus horas) · played_titles (tus horas) · minutes_played (tus horas) · save_achievements (tus logros) · import_library (tus horas; tus logros)

games/history.py: needs_update (historial de precios de Dónde comprarlo) · _amount (historial de precios) · _decimal (historial de precios) · shop_series (historial de precios) · update_history (historial de precios) · lowest_series (historial de precios: el más barato a lo largo del tiempo) · _year (gráfica y veredicto del historial de precios) · verdict (veredicto del precio de hoy en el historial) · last_big_sale (última gran rebaja en el historial) · chart (gráfica del historial de precios) · shop_options (selector de tienda de la gráfica) · selected_shops (tiendas que enseña la gráfica) · summary (todo el historial de precios)

games/igdb.py: is_configured (carrusel) · IGDBClient.__init__ (carrusel) · IGDBClient._token (carrusel) · IGDBClient.query (carrusel) · IGDBClient.top_games · IGDBClient.games_by_steam_ids (tus horas) · IGDBClient.games_by_ids · IGDBClient.games_by_names (tus horas) · IGDBClient.spanish_data (nombre en España e idiomas) · IGDBClient.game_media (cabecera y carrusel de imágenes y vídeos) · IGDBClient._with_editions (Dónde comprarlo: los IDs de tienda de sus ediciones) · sort_arts (cabecera y carrusel) · _upsert_ref (géneros y plataformas) · _normalize (Dónde comprarlo: los IDs de tienda con los que se buscan) · _store_ids (Dónde comprarlo: los IDs de

tienda con los que se buscan) · _store_alternatives (Dónde comprarlo: los IDs de tienda de sus ediciones, ports y remasters) · _xbox_product_ids (Dónde comprarlo: el precio de Xbox) · upsert_games

games/management/commands/import_igdb.py: Command.handle (Dónde comprarlo: con los IDs de tienda que trae) · Command._import_top · Command._refresh_without_store_ids (Dónde comprarlo: los juegos que no tenían con qué buscar sus precios)

games/management/commands/localize_games.py: Command.handle (nombre en España, sinopsis en español e idiomas) · Command._from_igdb (nombre en España e idiomas) · Command._from_stores (sinopsis en español e idiomas de Steam y de Microsoft)

games/management/commands/seed_demo.py: Command.handle

games/management/commands/update_prices.py: Command.handle (Dónde comprarlo)

games/media.py: steam_app_id (imagen y logo de la cabecera de Steam) · needs_update (si el carrusel va ya o se carga después con HTMX) · update_media (cabecera y carrusel de imágenes y vídeos) · _size (carrusel) · _video_order (orden de los vídeos del carrusel) · _exists (imagen y logo de la cabecera) · _logo_crop (logo de la cabecera) · _igdb_logo_url (logo de la cabecera) · _is_wide (cabecera y carrusel) · hero (imagen y logo de la cabecera) · _credit (de dónde salen la imagen y el logo de la cabecera) · _hero_image (imagen de la cabecera) · carousel (carrusel de imágenes y vídeos) · _image_item (miniaturas del carrusel) · _video_item (vídeos del carrusel) · banner_arts (botón del visor del carrusel para ponerla de cabecera)

games/models.py: spanish_genre_name (géneros) · igdb_image_url (cabecera y carrusel) · Genre.__str__ (géneros) · Platform.short_name (plataformas de la cabecera) · Game.get_absolute_url · Game.cover_url · Game.merged_languages (nombre en España, sinopsis en español e idiomas) · Game.refresh_spanish (nombre en España, sinopsis en español e idiomas) · Game.spain_name (nombre en España de la cabecera) · Game.language_sources (de dónde salen los idiomas) · Game.language_rows (tabla de idiomas) · Game.release_year · Game.price_lookups (Dónde comprarlo: los IDs de tienda con los que se buscan) · Game.xbox_lookup_ids (Dónde comprarlo: el precio de Xbox) · Game.on_current_xbox (Dónde comprarlo: Xbox) · Game.nintendo_lookup_nsuid (Dónde comprarlo: el precio de Nintendo) · Game.is_free (Dónde comprarlo: «Gratis») · Game.price_groups (Dónde comprarlo: destacadas, el resto y las plegadas) · Game.store_searches (Dónde comprarlo: enlaces a buscarlo en las tiendas de consola sin precio) · Game.community_rating (valoración de la comunidad) · Game.critic_score (nota de la crítica) · Game.critic_tier (color de la nota de la crítica) · Game.historical_low_symbol (mínimo histórico) · currency_symbol (Dónde comprarlo) · format_price (historial de precios) · GamePrice.symbol (Dónde comprarlo) · GamePrice.is_free (Dónde comprarlo: «Gratis») · GamePrice.is_featured (Dónde comprarlo: tiendas destacadas) · GamePrice.logo_template (Dónde comprarlo: logos de las tiendas)

games/nintendo.py: _country (Dónde comprarlo: el precio de Nintendo) · _amount (Dónde comprarlo: el precio de Nintendo) · fetch_prices (Dónde comprarlo: el precio de Nintendo) · _offer (Dónde comprarlo: el precio de Nintendo) · _normalize (Dónde comprarlo: el precio de Nintendo) · search_game (Dónde comprarlo: el precio de Nintendo) · store_url (Dónde comprarlo: enlace a la eShop) · _systems (Dónde comprarlo: el precio de Nintendo) · _store (Dónde comprarlo: el precio de Nintendo) · update_nintendo_prices (Dónde comprarlo: el precio de Nintendo)

games/prices.py: is_configured (Dónde comprarlo: sin clave de IsThereAnyDeal no hay precios de PC) · ITADClient.__init__ (Dónde comprarlo: los de PC; historial de precios) · ITADClient._post (Dónde comprarlo: los de PC) · ITADClient._get (historial de precios) · ITADClient._send (Dónde comprarlo: los de PC; historial de precios) · ITADClient.lookup (Dónde comprarlo: los de PC) · ITADClient.prices (Dónde comprarlo: los de PC) · ITADClient.history (historial de precios) · _amount (Dónde comprarlo: los de PC) · _batches (Dónde comprarlo: los de PC) · _cheapest_per_shop (Dónde comprarlo: los de PC) · _store_prices (Dónde comprarlo: los de PC) · _lookup (Dónde comprarlo: los de PC) · _fetch_prices (Dónde comprarlo: los de PC) · _merge (Dónde comprarlo: los de PC) · update_prices (Dónde comprarlo: los de PC) · fetch_missing (Dónde comprarlo de un juego cuyos precios nunca se han buscado) · lookup_ids (historial de precios)

games/spanish.py: igdb_languages (nombre en España, sinopsis en español e idiomas) · spanish_names (nombre en España, sinopsis en español e idiomas) · europe_cover · apply_igdb (nombre en España, sinopsis en español e idiomas) · steam_languages (idiomas de Steam) · clean_text (sinopsis en español) · steam_details (sinopsis en español e idiomas de Steam) · localize_games (nombre en España, sinopsis en español e idiomas)

games/views.py: detail · game_media (carrusel de imágenes y vídeos que se carga con HTMX) · _user_shops (Dónde comprarlo e historial: primero tus tiendas) · _prices_context (Dónde comprarlo) · game_prices (Dónde comprarlo, cuando se carga con HTMX) · _history_context (historial de precios) · price_history (historial de precios: «¿compro ahora o espero?»)

games/xbox.py: _market (Dónde comprarlo: el precio de Xbox; sinopsis de Microsoft) · _locale (Dónde comprarlo: el precio de Xbox; sinopsis de Microsoft) · fetch_products (Dónde comprarlo: el precio de Xbox) · fetch_descriptions (sinopsis en español de la tienda de Microsoft) · _console_offer (Dónde comprarlo: el precio de Xbox) · _normalize (Dónde comprarlo: el precio de Xbox) · search_games (Dónde comprarlo: el precio de Xbox) · store_url (Dónde comprarlo: enlace a la tienda de Xbox) · _store (Dónde comprarlo: el precio de Xbox) · update_xbox_prices (Dónde comprarlo: el precio de Xbox)

library/forms.py: stars_label · ReviewImagesForm.save (imágenes de las reseñas) · ReportForm.clean_detail

library/markup.py: _inline · render · render_inline · plain

library/models.py: LibraryEntry.imported_status (tus horas) · LibraryEntry.update_import (tus horas) · LibraryEntry.playtime_minutes (tus horas) · LibraryEntry.playtime_hours (tus horas) · LibraryEntry.hours_by_platform (tus horas en cada plataforma) · LibraryEntry.achievements (tus logros en cada plataforma) · ReviewQuerySet.published (reseñas: sin los borradores) · ReviewQuerySet.visible_to (reseñas: solo las que puede leer quien mira) · Review.get_absolute_url (reseñas) · Review.finished · Review.status_note · Review.hours_source_names · Review.hours_source_logos · Review.snapshot · AttachedImage.is_official · AttachedImage.url · AttachedImage.thumb_url

library/points.py: key (lo que más destacan y lo que menos gusta) · _tally (lo que más destacan y lo que menos gusta) · _visible (lo que más destacan y lo que menos gusta) · consensus (lo que más destacan y lo que menos gusta)

library/templatetags/reviews.py: report_reasons · review_html · review_inline · review_plain · stars_label

library/views/comentarios_y_denuncias.py: _report

library/views/escribir_resena.py: _review_editor (reseñas) · review_delete (reseñas)

library/views/ficha_del_juego.py: mark_liked (reseñas) · review_context (reseñas) · _respond (Tu biblioteca: lo que cambia tras cada botón) · set_status (Tu biblioteca: estado) · remove (Tu biblioteca: quitar el juego) · rate (tu nota y la valoración de la comunidad)

library/views/lista_de_deseos.py: _wish_response (botón «Lo quiero») · wish_toggle (botón «Lo quiero») · wish_target (precio para avisar)

library/views/pagina_de_la_resena.py: _readable_review · review_like · review_report

social/activity.py: status_changed (al cambiar el estado) · rated (al poner o quitar la nota) · game_removed (al quitar un juego de la biblioteca) · list_games_added (desplegable «Añadir a una lista») · list_game_removed (desplegable «Añadir a una lista»)

lists/forms.py: GameListForm.save (listas en las que sale)

lists/models.py: GameListQuerySet.visible_to (listas en las que sale) · GameListQuerySet.editable_by (desplegable «Añadir a una lista») · GameListQuerySet.with_preview (listas en las que sale) · GameList.get_absolute_url (listas en las que sale) · GameList.can_edit_games (desplegable «Añadir a una lista») · GameList.is_tiered · GameList.noun · GameList.mosaic_gaps · GameList.touch (desplegable «Añadir a una lista»; listas en las que sale) · GameList.add (desplegable «Añadir a una lista») · GameList.remove (desplegable «Añadir a una lista»)

lists/views/comun.py: _editable_list (desplegable «Añadir a una lista»)

lists/views/editar_lista.py: editor_context (al añadir desde «Añadir a una lista») · add (desplegable «Añadir a una lista») · remove (desplegable «Añadir a una lista») · _item_response (desplegable «Añadir a una lista»)

lists/views/listas_del_juego.py: _lists_with (listas en las que sale) · picker_context (desplegable «Añadir a una lista») · game_page_context (desplegable «Añadir a una lista» y «En N listas»)

challenges/progress.py: _start (al pasar un juego a «Completado» se mira si consigues un reto) · _game_filters (al pasar un juego a «Completado» se mira si consigues un reto) · _completed (al pasar un juego a «Completado» se mira si consigues un reto) · done_by (al pasar un juego a «Completado» se mira si consigues un reto) · check (al pasar un juego a «Completado»)

JavaScript: base.html (línea 292) (desplegable «Añadir a una lista») · games/_media.html (línea 59) (carrusel de imágenes y vídeos) · games/_price_history.html (línea 44) (historial de precios) · games/detail.html (línea 7) (cabecera)

Listas del juego

Todas las listas en las que está un juego («Ver todas» desde su ficha).

Dirección: /juegos/<slug:slug>/listas/ (lists:game → lists.views.listas_del_juego.game_lists)

Plantillas: lists/game_lists.html
Trozos que incluye: lists/_card.html, lists/_mosaic.html, _icon.html

Funciones que la afectan (17)

accounts/management/commands/seed_test_profile.py: Command.lists

accounts/models.py: Profile.visible_filter (lo de perfiles privados solo lo ve su dueño)

games/models.py: igdb_image_url · Game.get_absolute_url · Game.cover_url

games/spanish.py: europe_cover · apply_igdb

lists/forms.py: GameListForm.save

lists/models.py: GameListQuerySet.visible_to · GameListQuerySet.with_preview · GameList.get_absolute_url · GameList.kind_note · GameList.mosaic_gaps · GameList.touch

lists/views/listas_del_juego.py: _lists_with · game_lists

lists/views/tarjetas.py: mark_progress

Perfil

La página de cada jugador (/u/<usuario>/): cabecera personalizada (foto, fondo, color, redes, cuentas, seguidores), escaparate (favoritos, jugando ahora, reseña destacada, estadísticas, logros, listas...) y las pestañas Biblioteca, Reseñas, Listas, Tier lists y Actividad. También se ve dentro de Editar perfil, en su vista previa.

Dirección: /u/<str:username>/ (profile → accounts.views.perfil.profile)
/u/<str:username>/actividad/ (social:activity → social.views.activity)

Plantillas: accounts/profile.html, social/_activities.html
Trozos que incluye: accounts/_avatar.html, _icon.html, social/_follower_count.html, social/_follow_button.html, accounts/linking/_dialog.html, games/_card.html, library/_platform_mark.html, library/_review.html, lists/_card.html, lists/_tier_card.html, accounts/_unlock.html, challenges/_card.html, social/_activity.html, accounts/linking/_steam.html, accounts/linking/_xbox.html, accounts/linking/_psn.html, accounts/linking/_platforms.html, library/_stars.html, library/_review_body.html, library/_like.html, library/_report.html, lists/_mosaic.html, social/_post_body.html, social/_post_like.html, lists/_like.html, library/_review_images.html, social/_post_images.html

Funciones que la afectan (301)

accounts/achievements.py: progress_by_game (insignias de las portadas y cifras de logros) · badge (insignia «Platino» o «100 %» de las portadas de la biblioteca) · totals (platinos y juegos al 100 % de las cifras del perfil) · save_info (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · save_unlocked (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · showcase (sección «Logros» del escaparate)

accounts/admin.py: ExternalTitleAdmin.save_model (biblioteca y horas)

accounts/forms.py: ProfileForm.save (todo lo que se personaliza) · ProfileForm._apply_extras (redes y escaparate) · ProfileForm._save_favorites (favoritos del escaparate) · PlatformAccountForm.clean (modal «Vincular cuentas»)

accounts/management/commands/nightly.py: Command.handle (insignias de la sección «Retos»)

accounts/management/commands/rematch_titles.py: Command.handle (biblioteca y horas) · Command.rematch (biblioteca y horas)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.achievements (logros de ejemplo) · Command.demo_achievements (sección «Logros») · Command.review (pestaña Reseñas) · Command.lists (pestañas Listas y Tier lists) · Command.activity (pestaña Actividad) · Command.create_empty_user

accounts/management/commands/sync_libraries.py: Command.handle (biblioteca y horas; insignias de las portadas, cifras y sección «Logros»)

accounts/matching.py: clean_name (modal «Vincular cuentas» de su dueño; biblioteca y horas) · normalize (modal «Vincular cuentas» de su dueño; biblioteca y horas) · name_key (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _name_keys (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _catalog_games (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _title_parts (modal «Vincular cuentas» de su dueño; biblioteca y horas) · add_title (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _platform_names (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _fits (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _share_platform (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _seen_games (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _score (modal «Vincular cuentas» de su dueño; biblioteca y horas) · match_by_name (modal «Vincular cuentas» de su dueño; biblioteca y horas) · save_titles (modal «Vincular cuentas» de su dueño; biblioteca y horas) · update_library (modal «Vincular cuentas» de su dueño; biblioteca y horas) · reassign (biblioteca y horas)

accounts/models.py: normalize_handle (redes de la cabecera) · text_on (texto sobre el color del perfil) · Profile.get_absolute_url · Profile.visible_to · Profile.steam_accounts (cuentas de la cabecera y modal «Vincular cuentas») · Profile.steam_linked (modal «Vincular cuentas») · Profile.psn (cuentas de la cabecera) · Profile.platforms (cuentas de la cabecera y modal «Vincular cuentas») · Profile.xbox (cuentas de la cabecera) · Profile.accent (color del perfil) · Profile.accent_rgb (color del perfil) · Profile.accent_text (texto sobre el color del perfil) · Profile.banner_url (cabecera) · Profile.banner_blurred (cabecera) · Profile.background_url (fondo de la página) · Profile._uploaded_url (cabecera y fondo subidos) · Profile.background_blurred (fondo de la página) · Profile.background_dim_ratio (lo que se oscurece el fondo) · Profile.banner_framing (encuadre de la cabecera) · Profile.background_framing (encuadre del fondo) · Profile._framing (encuadre de la cabecera y el fondo) · Profile.section_layout (escaparate) · Profile.visible_sections (las secciones del escaparate que se enseñan) · Profile.social_links (redes de la cabecera) · SteamAccount.name (cuentas de la cabecera y modal «Vincular cuentas») · SteamAccount.url (cuentas de la cabecera y modal «Vincular cuentas») · PSNAccount.name (cuentas de la cabecera y modal «Vincular cuentas») · PSNAccount.url (enlace de la cuenta de PlayStation en la cabecera) · PlatformAccount.logo_template (cuentas de la cabecera) · PlatformAccount.url (enlace de la cuenta de GOG en la cabecera) · PlatformAccount.normalize (modal «Vincular cuentas») · PlatformAccount.is_valid_handle (modal «Vincular cuentas») · XboxAccount.name (cuentas de la cabecera y modal «Vincular cuentas») · XboxAccount.url (enlace de la cuenta de Xbox en la cabecera) · ImportedTitle.set_achievements (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · Achievement.rarity (sección «Logros»: el más raro)

accounts/psn.py: clean_npsso (modal «Vincular cuentas» de su dueño) · access_token (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · _get (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · _account_id_from_token (modal «Vincular cuentas» de su dueño; biblioteca y horas) · profile (modal «Vincular cuentas» de su dueño; cuenta de PlayStation de la cabecera) · played_games (modal «Vincular cuentas» de su dueño; biblioteca y horas) · trophy_titles (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · trophies_by_title_id (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · trophy_key (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · trophy_count (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · save_trophies (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · duration_minutes (modal «Vincular cuentas» de su dueño; biblioteca y horas) · import_library (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»)

accounts/steam.py: is_configured (modal «Vincular cuentas» de su dueño) · login_url (modal «Vincular cuentas» de su dueño) · verify_login (modal «Vincular cuentas» de su dueño) · _api (modal «Vincular cuentas» de su dueño; biblioteca y horas) · player_summary (cuentas de la cabecera) · owned_games (modal «Vincular cuentas» de su dueño; biblioteca y horas) · refresh_account (cuentas de la cabecera) · _match_games (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _timestamp (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · _played_at (modal «Vincular cuentas» de su dueño; biblioteca y horas) · import_library (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · player_achievements (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · _icon (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · achievement_schema (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · global_percentages (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · achievement_info (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · _needs_info (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · import_achievements (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»)

accounts/views/comun.py: _back

accounts/views/cuentas_vinculadas.py: linking_context (modal «Vincular cuentas» de su dueño) · _steam_return_to (modal «Vincular cuentas» de su dueño) · _report_import (modal «Vincular cuentas» de su dueño) · _achievements_note (modal «Vincular cuentas» de su dueño) · _with_unmatched (modal «Vincular cuentas» de su dueño) · _report_wishlist (modal «Vincular cuentas» de su dueño) · _fetch_library_prices (modal «Vincular cuentas» de su dueño) · _import_steam (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · steam_link (modal «Vincular cuentas» de su dueño) · steam_callback (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · steam_import (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · steam_auto_sync (modal «Vincular cuentas» de su dueño) · steam_wishlist (modal «Vincular cuentas» de su dueño) · steam_unlink (modal «Vincular cuentas» de su dueño; cuentas de la cabecera) · _psn_error (modal «Vincular cuentas» de su dueño) · psn_sync (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · psn_unlink (modal «Vincular cuentas» de su dueño; cuentas de la cabecera) · platform_add (modal «Vincular cuentas» de su dueño; cuentas de la cabecera) · platform_remove (modal «Vincular cuentas» de su dueño; cuentas de la cabecera) · _xbox_redirect_uri (modal «Vincular cuentas» de su dueño) · xbox_link (modal «Vincular cuentas» de su dueño) · xbox_callback (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros») · xbox_unlink (modal «Vincular cuentas» de su dueño; cuentas de la cabecera)

accounts/views/ficha_del_juego.py: favorite_toggle (favoritos del escaparate) · banner_from_game (cabecera)

accounts/views/juegos_importados.py: imported_title_assign (biblioteca y horas)

accounts/views/perfil.py: profile · _profile_owners (cabecera: perfil, juego de la cabecera y cuentas) · _profile_context (cabecera, escaparate y pestañas) · _profile_stats (cifras de la cabecera: juegos, completados y horas)

accounts/xbox.py: is_configured (modal «Vincular cuentas» de su dueño) · login_url (modal «Vincular cuentas» de su dueño) · _post (modal «Vincular cuentas» de su dueño; biblioteca y horas) · xbox_tokens (modal «Vincular cuentas» de su dueño; biblioteca y horas) · _get (modal «Vincular cuentas» de su dueño; biblioteca y horas) · avatar (cuenta de Xbox de la cabecera) · played_titles (modal «Vincular cuentas» de su dueño; biblioteca y horas) · minutes_played (modal «Vincular cuentas» de su dueño; biblioteca y horas) · save_achievements (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros») · import_library (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»)

games/igdb.py: is_configured (modal «Vincular cuentas» de su dueño) · IGDBClient.__init__ (modal «Vincular cuentas» de su dueño) · IGDBClient._token (modal «Vincular cuentas» de su dueño) · IGDBClient.query (modal «Vincular cuentas» de su dueño) · IGDBClient.top_games · IGDBClient.search (modal «Vincular cuentas» de su dueño) · IGDBClient.games_by_steam_ids (modal «Vincular cuentas» de su dueño; biblioteca y horas: juegos de Steam que aún no están en el catálogo) · IGDBClient.games_by_ids (modal «Vincular cuentas» de su dueño) · IGDBClient.games_by_names (modal «Vincular cuentas» de su dueño; biblioteca y horas: juegos de PlayStation y Xbox que aún no están en el catálogo) · IGDBClient.spanish_data (filtro de idioma de la biblioteca) · IGDBClient._with_editions · upsert_games (modal «Vincular cuentas» de su dueño)

games/management/commands/import_igdb.py: Command.handle · Command._import_top

games/management/commands/localize_games.py: Command.handle (filtro de idioma) · Command._from_igdb (filtro de idioma)

games/management/commands/seed_demo.py: Command.handle

games/media.py: banner_arts (cabecera)

games/models.py: spanish_genre_name (géneros favoritos del escaparate) · igdb_image_url (cabecera con la imagen de un juego) · language_filter (filtro de idioma de la biblioteca) · Platform.short_name (plataforma de cada juego) · Game.get_absolute_url · Game.cover_url · Game.merged_languages (filtro de idioma de la biblioteca) · Game.refresh_spanish (filtro de idioma de la biblioteca) · Game.release_year

games/prices.py: fetch_missing (modal «Vincular cuentas» de su dueño)

games/spanish.py: igdb_languages (filtro de idioma de la biblioteca) · spanish_names (filtro de idioma de la biblioteca; modal «Vincular cuentas» de su dueño) · europe_cover · apply_igdb (filtro de idioma de la biblioteca) · steam_languages (filtro de idioma) · localize_games (filtro de idioma de la biblioteca)

library/backlog.py: is_stale (aviso de lo que queda por repasar) · pending (aviso de lo que queda por repasar) · answer (biblioteca y pestaña Actividad) · value (pestaña Pendiente: lo que cuesta)

library/forms.py: stars_label · ReportForm.clean_detail

library/markup.py: _inline · render · render_inline · plain

library/models.py: LibraryEntry.imported_status (modal «Vincular cuentas» de su dueño; biblioteca y horas: el estado con el que llega cada juego importado) · LibraryEntry.update_import (modal «Vincular cuentas» de su dueño; biblioteca y horas) · LibraryEntry.playtime_minutes (horas de la biblioteca) · LibraryEntry.playtime_hours (horas de la biblioteca) · LibraryEntry.hours_by_platform (horas de cada plataforma) · LibraryEntry.stars (nota media de las cifras de la cabecera) · PlaySession.add (modal «Vincular cuentas» de su dueño) · ReviewQuerySet.published (pestaña Reseñas y reseña destacada) · ReviewQuerySet.visible_to (pestaña Reseñas y reseña destacada) · Review.get_absolute_url (pestaña Reseñas y reseña destacada) · Review.finished · Review.status_note · Review.hours_source_names · Review.hours_source_logos · Review.snapshot (datos de la partida) · AttachedImage.is_official · AttachedImage.url · AttachedImage.thumb_url

library/reminders.py: remind_outdated_reviews (modal «Vincular cuentas» de su dueño)

library/templatetags/reviews.py: report_reasons · review_html · review_inline · review_plain · stars_label

library/views/comentarios_y_denuncias.py: _report

library/views/escribir_resena.py: _review_editor (pestaña Reseñas y reseña destacada) · review_delete (pestaña Reseñas)

library/views/ficha_del_juego.py: mark_liked (pestaña Reseñas) · set_status (biblioteca) · remove (biblioteca) · rate (notas de la biblioteca)

library/views/pagina_de_la_resena.py: _readable_review · review_like · review_report (botón «Denunciar» de las reseñas)

library/views/repasar_la_biblioteca.py: tidy_answer (biblioteca)

library/wishlist.py: steam_deals (mensaje al traer la lista de deseos de Steam desde Cuentas vinculadas)

social/activity.py: _recent (pestaña Actividad) · _game_activity (pestaña Actividad) · status_changed (pestaña Actividad) · rated (pestaña Actividad) · review_published (pestaña Actividad) · game_removed (pestaña Actividad) · post_published (pestaña Actividad) · games_imported (modal «Vincular cuentas» de su dueño; pestaña Actividad) · _list_activity (pestaña Actividad) · list_created (pestaña Actividad) · list_games_added (pestaña Actividad) · list_game_removed (pestaña Actividad) · _challenge_activity (pestaña Actividad) · challenge_joined (pestaña Actividad) · challenge_won (pestaña Actividad) · library_tidied (pestaña Actividad) · challenge_left (pestaña Actividad)

social/forms.py: PostForm.save (pestaña Actividad)

social/models.py: FollowManager.follow · FollowManager.unfollow · Post.get_absolute_url (pestaña Actividad) · Activity.is_empty (pestaña Actividad) · Activity.has_content (pestaña Actividad) · Activity.list_created (pestaña Actividad) · Activity.list_added (pestaña Actividad) · Activity.challenge_verb (pestaña Actividad) · Activity.verb (pestaña Actividad) · Activity.shown_rating (pestaña Actividad) · Activity.platform_name (pestaña Actividad) · Activity.import_count (pestaña Actividad)

social/signals.py: forget_empty_activity (pestaña Actividad)

social/views.py: follow_context (botón «Seguir» y contadores) · follow · _batch (pestaña Actividad) · activity_context (pestaña Actividad) · _cursor (pestaña Actividad) · _parse_cursor (pestaña Actividad) · activity (pestaña Actividad: «Cargar más») · _posts (pestaña Actividad) · mark_liked_posts · _readable_post (404 al pulsar algo de una publicación que no se puede ver) · post_delete (pestaña Actividad) · post_like

lists/forms.py: GameListForm.save (pestañas Listas y Tier lists y escaparate)

lists/models.py: GameListQuerySet.visible_to (pestañas Listas y Tier lists y escaparate) · GameListQuerySet.with_preview (pestañas Listas y Tier lists y escaparate: lo que enseña cada tarjeta) · GameList.get_absolute_url (pestañas Listas y Tier lists y escaparate) · GameList.is_ranked · GameList.is_tiered (pestaña Tier lists) · GameList.kind_note · GameList.noun · GameList.accepts_versions (pestaña Tier lists: la marca «Abierta») · GameList.public_parent (pestaña Tier lists) · GameList.make_version (pestaña Tier lists) · GameList.tier_list (tablero en miniatura) · GameList.mosaic_gaps · GameList.touch (pestañas Listas y Tier lists y escaparate: la fecha de la última vez que cambió)

lists/views/editar_lista.py: delete (pestañas Listas y Tier lists)

lists/views/nueva_lista.py: _create (pestañas Listas y Tier lists)

lists/views/pagina_de_la_lista.py: _readable_list · make_version (pestaña Tier lists) · like

lists/views/tarjetas.py: mark_progress (pestaña Listas) · mark_liked_lists · profile_lists (pestañas Listas y Tier lists y escaparate) · mark_boards (tablero en miniatura de cada tier list)

challenges/closing.py: close_all (insignias de la sección «Retos»)

challenges/management/commands/close_challenges.py: Command.handle (insignias de la sección «Retos»)

challenges/progress.py: mark_completed (insignias de la sección «Retos») · _award (pestaña Actividad) · check (modal «Vincular cuentas» de su dueño; insignias de la sección «Retos») · showcase (sección «Retos» del escaparate)

challenges/views.py: new (pestaña Actividad) · join (pestaña Actividad) · leave (pestaña Actividad)

JavaScript: accounts/linking/_dialog.html (línea 40) (modal «Vincular cuentas» de su dueño) · accounts/linking/_platforms.html (línea 40) (modal «Vincular cuentas»)

Seguidores y seguidos

Las listas de gente que sigue a un jugador y de gente a la que sigue, con el botón de seguir de cada uno.

Dirección: /u/<str:username>/seguidores/ (social:followers → social.views.followers)
/u/<str:username>/siguiendo/ (social:following → social.views.following)

Plantillas: social/people.html
Trozos que incluye: accounts/_avatar.html, social/_person.html, social/_follow_button.html, _icon.html, social/_follower_count.html

Funciones que la afectan (14)

accounts/models.py: text_on · Profile.get_absolute_url · Profile.visible_to · Profile.visible_filter · Profile.accent · Profile.accent_rgb · Profile.accent_text

social/models.py: FollowManager.follow · FollowManager.unfollow

social/views.py: follow_context · follow · followers · following · _people

Publicar

La página para escribir una publicación (texto, imágenes y el juego del que hablas). Normalmente se publica desde la caja de Inicio; esta página sale sin JavaScript o si falta algo.

Dirección: /publicar/ (social:post_new → social.views.post_new)

Plantillas: social/post_form.html
Trozos que incluye: _icon.html, social/_composer.html, accounts/_avatar.html

Funciones que la afectan (10)

accounts/images.py: process (imágenes)

library/models.py: attached_image_upload_to · AttachedImage.upload_folder

social/activity.py: post_published

social/forms.py: PostForm.clean_text (el texto) · PostForm.clean (texto e imágenes) · PostForm.save

social/models.py: PostImage.upload_folder (carpeta de las imágenes de las publicaciones)

social/views.py: composer_context · post_new

JavaScript: social/_composer.html (línea 37)

Página de la publicación

Una publicación entera: su autor, el juego, el texto, las imágenes, los «me gusta», los comentarios y los botones de borrar y denunciar.

Dirección: /u/<str:username>/publicaciones/<int:post_id>/ (social:post → social.views.post_detail)

Plantillas: social/post_detail.html
Trozos que incluye: accounts/_avatar.html, social/_post_body.html, social/_post_like.html, _icon.html, library/_report.html, library/_comments.html, social/_post_images.html

Funciones que la afectan (36)

accounts/models.py: Profile.get_absolute_url · Profile.visible_to

games/models.py: igdb_image_url · Game.get_absolute_url · Game.cover_url

games/spanish.py: europe_cover · apply_igdb

library/forms.py: ReportForm.clean_detail

library/markup.py: _inline · render · plain

library/models.py: AttachedImage.is_official (imágenes de la ficha frente a las subidas) · AttachedImage.url · AttachedImage.thumb_url (miniaturas de las imágenes) · AttachedImage.delete_files (al quitar una imagen o borrar lo que la lleva)

library/templatetags/reviews.py: report_reasons · review_html · review_plain · can_delete

library/views/comentarios_y_denuncias.py: comment_thread · _report

social/forms.py: PostForm.save

social/models.py: Post.get_absolute_url · PostComment.can_delete (botón «Borrar» de los comentarios)

social/signals.py: delete_post_image_files (al borrarla)

social/views.py: _posts (las tarjetas de las publicaciones) · mark_liked_posts (tus «me gusta» marcados) · _readable_post · post_detail · _post_comments (comentarios) · post_comment (comentarios) · post_comment_delete (botón «Borrar» de un comentario) · post_comment_report (denunciar un comentario) · post_delete (botón «Borrar») · post_like (botón «me gusta») · post_report (botón «Denunciar»)

Escribir reseña

El editor de reseñas: nota, texto con formato y su vista previa, datos de la partida (plataforma, horas, cuándo la acabaste...), lo mejor y lo peor, imágenes, spoilers y borrador.

Dirección: /biblioteca/<int:game_id>/resenas/nueva/ (library:review_new → library.views.escribir_resena.review_new)
/biblioteca/resenas/<int:review_id>/editar/ (library:review_edit → library.views.escribir_resena.review_edit)
/biblioteca/resenas/vista-previa/ (library:review_preview → library.views.escribir_resena.review_preview)

Plantillas: library/review_form.html, library/_review_preview.html
Trozos que incluye: library/_star_input.html, _icon.html

Funciones que la afectan (59)

accounts/images.py: process (imágenes)

accounts/models.py: Profile.visible_filter

games/media.py: _is_wide (imágenes de la ficha para la reseña) · official_images (imágenes de la ficha que se pueden poner sin subir nada) · _image_item (imágenes de la ficha)

games/models.py: igdb_image_url (imágenes de la ficha) · Game.__str__ (el juego elegido en los formularios) · Game.get_absolute_url · Game.review_platforms (plataformas que se pueden elegir) · Game.cover_url

games/spanish.py: europe_cover · apply_igdb

library/autofill.py: _platforms (la plataforma de la partida, rellenada con lo importado) · _played_on (la plataforma y las horas de la partida, rellenadas con lo importado) · _duration (tiempo jugado de las notas de la partida) · play_details (plataforma y fechas de la partida, rellenadas con lo importado)

library/forms.py: stars_label (la nota en estrellas) · PointsWidget.__init__ («Lo mejor» y «Lo peor») · PointsWidget.decompress («Lo mejor» y «Lo peor») · PointsField.__init__ («Lo mejor» y «Lo peor») · PointsField.compress («Lo mejor» y «Lo peor») · ReviewForm.__init__ (plataformas que se pueden elegir) · ReviewForm.clean_text (el texto) · ReviewForm.point_slots (casillas de «Lo mejor» y «Lo peor») · ReviewForm.pro_slots (casillas de «Lo mejor») · ReviewForm.con_slots (casillas de «Lo peor») · ReviewForm.clean (fechas de la partida) · MultipleImageField.__init__ (imágenes) · MultipleImageField.clean (imágenes) · ReviewImagesForm.__init__ (imágenes subidas y de la ficha) · ReviewImagesForm.free_slots (cuántas imágenes más caben) · ReviewImagesForm.clean (imágenes) · ReviewImagesForm.save (imágenes)

library/markup.py: _inline (vista previa) · render (vista previa)

library/models.py: LibraryEntry.playtime_minutes (horas de la partida) · LibraryEntry.playtime_hours (horas de la partida) · LibraryEntry.playtime_sources (de dónde salen las horas al publicarla) · Review.snapshot (al publicar se guardan el estado y las horas) · attached_image_upload_to · AttachedImage.upload_folder · AttachedImage.is_official · AttachedImage.thumb_url · AttachedImage.delete_files · ReviewImage.upload_folder (carpeta de las imágenes de las reseñas)

library/points.py: key (sugerencias de «Lo mejor» y «Lo peor») · _tally (sugerencias de «Lo mejor» y «Lo peor») · _visible (sugerencias de «Lo mejor» y «Lo peor») · suggestions (sugerencias de «Lo mejor» y «Lo peor»)

library/signals.py: delete_review_image_files (al quitar una imagen o borrar la reseña)

library/templatetags/reviews.py: review_html (vista previa) · stars_label (la nota en estrellas)

library/views/escribir_resena.py: review_new · review_edit · _review_editor (publicar o guardar el borrador) · review_delete (botón «Borrar») · review_preview (vista previa del texto con formato)

social/activity.py: review_published (al publicar)

social/signals.py: forget_empty_activity (botón «Borrar»)

JavaScript: library/review_form.html (línea 181)

Página de la reseña

Una reseña entera: el juego, la nota, los datos de la partida, sus imágenes, los «me gusta», los comentarios y los botones de compartir, editar y denunciar.

Dirección: /u/<str:username>/resenas/<int:review_id>/ (review_detail → library.views.pagina_de_la_resena.review_detail)

Plantillas: library/review_detail.html
Trozos que incluye: _icon.html, library/_review.html, library/_comments.html, accounts/_avatar.html, library/_stars.html, library/_review_body.html, library/_like.html, library/_report.html, library/_review_images.html

Funciones que la afectan (50)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.review

accounts/models.py: Profile.get_absolute_url · Profile.visible_to

games/models.py: igdb_image_url · Platform.short_name (plataforma de la partida) · Game.get_absolute_url · Game.cover_url · Game.release_year

games/spanish.py: europe_cover · apply_igdb

library/forms.py: stars_label · ReviewImagesForm.save (imágenes) · ReportForm.clean_detail

library/markup.py: _inline · render · render_inline · plain

library/models.py: LibraryEntry.playtime_sources (sello de horas verificadas) · ReviewQuerySet.published · ReviewQuerySet.visible_to · Review.get_absolute_url · Review.finished · Review.status_note (cómo iba el autor con el juego al publicarla) · Review.hours_source_names (de dónde salen las horas) · Review.hours_source_logos (sello de horas verificadas) · Review.hours_since (propuesta de actualizarla) · Review.snapshot · ReviewComment.can_delete (botón «Borrar» de los comentarios) · AttachedImage.is_official · AttachedImage.url · AttachedImage.thumb_url

library/reminders.py: latest_reviews (propuesta de actualizarla)

library/templatetags/reviews.py: report_reasons · review_html · review_inline · review_plain · stars_label · can_delete (botón «Borrar» de cada comentario)

library/views/comentarios_y_denuncias.py: comment_thread · _report

library/views/escribir_resena.py: _review_editor

library/views/pagina_de_la_resena.py: _readable_review · review_detail · _comments_context (comentarios) · review_like (botón «me gusta» de las reseñas) · review_comment (comentarios) · comment_delete (botón «Borrar» de un comentario) · review_report · comment_report (denunciar un comentario)

Lista de deseos

«Deseos» en el menú: los juegos que quieres, con el mejor precio de hoy, su mínimo histórico, tu precio límite y si están en tu lista de Steam; se ordena y se filtra por idioma.

Dirección: /biblioteca/lista-de-deseos/ (library:wishlist → library.views.lista_de_deseos.wishlist)

Plantillas: library/wishlist.html
Trozos que incluye: _icon.html

Funciones que la afectan (91)

accounts/management/commands/nightly.py: Command.handle (mejor precio de hoy y mínimo histórico; filtro de idioma)

accounts/management/commands/sync_libraries.py: Command.handle · Command.sync_wishlist

accounts/models.py: Profile.steam_linked (aviso de traer la de Steam)

accounts/notices.py: wants (cómo te avisamos de las bajadas de precio)

accounts/steam.py: _api · wishlist · _match_games · _first · _wishlist_games · import_wishlist

accounts/views/cuentas_vinculadas.py: _fetch_library_prices (precios) · _import_steam · steam_wishlist

accounts/views/editar_perfil.py: shops_update (mejor precio en tus tiendas)

games/igdb.py: IGDBClient.top_games · IGDBClient.games_by_ids · IGDBClient.spanish_data (filtro de idioma) · IGDBClient._with_editions (mejor precio de hoy y mínimo histórico) · _normalize (mejor precio de hoy y mínimo histórico) · _store_ids (mejor precio de hoy y mínimo histórico) · _store_alternatives (mejor precio de hoy y mínimo histórico) · _xbox_product_ids (mejor precio de hoy y mínimo histórico) · upsert_games

games/management/commands/import_igdb.py: Command.handle (mejor precio de hoy y mínimo histórico) · Command._import_top · Command._refresh_without_store_ids (mejor precio de hoy y mínimo histórico)

games/management/commands/localize_games.py: Command.handle (filtro de idioma) · Command._from_igdb (filtro de idioma)

games/management/commands/seed_demo.py: Command.handle

games/management/commands/update_prices.py: Command.handle (mejor precio de hoy y mínimo histórico)

games/models.py: igdb_image_url · language_filter (filtro de idioma) · Game.get_absolute_url · Game.cover_url · Game.merged_languages (filtro de idioma) · Game.refresh_spanish (filtro de idioma) · Game.price_lookups (mejor precio de hoy y mínimo histórico) · Game.xbox_lookup_ids (mejor precio de hoy y mínimo histórico) · Game.nintendo_lookup_nsuid (mejor precio de hoy y mínimo histórico) · Game.best_price (mejor precio de hoy) · Game.cheapest (mejor precio de hoy en tus tiendas) · Game.historical_low_symbol (mínimo histórico) · currency_symbol · GamePrice.symbol (precios)

games/nintendo.py: _country (mejor precio de hoy y mínimo histórico) · _amount (mejor precio de hoy y mínimo histórico) · fetch_prices (mejor precio de hoy y mínimo histórico) · _offer (mejor precio de hoy y mínimo histórico) · _normalize (mejor precio de hoy y mínimo histórico) · search_game (mejor precio de hoy y mínimo histórico) · _systems (mejor precio de hoy y mínimo histórico) · _store (mejor precio de hoy y mínimo histórico) · update_nintendo_prices (mejor precio de hoy y mínimo histórico)

games/prices.py: is_configured (mejor precio de hoy y mínimo histórico) · ITADClient.__init__ (mejor precio de hoy y mínimo histórico) · ITADClient._post (mejor precio de hoy y mínimo histórico) · ITADClient._send (mejor precio de hoy y mínimo histórico) · ITADClient.lookup (mejor precio de hoy y mínimo histórico) · ITADClient.prices (mejor precio de hoy y mínimo histórico) · _amount (mejor precio de hoy y mínimo histórico) · _batches (mejor precio de hoy y mínimo histórico) · _cheapest_per_shop (mejor precio de hoy y mínimo histórico) · _store_prices (mejor precio de hoy y mínimo histórico) · _lookup (mejor precio de hoy y mínimo histórico) · _fetch_prices (mejor precio de hoy y mínimo histórico) · _merge (mejor precio de hoy y mínimo histórico) · update_prices (mejor precio de hoy y mínimo histórico) · fetch_missing

games/spanish.py: igdb_languages (filtro de idioma) · spanish_names (filtro de idioma) · europe_cover · apply_igdb (filtro de idioma) · steam_languages (filtro de idioma) · localize_games (filtro de idioma)

games/xbox.py: _market (mejor precio de hoy y mínimo histórico) · _locale (mejor precio de hoy y mínimo histórico) · fetch_products (mejor precio de hoy y mínimo histórico) · _console_offer (mejor precio de hoy y mínimo histórico) · _normalize (mejor precio de hoy y mínimo histórico) · search_games (mejor precio de hoy y mínimo histórico) · _store (mejor precio de hoy y mínimo histórico) · update_xbox_prices (mejor precio de hoy y mínimo histórico)

library/models.py: WishlistItem.on_steam (sello de la lista de Steam)

library/views/lista_de_deseos.py: _wish_response · wish_toggle · wish_target (tu precio límite) · _steam_order (orden «Como en Steam») · wishlist

library/wishlist.py: at_historical_low (en su mínimo histórico)

lists/views/pagina_de_la_lista.py: wish_missing

Repasar la biblioteca

Desde el aviso de tu perfil: los juegos que las importaciones no saben si terminaste (los de «Jugado» que nunca has repasado y los de «Jugando» que llevan meses sin tocarse), de uno en uno, con tus horas, tu última partida y tus logros, para contestar con un botón: lo terminé, lo dejé, sigo jugando, al pendiente o saltar. Al acabar, lo que has repasado hoy.

Dirección: /biblioteca/repasar/ (library:tidy → library.views.repasar_la_biblioteca.tidy)

Plantillas: library/tidy.html
Trozos que incluye: _icon.html, library/_tidy_card.html

Funciones que la afectan (14)

library/backlog.py: last_played (cuándo lo jugaste por última vez y la fecha con la que se apunta terminado) · is_stale (los de «Jugando» que se preguntan) · pending · queue (el siguiente juego y cuántos quedan) · _complete_games (los que se preguntan primero) · _number (las horas de cada juego) · hint (la tarjeta de cada juego) · answer (botones de cada tarjeta) · today_summary (lo que llevas hoy y el resumen al acabar)

library/views/repasar_la_biblioteca.py: _card_context · tidy · tidy_answer (botones de cada tarjeta)

social/activity.py: library_tidied (al contestar)

challenges/views.py: new (botón «Ponte un reto para este año»)

Página de la lista

Una lista o tier list: mosaico, título, autor y colaboradores, descripción, los juegos (en portadas, con comentarios o por niveles), cuánto llevas jugado de ella, «me gusta», comentarios y denuncias. En las tier lists abiertas a la comunidad, cuánta gente la ha hecho, «Haz tu versión» y la tier list de la comunidad; en las versiones, de cuál son.

Dirección: /u/<str:username>/listas/<int:list_id>/ (lists:detail → lists.views.pagina_de_la_lista.detail)

Plantillas: lists/detail.html
Trozos que incluye: lists/_mosaic.html, _icon.html, accounts/_avatar.html, lists/_like.html, library/_report.html, lists/_tier_board.html, lists/_drag.html, games/_card.html, library/_comments.html, library/_stars.html, library/_platform_mark.html

Funciones que la afectan (143)

accounts/management/commands/nightly.py: Command.handle (lo que cuesta lo que te falta)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.lists

accounts/models.py: text_on (texto sobre el color de cada nivel de las tier lists) · Profile.get_absolute_url · Profile.visible_to · Profile.accent · Profile.accent_rgb · Profile.accent_text

accounts/views/cuentas_vinculadas.py: _fetch_library_prices (lo que cuesta lo que te falta)

games/igdb.py: IGDBClient.top_games · IGDBClient.games_by_ids · IGDBClient._with_editions (lo que cuesta lo que te falta) · _normalize (lo que cuesta lo que te falta) · _store_ids (lo que cuesta lo que te falta) · _store_alternatives (lo que cuesta lo que te falta) · _xbox_product_ids (lo que cuesta lo que te falta) · upsert_games

games/management/commands/import_igdb.py: Command.handle (lo que cuesta lo que te falta) · Command._import_top · Command._refresh_without_store_ids (lo que cuesta lo que te falta)

games/management/commands/seed_demo.py: Command.handle

games/management/commands/update_prices.py: Command.handle (lo que cuesta lo que te falta)

games/models.py: igdb_image_url · Game.get_absolute_url · Game.cover_url · Game.release_year · Game.price_lookups (lo que cuesta lo que te falta) · Game.xbox_lookup_ids (lo que cuesta lo que te falta) · Game.nintendo_lookup_nsuid (lo que cuesta lo que te falta) · Game.best_price (lo que cuesta lo que te falta) · Game.cheapest (lo que cuesta lo que te falta) · currency_symbol (los símbolos de moneda) · format_price (lo que cuesta lo que te falta)

games/nintendo.py: _country (lo que cuesta lo que te falta) · _amount (lo que cuesta lo que te falta) · fetch_prices (lo que cuesta lo que te falta) · _offer (lo que cuesta lo que te falta) · _normalize (lo que cuesta lo que te falta) · search_game (lo que cuesta lo que te falta) · _systems (lo que cuesta lo que te falta) · _store (lo que cuesta lo que te

falta) · update_nintendo_prices (lo que cuesta lo que te falta)

games/prices.py: is_configured (lo que cuesta lo que te falta) · ITADClient.__init__ (lo que cuesta lo que te falta) · ITADClient._post (lo que cuesta lo que te falta) · ITADClient._send (lo que cuesta lo que te falta) · ITADClient.lookup (lo que cuesta lo que te falta) · ITADClient.prices (lo que cuesta lo que te falta) · _amount (lo que cuesta lo que te falta) · _batches (lo que cuesta lo que te falta) · _cheapest_per_shop (lo que cuesta lo que te falta) · _store_prices (lo que cuesta lo que te falta) · _lookup (lo que cuesta lo que te falta) · _fetch_prices (lo que cuesta lo que te falta) · _merge (lo que cuesta lo que te falta) · update_prices (lo que cuesta lo que te falta) · fetch_missing

games/spanish.py: europe_cover · apply_igdb

games/xbox.py: _market (lo que cuesta lo que te falta) · _locale (lo que cuesta lo que te falta) · fetch_products (lo que cuesta lo que te falta) · _console_offer (lo que cuesta lo que te falta) · _normalize (lo que cuesta lo que te falta) · search_games (lo que cuesta lo que te falta) · _store (lo que cuesta lo que te falta) · update_xbox_prices (lo que cuesta lo que te falta)

library/forms.py: stars_label · ReportForm.clean_detail (formularios de denuncia)

library/markup.py: _inline · render · render_inline («Lo mejor», «Lo peor» y otros textos cortos con formato) · plain (resúmenes sin formato y con los spoilers tapados)

library/models.py: LibraryEntry.playtime_minutes · LibraryEntry.playtime_hours · LibraryEntry.hours_by_platform

library/templatetags/reviews.py: report_reasons (formularios de denuncia) · review_html · review_inline («Lo mejor», «Lo peor» y otros textos cortos con formato) · review_plain (resúmenes sin formato y con los spoilers tapados) · stars_label · can_delete (comentarios)

library/views/comentarios_y_denuncias.py: comment_thread (comentarios) · _report (formularios de denuncia)

library/views/ficha_del_juego.py: set_status (estado de cada juego) · remove (estado de cada juego)

social/activity.py: list_games_added (tablero de las tier lists) · list_game_removed (tablero de las tier lists)

lists/community.py: counted_lists (la tier list de la comunidad) · middle_tier (el nivel de cada juego en la tier list de la comunidad) · community_board (la tier list de la comunidad)

lists/forms.py: GameListForm.save

lists/models.py: GameListQuerySet.visible_to (solo las que puede ver quien mira) · GameList.get_absolute_url · GameList.visible_to (404 si no puedes verla) · GameList.can_edit_games (botones para editar sus juegos) · GameList.is_ranked · GameList.is_tiered · GameList.noun · GameList.profile_tab · GameList.accepts_versions (botón «Haz tu versión» y la tier list de la comunidad) · GameList.public_parent (de cuál es versión) · GameList.make_version (botón «Haz tu versión») · GameList.tier_list (tablero) · GameList.tier_rows (tablero de la tier list) · GameList.mosaic_gaps · GameList.touch (tablero de las tier lists) · GameList.add (tablero de las tier lists y las versiones de las abiertas) · GameList.remove (tablero de las tier lists) · GameList.move · GameList.place (arrastrar por el tablero de una tier list) · GameList._save_order (el orden de los juegos) · ListComment.can_delete (botón «Borrar» de los comentarios)

lists/presets.py: Preset.title_for · Preset.tier_list · Preset.tier_of (tablero)

lists/views/colaboradores.py: invite (le pregunta si acepta) · accept (aceptar la invitación) · leave (rechazar la invitación)

lists/views/comun.py: _editable_list (tablero de las tier lists)

lists/views/editar_lista.py: editor_context · board_context (tablero de la tier list al soltar una portada) · add (tablero de las tier lists) · remove (tablero de las tier lists) · move · note (los comentarios de los juegos) · tier (arrastrar por el tablero) · _item_response (tablero de las tier lists)

lists/views/nueva_lista.py: from_library

lists/views/pagina_de_la_lista.py: detail · _readable_list · make_version (botón «Haz tu versión») · like · _list_comments (comentarios) · comment (comentarios) · comment_delete (botón «Borrar» de un comentario) · comment_report (denunciar un comentario) · report (botón «Denunciar») · list_progress (cuánto llevas jugado y lo que cuesta lo que te falta) · _missing (lo que te falta y lo que cuesta) · wish_missing (botón «Lo quiero» de lo que te falta)

lists/views/tarjetas.py: mark_liked_lists

JavaScript: lists/_drag.html (línea 6) (tablero de las tier lists)

Nueva lista

El formulario para crear una lista normal o numerada.

Dirección: /listas/nueva/ (lists:new → lists.views.nueva_lista.new)

Plantillas: lists/form.html
Trozos que incluye: _icon.html, lists/_fields.html

Funciones que la afectan (12)

social/activity.py: list_created

lists/forms.py: GameListForm.__init__ (con los niveles de las tier lists) · GameListForm.switches (interruptor de si la ven los demás) · GameListForm.tier_fields (filas de los niveles) · GameListForm.clean_title (el título) · GameListForm.clean_description (la descripción) · GameListForm.clean (los niveles y sus colores) · GameListForm.save (al guardar) · NewListForm.__init__ (los tipos que se pueden elegir)

lists/models.py: GameList.is_tiered

lists/views/nueva_lista.py: _create · new

Nueva tier list

La página para crear una tier list: en blanco con sus niveles (y, si quieres, abierta a la comunidad) o desde una plantilla rellena con tu biblioteca.

Dirección: /tier-lists/nueva/ (lists:new_tier_list → lists.views.nueva_lista.new_tier_list)

Plantillas: lists/tier_form.html
Trozos que incluye: _icon.html, lists/_mosaic.html, lists/_fields.html

Funciones que la afectan (38)

accounts/achievements.py: progress_by_game (plantilla de juegos con todos los logros)

accounts/models.py: text_on (niveles)

games/models.py: igdb_image_url · Game.cover_url

games/spanish.py: europe_cover · apply_igdb

library/models.py: PlaySession.add (plantillas por horas jugadas)

social/activity.py: list_created (al crearla)

lists/forms.py: GameListForm.__init__ · GameListForm.switches · GameListForm.tier_fields · GameListForm.clean_title · GameListForm.clean_description · GameListForm.clean · GameListForm.save · TierListForm.__init__ (niveles de serie y ejemplos) · TierListForm.clean_mode (siempre tier list)

lists/models.py: default_tiers (niveles de serie) · GameList.is_tiered · GameList.tier_list · GameList.mosaic_gaps (huecos del mosaico de portadas)

lists/presets.py: library (plantillas «Desde tu biblioteca») · year_bounds (plantillas «Desde tu biblioteca»: las del año) · played_in (plantillas «Desde tu biblioteca»: lo jugado este año) · completed_in (plantillas «Desde tu biblioteca»: lo terminado este año) · most_played (plantillas «Desde tu biblioteca»: lo más jugado) · full_achievements (plantillas «Desde tu biblioteca»: lo que tiene con todos los logros) · played_on (plantillas «Desde tu biblioteca»: las de cada plataforma) · backlog (plantillas «Desde tu biblioteca»: lo pendiente) · Preset.label_for (plantillas «Desde tu biblioteca»: nombre de cada plantilla) · Preset.title_for (plantillas «Desde tu biblioteca»: título de la tier list que crea) · Preset.choose (plantillas «Desde tu biblioteca»: los juegos que entran) · Preset.tier_list (plantillas «Desde tu biblioteca»: sus niveles) · Preset.tier_of (plantillas «Desde tu biblioteca»: el nivel en el que empieza cada juego) · choices (plantillas «Desde tu biblioteca»)

lists/views/nueva_lista.py: _create (al crearla) · new_tier_list · from_library (botón de cada plantilla)

Editar lista

El editor de una lista: sus datos (en las tier lists, también si está abierta a la comunidad), buscar y añadir juegos, ordenarlos (arrastrando), ponerlos en un nivel, comentarlos y quitarlos, e invitar a quien colabora.

Dirección: /listas/<int:list_id>/editar/ (lists:edit → lists.views.editar_lista.edit)
/listas/<int:list_id>/buscar/ (lists:search → lists.views.editar_lista.search)

Plantillas: lists/edit.html, lists/_search_results.html
Trozos que incluye: _icon.html, lists/_fields.html, lists/_editor_items.html, lists/_drag.html, lists/_collaborators.html, lists/_search_result.html, lists/_tier_board.html, lists/_note_form.html, accounts/_avatar.html

Funciones que la afectan (54)

accounts/models.py: text_on (niveles) · Profile.get_absolute_url

games/models.py: igdb_image_url · search_filter (buscar juegos para añadir) · Game.get_absolute_url · Game.cover_url · Game.release_year

games/spanish.py: europe_cover · apply_igdb

social/activity.py: list_games_added (los juegos: al añadir juegos) · list_game_removed (los juegos: al quitar un juego)

lists/forms.py: GameListForm.__init__ (los datos de la lista) · GameListForm._tiers_lock (los niveles de una tier list abierta con versiones y los de una versión) · GameListForm.switches (los datos de la lista) · GameListForm.tier_fields (los datos de la lista) · GameListForm.clean_title (los datos de la lista) · GameListForm.clean_description (los datos de la lista) · GameListForm.clean (los datos de la lista) · GameListForm.save (los datos de la lista)

lists/models.py: default_tiers (niveles al convertir una lista en tier list) · GameList.get_absolute_url · GameList.visible_to · GameList.can_edit_games · GameList.is_ranked · GameList.is_tiered · GameList.noun · GameList.profile_tab (a qué pestaña del perfil vuelven) · GameList.tier_list (niveles) · GameList.tier_rows (juegos por niveles) · GameList.touch (los juegos) · GameList.add (los juegos: añadir un juego) · GameList.remove (los juegos: quitar un juego) · GameList.move (cambiar de puesto un juego) · GameList.place (nivel de cada juego) · GameList._save_order

lists/views/colaboradores.py: collaborators_context («Quién puede editarla») · _collaborators_response («Quién puede editarla») · invite (invitar a alguien) · collaborator_remove (quitar a alguien o retirar su invitación) · accept · leave (dejar de colaborar)

lists/views/comun.py: _own_list (404 si no es tuya) · _editable_list (los juegos: 404 si no puedes editarla)

lists/views/editar_lista.py: editor_context (los juegos del editor) · search_context (buscador de juegos para añadir) · edit · delete (botón «Borrar») · search (resultados del buscador) · add (los juegos: añadir un juego) · remove (los juegos: quitar un juego) · move (subir, bajar o arrastrar un juego) · note (comentario de cada juego) · tier (nivel de cada juego) · _item_response (los juegos: lo que cambia tras añadir, quitar, mover o poner en un nivel)

JavaScript: lists/_drag.html (línea 6) (orden de los juegos y tablero de las tier lists)

Tier lists

«Tier lists» en el menú: las tuyas, las de la gente que sigues y las abiertas a la comunidad (las que más gente ha hecho y las últimas), cada una con su tablero en miniatura.

Dirección: /tier-lists/ (lists:tier_lists → lists.views.tier_lists.explore)

Plantillas: lists/explore.html
Trozos que incluye: _icon.html, lists/_tier_card.html

Funciones que la afectan (10)

accounts/management/commands/seed_test_profile.py: Command.lists

lists/models.py: GameListQuerySet.with_preview · GameList.accepts_versions (las abiertas) · GameList.public_parent · GameList.make_version

lists/views/pagina_de_la_lista.py: make_version

lists/views/tarjetas.py: mark_progress · profile_lists (las tuyas) · mark_boards (el de cada tarjeta)

lists/views/tier_lists.py: explore

Retos

«Retos» en el menú: tus retos con cómo vas, los de la gente que sigues, los más populares y los últimos creados.

Dirección: /retos/ (challenges:explore → challenges.views.explore)

Plantillas: challenges/explore.html
Trozos que incluye: _icon.html, challenges/_card.html

Funciones que la afectan (78)

accounts/management/commands/nightly.py: Command.handle (progreso)

accounts/management/commands/seed_test_profile.py: Command.handle · Command.challenges

accounts/management/commands/sync_libraries.py: Command.handle (progreso)

accounts/matching.py: clean_name (progreso) · normalize (progreso) · name_key (progreso) · _name_keys (progreso) · _catalog_games (progreso) · _title_parts (progreso) · add_title (progreso) · _platform_names (progreso) · _fits (progreso) · _share_platform (progreso) · _seen_games (progreso) · _score (progreso) · match_by_name (progreso) · save_titles (progreso) · update_library (progreso)

accounts/psn.py: access_token (progreso) · _get (progreso) · _account_id_from_token (progreso) · played_games (progreso) · duration_minutes (progreso) · import_library (progreso)

accounts/steam.py: _api (progreso) · owned_games (progreso) · _match_games (progreso) · _timestamp (progreso) · _played_at (progreso) · import_library (progreso)

accounts/views/cuentas_vinculadas.py: _import_steam (progreso) · steam_callback (progreso) · steam_import (progreso) · psn_sync (progreso) · xbox_callback (progreso)

accounts/xbox.py: _post (progreso) · xbox_tokens (progreso) · _get (progreso) · played_titles (progreso) · minutes_played (progreso) · import_library (progreso)

games/igdb.py: IGDBClient.games_by_steam_ids (progreso) · IGDBClient.games_by_names (progreso)

library/backlog.py: answer

library/models.py: LibraryEntry.imported_status (progreso) · LibraryEntry.update_import (progreso) · PlaySession.add (progreso de los de horas)

library/views/ficha_del_juego.py: set_status (progreso) · remove (progreso)

library/views/repasar_la_biblioteca.py: tidy_answer

challenges/closing.py: close_all

challenges/forms.py: ChallengeForm.save

challenges/management/commands/close_challenges.py: Command.handle

challenges/models.py: long_date · ChallengeQuerySet.visible_to · ChallengeQuerySet.with_counts · Challenge.get_absolute_url · Challenge.goal (cómo vas en cada uno) · Challenge.is_upcoming · Challenge.is_over · Challenge.days_left · Challenge.filters_text · Challenge.rules_text · Challenge.dates_text

challenges/progress.py: _start (cómo vas en cada uno) · _game_filters (cómo vas en cada uno) · _completed (cómo vas en cada uno) · done_by (cómo vas en cada uno) · summary (cómo vas en cada uno) · mark_completed (cómo vas en cada uno: quién lo ha conseguido) · check (cómo vas en cada uno)

challenges/views.py: _open (los que siguen abiertos) · _with_progress (cómo vas en cada uno) · explore · delete · join · leave

Página del reto

Un reto: qué hay que hacer y hasta cuándo, cómo vas tú y la clasificación de quienes se han unido, con los botones de unirse y dejarlo.

Dirección: /retos/<int:challenge_id>/ (challenges:detail → challenges.views.detail)

Plantillas: challenges/detail.html
Trozos que incluye: _icon.html, accounts/_avatar.html, games/_card.html, library/_stars.html, library/_platform_mark.html

Funciones que la afectan (106)

accounts/management/commands/nightly.py: Command.handle (progreso y quién lo ha conseguido)

accounts/management/commands/seed_test_profile.py: Command.handle (los datos de ejemplo de la cuenta de prueba) · Command.challenges

accounts/management/commands/sync_libraries.py: Command.handle (progreso)

accounts/matching.py: clean_name (progreso) · normalize (progreso) · name_key (progreso) · _name_keys (progreso) · _catalog_games (progreso) · _title_parts (progreso) · add_title (progreso) · _platform_names (progreso) · _fits (progreso) · _share_platform (progreso) · _seen_games (progreso) · _score (progreso) · match_by_name (progreso) · save_titles (progreso) · update_library (progreso)

accounts/models.py: Profile.get_absolute_url (los enlaces al perfil de cada jugador) · Profile.visible_to (los perfiles privados solo los ve su dueño) · Profile.visible_filter (clasificación)

accounts/psn.py: access_token (progreso) · _get (progreso) · _account_id_from_token (progreso) · played_games (progreso) · duration_minutes (progreso) · import_library (progreso)

accounts/steam.py: _api (progreso) · owned_games (progreso) · _match_games (progreso) · _timestamp (progreso) · _played_at (progreso) · import_library (progreso)

accounts/views/cuentas_vinculadas.py: _import_steam (progreso) · steam_callback (progreso) · steam_import (progreso) · psn_sync (progreso) · xbox_callback (progreso)

accounts/xbox.py: _post (progreso) · xbox_tokens (progreso) · _get (progreso) · played_titles (progreso) · minutes_played (progreso) · import_library (progreso)

games/igdb.py: IGDBClient.top_games (los datos de cada juego: nombre, portada, géneros y plataformas) · IGDBClient.games_by_steam_ids (progreso) · IGDBClient.games_by_ids (los datos de cada juego: nombre, portada, géneros y plataformas) · IGDBClient.games_by_names (progreso) · IGDBClient._with_editions (los datos de cada juego: nombre, portada, géneros y plataformas) · upsert_games (los datos de cada juego: nombre, portada, géneros y plataformas)

games/management/commands/import_igdb.py: Command.handle (los datos de cada juego: nombre, portada, géneros y plataformas) · Command._import_top (los datos de cada juego: nombre, portada, géneros y plataformas)

games/management/commands/seed_demo.py: Command.handle (los datos de cada juego: nombre, portada, géneros y plataformas)

games/models.py: igdb_image_url · Game.get_absolute_url · Game.cover_url · Game.release_year

games/spanish.py: europe_cover · apply_igdb

library/backlog.py: answer (progreso)

library/forms.py: stars_label

library/markup.py: _inline (el texto con formato de reseñas, comentarios, publicaciones, listas y retos) · render (el texto con formato de reseñas, comentarios, publicaciones, listas y retos)

library/models.py: LibraryEntry.imported_status (progreso) · LibraryEntry.update_import (progreso) · LibraryEntry.playtime_minutes (horas en las portadas) · LibraryEntry.playtime_hours (horas en las portadas) · LibraryEntry.hours_by_platform (horas en las portadas) · PlaySession.add (progreso) · Review.finished (si lo terminó al publicarla)

library/templatetags/reviews.py: review_html (el texto con formato de reseñas, comentarios, publicaciones, listas y retos) · stars_label

library/views/ficha_del_juego.py: set_status (progreso) · remove (progreso)

library/views/repasar_la_biblioteca.py: tidy_answer (progreso)

social/activity.py: challenge_joined (botón para unirse) · challenge_left (botón para dejarlo)

lists/models.py: GameList.get_absolute_url

challenges/closing.py: close_all (quién lo ha conseguido)

challenges/forms.py: ChallengeForm.save

challenges/management/commands/close_challenges.py: Command.handle (quién lo ha conseguido)

challenges/models.py: long_date · ChallengeQuerySet.visible_to (solo los públicos y los tuyos) · ChallengeQuerySet.with_counts (cuánta gente se ha unido) · Challenge.get_absolute_url · Challenge.visible_to (los privados solo los ve quien los crea) · Challenge.goal (tu progreso y la clasificación) · Challenge.is_upcoming (si aún no ha empezado) · Challenge.is_over (si ya ha terminado y ya no se puede unir nadie) · Challenge.days_left (los días que le quedan) · Challenge.filters_text · Challenge.rules_text · Challenge.dates_text

challenges/progress.py: _start (tu progreso y la clasificación) · _game_filters (tu progreso y la clasificación) · _completed (tu progreso y la clasificación) · done_by (tu progreso y la clasificación) · summary (tu progreso y la clasificación) · finished_games (los juegos que llevas de uno de juegos concretos) · mark_completed (tu progreso y la clasificación) · check (tu progreso y la clasificación)

challenges/views.py: _visible (404 si no puedes verlo) · detail · join (botón para unirse) · leave (botón para dejarlo)

Nuevo reto

El formulario para crear un reto (de número de juegos, de horas o de juegos concretos), también desde «Convertir en reto» de una lista.

Dirección: /retos/nuevo/ (challenges:new → challenges.views.new)

Plantillas: challenges/form.html
Trozos que incluye: _icon.html, challenges/_field.html

Funciones que la afectan (13)

games/igdb.py: _upsert_ref (géneros y plataformas)

games/models.py: spanish_genre_name (géneros) · Platform.__str__ (plataformas)

social/activity.py: challenge_joined (al crearlo)

lists/models.py: GameListQuerySet.editable_by (listas que se pueden convertir en reto) · GameList.__str__ (las listas que se pueden elegir)

challenges/forms.py: ChallengeEditForm.clean_title · ChallengeEditForm.clean_description · ChallengeForm.__init__ (las listas, los géneros y las plataformas que se pueden elegir) · ChallengeForm.clean (lo que pide cada tipo de reto y el orden de las fechas y los años) · ChallengeForm.save (al crearlo)

challenges/progress.py: done_by (al crearlo ya cuenta lo que llevas)

challenges/views.py: new

Editar reto

El formulario para cambiar el título, la descripción o si es público un reto tuyo, y para borrarlo.

Dirección: /retos/<int:challenge_id>/editar/ (challenges:edit → challenges.views.edit)

Plantillas: challenges/edit.html
Trozos que incluye: _icon.html, challenges/_field.html

Funciones que la afectan (9)

challenges/forms.py: ChallengeEditForm.clean_title (el título) · ChallengeEditForm.clean_description (la descripción)

challenges/models.py: long_date (las fechas en palabras) · Challenge.get_absolute_url (los enlaces a cada reto) · Challenge.filters_text (de qué juegos va) · Challenge.rules_text (lo que hay que hacer) · Challenge.dates_text (cuándo cuenta)

challenges/views.py: edit · delete (botón «Borrar»)

Avisos

La campana de la barra de arriba: tus avisos (bajadas de precio, seguidores nuevos, «me gusta», comentarios, invitaciones a listas...).

Dirección: /avisos/ (notifications → accounts.views.avisos.notifications)

Plantillas: accounts/notifications.html
Trozos que incluye: _icon.html, accounts/_avatar.html

Funciones que la afectan (55)

accounts/forms.py: ProfileForm.save (los que te llegan) · ProfileForm._apply_extras (los que te llegan)

accounts/management/commands/nightly.py: Command.handle (bajadas de precio; retos conseguidos, que se acaban y cómo has quedado)

accounts/notices.py: enabled · wants · notify

accounts/views/avisos.py: notifications

games/management/commands/update_prices.py: Command.handle (bajadas de precio de tu lista de deseos)

games/models.py: igdb_image_url · Game.cover_url · Game.cheapest (bajadas de precio) · format_price (bajadas de precio)

games/spanish.py: europe_cover · apply_igdb

library/backlog.py: answer (reto conseguido)

library/models.py: Review.get_absolute_url (los enlaces a cada reseña) · Review.hours_since (reseñas que se han quedado atrás) · ReviewComment.get_absolute_url (enlace a un comentario de una reseña)

library/reminders.py: latest_reviews (reseñas que se han quedado atrás) · remind_outdated_reviews (reseñas que se han quedado atrás)

library/views/comentarios_y_denuncias.py: _report (aviso a los administradores) · _notify_staff (de los administradores)

library/views/lista_de_deseos.py: wish_target (bajadas de precio)

library/views/pagina_de_la_resena.py: _notify («me gusta» y comentarios en tus reseñas) · review_like (aviso a su autor) · review_comment (aviso a su autor)

library/wishlist.py: at_historical_low (bajadas de precio) · alert_reason (bajadas de precio de tu lista de deseos) · alert_text (texto de cada aviso de bajada de precio, también en el correo) · check_alerts (bajadas de precio de tu lista de deseos, también por correo)

social/models.py: FollowManager.follow (aviso al seguido) · Post.get_absolute_url (los enlaces a cada publicación) · PostComment.get_absolute_url (enlace a un comentario de una publicación)

social/views.py: follow (aviso al seguido) · post_comment (aviso a su autor) · _notify_post_comment (comentarios en tus publicaciones) · post_like (aviso a su autor) · _notify_post_like (a quien le gusta tu publicación)

lists/models.py: GameList.get_absolute_url (los enlaces a cada lista) · GameList.noun («tier list» o «lista» en los textos) · ListComment.get_absolute_url (enlace a un comentario de una lista)

lists/views/colaboradores.py: invite (aviso al invitado)

lists/views/pagina_de_la_lista.py: like (aviso a su dueño) · _notify_like (a quien le gusta tu lista) · comment (aviso a su dueño) · _notify_comment (comentarios en tus listas)

challenges/closing.py: close_all (retos conseguidos, que se acaban y cómo has quedado) · _amount (retos que se acaban y cómo has quedado) · _remind (retos que se acaban) · _announce (cómo has quedado en un reto)

challenges/management/commands/close_challenges.py: Command.handle (retos conseguidos, que se acaban y cómo has quedado)

challenges/progress.py: mark_completed (reto conseguido) · _award (reto conseguido) · check (reto conseguido)

challenges/views.py: join (a quien lo creó)

Editar perfil

Tu configuración: datos, apariencia (foto, cabecera, fondo, color, diseño y su encuadre), escaparate, redes, privacidad y avisos, tiendas en las que compras y cuentas vinculadas (Steam, Xbox, PlayStation y otras), con la vista previa del perfil en directo.

Dirección: /cuenta/perfil/ (edit_profile → accounts.views.editar_perfil.edit_profile)
/cuenta/perfil/vista-previa/ (profile_preview → accounts.views.editar_perfil.profile_preview)
/cuenta/perfil/imagenes-de-juego/ (banner_arts → accounts.views.editar_perfil.banner_arts)

Plantillas: accounts/edit_profile.html, accounts/profile.html, accounts/_banner_arts.html
Trozos que incluye: accounts/_framer.html, _icon.html, accounts/linking/_steam.html, accounts/linking/_xbox.html, accounts/linking/_psn.html, accounts/linking/_platforms.html, accounts/_preview_devices.html, accounts/_avatar.html, social/_follower_count.html, social/_follow_button.html, accounts/linking/_dialog.html, games/_card.html, library/_platform_mark.html, library/_review.html, lists/_card.html, lists/_tier_card.html, accounts/_unlock.html, challenges/_card.html, social/_activities.html, library/_stars.html, library/_review_body.html, library/_like.html, library/_report.html, lists/_mosaic.html, social/_activity.html, library/_review_images.html, social/_post_body.html, social/_post_like.html, lists/_like.html, social/_post_images.html

Funciones que la afectan (166)

accounts/achievements.py: save_info (Cuentas vinculadas) · save_unlocked (Cuentas vinculadas)

accounts/forms.py: ProfileForm.__init__ (y su vista previa) · ProfileForm.privacy_fields (Privacidad) · ProfileForm.notice_rows (Avisos) · ProfileForm._clean_image (Apariencia: foto, cabecera y fondo) · ProfileForm.clean_avatar (Apariencia › Foto de perfil) · ProfileForm.clean_banner (Apariencia › Cabecera) · ProfileForm.clean_background (Apariencia › Fondo de la página) · ProfileForm.clean_accent_color (Apariencia › Color del perfil) · ProfileForm.clean_banner_art (Apariencia › Cabecera con una imagen de un juego) · ProfileForm.clean (Redes sociales y Apariencia) · ProfileForm.save (botón «Guardar») · ProfileForm.preview (vista previa) · ProfileForm._apply_extras (redes, secciones del escaparate y avisos, también en la vista previa) · ProfileForm._delete_replaced_images (al cambiar o quitar la foto, la cabecera o el fondo) · ProfileForm._save_favorites (Escaparate › Juegos favoritos) · ProfileForm._favorite_changes (Juegos favoritos y su vista previa) · PlatformAccountForm.clean (Cuentas vinculadas › Otras plataformas)

accounts/images.py: process (Apariencia: foto, cabecera y fondo)

accounts/management/commands/seed_test_profile.py: Command.create_empty_user (la cuenta «tester_steam» queda vacía y sin cuentas vinculadas)

accounts/matching.py: clean_name (Cuentas vinculadas) · normalize (Cuentas vinculadas) · name_key (Cuentas vinculadas) · _name_keys (Cuentas vinculadas) · _catalog_games (Cuentas vinculadas) · _title_parts (Cuentas vinculadas) · add_title (Cuentas vinculadas) · _platform_names (Cuentas vinculadas) · _fits (Cuentas vinculadas) · _share_platform (Cuentas vinculadas) · _seen_games (Cuentas vinculadas) · _score (Cuentas vinculadas) · match_by_name (Cuentas vinculadas) · save_titles (Cuentas vinculadas) · update_library (Cuentas vinculadas)

accounts/models.py: normalize_handle (Redes sociales) · profile_upload_to (dónde se guardan la foto, la cabecera y el fondo subidos) · Profile.steam_accounts (Cuentas vinculadas › Steam) · Profile.psn (Cuentas vinculadas › PlayStation) · Profile.platforms (Cuentas vinculadas › Otras plataformas) · Profile.xbox (Cuentas vinculadas › Xbox) · Profile.accent (Color del perfil) · Profile.banner_url (Apariencia › Cabecera) · Profile.banner_blurred (Apariencia › Cabecera) · Profile.background_url (Apariencia › Fondo de la página) · Profile._uploaded_url (Apariencia) · Profile.background_blurred (Apariencia › Fondo de la página) · Profile.section_layout (Escaparate › Secciones del perfil) · SteamAccount.name (Cuentas vinculadas › Steam) · SteamAccount.url (Cuentas vinculadas › Steam) · PSNAccount.name (Cuentas vinculadas › PlayStation) · PlatformAccount.logo_template (Cuentas vinculadas › Otras plataformas) · PlatformAccount.normalize (Cuentas vinculadas › Otras plataformas) · PlatformAccount.is_valid_handle (Cuentas vinculadas › Otras plataformas) · XboxAccount.name (Cuentas vinculadas › Xbox) · XboxAccount.url (Cuentas vinculadas › Xbox) · ImportedTitle.set_achievements (Cuentas vinculadas)

accounts/notices.py: enabled (Avisos: cómo sale cada interruptor)

accounts/psn.py: clean_npsso (Cuentas vinculadas: PlayStation: el código NPSSO pegado) · access_token (Cuentas vinculadas) · _get (Cuentas vinculadas) · _account_id_from_token (Cuentas vinculadas) · profile (Cuentas vinculadas) · played_games (Cuentas vinculadas) · trophy_titles (Cuentas vinculadas) · trophies_by_title_id (Cuentas vinculadas) · trophy_key (Cuentas vinculadas) · trophy_count (Cuentas vinculadas) · save_trophies (Cuentas vinculadas) · duration_minutes (Cuentas vinculadas) · import_library (Cuentas vinculadas)

accounts/steam.py: is_configured (Cuentas vinculadas: sin clave de Steam no se puede importar) · login_url (Cuentas vinculadas: botón para vincular Steam) · verify_login (Cuentas vinculadas: vuelta desde Steam) · _api (Cuentas vinculadas) · player_summary (Cuentas vinculadas › Steam: nombre y avatar) · owned_games (Cuentas vinculadas) · wishlist (Cuentas vinculadas › Steam) · refresh_account (Cuentas vinculadas › Steam: nombre y avatar) · _match_games (Cuentas vinculadas) · _timestamp (Cuentas vinculadas) · _played_at (Cuentas vinculadas) · import_library (Cuentas vinculadas) · player_achievements (Cuentas vinculadas) · _icon (Cuentas vinculadas) · achievement_schema (Cuentas vinculadas) · global_percentages (Cuentas vinculadas) · achievement_info (Cuentas vinculadas) · _needs_info (Cuentas vinculadas) · import_achievements (Cuentas vinculadas) · _first (Cuentas vinculadas › Steam) · _wishlist_games (Cuentas vinculadas › Steam) · import_wishlist (Cuentas vinculadas › Steam)

accounts/views/comun.py: _back

accounts/views/cuentas_vinculadas.py: linking_context (Cuentas vinculadas) · _steam_return_to (Cuentas vinculadas: Steam) · _report_import (Cuentas vinculadas: mensaje con el resultado de la importación de Steam) · _achievements_note (Cuentas vinculadas: mensaje con el resultado de la importación) · _with_unmatched (Cuentas vinculadas: mensaje de la importación, con el enlace a Juegos importados) · _report_wishlist (Cuentas vinculadas: mensaje con lo que se ha traído de la lista de deseos de Steam) · _fetch_library_prices (Cuentas vinculadas) · _import_steam (Cuentas vinculadas) · steam_link (Cuentas vinculadas: botón para vincular Steam) · steam_callback (Cuentas vinculadas) · steam_import (Cuentas vinculadas: botón «Importar» de Steam) · steam_auto_sync (Cuentas vinculadas: casilla de la sincronización de cada noche) · steam_wishlist (Cuentas vinculadas: casilla para traer la lista

de deseos de Steam) · steam_unlink (Cuentas vinculadas: botón para desvincular una cuenta de Steam) · _psn_error (Cuentas vinculadas: mensaje de error de PlayStation) · psn_sync (Cuentas vinculadas: formulario del código NPSSO de PlayStation) · psn_unlink (Cuentas vinculadas: botón para desvincular PlayStation) · platform_add (Cuentas vinculadas: Otras plataformas) · platform_remove (Cuentas vinculadas: Otras plataformas) · _xbox_redirect_uri (Cuentas vinculadas: Xbox) · xbox_link (Cuentas vinculadas: botón para vincular Xbox) · xbox_callback (Cuentas vinculadas) · xbox_unlink (Cuentas vinculadas: botón para desvincular Xbox)

accounts/views/editar_perfil.py: profile_preview (vista previa del perfil) · edit_profile · _posted_sections (Escaparate › Secciones del perfil, cuando no se ha podido guardar) · game_arts (Apariencia › Cabecera: imágenes del juego elegido) · banner_arts (Apariencia › Cabecera: imágenes del juego elegido) · shops_update (Tiendas en las que compras)

accounts/views/ficha_del_juego.py: favorite_toggle (Juegos favoritos)

accounts/views/perfil.py: _profile_owners (vista previa) · _profile_context (vista previa)

accounts/xbox.py: is_configured (Cuentas vinculadas: sin credenciales de Azure no se puede vincular Xbox) · login_url (Cuentas vinculadas: botón para vincular Xbox) · _post (Cuentas vinculadas) · xbox_tokens (Cuentas vinculadas) · _get (Cuentas vinculadas) · avatar (Cuentas vinculadas › Xbox) · played_titles (Cuentas vinculadas) · minutes_played (Cuentas vinculadas) · save_achievements (Cuentas vinculadas) · import_library (Cuentas vinculadas)

games/igdb.py: is_configured (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo) · IGDBClient.__init__ (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo) · IGDBClient._token (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo) · IGDBClient.query (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo) · IGDBClient.search (Cuentas vinculadas: títulos de PlayStation y Xbox que no están en el catálogo) · IGDBClient.games_by_steam_ids (Cuentas vinculadas) · IGDBClient.games_by_ids (Cuentas vinculadas: juegos de Steam que aún no están en el catálogo) · IGDBClient.games_by_names (Cuentas vinculadas) · IGDBClient.artworks (Apariencia › Cabecera: imágenes del juego elegido) · sort_arts (imágenes de un juego para la cabecera) · upsert_games (Cuentas vinculadas: juegos importados que aún no están en el catálogo)

games/media.py: _is_wide (imágenes de un juego para la cabecera)

games/models.py: igdb_image_url (las portadas de los juegos) · Game.cover_url (las portadas de los juegos) · known_shops (Tiendas en las que compras) · format_price (mensaje al traer la lista de deseos de Steam)

games/prices.py: fetch_missing (Cuentas vinculadas: al importar)

games/spanish.py: spanish_names (Cuentas vinculadas: nombres con los que se emparejan los juegos importados) · europe_cover (las portadas de los juegos) · apply_igdb (las portadas de los juegos)

library/models.py: LibraryEntry.imported_status (Cuentas vinculadas) · LibraryEntry.update_import (Cuentas vinculadas) · PlaySession.add (Cuentas vinculadas: al importar horas)

library/reminders.py: remind_outdated_reviews (Cuentas vinculadas: se mira al importar horas)

library/wishlist.py: at_historical_low (mensaje al traer la lista de deseos de Steam) · steam_deals

social/activity.py: games_imported (Cuentas vinculadas: al importar)

challenges/progress.py: check (Cuentas vinculadas: al importar horas) · showcase (vista previa)

JavaScript: accounts/edit_profile.html (línea 380) · accounts/linking/_platforms.html (línea 40) (Cuentas vinculadas › Otras plataformas)

Juegos importados

Los juegos importados de Steam, PlayStation o Xbox que no se han encontrado en el catálogo (o que se han emparejado con el que no es), para asignarlos a mano u ocultarlos.

Dirección: /cuenta/juegos-importados/ (imported_titles → accounts.views.juegos_importados.imported_titles)

Plantillas: accounts/imported_titles.html

Funciones que la afectan (67)

accounts/admin.py: ExternalTitleAdmin.save_model

accounts/management/commands/nightly.py: Command.handle (los que no se encuentran)

accounts/management/commands/rematch_titles.py: Command.handle · Command.rematch

accounts/management/commands/sync_libraries.py: Command.handle (los que no se encuentran)

accounts/matching.py: clean_name (los que no se encuentran) · normalize (los que no se encuentran) · name_key (los que no se encuentran) · _name_keys (los que no se encuentran) · _catalog_games (los que no se encuentran) · _title_parts (los que no se encuentran) · add_title (los que no se encuentran) · _platform_names (los que no se encuentran) · _fits (los que no se encuentran) · _share_platform (los que no se encuentran) · _seen_games (los que no se encuentran) · _score (los que no se encuentran) · match_by_name (los que no se encuentran) · save_titles (los que no se encuentran) · update_library (los que no se encuentran) · reassign

accounts/models.py: ImportedTitle.hours (horas de cada título)

accounts/psn.py: access_token (los que no se encuentran) · _get (los que no se encuentran) · _account_id_from_token (los que no se encuentran) · played_games (los que no se encuentran) · duration_minutes (los que no se encuentran) · import_library (los que no se encuentran)

accounts/steam.py: _api (los que no se encuentran) · owned_games (los que no se encuentran) · _match_games (los que no se encuentran) · _timestamp (los que no se encuentran) · _played_at (los que no se encuentran) · import_library (los que no se encuentran)

accounts/views/comun.py: _back (vuelve a la página desde la que se envió el formulario)

accounts/views/cuentas_vinculadas.py: _import_steam (los que no se encuentran) · steam_callback (los que no se encuentran) · steam_import (los que no se encuentran) · psn_sync (los que no se encuentran) · xbox_callback (los que no se encuentran)

accounts/views/juegos_importados.py: _search_games (buscador para elegir el juego de un título) · imported_titles · imported_title_assign · imported_title_hide (ocultar un título o volver a mostrarlo)

accounts/xbox.py: _post (los que no se encuentran) · xbox_tokens (los que no se encuentran) · _get (los que no se encuentran) · played_titles (los que no se encuentran) · minutes_played (los que no se encuentran) · import_library (los que no se encuentran)

games/igdb.py: is_configured (buscar en IGDB) · IGDBClient.__init__ (buscar en IGDB) · IGDBClient._token (buscar en IGDB) · IGDBClient.query (buscar en IGDB) · IGDBClient.search (buscar el juego de un título) · IGDBClient.games_by_steam_ids (los que no se encuentran) · IGDBClient.games_by_names (los que no se encuentran) · upsert_games (buscar en IGDB)

games/models.py: igdb_image_url · Platform.short_name · Game.get_absolute_url (los enlaces a la ficha de cada juego) · Game.cover_url · Game.release_year (el año de cada juego)

games/spanish.py: europe_cover · apply_igdb

library/models.py: LibraryEntry.imported_status (los que no se encuentran) · LibraryEntry.update_import (los que no se encuentran)

Registro

«Crear cuenta»: el formulario de registro.

Dirección: /cuenta/registro/ (signup → accounts.views.registro.signup)

Plantillas: registration/signup.html
Trozos que incluye: _form.html

Funciones que la afectan (3)

accounts/forms.py: SignupForm.clean_email

accounts/signals.py: create_profile (crea el perfil de la cuenta nueva)

accounts/views/registro.py: signup

Iniciar sesión

«Entrar»: el formulario de inicio de sesión. La vista es la de Django (django.contrib.auth); NextPlay solo pone la plantilla, así que ninguna función del proyecto la toca.

Dirección: /cuenta/login/ (login → django.contrib.auth (login))

Plantillas: registration/login.html
Trozos que incluye: _form.html

Funciones que la afectan (0)

Ninguna.

Recuperar contraseña

Los cuatro pasos para cambiar la contraseña olvidada (pedirla, revisar el correo, escribir la nueva y listo) y el correo que se manda. Las vistas son las de Django (django.contrib.auth); NextPlay solo pone las plantillas, así que ninguna función del proyecto las toca.

Dirección: /cuenta/password_reset/ (password_reset → django.contrib.auth (password_reset))
/cuenta/password_reset/done/ (password_reset_done → django.contrib.auth (password_reset_done))
/cuenta/reset/<uidb64>/<token>/ (password_reset_confirm → django.contrib.auth (password_reset_confirm))
/cuenta/reset/done/ (password_reset_complete → django.contrib.auth (password_reset_complete))

Plantillas: registration/password_reset_form.html, registration/password_reset_done.html, registration/password_reset_confirm.html, registration/password_reset_complete.html
Trozos que incluye: _form.html

Funciones que la afectan (0)

Ninguna.

Panel de administración

El admin de Django, solo para el administrador: juegos, títulos importados, reseñas, listas, retos, denuncias...

Dirección: /admin/

Funciones que la afectan (74)

accounts/admin.py: ExternalTitleAdmin.owners (columna «usuarios» de los títulos importados) · ExternalTitleAdmin.save_model (al corregir el juego de un título importado) · ExternalTitleAdmin.lock (acción «bloquear» de los títulos importados) · ExternalTitleAdmin.unlock (acción «desbloquear» de los títulos importados)

accounts/matching.py: update_library (al corregir un título importado)

accounts/models.py: Profile.__str__ · FavoriteGame.__str__ · SteamAccount.__str__ · PSNAccount.__str__ · PlatformAccount.__str__ · XboxAccount.__str__ · ExternalTitle.__str__ · ImportedTitle.__str__ · Achievement.__str__ · UnlockedAchievement.__str__ · Notification.__str__

accounts/signals.py: create_profile (también al crear un usuario allí) · delete_profile_images (al borrar un usuario o un perfil)

games/admin.py: GamePriceInline.has_add_permission (precios de cada juego: no se añaden a mano)

games/models.py: Genre.__str__ · Platform.__str__ · Game.__str__ · GamePrice.__str__

library/admin.py: ReviewImageInline.preview (miniaturas de las imágenes de una reseña) · ReviewAdmin.get_queryset (reseñas con sus denuncias pendientes) · ReviewAdmin.pending_reports (columna de denuncias pendientes de las reseñas) · PendingFilter.lookups (filtro de denuncias revisadas o sin revisar) · PendingFilter.queryset (filtro de denuncias revisadas o sin revisar) · ReportAdmin.has_add_permission (denuncias: no se crean a mano) · ReportAdmin.kind (columna de qué se ha denunciado) · ReportAdmin.target_link (enlaces a lo denunciado) · ReportAdmin.mark_resolved (acción de marcar denuncias como revisadas) · ReportAdmin.mark_pending (acción de volver a dejarlas pendientes)

library/forms.py: ReportForm.clean_detail (denuncias)

library/models.py: LibraryEntry.__str__ · PlaySession.__str__ · Review.__str__ · ReviewLike.__str__ · ReviewComment.__str__ · AttachedImage.delete_files · ReviewImage.__str__ · Report.__str__ · Report.target_kind (columna de qué se ha denunciado) · WishlistItem.__str__

library/signals.py: delete_review_image_files

library/templatetags/reviews.py: report_reasons (denuncias)

library/views/comentarios_y_denuncias.py: _report (denuncias) · _notify_staff (denuncias)

library/views/pagina_de_la_resena.py: review_report (denuncias) · comment_report (denuncias)

social/admin.py: PostAdmin.get_queryset (publicaciones con sus denuncias pendientes) · PostAdmin.pending_reports (columna de denuncias pendientes de las publicaciones)

social/models.py: Follow.__str__ · Post.__str__ · PostImage.__str__ · PostLike.__str__ · PostComment.__str__ · Activity.__str__ · Activity.summary (lo que hizo cada actividad, en una línea)

social/signals.py: delete_post_image_files

social/views.py: post_comment_report (denuncias) · post_report (denuncias)

lists/admin.py: GameListAdmin.get_queryset (listas con cuántos juegos y denuncias pendientes tienen) · GameListAdmin.game_count (columna de cuántos juegos tiene cada lista) · GameListAdmin.pending_reports (columna de denuncias pendientes de las listas)

lists/models.py: GameList.__str__ · ListItem.__str__ · ListCollaborator.__str__ · ListLike.__str__ · ListComment.__str__

lists/views/pagina_de_la_lista.py: comment_report (denuncias) · report (denuncias)

challenges/models.py: Challenge.__str__ · Participation.__str__

En todas las pantallas

Lo que sale en cualquier pantalla: la barra de arriba (la campana de avisos y el menú de la cuenta), y la foto y el nombre de cada jugador.

accounts/context_processors.py: notifications (la campana de avisos de la barra de arriba)

accounts/forms.py: ProfileForm.save (tu nombre y tu foto en la barra de arriba)

accounts/models.py: Profile.name (el nombre de cada jugador, también en el menú de la cuenta de la barra de arriba) · Profile.steam_avatar (la foto del perfil cuando es la de Steam) · Profile.psn (la foto del perfil cuando es la de PlayStation) · Profile.xbox (la foto del perfil cuando es la de Xbox) · Profile.avatar_url (la foto de cada jugador, también la tuya en la barra de arriba) · Profile._uploaded_url (la foto subida) · Profile.avatar_image (la foto de cada jugador, con su encuadre, también la tuya en la barra de arriba) · Profile._framing (encuadre de la foto de cada jugador)

accounts/notices.py: enabled (el número de la campana) · wants (el número de la campana) · notify (el número de la campana)

accounts/psn.py: profile (la foto del perfil cuando es la de PlayStation)

accounts/steam.py: player_summary (la foto del perfil cuando es la de Steam) · refresh_account (la foto del perfil cuando es la de Steam)

accounts/views/avisos.py: notifications (la campana de la barra de arriba se queda sin número al abrirla)

accounts/xbox.py: avatar (la foto del perfil cuando es la de Xbox)

library/reminders.py: remind_outdated_reviews (el número de la campana)

library/views/pagina_de_la_resena.py: _notify (el número de la campana)

library/wishlist.py: check_alerts (el número de la campana)

social/views.py: _notify_post_comment (el número de la campana) · _notify_post_like (el número de la campana)

lists/views/pagina_de_la_lista.py: _notify_like (el número de la campana) · _notify_comment (el número de la campana)

challenges/closing.py: _remind (el número de la campana) · _announce (el número de la campana)

challenges/progress.py: _award (el número de la campana)

JavaScript: base.html (línea 84) (se nota donde hay reseñas, comentarios o publicaciones con spoilers y el botón «Copiar enlace») · base.html (línea 118) (las que tienen pestañas: Inicio, Perfil, Seguidores y seguidos y Juegos importados) · base.html (línea 145) (se nota donde hay imágenes de reseñas o de publicaciones: Ficha del juego,

Perfil, Inicio, Página de la reseña y Página de la publicación) · base.html (línea 232) (lo que se carga o se pulsa con HTMX) · base.html (línea 292) (el menú de la cuenta de la barra de arriba)

En ninguna pantalla

Lo que no se ve en la web: el arranque de Django, las opciones de los comandos de consola, lo que ya no usa nadie...

accounts/apps.py: AccountsConfig.ready (se ejecuta al arrancar Django y conecta las señales de accounts/signals.py)

accounts/management/commands/rematch_titles.py: Command.add_arguments (opciones del comando rematch_titles)

accounts/management/commands/seed_test_profile.py: Command.add_arguments (opciones del comando seed_test_profile)

accounts/management/commands/sync_libraries.py: Command.add_arguments (opciones del comando sync_libraries)

games/management/commands/import_igdb.py: Command.add_arguments (opciones del comando import_igdb)

games/management/commands/localize_games.py: Command.add_arguments (opciones del comando localize_games)

games/management/commands/update_prices.py: Command.add_arguments (opciones del comando update_prices)

library/apps.py: LibraryConfig.ready (se ejecuta al arrancar Django y conecta las señales de library/signals.py)

library/models.py: Review.stars (ahora no la usa nadie: las estrellas se pintan desde rating con library/_stars.html)

library/wishlist.py: _send_email (manda por correo los avisos de precio a quien los tiene encendidos en Editar perfil › Avisos)

social/apps.py: SocialConfig.ready (se ejecuta al arrancar Django y conecta las señales de social/signals.py)

Parte 2 · Funciones por archivo

Archivo	Funciones
accounts/achievements.py	8
accounts/admin.py	4
accounts/apps.py	1
accounts/context_processors.py	1
accounts/forms.py	18
accounts/images.py	1
accounts/management/commands/nightly.py	1
accounts/management/commands/rematch_titles.py	3
accounts/management/commands/seed_test_profile.py	9
accounts/management/commands/sync_libraries.py	3
accounts/matching.py	16
accounts/models.py	54
accounts/notices.py	3
accounts/psn.py	13
accounts/signals.py	2
accounts/steam.py	22
accounts/views/avisos.py	1
accounts/views/comun.py	1
accounts/views/cuentas_vinculadas.py	23
accounts/views/editar_perfil.py	6
accounts/views/ficha_del_juego.py	4
accounts/views/juegos_importados.py	4
accounts/views/perfil.py	4
accounts/views/registro.py	1
accounts/xbox.py	10
games/admin.py	1
games/history.py	13
games/igdb.py	20
games/management/commands/import_igdb.py	4
games/management/commands/localize_games.py	4
games/management/commands/seed_demo.py	1
games/management/commands/update_prices.py	2
games/media.py	17
games/models.py	38
games/nintendo.py	10
games/prices.py	18
games/spanish.py	8

Archivo	Funciones
games/views.py	11
games/xbox.py	10
library/admin.py	10
library/apps.py	1
library/autofill.py	4
library/backlog.py	10
library/forms.py	18
library/markup.py	4
library/models.py	38
library/points.py	5
library/reminders.py	2
library/signals.py	1
library/templatetags/reviews.py	6
library/views/comentarios_y_denuncias.py	3
library/views/escribir_resena.py	5
library/views/ficha_del_juego.py	6
library/views/lista_de_deseos.py	5
library/views/pagina_de_la_resena.py	9
library/views/repasar_la_biblioteca.py	3
library/wishlist.py	6
social/activity.py	17
social/admin.py	2
social/apps.py	1
social/forms.py	3
social/models.py	22
social/signals.py	2
social/views.py	28
lists/admin.py	3
lists/community.py	3
lists/forms.py	11
lists/models.py	31
lists/presets.py	14
lists/views/colaboradores.py	6
lists/views/comun.py	2
lists/views/editar_lista.py	12
lists/views/listas_del_juego.py	4
lists/views/nueva_lista.py	4
lists/views/pagina_de_la_lista.py	14
lists/views/tarjetas.py	4

Archivo	Funciones
lists/views/tier_lists.py	1
challenges/admin.py	0
challenges/closing.py	4
challenges/forms.py	5
challenges/management/commands/close_challenges.py	1
challenges/models.py	14
challenges/progress.py	10
challenges/views.py	10

App «accounts»

Usuarios: registro, inicio de sesión, perfil público o privado, vinculación con Steam, Xbox y PlayStation y cuentas de otras plataformas.

accounts/achievements.py

Logros importados de Steam, de PlayStation (sus trofeos) y de Xbox, juntados por juego.

Cada plataforma los apunta en los títulos importados del usuario (models.ImportedTitle.set_achievements): Steam los lee juego a juego (steam.import_achievements), PlayStation con su lista de trofeos (psn.save_trophies) y Xbox con su historial de juegos (xbox.save_achievements). Aquí se juntan por juego para la ficha, las portadas y las cifras del perfil.

De Steam se guardan además uno a uno: los logros de cada juego (models.Achievement, con save_info) y cuáles tiene cada usuario y cuándo los consiguió (models.UnlockedAchievement, con save_unlocked). De ahí salen «Tus logros» de la ficha (detail) y la sección «Logros» del perfil (showcase).
Pantallas de este archivo: Ficha del juego, Perfil, Nueva tier list, Editar perfil

```
def progress_by_game(user, game_ids)
```

Cómo va user con los logros de los juegos game_ids: {id del juego: [progreso en cada plataforma]}, solo de los juegos que tienen logros y ya se han leído.

Cada progreso es un dict con la plataforma («platform»), su nombre («label») y cómo llama a sus logros («noun»: «trofeos» en PlayStation), los conseguidos («unlocked») y los que hay («total»), el porcentaje («percent», sin redondear hacia arriba: el 100 % es tenerlos todos), si los tiene todos («complete») y si tiene el platino de PlayStation («platinum»). Con varias cuentas o versiones del juego en una plataforma (la de PS4 y la de PS5), cuenta la que más lleva. «detailed» dice si se conocen también uno a uno (detail), y «imported_id» y «title_id» son su título importado y el de la plataforma.

Pantallas: Perfil (insignias de las portadas y cifras de logros), Ficha del juego (tus logros), Nueva tier list (plantilla de juegos con todos los logros).

accounts/achievements.py, línea 24

```
def badge(progress)
```

Insignia de la portada de un juego según su progreso (un valor de progress_by_game): «Platino» si tiene el de PlayStation, «100 %» si tiene todos los logros en alguna plataforma y, si no, vacía.

Pantallas: Perfil (insignia «Platino» o «100 %» de las portadas de la biblioteca).

accounts/achievements.py, línea 66

```
def totals(progress)
```

Cifras del perfil a partir de progress_by_game: cuántos platinos tiene («platinums») y en cuántos juegos tiene todos los logros de alguna plataforma («complete»).

Pantallas: Perfil (platinos y juegos al 100 % de las cifras del perfil).

accounts/achievements.py, línea 79

```
def save_info(title, items, percentages, at)
```

Guarda los logros del juego de title (un ExternalTitle) tal como los da su plataforma y apunta que se han leído en at.

items son sus logros en orden, cada uno con «api_name», «name», «description», «icon», «icon_locked» y «hidden»; percentages, el porcentaje de los jugadores que tiene cada uno ({clave: porcentaje}), o None si no se ha podido leer: entonces se queda el que había. Los logros que ya no da la plataforma no se borran: alguien puede tenerlos.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/achievements.py, línea 92

```
def save_unlocked(found)
```

Apunta uno a uno los logros conseguidos en unos títulos importados: found es [(ImportedTitle, logros)], con todos los logros de su juego como [(clave, conseguido, fecha o None)], que es lo que da steam.player_achievements.

Los que no salían en los logros del juego (save_info) se añaden con su clave por nombre, para no perderlos; de los que ya estaban apuntados se borran los que ya no tiene y se corrige la fecha si ha cambiado.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/achievements.py, línea 120

```
def _by_rarity(achievement)
```

Clave para ordenar del logro más raro al más común; los que no se sabe, al final.

Pantallas: Ficha del juego (tus logros, del más raro al más común).

accounts/achievements.py, línea 180

```
def detail(user, game, order='')
```

«Tus logros» de user en game, uno a uno, de la plataforma en que se conocen así (de momento, Steam; con varias versiones, la que más lleva), o None si no hay ninguna.

Devuelve un dict con la plataforma («label»), los que tiene («unlocked», cada logro con su «unlocked_at») y los que le faltan («locked»), cada grupo cortado en DETAIL_SHOWN («…_shown») y el resto aparte («…_more»), cuántos hay («total»), el más raro de los que tiene («rarest»), las fechas del primero y del último («first» y «last») y los órdenes (ORDERS) con el elegido («order»). Con order «raros» los dos grupos van del más raro al más común y con «juego», en el orden del juego; si no, los que tiene del más reciente al más antiguo (los que no tienen fecha, al final) y los que le faltan del más común al más raro, que suelen ser los más fáciles.

Pantallas: Ficha del juego («Tus logros», uno a uno).

accounts/achievements.py, línea 188

```
def showcase(user)
```

Sección «Logros» del perfil de user: sus últimos logros conseguidos («latest», SHOWCASE_SHOWN, del más reciente al más antiguo y con su juego en achievement.title.game), el más raro («rarest») y cuántos tiene guardados uno a uno («count»); None si no tiene ninguno.

Pantallas: Perfil (sección «Logros» del escaparate).

accounts/achievements.py, línea 238

accounts/admin.py

Panel de administración de las cuentas: usuarios, perfiles, favoritos, cuentas vinculadas (Steam, PlayStation, Xbox y otras plataformas), títulos importados (donde se corrige a mano el juego de cada uno), logros y avisos.
Pantallas de este archivo: Ficha del juego, Perfil, Juegos importados, Panel de administración

Clase ExternalTitleAdmin

Revisión de los emparejamientos: los cambiados por usuarios se filtran por «emparejado por».

```
@admin.display(description='usuarios')
def owners(self, obj)
```

Cuántos usuarios tienen este título importado.

Pantallas: Panel de administración (columna «usuarios» de los títulos importados).

accounts/admin.py, línea 75

```
def save_model(self, request, obj, form, change)
```

Si cambia el juego, lo marca como corregido por el administrador y recoloca ya las horas de todos los usuarios que tienen el título (las de un usuario normal esperan a su próxima importación).

Pantallas: Panel de administración (al corregir el juego de un título importado), Juegos importados, Perfil (biblioteca y horas), Ficha del juego (tus horas).

accounts/admin.py, línea 82

```
@admin.action(description='Bloquear: los usuarios ya no podrán cambiarlos')
def lock(self, request, queryset)
```

Bloquea los títulos elegidos.

Pantallas: Panel de administración (acción «bloquear» de los títulos importados).

accounts/admin.py, línea 98

```
@admin.action(description='Desbloquear')
def unlock(self, request, queryset)
```

Desbloquea los títulos elegidos.

Pantallas: Panel de administración (acción «desbloquear» de los títulos importados).

accounts/admin.py, línea 106

Clase AchievementAdmin

Logros de cada juego tal como los da su plataforma; se rellenan solos al leer los de los jugadores.

accounts/apps.py

Configuración de la app accounts (usuarios, perfiles y cuentas vinculadas): al arrancar Django conecta sus señales (accounts/signals.py).

Clase AccountsConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (crean el perfil de cada usuario nuevo).

Pantallas: ninguna (se ejecuta al arrancar Django y conecta las señales de accounts/signals.py).

accounts/apps.py, línea 11

accounts/context_processors.py

Variables que reciben todas las plantillas: el número de avisos sin leer de la campana de la barra de arriba, que se ve en todas las pantallas.

```
def notifications(request)
```

Número de avisos sin leer del usuario con sesión, para la campana de la cabecera.

Pantallas: todas (la campana de avisos de la barra de arriba).

accounts/context_processors.py, línea 8

accounts/forms.py

Formularios de las cuentas: el de Registro (SignupForm), el de Editar perfil (ProfileForm: datos, apariencia, escaparate, redes, privacidad y avisos, también para su vista previa) y el de la cuenta de otra plataforma (PlatformAccountForm), en Cuentas vinculadas de Editar perfil y en el modal «Vincular cuentas» del Perfil.
Pantallas de este archivo: Ficha del juego, Perfil, Avisos, Editar perfil, Registro

Clase SignupForm

```
def clean_email(self)
```

Pasa el correo a minúsculas y rechaza el registro si ya hay una cuenta con ese correo.

Pantallas: Registro.

accounts/forms.py, línea 42

Clase ProfileForm

Todo lo que se cambia en Editar perfil salvo las tiendas y las cuentas vinculadas.

Datos (nombre, estado, pronombres, ubicación, biografía), apariencia (foto, cabecera, fondo, color y diseño), escaparate (favoritos, reseña destacada y secciones), redes sociales, privacidad y avisos. Las redes llegan en campos link_<red> y se guardan juntas en Profile.links; los avisos, en campos aviso_<grupo> y correo_<grupo> que se guardan juntos en Profile.notification_prefs. Los favoritos y las secciones llegan como listas en el orden en que están en la página.

```
def __init__(self, *args, user, **kwargs)
```

Prepara el formulario para user.

Los juegos de la cabecera son los de su biblioteca (y el que ya tenga puesto), las reseñas que puede destacar son las suyas y las fotos que puede elegir son las de las cuentas que tiene vinculadas. Añade un campo por red social con lo que tenga guardado y los interruptores de «Avisos».

Pantallas: Editar perfil (y su vista previa).

accounts/forms.py, línea 105

```
@property
def privacy_fields(self)
```

Los campos de «Privacidad», que la plantilla pinta como interruptores.

Pantallas: Editar perfil (Privacidad).

accounts/forms.py, línea 184

```
@property
def notice_rows(self)
```

Los interruptores de «Avisos», grupo a grupo: (el de la campana, el del correo o None).

Pantallas: Editar perfil (Avisos).

accounts/forms.py, línea 192

```
def _clean_image(self, name)
```

Pasa la imagen nueva del campo name por accounts.images; sin imagen nueva deja lo que había (False = quitarla).

Pantallas: Editar perfil (Apariencia: foto, cabecera y fondo).

accounts/forms.py, línea 202

```
def clean_avatar(self)
```

Foto de perfil nueva, recortada en cuadrado y en WebP.

Pantallas: Editar perfil (Apariencia › Foto de perfil).

accounts/forms.py, línea 212

```
def clean_banner(self)
```

Imagen de cabecera nueva, recortada en apaisado (3:1) y en WebP.

Pantallas: Editar perfil (Apariencia › Cabecera).

accounts/forms.py, línea 219

```
def clean_background(self)
```

Imagen de fondo nueva, reducida a 1920 × 1200 como mucho y en WebP.

Pantallas: Editar perfil (Apariencia › Fondo de la página).

accounts/forms.py, línea 226

```
def clean_accent_color(self)
```

Color en minúsculas; vacío si es el violeta por defecto. Rechaza lo que no sea #rrggbb.

Pantallas: Editar perfil (Apariencia › Color del perfil).

accounts/forms.py, línea 233

```
def clean_banner_art(self)
```

ID de la imagen de IGDB elegida para la cabecera; solo minúsculas y números, como los de IGDB.

Pantallas: Editar perfil (Apariencia › Cabecera con una imagen de un juego).

accounts/forms.py, línea 243

```
def clean(self)
```

Comprueba las redes sociales y que la cabecera y el fondo elegidos tengan su imagen o su juego.

Deja en cleaned_data["links"] el diccionario que se guardará en Profile.links.

Pantallas: Editar perfil (Redes sociales y Apariencia).

accounts/forms.py, línea 253

```
def save(self, commit=True)
```

Guarda el perfil con sus redes, el orden de sus secciones y los avisos que quiere recibir.

Con commit también borra del disco las imágenes sustituidas o quitadas y guarda el orden de los favoritos (quitando los marcados).

Pantallas: Editar perfil (botón «Guardar»), Perfil (todo lo que se personaliza), todas (tu nombre y tu foto en la barra de arriba), Avisos (los que te llegan).

accounts/forms.py, línea 291

```
def preview(self)
```

Perfil y favoritos tal como quedarían con lo enviado, para la vista previa de Editar perfil.

No guarda nada: valida el formulario y lo que no sea válido se queda como estaba. Las imágenes nuevas no llegan, porque no se suben hasta guardar; en su lugar vienen en local_avatar, local_banner y local_background las direcciones blob: con las que el navegador ya las enseña. Devuelve (perfil sin guardar, favoritos en su nuevo orden).

Pantallas: Editar perfil (vista previa).

accounts/forms.py, línea 308

```
def _apply_extras(self, profile)
```

Pone en profile lo que no se guarda campo a campo: las redes (en links), el orden y la visibilidad de las secciones y los avisos que quiere recibir (en notification_prefs).

Si las secciones enviadas no son válidas, se dejan como estaban. Los avisos solo cambian si el envío trae la sección «Avisos» (lo dice su campo oculto «avisos»): un envío sin ella no apaga ninguno.

Pantallas: Editar perfil (redes, secciones del escaparate y avisos, también en la vista previa), Perfil (redes y escaparate), Avisos (los que te llegan).

accounts/forms.py, línea 326

```
def _delete_replaced_images(self, profile)
```

Borra del disco las imágenes que tenía el perfil y ya no usa (sustituidas por otra o quitadas).

Pantallas: Editar perfil (al cambiar o quitar la foto, la cabecera o el fondo).

accounts/forms.py, línea 349

```
def _save_favorites(self)
```

Quita los favoritos marcados y guarda el orden en que llegaron los demás (los que no llegaron, al final).

Pantallas: Editar perfil (Escaparate › Juegos favoritos), Perfil (favoritos del escaparate), Ficha del juego (botón de favoritos).

accounts/forms.py, línea 359

```
def _favorite_changes(self)
```

(ids de los juegos que se quitan, favoritos que quedan) según el envío, sin guardar nada.

Los que quedan van en el orden en que llegaron en favoritos y los que no llegaron, al final.

Pantallas: Editar perfil (Juegos favoritos y su vista previa).

accounts/forms.py, línea 373

Clase PlatformAccountForm

```
def clean(self)
```

Normaliza el nombre o código y comprueba que tiene el formato de la plataforma elegida.

Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (modal «Vincular cuentas»).

accounts/forms.py, línea 392

accounts/images.py

Tratamiento de las imágenes que suben los usuarios: la foto, la cabecera y el fondo del perfil y las de las reseñas y publicaciones.

Cada imagen se comprueba con Pillow, se gira según su EXIF (las fotos del móvil vienen tumbadas), se reduce hasta caber en el tamaño de su uso y se guarda en WebP con un nombre aleatorio. No se recorta: se guarda entera y el usuario elige qué parte se ve, o que se vea entera, con el encuadre del perfil (Profile.avatar_x, avatar_y, avatar_zoom, avatar_fit y los de la cabecera y el fondo). De las imágenes de las reseñas y de las publicaciones se guardan dos tamaños: la grande, para el visor, y una miniatura. Al volver a codificarla se pierden los metadatos EXIF, que en las fotos del móvil pueden llevar la ubicación.
Pantallas de este archivo: Inicio, Publicar, Escribir reseña, Editar perfil

Clase Kind

Medidas máximas (en píxeles) y peso máximo de la subida (en MB) de un tipo de imagen.

```
def process(upload, kind)
```

Devuelve la imagen subida lista para guardar: un ContentFile en WebP con nombre «<tipo>-<aleatorio>.webp».

kind es una clave de KINDS. Lanza ValidationError con un mensaje para el usuario si no es una imagen, es de otro formato, pesa más de lo permitido o tiene demasiados píxeles. De los GIF animados se queda con el primer fotograma.

Pantallas: Editar perfil (Apariencia: foto, cabecera y fondo), Escribir reseña (imágenes), Publicar (imágenes), Inicio (imágenes de la caja para publicar).

accounts/images.py, línea 45

accounts/management/commands/nightly.py

Comando «manage.py nightly»: las tareas de cada noche en orden (sync_libraries, close_challenges, update_prices y localize_games). No pinta ninguna pantalla; lo que guarda se ve sobre todo en el Perfil, la Ficha del juego, los Retos y la Lista de deseos.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Lista de deseos, Página de la lista, Retos, Página del reto, Avisos, Juegos importados

Clase Command

```
def handle(self, *args, **options)
```

Lanza cada tarea aunque falle la anterior y da error al final si alguna ha fallado.

Pantallas: Perfil (biblioteca y horas; insignias de las portadas, cifras y sección «Logros»; filtro de idioma de la biblioteca), Ficha del juego (tus horas; tus logros; botón «Lo quiero»; Dónde comprarlo; nombre en España, sinopsis en español e idiomas), Juegos importados (los que no se encuentran), Inicio (feed: la importación y «consiguió el reto»), Retos (progreso), Página del reto (progreso y quién lo ha conseguido), Perfil (insignias de la sección «Retos»), Lista de deseos (mejor precio de hoy y mínimo histórico; filtro de idioma), Página de la lista (lo que cuesta lo que te falta), Avisos (bajadas de precio; retos conseguidos, que se acaban y cómo has quedado), Catálogo (búsqueda por nombre en España y filtro de idioma).

accounts/management/commands/nightly.py, línea 20

accounts/management/commands/rematch_titles.py

Comando «manage.py rematch_titles»: vuelve a buscar en el catálogo los títulos de PlayStation y Xbox importados que no se encontraron. Lo que empareja deja de salir en Juegos importados y pasa a la biblioteca del Perfil con sus horas.
Pantallas de este archivo: Ficha del juego, Perfil, Juegos importados

Clase Command

```
def add_arguments(self, parser)
```

Opción: --dry-run, que enseña lo que haría sin guardar nada.

Pantallas: ninguna (opciones del comando rematch_titles).

accounts/management/commands/rematch_titles.py, línea 23

```
def handle(self, *args, **options)
```

Con --dry-run lo hace todo dentro de una transacción que se deshace, también los juegos que traiga de IGDB.

Pantallas: Juegos importados, Perfil (biblioteca y horas), Ficha del juego (tus horas).

accounts/management/commands/rematch_titles.py, línea 30

```
def rematch(self)
```

Busca los títulos pendientes, apunta su juego (para todos, como una importación) y actualiza las bibliotecas.

Las horas pasan como con una corrección del emparejamiento: sin apuntar actividad ni sesiones, porque no son de ahora.

Pantallas: Juegos importados, Perfil (biblioteca y horas), Ficha del juego (tus horas).

accounts/management/commands/rematch_titles.py, línea 43

accounts/management/commands/seed_test_profile.py

Comando «manage.py seed_test_profile»: crea la cuenta de prueba con biblioteca, reseñas, listas, retos, logros y actividad de ejemplo (o, con --vacio, la cuenta vacía «tester_steam» para probar las importaciones). Solo para desarrollo.
Pantallas de este archivo: Inicio, Ficha del juego, Listas del juego, Perfil, Página de la reseña, Página de la lista, Tier lists, Retos, Página del reto, Editar perfil

Clase Command

```
def add_arguments(self, parser)
```

Opción --vacio para crear la cuenta vacía en vez de la de ejemplo.

Pantallas: ninguna (opciones del comando seed_test_profile).

accounts/management/commands/seed_test_profile.py, línea 149

```
def handle(self, *args, **options)
```

Crea el usuario de prueba con su biblioteca y reseñas, y añade reseñas de los usuarios demo.

Solo funciona con DJANGO_DEBUG=1. Si el usuario ya existía, rehace su biblioteca desde cero. Con --vacio crea (o deja vacía) la cuenta «tester_steam».

Pantallas: Perfil, Inicio, Ficha del juego, Página de la reseña, Página de la lista, Retos, Página del reto (los datos de ejemplo de la cuenta de prueba).

accounts/management/commands/seed_test_profile.py, línea 158

```
def achievements(self, user, games)
```

Rehace los títulos importados de prueba del usuario con sus logros (ACHIEVEMENTS), sin tocar los de verdad. Los de Steam van también uno a uno (demo_achievements), para ver «Tus logros» desplegados en la ficha y la sección «Logros» del perfil.

Pantallas: Perfil (logros de ejemplo), Ficha del juego (tus logros).

accounts/management/commands/seed_test_profile.py, línea 249

```
def demo_achievements(self, imported, unlocked, total, now)
```

Rehace los logros de ejemplo del título de Steam de imported: total, con los nombres de DEMO_ACHIEVEMENTS, uno de cada ocho oculto y del más común al más raro, y los unlocked primeros conseguidos a lo largo de los dos últimos años, del más antiguo al más reciente.

Pantallas: Ficha del juego (tus logros de ejemplo), Perfil (sección «Logros»).

accounts/management/commands/seed_test_profile.py, línea 271

```
def review(self, user, entry, text, spoilers, when)
```

Publica una reseña de user sobre el juego de entry con fecha when, con su nota, horas y estado.

Pantallas: Página de la reseña, Ficha del juego (reseñas), Perfil (pestaña Reseñas), Inicio (feed).

accounts/management/commands/seed_test_profile.py, línea 298

```
def lists(self, user, catalog)
```

Rehace las listas de ejemplo (LISTS) del usuario de prueba y de los usuarios demo que existan, con sus juegos (catalog: los de su biblioteca y los extra, en orden), sus comentarios y sus niveles, cada una con su actividad (creada hace unas horas, con sus juegos), con los «me gusta» y comentarios de LIST_REACTIONS y con quien colabora en ellas (LIST_COLLABORATORS). Las de OPEN_TIER_LISTS quedan abiertas a la comunidad, con las versiones de TIER_LIST_VERSIONS y su actividad.

Pantallas: Perfil (pestañas Listas y Tier lists), Página de la lista, Tier lists, Ficha del juego (listas en las que sale), Listas del juego.

accounts/management/commands/seed_test_profile.py, línea 308

```
def challenges(self, user, games)
```

Rehace los retos de ejemplo (CHALLENGES) con quienes se unen a ellos, y las sesiones de juego del usuario de prueba (SESSIONS, sobre games, los juegos de su biblioteca) para que el reto de horas tenga progreso.

Pantallas: Retos, Página del reto.

accounts/management/commands/seed_test_profile.py, línea 373

```
def activity(self, user, game, when, status, rating, review)
```

Apunta en la actividad de user lo que hizo con game en el momento when: su estado, su nota y su reseña (o None).

Pantallas: Inicio (feed), Perfil (pestaña Actividad).

accounts/management/commands/seed_test_profile.py, línea 406

```
def create_empty_user(self)
```

Crea «tester_steam» o lo deja como recién creado: sin juegos ni cuentas de Steam, PlayStation, Xbox u otras plataformas.

Pantallas: Perfil, Editar perfil (la cuenta «tester_steam» queda vacía y sin cuentas vinculadas).

accounts/management/commands/seed_test_profile.py, línea 416

accounts/management/commands/sync_libraries.py

Comando «manage.py sync_libraries» (parte de la tarea de cada noche): vuelve a importar la biblioteca, los logros y, si se ha pedido, la lista de deseos de Steam de quien tiene activada la sincronización automática en Editar perfil › Cuentas vinculadas.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Lista de deseos, Retos, Página del reto, Juegos importados

Clase Command

```
def add_arguments(self, parser)
```

Opciones: --pause (segundos entre usuarios), --min-hours (saltar a los recién sincronizados) y --user.

Pantallas: ninguna (opciones del comando sync_libraries).

accounts/management/commands/sync_libraries.py, línea 31

```
def handle(self, *args, **options)
```

Recorre los usuarios con cuentas de Steam, del que lleva más tiempo sin sincronizar al que menos.

Un usuario que falla (perfil privado, Steam o IGDB caídos) no para a los demás: se apunta en el log y se sigue. Solo da error al final si no ha podido sincronizar a nadie. Que falle la lista de deseos de alguien no cuenta como fallo de su biblioteca.

Pantallas: Perfil (biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros; botón «Lo quiero»), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso), Lista de deseos.

accounts/management/commands/sync_libraries.py, línea 46

```
def sync_wishlist(self, user)
```

Trae a «Lo quiero» la lista de deseos de Steam del usuario y lo cuenta; si falla, lo apunta en el log.

Pantallas: Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/management/commands/sync_libraries.py, línea 106

accounts/matching.py

Emparejamiento de los juegos importados (Steam, PlayStation, Xbox) con el catálogo y su paso a la biblioteca.

Cada título importado se guarda en ExternalTitle (compartido por todos los usuarios) y en ImportedTitle (del usuario, con sus minutos). Los títulos ya emparejados —por una importación anterior o porque alguien lo corrigió a mano— no se vuelven a buscar; los demás se buscan con la función de cada plataforma: Steam por su ID de app y las demás por nombre, sin fijarse en cómo está escrito (name_key), en el catálogo local y, si falta, en IGDB. Las horas de cada plataforma en la biblioteca salen después de todos los ImportedTitle del usuario.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Retos, Página del reto, Editar perfil, Juegos importados, Panel de administración

```
def clean_name(name)
```

Nombre de un juego sin ™ ® © ni coletillas de plataforma, para buscarlo en el catálogo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 49

```
def normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 61

```
def name_key(name)
```

Nombre para comparar sin fijarse en cómo está escrito: sin ™ ® ©, signos, espacios, tildes ni mayúsculas, para que «WATCH_DOGS 2» y «Watch Dogs 2», «Tom Clancy’s» y «Tom Clancy's» o «Ghostrunner2» y «Ghostrunner 2» sean el mismo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 71

```
def _name_keys(name, name_es, search_names)
```

Claves (name_key) de todos los nombres de un juego: el suyo, el de España y los otros que guarda para el buscador.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 84

```
def _catalog_games(keys)
```

Juegos del catálogo que tienen alguna de las claves keys entre sus nombres, con sus plataformas ya cargadas.

La comparación no cabe en una consulta, así que recorre los nombres de todo el catálogo. Solo se hace cuando hay títulos que buscar, no en cada visita: si un día el catálogo tiene cientos de miles de juegos, habrá que guardar las claves.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 94

```
def _title_parts(name)
```

Lo que se busca de un título: (clave de su nombre, clave sin el año o la región del final, nombre sin ellos, año o None).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 109

```
def add_title(titles, external_id, name, minutes=0, last_played=None, platforms=(),
first_played=None, deck_minutes=0)
```

Añade un título importado al dict titles ({ID externo: datos}).

Si el ID ya estaba (el mismo juego en dos cuentas de Steam, o dos versiones con el mismo nombre) suma los minutos (y los jugados en una Steam Deck), se queda con la última partida más reciente y con la primera más antigua, y junta las plataformas, que son nombres de plataforma de IGDB («PlayStation 5», «Xbox One»...): sirven para desempatar al buscar por nombre y para rellenar la plataforma de las reseñas.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 121

```
def _platform_names(game)
```

Nombres de las plataformas del juego (de IGDB), aprovechando el prefetch si lo hay.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 146

```
def _fits(game, platforms)
```

Indica si el juego puede ser uno jugado en esas plataformas.

Sin plataformas conocidas (del título o del juego) cualquiera vale.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 156

```
def _share_platform(names, platforms)
```

Indica si un juego que sale en las plataformas names puede ser uno jugado en platforms (ver _fits).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 168

```
def _seen_games(keys)
```

Juegos de los títulos ya emparejados, de cualquier plataforma, que se llaman como alguna de las claves keys, como {clave: [Game]}: Steam los empareja por su ID de app y los demás pueden estar corregidos a mano, así que son una pista fiable.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 178

```
def _score(game, group, platforms, year)
```

Lo bueno que es game para un título, para quedarse con el mayor: primero que sea del año que dice el título, después que salga en sus plataformas, después cómo se ha encontrado (group: cuanto más alto, más se parece el nombre), después el año más cercano y, por último, el más popular.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 198

```
def match_by_name(titles)
```

Busca en el catálogo los juegos de {clave: (nombre limpio, plataformas)} y devuelve {clave: Game}.

Los nombres se comparan sin fijarse en cómo están escritos (name_key), con el nombre de cada juego, con sus otros nombres (el de España y los alternativos que guarda para el buscador) y con los títulos ya emparejados de otras plataformas (_seen_games). Un título que acaba en un año o una región («resident evil 4 (2005)», «LittleBigPlanet3 (LATAM)») también vale sin ellos. Primero mira lo que ya hay; después pide a IGDB por nombre exacto (tal cual y con mayúsculas de título, porque a veces vienen en mayúsculas) los que no tengan ningún juego que se llame así en sus plataformas y, para los que sigan sin ningún candidato, hace hasta MAX_SEARCHES búsquedas sueltas quedándose solo con los resultados que tienen ese nombre entre los suyos y salen en sus plataformas. Los lanzadores (LAUNCHERS) no se buscan. Si varios juegos encajan (remakes como God of War o Resident Evil 4), decide _score.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 217

```
def save_titles(user, platform, titles, auto_match)
```

Guarda los títulos importados de un usuario en una plataforma y devuelve {ID externo: Game o None}.

titles es el dict que llena add_title. Los títulos que ya tienen juego, o que alguien dejó sin juego a mano, se respetan; los demás se buscan con auto_match(ids), que devuelve {ID: Game}. Los minutos, las partidas y las plataformas del usuario se sobrescriben con los de esta importación.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/matching.py, línea 294

```
def update_library(user, platform, opened_only=False, record_activity=False)
```

Pasa a la biblioteca las horas de una plataforma a partir de todos los títulos importados del usuario.

Por juego suma los minutos de sus títulos (puede venir de varias cuentas o de la versión de PS4 y la de PS5) y toma la última partida más reciente. Los juegos nuevos entran como «Jugando» si se jugaron en los últimos días y como «Jugado» si no; sin horas ni partida, como «Pendiente», salvo con opened_only (plataformas que solo listan juegos abiertos alguna vez). Los que ya estaban conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas). Si un título ha cambiado de juego desde la última vez (porque se corrigió el emparejamiento), al juego anterior se le quitan sus horas de esta plataforma. Si han cambiado horas, avisa al usuario de las reseñas que se le han quedado atrás (library.reminders). Con record_activity (lo piden las importaciones, no las correcciones del emparejamiento), los juegos nuevos se apuntan en su actividad como una sola importación (social.activity.games_imported) y lo que ha subido el tiempo de los que ya tenían horas de esta plataforma se apunta como sesiones de juego (library.models.PlaySession), con las que se mira si ha conseguido algún reto de horas (challenges.progress.check). Devuelve (juegos añadidos, juegos actualizados).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso), Panel de administración (al corregir un título importado).

accounts/matching.py, línea 350

```
def reassign(imported, game, by)
```

Cambia el juego de un título importado (None = ninguno) para todos y recoloca las horas de quien lo cambia.

Lo hecho por un administrador queda como suyo («admin»); lo de los demás, como «user». Los demás usuarios con ese título lo verán en su próxima importación. Si el juego anterior se queda en la biblioteca de quien lo cambia sin horas, valoración ni reseña, se quita: solo estaba por el error. Lanza PermissionError si el título está bloqueado y quien lo cambia no es administrador.

Devuelve un dict con los juegos añadidos («added») y actualizados («updated») en su biblioteca y si se ha quitado el juego anterior («removed»).

Pantallas: Juegos importados, Perfil (biblioteca y horas), Ficha del juego (tus horas).

accounts/matching.py, línea 426

accounts/models.py

Modelos de las cuentas: el usuario y su perfil (lo que se personaliza en Editar perfil y se ve en el Perfil y, con su foto y su nombre, en todas las pantallas), sus juegos favoritos, las cuentas vinculadas de Steam, PlayStation, Xbox y otras plataformas, los títulos importados de cada una con sus horas y logros, y los avisos de la campana.
Pantallas de este archivo: Inicio, Ficha del juego, Listas del juego, Perfil, Seguidores y seguidos, Página de la publicación, Escribir reseña, Página de la reseña, Lista de deseos, Página de la lista, Nueva tier list, Editar lista, Página del reto, Editar perfil, Juegos importados, Panel de administración

```
def normalize_handle(network, value)
```

Limpia el usuario de una red social tal como lo escribe la gente.

Quita los espacios y la @ del principio y, si pega la URL de su perfil, se queda con el usuario (en Bluesky, lo que va después de /profile/). El usuario de Discord se pasa a minúsculas.

Pantallas: Editar perfil (Redes sociales), Perfil (redes de la cabecera).

accounts/models.py, línea 54

```
def text_on(color)
```

Color del texto encima de un fondo del color color («#rrggbb»): blanco, como en el resto de la web, salvo en los colores claros (contraste con el blanco por debajo de 3:1, el mínimo de WCAG para texto en negrita), que llevan texto casi negro.

Pantallas: Perfil (texto sobre el color del perfil), Seguidores y seguidos, Página de la lista (texto sobre el color de cada nivel de las tier lists), Editar lista (niveles), Nueva tier list (niveles).

accounts/models.py, línea 72

```
def profile_upload_to(instance, filename)
```

Ruta de las imágenes de un perfil: perfiles/<id del usuario>/<nombre aleatorio que pone accounts.images>.

Pantallas: Editar perfil (dónde se guardan la foto, la cabecera y el fondo subidos).

accounts/models.py, línea 91

Clase Profile

```
def __str__(self)
```

Texto del perfil en el admin y en la consola: su nombre visible.

Pantallas: Panel de administración.

accounts/models.py, línea 218

```
@property
def name(self)
```

Nombre que se muestra: el nombre visible si lo tiene, si no el nombre de usuario.

Pantallas: todas (el nombre de cada jugador, también en el menú de la cuenta de la barra de arriba).

accounts/models.py, línea 226

```
def get_absolute_url(self)
```

URL del perfil público del usuario (/u/<usuario>/).

Pantallas: Inicio, Ficha del juego, Perfil, Seguidores y seguidos, Página de la publicación, Página de la reseña, Página de la lista, Editar lista, Página del reto (los enlaces al perfil de cada jugador).

accounts/models.py, línea 233

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el perfil: siempre si es público, y si es privado solo su dueño.

Es la regla de quién ve qué en todo NextPlay (perfil, biblioteca, reseñas, seguidores...); en las consultas la aplica visible_filter, que tiene que decir lo mismo.

Pantallas: Perfil, Seguidores y seguidos, Página de la reseña, Página de la publicación, Página de la lista, Página del reto (los perfiles privados solo los ve su dueño).

accounts/models.py, línea 242

```
@staticmethod
def visible_filter(viewer, path='user')
```

La regla de visible_to como condición de consulta (Q): lo de los perfiles públicos y lo de viewer.

path es el camino desde lo que se filtra hasta su usuario: «user» en las reseñas (el de por defecto), «follower» en los seguidores...; vacío para filtrar los propios usuarios.

Pantallas: Inicio (feed y sugerencias), Ficha del juego (reseñas y listas), Escribir reseña, Página del reto (clasificación), Seguidores y seguidos, Listas del juego (lo de perfiles privados solo lo ve su dueño).

accounts/models.py, línea 254

```
@property
def steam_accounts(self)
```

Cuentas de Steam vinculadas, de la primera a la última (aprovecha el prefetch si lo hay).

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 270

```
@property
def steam_linked(self)
```

Indica si el usuario tiene al menos una cuenta de Steam vinculada.

Pantallas: Perfil (modal «Vincular cuentas»), Lista de deseos (aviso de traer la de Steam).

accounts/models.py, línea 279

```
@property
def steam_avatar(self)
```

Avatar de la primera cuenta de Steam vinculada, o cadena vacía si no tiene ninguna.

Pantallas: todas (la foto del perfil cuando es la de Steam).

accounts/models.py, línea 287

```
@property
def psn(self)
```

Cuenta de PlayStation vinculada, o None si no tiene.

Pantallas: Editar perfil (Cuentas vinculadas › PlayStation), Perfil (cuentas de la cabecera), todas (la foto del perfil cuando es la de PlayStation).

accounts/models.py, línea 296

```
@property
def platforms(self)
```

Cuentas de otras plataformas (Xbox, Switch, Epic...) que muestra en su perfil.

Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 308

```
@property
def xbox(self)
```

Cuenta de Xbox vinculada, o None si no tiene.

Pantallas: Editar perfil (Cuentas vinculadas › Xbox), Perfil (cuentas de la cabecera), todas (la foto del perfil cuando es la de Xbox).

accounts/models.py, línea 317

```
@property
def avatar_url(self)
```

Foto del perfil según avatar_source: la subida o la de Steam, PlayStation o Xbox; vacía para la inicial.

En automático (o si la elegida ya no existe, p. ej. porque desvinculó la cuenta) prueba por orden la subida, la de Steam, la de PlayStation y la de Xbox. Cada una se mira solo si hace falta.

Pantallas: todas (la foto de cada jugador, también la tuya en la barra de arriba).

accounts/models.py, línea 329

```
@property
def accent(self)
```

Color del perfil en hexadecimal; el violeta de NextPlay si no ha elegido otro.

Pantallas: Perfil (color del perfil), Seguidores y seguidos, Página de la lista, Editar perfil (Color del perfil).

accounts/models.py, línea 351

```
@property
def accent_rgb(self)
```

El color del perfil como «r, g, b», para usarlo con transparencia en el CSS: rgba(var(--accent-rgb), .3).

Pantallas: Perfil (color del perfil), Seguidores y seguidos, Página de la lista.

accounts/models.py, línea 360

```
@property
def accent_text(self)
```

Color del texto encima del color del perfil (ver text_on).

Pantallas: Perfil (texto sobre el color del perfil), Seguidores y seguidos, Página de la lista.

accounts/models.py, línea 368

```
@property
def banner_url(self)
```

Imagen de la cabecera según banner_mode, o vacía para el degradado con el color del perfil.

Con un juego usa el arte de IGDB elegido o, si no hay, su portada (que se enseña desenfocada).

Pantallas: Perfil (cabecera), Editar perfil (Apariencia › Cabecera).

accounts/models.py, línea 376

```
@property
def banner_blurred(self)
```

Indica si la cabecera es la portada de un juego: es vertical y se desenfoca para que no se vea pixelada.

Pantallas: Perfil (cabecera), Editar perfil (Apariencia › Cabecera).

accounts/models.py, línea 390

```
@property
def background_url(self)
```

Imagen de fondo de la página del perfil según background_mode, o vacía si no tiene.

Pantallas: Perfil (fondo de la página), Editar perfil (Apariencia › Fondo de la página).

accounts/models.py, línea 398

```
def _uploaded_url(self, name)
```

URL de la imagen subida del campo name (avatar, banner o background), o vacía si no hay.

En la vista previa de Editar perfil manda la que se acaba de elegir y aún no se ha subido.

Pantallas: todas (la foto subida), Perfil (cabecera y fondo subidos), Editar perfil (Apariencia).

accounts/models.py, línea 409

```
@property
def background_blurred(self)
```

Indica si el fondo es la portada de un juego (la de la cabecera) y hay que desenfocarlo.

Pantallas: Perfil (fondo de la página), Editar perfil (Apariencia › Fondo de la página).

accounts/models.py, línea 420

```
@property
def background_dim_ratio(self)
```

Lo que se oscurece el fondo como número entre 0 y 1 con punto decimal, para el CSS.

Pantallas: Perfil (lo que se oscurece el fondo).

accounts/models.py, línea 428

```
@property
def avatar_image(self)
```

La foto que se enseña, para accounts/_avatar.html: {"url", "framing", "fit"}, o {} si va la inicial.

El encuadre (ver _framing) solo vale para la foto subida: las de Steam, PlayStation y Xbox se enseñan tal cual.

Pantallas: todas (la foto de cada jugador, con su encuadre, también la tuya en la barra de arriba).

accounts/models.py, línea 436

```
@property
def banner_framing(self)
```

Encuadre de la cabecera como variables de CSS (ver _framing).

Pantallas: Perfil (encuadre de la cabecera).

accounts/models.py, línea 452

```
@property
def background_framing(self)
```

Encuadre del fondo de la página como variables de CSS (ver _framing).

Pantallas: Perfil (encuadre del fondo).

accounts/models.py, línea 460

```
def _framing(self, name)
```

Encuadre de la imagen name (avatar, banner o background) como variables de CSS para su atributo style.

«--focus-x: 30%; --focus-y: 20%; --zoom: 1.5;», que app.css pasa a object-position y a la escala de la imagen. Vacío con el encuadre por defecto: centrada y sin zoom.

Pantallas: todas (encuadre de la foto de cada jugador), Perfil (encuadre de la cabecera y el fondo).

accounts/models.py, línea 467

```
@property
def section_layout(self)
```

Secciones del escaparate en el orden elegido: [(clave, nombre, visible)].

Las que no estén en sections (nuevas o nunca configuradas) van al final y visibles; las claves que ya no existen o repetidas se ignoran.

Pantallas: Editar perfil (Escaparate › Secciones del perfil), Perfil (escaparate).

accounts/models.py, línea 481

```
@property
def visible_sections(self)
```

Claves de las secciones del escaparate que se enseñan, en su orden.

Pantallas: Perfil (las secciones del escaparate que se enseñan).

accounts/models.py, línea 498

```
@property
def social_links(self)
```

Redes del perfil para enseñarlas: [(clave, nombre, usuario o dominio, URL o vacía)].

Van en el orden de SOCIAL_LINKS y la web personal al final, con su dominio como texto.

Pantallas: Perfil (redes de la cabecera).

accounts/models.py, línea 506

Clase FavoriteGame

Juego que el usuario enseña en «Juegos favoritos» de su perfil; como mucho MAX_PER_USER, en el orden que elija.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

Pantallas: Panel de administración.

accounts/models.py, línea 540

Clase SteamAccount

Cuenta de Steam vinculada a un usuario; un usuario puede tener varias.

```
def __str__(self)
```

Texto de la cuenta en el admin: su nombre en Steam, o el SteamID si no se conoce.

Pantallas: Panel de administración.

accounts/models.py, línea 568

```
@property
def name(self)
```

Nombre que se muestra: el de Steam, o el SteamID si aún no se ha podido leer.

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 576

```
@property
def url(self)
```

URL del perfil en Steam; la construye con el SteamID si Steam no la ha dado.

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 585

Clase PSNAccount

Cuenta de PlayStation Network vinculada a un usuario (una por usuario).

No se guarda ningún código ni token de Sony: el usuario pega su código NPSSO cada vez que sincroniza.

```
def __str__(self)
```

Texto de la cuenta en el admin: su PSN ID, o el ID de cuenta si no se conoce.

Pantallas: Panel de administración.

accounts/models.py, línea 612

```
@property
def name(self)
```

Nombre que se muestra: el PSN ID, o «PlayStation» si Sony no lo ha devuelto.

Pantallas: Editar perfil (Cuentas vinculadas › PlayStation), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 620

```
@property
def url(self)
```

Perfil público en PSNProfiles (Sony no tiene página pública de perfiles), o vacío sin PSN ID.

Pantallas: Perfil (enlace de la cuenta de PlayStation en la cabecera).

accounts/models.py, línea 629

Clase PlatformAccount

Cuenta de otra plataforma que el usuario muestra en su perfil (sin importar juegos).

No se verifica que sea suya: es lo que escribe, como en la mayoría de webs. Una por plataforma.

```
def __str__(self)
```

Texto de la cuenta en el admin: «Plataforma: nombre».

Pantallas: Panel de administración.

accounts/models.py, línea 687

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la plataforma.

Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (cuentas de la cabecera).

accounts/models.py, línea 695

```
@property
def url(self)
```

Perfil público en la web de la plataforma cuando existe (solo GOG); si no, vacío.

Pantallas: Perfil (enlace de la cuenta de GOG en la cabecera).

accounts/models.py, línea 703

```
@classmethod
def normalize(cls, platform, handle)
```

Limpia lo que escribe el usuario: quita espacios de los extremos y pone el código de Switch en formato SW-1234-5678-9012 aunque lo escriba solo con números o con espacios.

Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (modal «Vincular cuentas»).

accounts/models.py, línea 713

```
@classmethod
def is_valid_handle(cls, platform, handle)
```

Indica si el nombre o código cumple el formato de esa plataforma.

Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (modal «Vincular cuentas»).

accounts/models.py, línea 727

Clase XboxAccount

Cuenta de Xbox (cuenta Microsoft) vinculada a un usuario (una por usuario).

No se guarda ningún token de Microsoft: para sincronizar el usuario vuelve a iniciar sesión.

```
def __str__(self)
```

Texto de la cuenta en el admin: su gamertag.

Pantallas: Panel de administración.

accounts/models.py, línea 753

```
@property
def name(self)
```

Nombre que se muestra: el gamertag, o «Xbox» si Microsoft no lo ha devuelto.

Pantallas: Editar perfil (Cuentas vinculadas › Xbox), Perfil (cuentas de la cabecera y modal «Vincular cuentas»).

accounts/models.py, línea 761

```
@property
def url(self)
```

Perfil público del gamertag en xbox.com, o vacío si no se conoce.

Pantallas: Editar perfil (Cuentas vinculadas › Xbox), Perfil (enlace de la cuenta de Xbox en la cabecera).

accounts/models.py, línea 770

Clase ExternalTitle

Juego tal como lo identifica una plataforma (app de Steam, titleId de PlayStation o de Xbox) y el juego del catálogo que le corresponde (vacío si no se encontró).

Es compartido entre usuarios: la importación lo mira antes de buscar por nombre, así que una corrección hecha a mano sirve para todos. Un administrador puede bloquearlo para que nadie más lo cambie.

```
def __str__(self)
```

Texto del título en el admin: «Plataforma: nombre».

Pantallas: Panel de administración.

accounts/models.py, línea 825

Clase ImportedTitle

Título que tiene un usuario en una plataforma, con los minutos y la última partida de su última importación.

De aquí salen las horas de cada plataforma en la biblioteca, así que al corregir el juego de un título se pueden recolocar sin volver a importar.

```
def __str__(self)
```

Texto en el admin: «usuario · título».

Pantallas: Panel de administración.

accounts/models.py, línea 872

```
@property
def hours(self)
```

Horas jugadas redondeadas a una décima.

Pantallas: Juegos importados (horas de cada título).

accounts/models.py, línea 880

```
def set_achievements(self, unlocked, total, platinum=False, at=None)
```

Apunta los logros leídos de la plataforma (sin guardar): unlocked de total, con o sin platino, leídos en at (ahora, si no se da). Devuelve True si ha cambiado algo de lo que se enseña.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/models.py, línea 887

Clase Achievement

Logro de un juego en su plataforma (de momento, solo de Steam), compartido por todos los que lo tienen: su nombre y su descripción en castellano, sus iconos, si es oculto y qué parte de los jugadores de la plataforma lo tiene (percent, de donde sale su rareza). Lo guarda steam.import_achievements la primera vez que alguien tiene logros de ese juego; lo que ha conseguido cada uno va en UnlockedAchievement.

```
def __str__(self)
```

Texto en el admin: «Plataforma: juego · logro».

Pantallas: Panel de administración.

accounts/models.py, línea 937

```
@property
def rarity(self)
```

Nombre de su rareza (RARITY_TIERS): «Rarísimo», «Muy raro» o «Raro», o vacío si es común o no se sabe.

Pantallas: Ficha del juego (tus logros: su rareza), Perfil (sección «Logros»: el más raro).

accounts/models.py, línea 945

Clase UnlockedAchievement

Logro que ha conseguido un usuario en uno de sus títulos importados y cuándo. Algunos logros muy antiguos de Steam no tienen fecha. Lo apunta steam.import_achievements cada vez que lee los logros de ese juego.

```
def __str__(self)
```

Texto en el admin: «usuario · logro».

Pantallas: Panel de administración.

accounts/models.py, línea 977

Clase Notification

Aviso para un usuario: se ve en la campana de la cabecera hasta que abre la página de avisos.

```
def __str__(self)
```

Texto del aviso en el admin: «usuario · texto».

Pantallas: Panel de administración.

accounts/models.py, línea 1028

accounts/notices.py

Un solo sitio para crear los avisos de la campana (accounts.models.Notification) y decidir si llegan.

Todo lo que avisa a alguien en NextPlay pasa por notify(): las bajadas de precio, los nuevos seguidores, los «me gusta», los comentarios, las invitaciones a editar listas, las reseñas que se han quedado atrás y las denuncias. notify() decide lo mismo para todos: nadie recibe avisos de lo que hace él mismo, no se repiten (cada tipo tiene su regla en REPEAT) y solo llega lo que cada uno ha dejado encendido en la sección «Avisos» de Editar perfil (GROUPS y wants()). Un tipo de aviso nuevo necesita su regla y su grupo: los tests lo comprueban.

Pantallas: Avisos, todas (el número de la campana), Editar perfil (Avisos).
Pantallas de este archivo: Lista de deseos, Avisos, Editar perfil

Clase Repeat

Regla para no repetir un aviso: no se crea si la misma persona ya tiene otro del mismo tipo que coincide en fields («actor», quién lo provoca, y «url», aquello de lo que avisa) y, con unread, que aún no ha leído. Sin fields se avisa siempre, porque quien avisa ya sabe si toca.

Clase Group

Una línea de la sección «Avisos» de Editar perfil: su nombre, lo que explica, los tipos de aviso que enciende o apaga y si también pueden llegar por correo.

```
def enabled(prefs, key, channel=BELL)
```

Si el grupo key está encendido para channel en prefs (lo guardado en Profile.notification_prefs). Lo que no está guardado, sí.

Pantallas: Editar perfil (Avisos: cómo sale cada interruptor), Avisos, todas (el número de la campana).

accounts/notices.py, línea 115

```
def wants(user, kind, channel=BELL)
```

Si user quiere los avisos de tipo kind por channel (BELL o EMAIL), según lo que eligió en Editar perfil.

Por correo solo llegan los grupos que lo admiten (hoy, las bajadas de precio). Los tipos sin grupo, las denuncias, llegan siempre a la campana y nunca por correo.

Pantallas: Avisos, todas (el número de la campana), Lista de deseos (cómo te avisamos de las bajadas de precio).

accounts/notices.py, línea 124

```
def notify(user, kind, text, url='', *, game=None, actor=None, since=None)
```

Avisa a user en la campana: crea y devuelve el aviso de tipo kind con text, url, el juego cuya portada sale al lado y quién lo ha provocado (actor). No lo crea, y devuelve None, si:

- actor es el propio user: nadie recibe avisos de lo que hace él mismo; - user ha apagado ese tipo de aviso en Editar perfil (wants); - ya tiene uno que la regla de ese tipo no deja repetir (REPEAT); con since, solo cuentan los de desde entonces.

El texto se corta a lo que cabe en Notification.text. Cuando existan los bloqueos y los silencios, este es el único sitio que tendrá que mirarlos.

Pantallas: Avisos, todas (el número de la campana).

accounts/notices.py, línea 141

accounts/psn.py

Importación de juegos y horas de PlayStation (PS4/PS5) con la API no oficial de la app de PlayStation.

Sony no tiene API pública: se usa la misma que la app oficial, a la que se accede con el código NPSSO de la sesión del usuario en playstation.com. El código no se guarda; se pide en cada sincronización. Es experimental: Sony puede cambiarla sin avisar.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Retos, Página del reto, Editar perfil, Juegos importados

Clase PSNError

Fallo al hablar con PlayStation (red, respuesta inesperada o API cambiada).

Clase PSNAuthError

Sony no acepta el código NPSSO (mal copiado o caducado).

Clase PSNRejectedError

PlayStation rechaza lo que se le pide (400 o 404): p. ej. un ID de juego que no conoce.

```
def clean_npsso(value)
```

Saca el código NPSSO de lo que pegue el usuario: el código suelto o el JSON entero de la página de Sony.

Devuelve el código o None si no tiene el formato esperado (64 letras y números).

Pantallas: Editar perfil (Cuentas vinculadas: PlayStation: el código NPSSO pegado), Perfil (modal «Vincular cuentas» de su dueño).

accounts/psn.py, línea 58

```
def access_token(npsso)
```

Cambia el código NPSSO por un token de acceso de la API de PlayStation.

Lanza PSNAuthError si Sony no da el código de autorización (NPSSO incorrecto o caducado).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 75

```
def _get(url, token, **params)
```

GET autenticado contra la API de PlayStation; devuelve el JSON. Lanza PSNRejectedError si PlayStation rechaza lo que se le pide (400 o 404) y PSNError con cualquier otro fallo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 121

```
def _account_id_from_token(token)
```

ID de cuenta guardado dentro del token JWT, por si el perfil no lo devuelve; None si no está.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 142

```
def profile(token)
```

Datos de la cuenta del token: {"account_id", "online_id", "avatar"}.

Si el perfil no responde, intenta sacar al menos el ID de cuenta del propio token.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; cuenta de PlayStation de la cabecera), todas (la foto del perfil cuando es la de PlayStation).

accounts/psn.py, línea 158

```
def played_games(token)
```

Juegos de PS4 y PS5 que ha jugado la cuenta, con «name», «playDuration», «lastPlayedDateTime»...

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 181

```
def trophy_titles(token)
```

Juegos con trofeos de la cuenta (también de PS3 y PS Vita): cada uno con su nombre («trophyTitleName») y, por tipo (bronze, silver, gold y platinum), los trofeos que tiene («definedTrophies») y los conseguidos («earnedTrophies»).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/psn.py, línea 199

```
def trophies_by_title_id(token, title_ids)
```

Listas de trofeos de los juegos title_ids a partir de su ID de PSN: {ID: [listas como las de trophy_titles]}.

Solo se piden los ID de verdad (TITLE_ID: los títulos que llegan sin ID se importan con su nombre), TITLE_IDS_PER_REQUEST por petición y TROPHY_WORKERS peticiones a la vez. Una colección trae una lista por cada juego que incluye. Si PlayStation rechaza una petición (un ID que no conoce la tumba entera), sus juegos se piden de uno en uno; los que aun así fallan, y los de una petición que falla por otra cosa (la red, el token), se quedan fuera y se buscan por nombre (save_trophies).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/psn.py, línea 216

```
def trophy_key(name)
```

Clave para emparejar un juego con su lista de trofeos por el nombre: sin coletillas de plataforma y sin fijarse en cómo está escrito (name_key), para que «Tom Clancy’s» y «Tom Clancy's» o «Ghostrunner2» y «Ghostrunner 2» sean el mismo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/psn.py, línea 245

```
def trophy_count(lists)
```

(conseguidos, totales, platino) de una o varias listas de trofeos sumadas (las de los juegos de una colección), o None si no tienen ningún trofeo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/psn.py, línea 255

```
def save_trophies(user, trophies, by_title=None)
```

Apunta en los títulos de PlayStation importados del usuario sus trofeos y devuelve en cuántos.

Primero, los de su ID de juego (by_title, de trophies_by_title_id): una colección suma las listas de sus juegos y cuenta como platino si tiene alguno. Los que no salen así se buscan por nombre (trophy_key) en la lista de trofeos de la cuenta (trophies, de trophy_titles): el del título o el del juego del catálogo que tiene y, si con ellos no aparece, sin el año o la región del final (YEAR_OR_REGION). Si un juego tiene trofeos aparte en PS4 y en PS5, por nombre cuenta la versión que más lleva (primero el platino). Los títulos sin trofeos leídos se quedan como estaban.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/psn.py, línea 269

```
def duration_minutes(value)
```

Minutos de una duración ISO 8601 de PSN («PT228H56M33S» → 13736); 0 si no se entiende.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 306

```
def import_library(user, npsso)
```

Vincula (o actualiza) la cuenta de PSN del NPSSO e importa sus juegos de PS4/PS5 con sus horas.

Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas de PlayStation. La versión de PS4 y la de PS5 de un juego se suman. Lanza PSNAuthError si el código no vale y ValueError si esa cuenta de PSN ya es de otro usuario.

Con el mismo token lee después los trofeos de cada juego (trophies_by_title_id, trophy_titles y save_trophies); si fallan, la importación de los juegos sigue valiendo.

Devuelve un dict con la cuenta («account»), cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), y de cuántos se han leído los trofeos («trophies»).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/psn.py, línea 324

accounts/signals.py

Señales de las cuentas: crean el perfil de cada usuario nuevo (al registrarse o desde el Panel de administración) y borran del disco las imágenes de un perfil que se elimina.
Pantallas de este archivo: Registro, Panel de administración

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, raw=False, **kwargs)
```

Crea el perfil de un usuario justo después de crearlo.

No hace nada al guardar un usuario ya existente ni al cargar fixtures (raw=True).

Pantallas: Registro (crea el perfil de la cuenta nueva), Panel de administración (también al crear un usuario allí).

accounts/signals.py, línea 13

```
@receiver(post_delete, sender=Profile)
def delete_profile_images(sender, instance, **kwargs)
```

Borra del disco la foto, la cabecera y el fondo de un perfil eliminado (p. ej. al borrar la cuenta).

Pantallas: Panel de administración (al borrar un usuario o un perfil).

accounts/signals.py, línea 26

accounts/steam.py

Vinculación con Steam (inicio de sesión OpenID) e importación de los juegos de su biblioteca y de su lista de deseos.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Lista de deseos, Retos, Página del reto, Editar perfil, Juegos importados

Clase SteamError

Fallo al hablar con Steam (red, clave de la API o respuesta no válida).

Clase SteamPrivateError

Steam no devuelve los juegos porque los detalles del perfil del usuario no son públicos.

```
def is_configured()
```

Indica si está puesta la clave de la Web API de Steam, necesaria para leer la biblioteca.

Pantallas: Editar perfil (Cuentas vinculadas: sin clave de Steam no se puede importar), Perfil (modal «Vincular cuentas» de su dueño).

accounts/steam.py, línea 56

```
def login_url(return_to, realm)
```

URL de Steam donde el usuario inicia sesión; Steam lo devuelve después a return_to.

Pantallas: Editar perfil (Cuentas vinculadas: botón para vincular Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/steam.py, línea 65

```
def verify_login(params, return_to)
```

Comprueba con Steam la respuesta de OpenID y devuelve el SteamID64 del usuario, o None si no vale.

Rechaza respuestas de otro proveedor, para otra URL de vuelta o que Steam no confirme como suyas (así nadie puede inventarse una URL de vuelta con el SteamID de otro).

Pantallas: Editar perfil (Cuentas vinculadas: vuelta desde Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/steam.py, línea 82

```
def _api(path, **params)
```

Llama a un método GET de la Web API de Steam con la clave de settings y devuelve su «response».

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas; botón «Lo quiero»), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso), Lista de deseos.

accounts/steam.py, línea 109

```
def player_summary(steam_id)
```

Nombre, avatar y URL del perfil de Steam: {"persona", "avatar", "profile_url"}, o {} si no lo encuentra.

Pantallas: Editar perfil (Cuentas vinculadas › Steam: nombre y avatar), Perfil (cuentas de la cabecera), todas (la foto del perfil cuando es la de Steam).

accounts/steam.py, línea 128

```
def owned_games(steam_id)
```

Juegos de la cuenta de Steam: [{"appid", "name", "playtime_forever", "rtime_last_played"...}].

Lanza SteamPrivateError si el perfil no deja ver sus juegos (Steam responde sin «games»).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/steam.py, línea 145

```
def wishlist(steam_id)
```

Lista de deseos de la cuenta de Steam: [{"appid", "priority", "date_added"}], o None si no se puede leer.

No necesita la clave de la API (se manda igual). Steam contesta lo mismo, sin «items», a una lista privada que a una vacía, así que las dos dan None.

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/steam.py, línea 167

```
def refresh_account(account)
```

Actualiza el nombre, el avatar y la URL guardados de una cuenta de Steam (sin fallar si Steam no responde).

Pantallas: Editar perfil (Cuentas vinculadas › Steam: nombre y avatar), Perfil (cuentas de la cabecera), todas (la foto del perfil cuando es la de Steam).

accounts/steam.py, línea 178

```
def _match_games(app_ids)
```

Busca los juegos del catálogo de una lista de IDs de Steam y devuelve {ID de Steam: Game}.

Primero mira el catálogo local; los que falten los busca en IGDB y los importa (si IGDB está configurado).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas; botón «Lo quiero»), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso), Lista de deseos.

accounts/steam.py, línea 194

```
def _timestamp(value)
```

Fecha de una marca de tiempo de Steam (segundos desde 1970), o None si no viene o es 0.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/steam.py, línea 221

```
def _played_at(data)
```

Fecha de la última partida de un juego de Steam, o None si nunca se ha jugado.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/steam.py, línea 231

```
def import_library(user, include_unplayed=False)
```

Importa a la biblioteca del usuario los juegos que ha jugado en todas sus cuentas de Steam vinculadas.

Si un juego está en varias cuentas (o en varias apps de Steam, como un relanzamiento) se suman sus horas y se toma la última partida más reciente. Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas. Con include_unplayed también entran los que nunca ha abierto, como «Pendiente». Cada app queda guardada como título importado, así que las que no se encuentren se pueden emparejar a mano. Las cuentas que no dejan ver sus juegos se saltan; si no deja ninguna, lanza SteamPrivateError.

Después lee los logros de los juegos jugados hace menos (import_achievements).

Devuelve un dict con cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), los nombres de las cuentas privadas («private») y de cuántos juegos se han leído los logros («achievements»).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/steam.py, línea 241

```
def player_achievements(steam_id, app_id)
```

Logros de un juego en una cuenta de Steam, todos y en el orden de Steam: [(clave, conseguido, fecha)], con la fecha en que lo consiguió o None si no lo tiene (o si es de los muy antiguos, de los que Steam no guarda la fecha); [] si el juego no tiene logros o None si no se pueden leer (perfil o detalles de juego privados, o Steam no responde).

A un juego sin logros Steam le contesta con un 400 y «Requested app has no stats», así que no se mira el código de la respuesta sino su «playerstats».

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 301

```
def _icon(url)
```

Dirección actual de un icono de logro de Steam (ICON_URL en vez de OLD_ICON_PATH), o vacía si no viene.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 329

```
def achievement_schema(app_id)
```

Logros de un juego de Steam con sus textos en castellano, en el orden de Steam: [{"api_name", "name", "description", "icon", "icon_locked", "hidden"}], [] si no tiene o None si Steam no responde.

De los ocultos Steam no da la descripción (tampoco en los logros de cada jugador), así que se queda vacía. Los iconos van con su dirección actual (_icon).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 338

```
def global_percentages(app_id)
```

Porcentaje de los jugadores de Steam que tiene cada logro de un juego, redondeado a una décima: {clave: porcentaje}, o None si Steam no lo da. No necesita la clave de la API, y el porcentaje llega como texto («63.1»).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 371

```
def achievement_info(app_id)
```

Logros de un juego de Steam con su rareza, para achievements.save_info: (esquema, porcentajes) como los dan achievement_schema y global_percentages (los porcentajes, None si no se han podido leer), o None si Steam no da el esquema.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 387

```
def _needs_info(title, result, known, now)
```

Si hay que (volver a) leer los logros del juego de title: si nunca se han leído, si se leyeron hace más de ACHIEVEMENTS_INFO_DAYS días o si el jugador trae alguno que no está. result es lo que da player_achievements y known, las (título, clave) que ya hay.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 401

```
def import_achievements(user, limit=ACHIEVEMENTS_PER_IMPORT)
```

Lee de Steam los logros de los juegos del usuario que ha jugado desde la última vez que se leyeron (o que nunca se han leído), de la partida más reciente a la más antigua y como mucho limit (None: todos), y los apunta en sus títulos importados (ImportedTitle): cuántos tiene y, uno a uno, cuáles y cuándo los consiguió (achievements.save_unlocked). Con varias cuentas de Steam se queda con la que más lleva de cada juego.

De los juegos cuyos logros aún no se conocen (o con más de ACHIEVEMENTS_INFO_DAYS días, o si el jugador trae uno nuevo) lee también cuáles tienen, con sus textos en castellano y su rareza (achievement_info). Lo que no se puede leer se intenta la próxima vez: también los logros de un jugador si no se sabe aún cuáles son los de su juego. Van ACHIEVEMENTS_WORKERS peticiones a la vez.

Devuelve de cuántos juegos se han leído.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/steam.py, línea 414

```
def _first(values)
```

El menor de los valores que no son None (el mejor puesto o la fecha más antigua), o None si no hay ninguno.

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/steam.py, línea 481

```
def _wishlist_games(app_ids)
```

Juegos del catálogo de unos IDs de Steam de la lista de deseos: {ID: Game o None}.

Si alguien corrigió a mano el juego de esa app (ExternalTitle, también para decir que no es un juego), manda la corrección; los demás se buscan como los de la biblioteca (_match_games).

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/steam.py, línea 490

```
def import_wishlist(user)
```

Trae a «Lo quiero» los juegos de la lista de deseos de todas las cuentas de Steam del usuario.

Los juegos se buscan por su ID de Steam como los de la biblioteca. No se añaden los que ya están en su biblioteca ni los que quitó a mano de «Lo quiero» (Profile.steam_wishlist_skip); de los que ya estaban en «Lo quiero» solo se apuntan su puesto y su fecha en Steam. Si se han podido leer todas sus listas, se quitan los que trajo una importación anterior y ya no están en Steam (los ha comprado o quitado allí); los que añadió él en NextPlay nunca se quitan. Un juego en varias cuentas (o en varias apps, como un relanzamiento) se queda con el mejor puesto y la fecha más antigua.

Devuelve un dict con los deseados de Steam («total»), los WishlistItem nuevos («added», una lista), cuántos se han quitado («removed»), cuántos no se traen porque ya los tiene («owned») o porque los quitó él («skipped»), cuántos no están en el catálogo («unmatched») y los nombres de las cuentas cuya lista no se pudo leer, porque es privada o está vacía («unreadable»).

Pantallas: Editar perfil (Cuentas vinculadas › Steam), Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/steam.py, línea 508

accounts/views/avisos.py

Vista de Avisos (avisos/): la página de la campana de la barra de arriba.
Pantallas de este archivo: Avisos

```
@login_required
def notifications(request)
```

Últimos avisos del usuario (bajadas de precio, nuevos seguidores...). Al abrir la página, los no leídos pasan a leídos.

Los que tienen quién los ha provocado (actor) traen ya lo que hace falta para enseñar su foto.

Pantallas: Avisos, todas (la campana de la barra de arriba se queda sin número al abrirla).

accounts/views/avisos.py, línea 11

accounts/views/comun.py

Lo que comparten varias vistas de las cuentas: volver a la página desde la que se envió un formulario (Editar perfil, el Perfil o Juegos importados).
Pantallas de este archivo: Perfil, Editar perfil, Juegos importados

```
def _back(request, default)
```

Redirige a la página de la que vino el formulario (campo «next») si es de esta web; si no, a default.

Así las acciones lanzadas desde el modal del perfil vuelven al perfil y no a Editar perfil.

Pantallas: Editar perfil, Perfil, Juegos importados (vuelve a la página desde la que se envió el formulario).

accounts/views/comun.py, línea 9

accounts/views/cuentas_vinculadas.py

Vistas de las cuentas vinculadas: vincular, importar y desvincular Steam, PlayStation y Xbox y añadir o quitar las cuentas de otras plataformas. Se pulsan en Editar perfil › Cuentas vinculadas y en el modal «Vincular cuentas» del Perfil, y lo que importan sale en el Perfil, la Ficha del juego, Juegos importados, Inicio y los retos.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Lista de deseos, Página de la lista, Retos, Página del reto, Editar perfil, Juegos importados

```
def linking_context()
```

Variables que necesitan los paneles de vinculación de cuentas (en Editar perfil y en el modal del perfil).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 41

```
def _steam_return_to(request, state)
```

URL absoluta a la que Steam devuelve al usuario tras iniciar sesión, con el código anti-CSRF state.

Pantallas: Editar perfil (Cuentas vinculadas: Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 59

```
def _report_import(request, result)
```

Avisa al usuario de cuántos juegos se han importado de Steam, cuántos no estaban en el catálogo y qué cuentas se han saltado por ser privadas.

Pantallas: Editar perfil (Cuentas vinculadas: mensaje con el resultado de la importación de Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 67

```
def _achievements_note(count, noun='logros')
```

Coletilla del mensaje de una importación con los juegos de los que se han leído los logros (o los trofeos).

Pantallas: Editar perfil (Cuentas vinculadas: mensaje con el resultado de la importación), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 86

```
def _with_unmatched(text, unmatched)
```

Mensaje de resultado de una importación, con el aviso de los juegos no encontrados y un enlace para emparejarlos a mano (HTML seguro: el texto se escapa).

Pantallas: Editar perfil (Cuentas vinculadas: mensaje de la importación, con el enlace a Juegos importados), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 95

```
def _report_wishlist(request, result)
```

Avisa de lo que se ha traído de la lista de deseos de Steam y, con los precios de hoy, de cuántos de sus deseados están en su mínimo histórico o son más baratos fuera de Steam (HTML seguro: el texto se escapa).

Los precios de los juegos nuevos se piden en ese momento (games.prices.fetch_missing).

Pantallas: Editar perfil (Cuentas vinculadas: mensaje con lo que se ha traído de la lista de deseos de Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 110

```
def _fetch_library_prices(user)
```

Pide ya los precios de los juegos de la biblioteca del usuario que nunca se han buscado (games.prices.fetch_missing), para que lo recién importado tenga precio sin esperar a la tarea de cada noche.

Si una tienda no responde, la importación vale igual: lo que falte llega con update_prices. La sincronización de cada noche (sync_libraries) no lo hace, porque justo después va update_prices con todo el catálogo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño), Ficha del juego (Dónde comprarlo de lo recién importado), Lista de deseos (precios), Página de la lista (lo que cuesta lo que te falta).

accounts/views/cuentas_vinculadas.py, línea 147

```
def _import_steam(request, include_unplayed=False)
```

Lanza la importación de todas las cuentas de Steam del usuario y le avisa del resultado o del fallo.

Si lo tiene activado (Profile.steam_wishlist), trae también su lista de deseos de Steam a «Lo quiero»; que falle eso no estropea lo de la biblioteca. Si falla por Steam o IGDB deja el error completo en el log del servidor para poder revisarlo. Si sale bien, pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros; botón «Lo quiero»), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso), Lista de deseos.

accounts/views/cuentas_vinculadas.py, línea 163

```
@require_POST
@login_required
def steam_link(request)
```

Manda al usuario a iniciar sesión en Steam para vincular una cuenta más (Steam nunca nos da su contraseña).

No deja pasar de SteamAccount.MAX_PER_USER cuentas.

Pantallas: Editar perfil (Cuentas vinculadas: botón para vincular Steam), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 199

```
@login_required
def steam_callback(request)
```

Vuelta desde Steam: verifica el inicio de sesión, añade la cuenta a las del usuario e importa sus juegos.

Rechaza la vuelta si el código anti-CSRF no coincide, si Steam no la confirma o si esa cuenta de Steam ya está vinculada a otro usuario. Si ya era suya, solo avisa.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/views/cuentas_vinculadas.py, línea 215

```
@require_POST
@login_required
def steam_import(request)
```

Vuelve a importar los juegos de todas las cuentas de Steam vinculadas (con «unplayed» también los no jugados).

Pantallas: Editar perfil (Cuentas vinculadas: botón «Importar» de Steam), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/views/cuentas_vinculadas.py, línea 261

```
@require_POST
@login_required
def steam_auto_sync(request)
```

Activa (auto_sync=1) o desactiva la sincronización nocturna de la biblioteca de Steam del usuario.

Pantallas: Editar perfil (Cuentas vinculadas: casilla de la sincronización de cada noche), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 281

```
@require_POST
@login_required
def steam_wishlist(request)
```

Activa (steam_wishlist=1) o desactiva que la importación de Steam traiga la lista de deseos a «Lo quiero».

Con restore=1, olvida los juegos que quitó a mano para que la próxima importación los vuelva a traer.

Pantallas: Editar perfil (Cuentas vinculadas: casilla para traer la lista de deseos de Steam), Perfil (modal «Vincular cuentas» de su dueño), Lista de deseos, Ficha del juego (botón «Lo quiero»).

accounts/views/cuentas_vinculadas.py, línea 299

```
@require_POST
@login_required
def steam_unlink(request, account_id)
```

Desvincula una cuenta de Steam del usuario; los juegos importados se quedan en su biblioteca.

Solo puede desvincular cuentas suyas: con el ID de una cuenta ajena responde 404.

Pantallas: Editar perfil (Cuentas vinculadas: botón para desvincular una cuenta de Steam), Perfil (modal «Vincular cuentas» de su dueño; cuentas de la cabecera).

accounts/views/cuentas_vinculadas.py, línea 324

```
def _psn_error(request, text)
```

Avisa del error de PlayStation y vuelve a donde estaba el formulario (por defecto, Editar perfil).

Pantallas: Editar perfil (Cuentas vinculadas: mensaje de error de PlayStation), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 340

```
@require_POST
@login_required
def psn_sync(request)
```

Vincula la cuenta de PlayStation del código NPSSO pegado (o la actualiza) e importa sus juegos de PS4/PS5.

El código solo se usa durante esta petición: no se guarda. Avisa si el código no tiene el formato esperado, si Sony no lo acepta o si esa cuenta de PlayStation ya es de otro usuario. Después pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

Pantallas: Editar perfil (Cuentas vinculadas: formulario del código NPSSO de PlayStation), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/views/cuentas_vinculadas.py, línea 352

```
@require_POST
@login_required
def psn_unlink(request)
```

Desvincula la cuenta de PlayStation del usuario; los juegos importados se quedan en su biblioteca.

Pantallas: Editar perfil (Cuentas vinculadas: botón para desvincular PlayStation), Perfil (modal «Vincular cuentas» de su dueño; cuentas de la cabecera).

accounts/views/cuentas_vinculadas.py, línea 393

```
@require_POST
@login_required
def platform_add(request)
```

Añade la cuenta de otra plataforma (Xbox, Switch, Epic, GOG, Battle.net) o cambia la que ya tenía.

Si el formato no es el de esa plataforma avisa del error y no guarda nada.

Pantallas: Editar perfil (Cuentas vinculadas: Otras plataformas), Perfil (modal «Vincular cuentas» de su dueño; cuentas de la cabecera).

accounts/views/cuentas_vinculadas.py, línea 406

```
@require_POST
@login_required
def platform_remove(request, platform)
```

Quita del perfil del usuario su cuenta de esa plataforma (no da error si no la tenía).

Pantallas: Editar perfil (Cuentas vinculadas: Otras plataformas), Perfil (modal «Vincular cuentas» de su dueño; cuentas de la cabecera).

accounts/views/cuentas_vinculadas.py, línea 430

```
def _xbox_redirect_uri(request)
```

URL absoluta de vuelta desde Microsoft; tiene que coincidir con la registrada en la aplicación de Azure.

Pantallas: Editar perfil (Cuentas vinculadas: Xbox), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 444

```
@require_POST
@login_required
def xbox_link(request)
```

Manda al usuario a iniciar sesión con su cuenta Microsoft para vincular Xbox o volver a sincronizarla.

Sirve para las dos cosas porque no se guarda ningún token: cada sincronización pide iniciar sesión.

Pantallas: Editar perfil (Cuentas vinculadas: botón para vincular Xbox), Perfil (modal «Vincular cuentas» de su dueño).

accounts/views/cuentas_vinculadas.py, línea 454

```
@login_required
def xbox_callback(request)
```

Vuelta desde Microsoft: comprueba el código anti-CSRF, vincula la cuenta de Xbox e importa sus juegos.

Si el usuario canceló el inicio de sesión, si el código no coincide o si Xbox Live falla, avisa y vuelve a Editar perfil. Después pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/views/cuentas_vinculadas.py, línea 470

```
@require_POST
@login_required
def xbox_unlink(request)
```

Desvincula la cuenta de Xbox del usuario; los juegos importados se quedan en su biblioteca.

Pantallas: Editar perfil (Cuentas vinculadas: botón para desvincular Xbox), Perfil (modal «Vincular cuentas» de su dueño; cuentas de la cabecera).

accounts/views/cuentas_vinculadas.py, línea 510

accounts/views/editar_perfil.py

Vistas de Editar perfil (cuenta/perfil/): el formulario de datos, apariencia, escaparate, redes, privacidad y avisos, su vista previa en directo, las imágenes de un juego para la cabecera y las tiendas en las que compras. Las cuentas vinculadas van aparte, en cuentas_vinculadas.py.
Pantallas de este archivo: Ficha del juego, Lista de deseos, Editar perfil

```
@require_POST
@login_required
def profile_preview(request)
```

Vista previa de Editar perfil: la página del perfil tal como quedaría con lo que hay en el formulario.

La pide el JavaScript de Editar perfil cada vez que se cambia algo y la enseña a la vez en un marco del ancho de un ordenador y en otro del de un móvil. No guarda nada (ver ProfileForm.preview). Sale como la ven los demás, sin los botones del dueño, aunque el perfil sea privado.

Pantallas: Editar perfil (vista previa del perfil).

accounts/views/editar_perfil.py, línea 31

```
@login_required
def edit_profile(request)
```

Editar perfil: datos, apariencia (foto, cabecera, fondo, color y diseño), escaparate, redes, privacidad y avisos.

El formulario trabaja con una copia del perfil para que, si no es válido, la página (las imágenes subidas, la barra superior) siga enseñando lo guardado. Al lado va la vista previa del perfil en un ordenador y en un móvil, que pinta profile_preview con lo que hay en el formulario. Las tiendas y las cuentas vinculadas tienen sus propios formularios en la misma página.

Pantallas: Editar perfil.

accounts/views/editar_perfil.py, línea 57

```
def _posted_sections(form)
```

Secciones tal como venían en un envío no válido (orden y visibles), para no perder lo que movió; [] si no hay.

Pantallas: Editar perfil (Escaparate › Secciones del perfil, cuando no se ha podido guardar).

accounts/views/editar_perfil.py, línea 98

```
def game_arts(game)
```

Imágenes apaisadas de IGDB (arte y capturas) de un juego para la cabecera: [{"id", "thumb", "full"}].

«thumb» es la miniatura para elegirla y «full», el tamaño con el que se ve en la cabecera.

Se guardan un día en caché. Lista vacía si no hay juego, el juego no viene de IGDB, IGDB no está configurado o no responde.

Pantallas: Editar perfil (Apariencia › Cabecera: imágenes del juego elegido).

accounts/views/editar_perfil.py, línea 115

```
@login_required
def banner_arts(request)
```

Opciones de imagen para la cabecera con el juego elegido en Editar perfil (fragmento HTML).

El formulario lo pide con HTMX al cargar y al cambiar de juego (?banner_game=). Solo vale con juegos de la biblioteca del usuario o con el que ya tiene en la cabecera.

Pantallas: Editar perfil (Apariencia › Cabecera: imágenes del juego elegido).

accounts/views/editar_perfil.py, línea 143

```
@require_POST
@login_required
def shops_update(request)
```

Guarda las tiendas en las que compra el usuario (casillas «shops»). Ninguna o todas = enseñar todas, así las tiendas que aparezcan más adelante también se ven.

Pantallas: Editar perfil (Tiendas en las que compras), Ficha del juego (Dónde comprarlo: primero tus tiendas), Lista de deseos (mejor precio en tus tiendas).

accounts/views/editar_perfil.py, línea 174

accounts/views/ficha_del_juego.py

Botones de las cuentas en la Ficha del juego: «Añadir a mis favoritos», «Tus logros» (que se cargan con HTMX) y poner de cabecera del perfil una imagen del carrusel.
Pantallas de este archivo: Ficha del juego, Perfil, Editar perfil

```
def favorite_context(user, game, error='')
```

Variables del botón «Añadir a mis favoritos» de la ficha de game para user.

Pantallas: Ficha del juego (botón «Añadir a mis favoritos»).

accounts/views/ficha_del_juego.py, línea 20

```
@require_POST
@login_required
def favorite_toggle(request, game_id)
```

Añade el juego a los favoritos del perfil del usuario o lo quita si ya estaba.

Como mucho FavoriteGame.MAX_PER_USER: con el cupo lleno avisa de que quite alguno. Los nuevos van al final. Con HTMX devuelve el botón actualizado; si no, vuelve a la ficha del juego.

Pantallas: Ficha del juego (botón «Añadir a mis favoritos»), Perfil (favoritos del escaparate), Editar perfil (Juegos favoritos).

accounts/views/ficha_del_juego.py, línea 36

```
@login_required
def game_achievements(request, game_id)
```

«Tus logros» de un juego, uno a uno (fragmento HTML que la ficha pide con HTMX al desplegarlos y al cambiar de orden con ?orden=): los que tiene quien mira, con cuándo los consiguió, y los que le faltan, con su rareza (accounts.achievements.detail). Da 404 si no tiene logros de ese juego leídos uno a uno.

Pantallas: Ficha del juego («Tus logros», al desplegarlos y al cambiar el orden).

accounts/views/ficha_del_juego.py, línea 63

```
@require_POST
@login_required
def banner_from_game(request, game_id)
```

Pone de cabecera del perfil una imagen de la ficha de un juego (el botón del visor del carrusel).

Como en Editar perfil, solo con juegos de la biblioteca del usuario, y solo con imágenes apaisadas de la ficha (games.media.banner_arts). El encuadre vuelve al centro, como con cualquier imagen nueva. Con HTMX devuelve el aviso del visor; si no, vuelve a la ficha con un mensaje.

Pantallas: Ficha del juego (botón del visor del carrusel para ponerla de cabecera), Perfil (cabecera).

accounts/views/ficha_del_juego.py, línea 79

accounts/views/juegos_importados.py

Vistas de Juegos importados (cuenta/juegos-importados/): los títulos importados de Steam, PlayStation o Xbox que no se encontraron en el catálogo, para asignarles su juego a mano u ocultarlos, y los que han llevado a un juego, para corregirlo.
Pantallas de este archivo: Ficha del juego, Perfil, Juegos importados

```
def _search_games(query)
```

Juegos del catálogo cuyo nombre contiene query, de más a menos popular, completando con IGDB si salen pocos.

Pantallas: Juegos importados (buscador para elegir el juego de un título).

accounts/views/juegos_importados.py, línea 28

```
@login_required
def imported_titles(request)
```

Juegos importados para revisar a mano.

Sin parámetros, los títulos de Steam, PlayStation o Xbox del usuario que no se encontraron en el catálogo (con ?ocultos=1, los que ocultó). Con ?juego=<id>, los que han llevado a ese juego de su biblioteca, para corregirlo si no es el que jugó. Con ?titulo=<id>&q= busca candidatos para ese título (por defecto, por su nombre).

Pantallas: Juegos importados.

accounts/views/juegos_importados.py, línea 45

```
@require_POST
@login_required
def imported_title_assign(request, pk)
```

Asigna a un título importado del usuario el juego elegido (o ninguno si «game» llega vacío).

La corrección sirve para todos los usuarios con ese título y sus horas pasan al juego nuevo en su biblioteca. Responde 404 si el título no es suyo o el juego no existe.

Pantallas: Juegos importados, Perfil (biblioteca y horas), Ficha del juego (tus horas).

accounts/views/juegos_importados.py, línea 89

```
@require_POST
@login_required
def imported_title_hide(request, pk)
```

Oculta (hidden=1) un título sin emparejar del usuario —herramientas, demos, apps— o lo vuelve a mostrar.

Pantallas: Juegos importados (ocultar un título o volver a mostrarlo).

accounts/views/juegos_importados.py, línea 118

accounts/views/perfil.py

Vista del Perfil (u/<usuario>/): cabecera, escaparate y pestañas Biblioteca, Reseñas, Listas, Tier lists y Actividad. Lo mismo pinta la vista previa de Editar perfil (editar_perfil.profile_preview).
Pantallas de este archivo: Perfil, Editar perfil

```
def profile(request, username)
```

Perfil público de un usuario: cabecera personalizada, escaparate, su biblioteca y sus reseñas.

El escaparate (favoritos, lo que está jugando, reseña destacada, estadísticas y géneros) sale en el orden y con las secciones que haya elegido. La biblioteca tiene pestañas por estado (Jugando, Completado...) con su contador y se filtra con ?estado= y por idioma con ?idioma= (voces o textos en castellano); se ordena con ?orden= (recientes, nota, horas o nombre). Con ?vista=resenas muestra sus reseñas en vez de la biblioteca, con ?vista=listas sus listas, con ?vista=tier-lists sus tier lists y con ?vista=actividad lo que ha hecho (desde ?antes=, el cursor de «Cargar más»). Un perfil privado da 404 a cualquiera que no sea su dueño.

Pantallas: Perfil.

accounts/views/perfil.py, línea 28

```
def _profile_owners()
```

Usuarios con lo que enseña la cabecera del perfil ya cargado: perfil, juego de la cabecera y cuentas vinculadas.

Pantallas: Perfil (cabecera: perfil, juego de la cabecera y cuentas), Editar perfil (vista previa).

accounts/views/perfil.py, línea 48

```
def _profile_context(request, owner, favorites=None, preview=False)
```

Variables de la plantilla del perfil de owner, con la biblioteca filtrada y ordenada según ?estado=, ?idioma= y ?orden=.

favorites son sus juegos favoritos en orden (si no se dan, los guardados). Con preview es la vista previa de Editar perfil: se ve como la ven los demás (también sus listas: solo las públicas), el usuario de la barra superior es owner con los cambios sin guardar y la biblioteca se corta en PREVIEW_LIBRARY juegos. También lleva cuántos seguidores y seguidos tiene owner y si quien mira sigue a owner o al revés (social.views.follow_context), y sus listas y, aparte, sus tier lists (lists.views.profile_lists), para sus pestañas y el escaparate; las tier lists que se enseñan llevan su tablero en miniatura (lists.views.mark_boards). Con sus logros importados (accounts.achievements), cada juego de la biblioteca lleva su insignia de «Platino» o «100 %» (achievement_badge) y las cifras de la cabecera, sus platinos y sus juegos al 100 %; si el escaparate enseña la sección «Logros», lleva también sus últimos logros y el más raro (achievements.showcase). Si enseña la sección «Retos», sus insignias y los retos que lleva en marcha, solo los que quien mira puede ver (challenges.progress.showcase). A su dueño le dice cuántos juegos le quedan por repasar (library.backlog.pending, para el aviso que lleva a Repasar la biblioteca) y, en la pestaña Pendiente, lo que cuesta hoy (backlog.value).

Pantallas: Perfil (cabecera, escaparate y pestañas), Editar perfil (vista previa).

accounts/views/perfil.py, línea 58

```
def _profile_stats(owner, entries, counts, reviews)
```

Cifras de la cabecera del perfil: juegos, completados, horas jugadas (Steam + PlayStation + Xbox) y las de cada plataforma (hours_by_platform, de la que tiene más a la que menos y solo las que tienen alguna), nota media (en estrellas: las notas van en medias estrellas) y reseñas publicadas (reviews, ya contadas).

También calcula los tres géneros más repetidos de su biblioteca.

Pantallas: Perfil (cifras de la cabecera: juegos, completados y horas).

accounts/views/perfil.py, línea 191

accounts/views/registro.py

Vista de Registro (cuenta/registro/): el formulario para crear una cuenta.
Pantallas de este archivo: Registro

```
def signup(request)
```

Registro de cuenta nueva.

Si el usuario ya ha iniciado sesión lo manda a su perfil. Al registrarse bien inicia su sesión y lo lleva al catálogo para que añada sus primeros juegos.

Pantallas: Registro.

accounts/views/registro.py, línea 10

accounts/xbox.py

Vinculación con Xbox (inicio de sesión con cuenta Microsoft) e importación de sus juegos y minutos jugados.

Flujo: el usuario inicia sesión en Microsoft (OAuth 2.0 de una aplicación de Azure con el permiso XboxLive.signin) → con el token de Microsoft se pide uno de usuario de Xbox Live → con ese, uno XSTS que trae el XUID y el gamertag → con el XSTS se leen el historial de juegos y la estadística MinutesPlayed. Ningún token se guarda: se usan durante la petición y se tiran.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Retos, Página del reto, Editar perfil, Juegos importados

Clase XboxError

Fallo al hablar con Microsoft o Xbox Live (red, credenciales o respuesta inesperada).

```
def is_configured()
```

Indica si están puestas las credenciales de la aplicación de Azure para iniciar sesión con Microsoft.

Pantallas: Editar perfil (Cuentas vinculadas: sin credenciales de Azure no se puede vincular Xbox), Perfil (modal «Vincular cuentas» de su dueño).

accounts/xbox.py, línea 51

```
def login_url(redirect_uri, state)
```

URL de Microsoft donde el usuario inicia sesión; Microsoft lo devuelve a redirect_uri con un código.

Pantallas: Editar perfil (Cuentas vinculadas: botón para vincular Xbox), Perfil (modal «Vincular cuentas» de su dueño).

accounts/xbox.py, línea 60

```
def _post(url, **kwargs)
```

POST a Microsoft o Xbox Live que devuelve el JSON; lanza XboxError si falla.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 77

```
def xbox_tokens(code, redirect_uri)
```

Cambia el código de Microsoft por los datos de Xbox Live: {"xuid", "gamertag", "auth"}.

«auth» es la cabecera Authorization (XBL3.0) para llamar a las APIs de Xbox Live.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 102

```
def _get(url, auth, contract, **params)
```

GET autenticado contra una API de Xbox Live con su versión de contrato; devuelve el JSON.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 144

```
def avatar(xuid, auth)
```

URL del avatar (gamerpic) de la cuenta, o vacío si Xbox Live no la da.

Pantallas: Editar perfil (Cuentas vinculadas › Xbox), Perfil (cuenta de Xbox de la cabecera), todas (la foto del perfil cuando es la de Xbox).

accounts/xbox.py, línea 164

```
def played_titles(xuid, auth)
```

Juegos del historial de la cuenta (solo de tipo «Game», sin apps como Netflix), con su última partida.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 178

```
def minutes_played(xuid, auth, title_ids)
```

Minutos jugados por título según la estadística MinutesPlayed: {titleId: minutos}.

Los juegos que no publican esa estadística (muchos de Xbox 360 y algunos de PC) no aparecen.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 189

```
def save_achievements(user, played)
```

Apunta en los títulos de Xbox importados del usuario los logros que llegan con su historial (played, de played_titles: «currentAchievements» de «totalAchievements») y devuelve en cuántos.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus logros).

accounts/xbox.py, línea 218

```
def import_library(user, code, redirect_uri)
```

Vincula (o actualiza) la cuenta de Xbox del inicio de sesión e importa sus juegos con los minutos jugados.

Los juegos se emparejan con el catálogo por nombre (y por los dispositivos en los que se jugaron). Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas) y se les actualizan los minutos de Xbox. Si el usuario tenía puesto a mano un gamertag en «Otras plataformas», se quita porque ya está la cuenta verificada. Lanza ValueError si esa cuenta de Xbox ya es de otro usuario.

Devuelve un dict con la cuenta («account»), cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), y de cuántos se han leído los logros («achievements»: llegan con el historial, ver save_achievements).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas; insignias de las portadas, cifras y sección «Logros»), Ficha del juego (tus horas; tus logros), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

accounts/xbox.py, línea 241

App «games»

Catálogo de juegos, importación desde IGDB y precios de PC (IsThereAnyDeal), Xbox y Nintendo.

games/admin.py

Panel de administración de los juegos: sus datos, los IDs manuales de Steam, Xbox y Nintendo para cuando IGDB o el emparejado se equivocan, sus precios (solo para mirarlos), los géneros y las plataformas.
Pantallas de este archivo: Panel de administración

Clase GamePriceInline

```
def has_add_permission(self, request, obj=None)
```

Impide añadir precios a mano: solo los escribe update_prices.

Pantallas: Panel de administración (precios de cada juego: no se añaden a mano).

games/admin.py, línea 17

games/history.py

Historial de precios de PC de un juego (de IsThereAnyDeal), veredicto «¿compro ahora o espero?» y gráfica.

El historial se guarda en Game.price_history por tienda, para poder enseñarlo de todas o solo de algunas (las del usuario, o una concreta):

{"since": fecha ISO, "shops": {"61": {"name": "Steam", "points": [[fecha ISO, precio o None, % de rebaja], ...]}}}

Cada tienda tiene un punto por cada cambio de precio, de más viejo a más nuevo; precio None es que dejó de venderlo. Se pide a ITAD al abrir la ficha y se reutiliza durante MAX_AGE.
Pantallas de este archivo: Ficha del juego

```
def needs_update(game, now=None)
```

Indica si hay que volver a pedir el historial: nunca se pidió, tiene más de MAX_AGE o es del formato antiguo (solo el más barato, sin separar por tienda).

Pantallas: Ficha del juego (historial de precios de Dónde comprarlo).

games/history.py, línea 37

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

Pantallas: Ficha del juego (historial de precios).

games/history.py, línea 51

```
def _decimal(value)
```

Precio guardado en el JSON (float) como Decimal con dos decimales.

Pantallas: Ficha del juego (historial de precios).

games/history.py, línea 59

```
def shop_series(events, seed, start)
```

Historial de cada tienda a partir de los cambios de ITAD: {"since", "shops"} (ver el docstring del módulo).

seed es {id de tienda: (precio, rebaja, nombre)} con lo que cuesta hoy en cada tienda; se usa como precio de partida de las tiendas sin cambios en el periodo. Las que sí tienen cambios empiezan con el precio normal de su primer cambio (antes de una rebaja se vendía a precio normal); si su primer cambio es que dejó de venderlo, no se sabe a cuánto estaba y empiezan ahí.

Pantallas: Ficha del juego (historial de precios).

games/history.py, línea 67

```
def update_history(game, client=None, now=None)
```

Pide a ITAD el historial de los dos últimos años del juego y lo guarda. Lanza ITADError si falla.

Pantallas: Ficha del juego (historial de precios).

games/history.py, línea 94

```
def lowest_series(history, shops=None)
```

Precio más barato entre esas tiendas (ids; None = todas) a lo largo del tiempo.

Devuelve [(fecha, precio, rebaja, tienda)] de más viejo a más nuevo, con un punto solo cuando cambia el más barato; si varias tiendas cambian a la vez, queda el resultado final.

Pantallas: Ficha del juego (historial de precios: el más barato a lo largo del tiempo).

games/history.py, línea 115

```
def _year(points, now)
```

Puntos (fecha, precio, rebaja, tienda) del último año, empezando con el que estaba vigente hace un año (con esa fecha) si se sabe.

Pantallas: Ficha del juego (gráfica y veredicto del historial de precios).

games/history.py, línea 147

```
def verdict(points, current, low, now, all_shops=True)
```

Veredicto para el precio de hoy: {"code", "title", "text", "year_low"}.

«minimo» si está en su mínimo (low: el histórico con todas las tiendas, o el de los dos años de historial con unas tiendas concretas); «bueno» si en el último año no ha estado más barato o está cerca de lo más barato del año; «espera» si ha bajado de ese precio varias veces.

Pantallas: Ficha del juego (veredicto del precio de hoy en el historial).

games/history.py, línea 159

```
def last_big_sale(points)
```

Última vez que estuvo rebajado al menos BIG_SALE_CUT %: {"date", "cut", "price", "shop"}, o None.

Pantallas: Ficha del juego (última gran rebaja en el historial).

games/history.py, línea 198

```
def chart(points, now, current=None, low_line=None, width=320, height=90, pad=6)
```

Datos de la gráfica escalonada del último año para dibujarla en SVG, o None si no hay historial.

Termina en current (el GamePrice más barato de hoy, el que enseña la ficha) si se da: el historial de ITAD puede contar alguna tienda que no sale entre los precios de hoy. low_line es el precio de la línea del mínimo.

Devuelve el tamaño, los puntos de la línea («line») y del área bajo ella («area»), el precio más alto y más bajo de la gráfica, la altura de la línea del mínimo («low_y», None si queda fuera) y, en JSON («hover»), cada tramo con su fecha, posición, precio, rebaja y tienda para la etiqueta que sale al pasar por encima. Las coordenadas van ya en texto con punto decimal: en una plantilla en español, floatformat pondría coma y el SVG no sería válido.

Pantallas: Ficha del juego (gráfica del historial de precios).

games/history.py, línea 210

```
def shop_options(game, user_shops)
```

Opciones del selector de tienda de la gráfica: [(valor de ?tienda=, etiqueta)].

«Tus tiendas» (si el usuario las tiene elegidas), «Todas las tiendas» y cada tienda con historial, las más conocidas primero.

Pantallas: Ficha del juego (selector de tienda de la gráfica).

games/history.py, línea 269

```
def selected_shops(value, user_shops)
```

Tiendas que enseñar según ?tienda= y las del usuario: (ids o None para todas, valor elegido).

Sin valor: las del usuario si las tiene, si no todas. «todas»: todas. Un id: solo esa tienda.

Pantallas: Ficha del juego (tiendas que enseña la gráfica).

games/history.py, línea 290

```
def summary(game, now=None, shops=None)
```

Todo lo que enseña la ficha sobre el historial de esas tiendas (ids; None = todas).

{"current": precio más barato de hoy en esas tiendas de PC, "verdict", "chart", "last_big_sale", "all_shops"}, o None si no hay historial, el juego es gratis o ninguna de esas tiendas lo vende.

Pantallas: Ficha del juego (todo el historial de precios).

games/history.py, línea 305

games/igdb.py

Cliente de IGDB y la función que vuelca sus juegos en el catálogo local.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Lista de deseos, Página de la lista, Retos, Página del reto, Nuevo reto, Editar perfil, Juegos importados

Clase IGDBError

Fallo al hablar con IGDB o con Twitch (credenciales, red o respuesta no válida).

```
def is_configured()
```

Indica si están puestas las credenciales de Twitch para usar IGDB.

Pantallas: Catálogo (botón para traer de IGDB lo que buscas), Juegos importados (buscar en IGDB), Editar perfil (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo), Ficha del juego (carrusel), Perfil (modal «Vincular cuentas» de su dueño).

games/igdb.py, línea 60

Clase IGDBClient

```
def __init__(self, client_id=None, client_secret=None)
```

Prepara el cliente con las credenciales dadas o las de settings; falla si no hay ninguna.

Pantallas: Catálogo (traer de IGDB), Juegos importados (buscar en IGDB), Editar perfil (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo), Ficha del juego (carrusel), Perfil (modal «Vincular cuentas» de su dueño).

games/igdb.py, línea 71

```
def _token(self)
```

Token de acceso de Twitch: lo saca de la caché o pide uno nuevo y lo guarda hasta que caduque.

Pantallas: Catálogo (traer de IGDB), Juegos importados (buscar en IGDB), Editar perfil (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo), Ficha del juego (carrusel), Perfil (modal «Vincular cuentas» de su dueño).

games/igdb.py, línea 83

```
def query(self, endpoint, body)
```

Lanza una consulta en Apicalypse (el lenguaje de IGDB) contra endpoint y devuelve el JSON.

Si IGDB responde 401 borra el token de la caché para que la próxima llamada pida uno nuevo.

Pantallas: Catálogo (traer de IGDB), Juegos importados (buscar en IGDB), Editar perfil (imágenes de un juego para la cabecera; Cuentas vinculadas: juegos importados que aún no están en el catálogo), Ficha del juego (carrusel), Perfil (modal «Vincular cuentas» de su dueño).

games/igdb.py, línea 111

```
def top_games(self, limit=MAX_PAGE_SIZE, offset=0)
```

Una página de los juegos más valorados por número de votos, sin ediciones sueltas, con sus ediciones.

Pantallas: Catálogo (los juegos populares que trae el comando import_igdb), Ficha del juego, Inicio, Perfil, Lista de deseos, Página de la lista, Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas).

games/igdb.py, línea 138

```
def search(self, term, limit=20)
```

Busca juegos por texto en IGDB (hasta limit resultados, sin ediciones sueltas), con sus ediciones.

Pantallas: Catálogo (botón para traer de IGDB lo que buscas), Juegos importados (buscar el juego de un título), Editar perfil (Cuentas vinculadas: títulos de PlayStation y Xbox que no están en el catálogo), Perfil (modal «Vincular cuentas» de su dueño).

games/igdb.py, línea 152

```
def games_by_steam_ids(self, app_ids)
```

Juegos de IGDB (con sus ediciones) de una lista de IDs de Steam y un dict {ID de Steam: id de IGDB}.

Los IDs que IGDB no conoce (herramientas, bandas sonoras, servidores...) no aparecen en el dict.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas: juegos de Steam que aún no están en el catálogo), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

games/igdb.py, línea 166

```
def games_by_ids(self, igdb_ids)
```

Juegos de IGDB (con sus ediciones) a partir de sus ids, en lotes de MAX_PAGE_SIZE.

Pantallas: Catálogo, Ficha del juego, Inicio, Perfil (modal «Vincular cuentas» de su dueño), Lista de deseos, Página de la lista, Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas), Editar perfil (Cuentas vinculadas: juegos de Steam que aún no están en el catálogo).

games/igdb.py, línea 189

```
def games_by_names(self, names)
```

Juegos de IGDB (con sus ediciones) cuyo nombre es exactamente uno de names, sin ediciones sueltas.

Va en lotes para no hacer una petición por juego. Si dos juegos se llaman igual vienen los dos; quien llama decide con cuál quedarse (p. ej. el más popular).

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas: juegos de PlayStation y Xbox que aún no están en el catálogo), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

games/igdb.py, línea 203

```
def spanish_data(self, igdb_ids)
```

Solo los campos de castellano (idiomas, nombres y versiones regionales) de esos juegos de IGDB, en lotes.

Sirve para completar los juegos que se importaron antes de que se pidieran esos campos.

Pantallas: Ficha del juego (nombre en España e idiomas), Catálogo (filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/igdb.py, línea 227

```
def artworks(self, igdb_id, limit=8)
```

IDs de imagen del arte promocional de un juego y, si no llegan a limit, de sus capturas.

Son imágenes apaisadas que sirven de cabecera o de fondo del perfil (la portada es vertical). Del arte se quitan los logos, iconos, portadas e infografías (NOT_ART_TYPES) y va primero el principal.

Pantallas: Editar perfil (Apariencia › Cabecera: imágenes del juego elegido).

games/igdb.py, línea 244

```
def game_media(self, igdb_id)
```

Artes, capturas y vídeos de un juego en una sola consulta: {"artworks", "screenshots", "videos"}.

Son las listas tal como las da IGDB (los artes con su tipo y tamaño, los vídeos con su ID de YouTube); vacío si IGDB no conoce el juego.

Pantallas: Ficha del juego (cabecera y carrusel de imágenes y vídeos).

games/igdb.py, línea 263

```
def _with_editions(self, games)
```

Añade a cada juego sus ediciones (GOTY, Definitive...) en la clave «editions», para tener sus IDs de tienda.

Pantallas: Catálogo, Ficha del juego (Dónde comprarlo: los IDs de tienda de sus ediciones), Inicio, Perfil, Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta), Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas).

games/igdb.py, línea 274

```
def sort_arts(artworks)
```

Artes de IGDB que son ilustraciones (sin los tipos de NOT_ART_TYPES), con el arte principal primero.

Los que no traen tipo se quedan: IGDB no los ha clasificado, pero suelen ser ilustraciones.

Pantallas: Ficha del juego (cabecera y carrusel), Editar perfil (imágenes de un juego para la cabecera).

games/igdb.py, línea 301

```
def _upsert_ref(model, data, **extra)
```

Busca o crea un Genre o Platform de IGDB, aprovechando uno que no venga de IGDB con el mismo slug.

Pantallas: Catálogo (filtros de género y plataforma), Ficha del juego (géneros y plataformas), Nuevo reto (géneros y plataformas).

games/igdb.py, línea 312

```
def _normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

Pantallas: Ficha del juego (Dónde comprarlo: los IDs de tienda con los que se buscan), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/igdb.py, línea 332

```
def _store_ids(data)
```

IDs de tienda de un juego de IGDB: {"steam": [...], "gog": [...]}.

IGDB puede tener varias entradas por tienda: relanzamientos con el mismo nombre que el juego (se quedan todos y sus ofertas se juntan) y otras raras como «Mac Portal» (solo se usa la primera, y únicamente si ninguna se llama como el juego).

Pantallas: Ficha del juego (Dónde comprarlo: los IDs de tienda con los que se buscan), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/igdb.py, línea 341

```
def _store_alternatives(data, own_ids)
```

IDs de tienda de las ediciones del juego y después de las fichas de RELATED_STORES (sus ports, sus ediciones ampliadas y sus remasters), sin repetir los suyos.

Sirven para encontrar precios cuando el juego original ya no se vende o cuando IGDB guarda sus IDs de tienda en otra ficha, como la versión de PC de Red Dead Redemption o la edición ampliada de The Witcher 2.

Pantallas: Ficha del juego (Dónde comprarlo: los IDs de tienda de sus ediciones, ports y remasters), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/igdb.py, línea 365

```
def _xbox_product_ids(data)
```

IDs de producto de Microsoft Store del juego y después de sus ediciones y remasters, sin repetidos.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/igdb.py, línea 389

```
@transaction.atomic
def upsert_games(payload)
```

Crea o actualiza los juegos locales a partir de los de IGDB y devuelve la lista de Game.

Guarda datos, nota, portada, IDs de tienda, géneros, plataformas y lo de castellano (idiomas, nombre en España, otros nombres y portada europea, ver games.spanish). Aprovecha juegos de demo o creados a mano con el mismo nombre o slug en vez de duplicarlos. Si cambian los IDs de tienda, borra los precios de PC, que se buscaron con los IDs viejos. Todo va en una transacción.

Pantallas: Catálogo (botón para traer de IGDB), Ficha del juego, Inicio, Perfil (modal «Vincular cuentas» de su dueño), Lista de deseos, Página de la lista, Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas), Juegos importados (buscar en IGDB), Editar perfil (Cuentas vinculadas: juegos importados que aún no están en el catálogo).

games/igdb.py, línea 406

games/management/commands/import_igdb.py

Comando «manage.py import_igdb»: trae juegos de IGDB al catálogo (los más populares, una búsqueda o los que no tienen IDs de tienda). Lo que guarda se ve en el Catálogo, la Ficha del juego y en todas las pantallas que enseñan juegos.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Lista de deseos, Página de la lista, Página del reto

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (cuántos populares), --search (importar una búsqueda) y --sin-ids-de-tienda (repasar los del catálogo sin IDs de Steam ni de GOG).

Pantallas: ninguna (opciones del comando import_igdb).

games/management/commands/import_igdb.py, línea 19

```
def handle(self, *args, **options)
```

Punto de entrada: importa una búsqueda, los más populares o repasa los que no tienen IDs de tienda, y dice cuántos juegos guardó.

Pantallas: Catálogo, Ficha del juego (Dónde comprarlo: con los IDs de tienda que trae), Inicio, Perfil, Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta), Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas).

games/management/commands/import_igdb.py, línea 32

```
def _import_top(self, client, limit)
```

Importa los limit juegos más populares en páginas de hasta 500. Devuelve cuántos importó.

Pantallas: Catálogo (los juegos más populares de IGDB), Ficha del juego, Inicio, Perfil, Lista de deseos, Página de la lista, Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas).

games/management/commands/import_igdb.py, línea 56

```
def _refresh_without_store_ids(self, client)
```

Vuelve a pedir a IGDB los juegos del catálogo que no tienen con qué buscar sus precios de PC (sin IDs de Steam ni de GOG, ni suyos ni de sus ediciones), por si los tienen en un port o una edición ampliada (igdb.RELATED_STORES). Devuelve (cuántos tienen ahora con qué buscarlos, cuántos se han pedido); sus precios llegan con update_prices.

Pantallas: Ficha del juego (Dónde comprarlo: los juegos que no tenían con qué buscar sus precios), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/management/commands/import_igdb.py, línea 73

games/management/commands/localize_games.py

Comando «manage.py localize_games» (parte de la tarea de cada noche): completa los datos en castellano de los juegos (nombre en España, idiomas, portada europea y sinopsis en español) para la Ficha del juego y los filtros de idioma del Catálogo, el Perfil y la Lista de deseos.
Pantallas de este archivo: Catálogo, Ficha del juego, Perfil, Lista de deseos

Clase Command

```
def add_arguments(self, parser)
```

Opciones: --limit, --dias, --igdb, --sin-tiendas y --pausa.

Pantallas: ninguna (opciones del comando localize_games).

games/management/commands/localize_games.py, línea 26

```
def handle(self, *args, **options)
```

Primero IGDB (rápido, en lotes de 500) y después las tiendas (Steam de uno en uno, con pausas).

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (filtro de idioma y portadas europeas), Perfil (filtro de idioma), Lista de deseos (filtro de idioma).

games/management/commands/localize_games.py, línea 43

```
def _from_igdb(self, everything)
```

Pide a IGDB los idiomas, los nombres y la portada europea de los juegos que no los tienen (o de todos).

Los que IGDB no devuelve se quedan con los datos de IGDB vacíos, para no volver a pedirlos cada noche.

Pantallas: Ficha del juego (nombre en España e idiomas), Catálogo (filtro de idioma y portadas europeas), Perfil (filtro de idioma), Lista de deseos (filtro de idioma).

games/management/commands/localize_games.py, línea 53

```
def _from_stores(self, limit, days, pause)
```

Pide la sinopsis en español y los idiomas de Steam (y la descripción de Microsoft) de los más populares.

Pantallas: Ficha del juego (sinopsis en español e idiomas de Steam y de Microsoft).

games/management/commands/localize_games.py, línea 80

games/management/commands/seed_demo.py

Comando «manage.py seed_demo»: crea 12 juegos de prueba con sus géneros y plataformas para usar la web sin credenciales de IGDB. Solo para desarrollo.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Lista de deseos, Página de la lista, Página del reto

Clase Command

```
def handle(self, *args, **options)
```

Crea los 12 juegos de GAMES con sus géneros y plataformas; se salta los que ya existen.

Pantallas: Catálogo (los 12 juegos de prueba), Ficha del juego, Inicio, Perfil, Lista de deseos, Página de la lista, Página del reto (los datos de cada juego: nombre, portada, géneros y plataformas).

games/management/commands/seed_demo.py, línea 41

games/management/commands/update_prices.py

Comando «manage.py update_prices» (parte de la tarea de cada noche): actualiza los precios de PC, Xbox y Nintendo de «Dónde comprarlo» de la Ficha del juego, la Lista de deseos y la Página de la lista, y crea los avisos de bajada de precio.
Pantallas de este archivo: Ficha del juego, Lista de deseos, Página de la lista, Avisos

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (solo los N más populares) y --game (un solo juego por slug).

Pantallas: ninguna (opciones del comando update_prices).

games/management/commands/update_prices.py, línea 20

```
def handle(self, *args, **options)
```

Actualiza los precios de PC (si hay ITAD_API_KEY), los de Xbox y los de Nintendo de los juegos elegidos, y después crea los avisos de la lista de deseos.

Pantallas: Ficha del juego (Dónde comprarlo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta), Avisos (bajadas de precio de tu lista de deseos).

games/management/commands/update_prices.py, línea 28

games/media.py

Imágenes y vídeos de cada juego para la cabecera y el carrusel de la ficha.

Los artes, capturas, logos y vídeos se piden a IGDB en una sola consulta la primera vez que alguien abre la ficha (y otra vez pasado MAX_AGE) y se guardan en Game.media:

{"arts": [{"id", "type", "w", "h"}], "logos": [{"id", "type", "crop"}], "shots": [{"id", "w", "h"}], "videos": [{"id", "name"}], "steam": {"app_id", "hero", "logo"}}

«arts» son solo ilustraciones (sin logos, iconos, portadas ni infografías) con el arte principal primero; «logos», el blanco y el de color, que pueden ir encima de la cabecera; «videos», tráileres de YouTube con el nombre que les da IGDB (en inglés). Si el juego está en Steam, «steam» dice si su biblioteca tiene imagen de cabecera (True, False o None si no se pudo mirar) y el recorte de su logo. Los recortes («crop», ver _logo_crop) quitan el margen transparente de los logos para que encajen en la cabecera.
Pantallas de este archivo: Ficha del juego, Perfil, Escribir reseña, Editar perfil

`def steam_app_id(game)`

ID de Steam de la imagen y el logo de la cabecera: el manual si lo hay; None si no está en Steam.

Pantallas: Ficha del juego (imagen y logo de la cabecera de Steam).

games/media.py, línea 39

`def needs_update(game, now=None)`

Indica si hay que pedir a IGDB las imágenes y vídeos del juego.

Hay que pedirlos si el juego viene de IGDB y nunca se pidieron, tienen más de MAX_AGE o se pidieron con otro ID de Steam (o sin él).

Pantallas: Ficha del juego (si el carrusel va ya o se carga después con HTMX).

games/media.py, línea 47

`def update_media(game, client=None, now=None)`

Pide a IGDB los artes, capturas, logos y vídeos del juego y los guarda en game.media (formato arriba).

Si el juego está en Steam, mira también si su biblioteca tiene imagen de cabecera y logo. Del logo que se vaya a usar (el de Steam o, si no hay, el de IGDB) guarda el recorte sin el margen transparente.

Pantallas: Ficha del juego (cabecera y carrusel de imágenes y vídeos).

games/media.py, línea 63

`def _size(image)`

ID y tamaño de una imagen de IGDB tal como se guardan en Game.media.

Pantallas: Ficha del juego (carrusel).

games/media.py, línea 101

`def _video_order(video)`

Orden de los vídeos: el tráiler de lanzamiento (o el de la fecha de salida), los demás tráileres, los de gameplay y el resto; dentro de cada grupo, el que antes subió IGDB.

Pantallas: Ficha del juego (orden de los vídeos del carrusel).

games/media.py, línea 109

`def _exists(url)`

Indica si la imagen de url existe (True o False), o None si no se pudo comprobar.

Pantallas: Ficha del juego (imagen y logo de la cabecera).

games/media.py, línea 125

```
def _logo_crop(url)
```

Recorte del logo de url sin su margen transparente, o None si no existe, no se puede leer o no tiene transparencia (lleva fondo y taparía la cabecera con un recuadro).

Son fracciones de la imagen entera: izquierda «l», arriba «t», ancho «w» y alto «h» de lo que no es transparente, y la proporción ancho/alto de ese trozo («ratio»). Los logos de la biblioteca de Steam vienen en un lienzo de 16:9 con mucho margen arriba y abajo.

Pantallas: Ficha del juego (logo de la cabecera).

games/media.py, línea 137

```
def _igdb_logo_url(logo)
```

URL del logo de IGDB en PNG (con su transparencia), a un tamaño que se ve nítido en la cabecera.

Pantallas: Ficha del juego (logo de la cabecera).

games/media.py, línea 172

```
def _is_wide(image)
```

Indica si la imagen es apaisada (o no se sabe su tamaño); las verticales se enseñan enteras, sin recortar.

Pantallas: Ficha del juego (cabecera y carrusel), Escribir reseña (imágenes de la ficha para la reseña), Editar perfil (imágenes de un juego para la cabecera).

games/media.py, línea 180

```
def hero(game)
```

Imagen y logo de la cabecera de la ficha: {"image", "fallback", "credit", "fallback_credit", "logo", "logo_style"}.

La imagen es la de la biblioteca de Steam si el juego está en Steam y la tiene (o aún no se sabe: la página pasa a «fallback» si no carga); si no, el arte principal de IGDB sin logo, el arte apaisado más ancho o la primera captura. Sin imagen, la ficha se queda con la carátula desenfocada. El logo, el de Steam o el blanco o de color de IGDB, solo va cuando se sabe su recorte (logo_style, variables CSS); si no, el nombre en texto. «credit» dice de dónde salen; «fallback_credit», si se pasa a «fallback».

Pantallas: Ficha del juego (imagen y logo de la cabecera).

games/media.py, línea 189

```
def _credit(image_source, logo_source)
```

Texto que dice de dónde salen la imagen y el logo de la cabecera («Imagen y logo: biblioteca de Steam»).

Pantallas: Ficha del juego (de dónde salen la imagen y el logo de la cabecera).

games/media.py, línea 227

```
def _hero_image(media)
```

URL de la mejor imagen de IGDB para la cabecera, o vacía si no hay ninguna.

Es el arte principal sin logo; si no hay, el arte apaisado más ancho que no lleve el logo (repetiría el título); y si tampoco, la primera captura.

Pantallas: Ficha del juego (imagen de la cabecera).

games/media.py, línea 238

```
def carousel(game)
```

Miniaturas y filtros del carrusel de la ficha: {"items": [...], "filters": [(tipo, nombre, cuántas)]}.

Orden: el primer vídeo, el arte principal, las capturas, el resto de artes y el resto de vídeos. Cada miniatura es un dict con kind («video», «captura» o «arte»), tag (su etiqueta), heading y detail (el título del visor), label (para lectores de pantalla), thumb y full (URLs de la miniatura y de la imagen grande), tall (vertical: se enseña entera), youtube (ID del vídeo) y art (ID de IGDB que se puede poner de cabecera del perfil; vacío en los vídeos y en las imágenes verticales). Solo hay filtros si hay más de un tipo.

Pantallas: Ficha del juego (carrusel de imágenes y vídeos).

games/media.py, línea 256

```
def official_images(game)
```

Capturas y artes de la ficha que se pueden poner en una reseña sin subir nada, primero las capturas.

Son miniaturas como las del carrusel (ver carousel), con «id», el ID de la imagen en IGDB. Vacía si aún no se han pedido las imágenes del juego.

Pantallas: Escribir reseña (imágenes de la ficha que se pueden poner sin subir nada).

games/media.py, línea 278

```
def _image_item(kind, image, n, total)
```

Miniatura del carrusel de un arte o una captura (ver carousel), con su ID de IGDB en «id».

Pantallas: Ficha del juego (miniaturas del carrusel), Escribir reseña (imágenes de la ficha).

games/media.py, línea 293

```
def _video_item(video, n, total)
```

Miniatura del carrusel de un vídeo de YouTube (ver carousel); el vídeo se carga al pulsarlo en el visor.

Pantallas: Ficha del juego (vídeos del carrusel).

games/media.py, línea 316

```
def banner_arts(game)
```

IDs de IGDB de las imágenes de la ficha que valen de cabecera del perfil: artes y capturas apaisados.

Pantallas: Ficha del juego (botón del visor del carrusel para ponerla de cabecera), Perfil (cabecera).

games/media.py, línea 336

games/models.py

Modelos del catálogo: cada juego (con lo que trae de IGDB y de las tiendas: portadas, idiomas, IDs de tienda e imágenes), sus géneros, sus plataformas y sus precios por tienda. Salen en casi todas las pantallas, sobre todo en el Catálogo y en la Ficha del juego.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Listas del juego, Perfil, Página de la publicación, Escribir reseña, Página de la reseña, Lista de deseos, Página de la lista, Nueva tier list, Editar lista, Página del reto, Nuevo reto, Avisos, Editar perfil, Juegos importados, Panel de administración

```
def spanish_genre_name(name, slug='')
```

Nombre en español de un género de IGDB; si no está en GENRE_NAMES_ES, el original.

Pantallas: Catálogo (filtro de género), Ficha del juego (géneros), Inicio (géneros), Nuevo reto (géneros), Perfil (géneros favoritos del escaparate).

games/models.py, línea 41

```
def igdb_image_url(image_id, size='t_cover_big', ext='jpg')
```

URL de una imagen del CDN de IGDB (portada, arte o captura) en ese tamaño; vacía si no hay imagen.

Con ext «png» conserva la transparencia, que hace falta en los logos.

Pantallas: Inicio, Catálogo, Ficha del juego (cabecera y carrusel), Listas del juego, Perfil (cabecera con la imagen de un juego), Lista de deseos, Escribir reseña (imágenes de la ficha), Página de la reseña, Página de la publicación, Página de la lista, Editar lista, Nueva tier list, Página del reto, Avisos, Juegos importados, Editar perfil (las portadas de los juegos).

games/models.py, línea 53

Clase Spanish

Cuánto español tiene un juego en voces o en textos. Vacío = no se sabe (ninguna fuente da sus idiomas).

```
def language_filter(key, prefix='')
```

Condición (Q) del filtro de idioma ?idioma=key, o None si key no es uno de LANGUAGE_FILTERS.

prefix va delante de los campos para filtrar desde otro modelo (p. ej. «game__» en la lista de deseos). Con voces en castellano también se cuentan los textos: si está doblado, está traducido.

Pantallas: Catálogo (filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/models.py, línea 96

```
def search_filter(query)
```

Condición (Q) de la búsqueda de juegos por nombre: el de IGDB, el de España y sus otros nombres (siglas como «tw3»).

Pantallas: Catálogo (búsqueda por nombre, también desde la barra de arriba), Editar lista (buscar juegos para añadir).

games/models.py, línea 114

Clase Genre

```
def __str__(self)
```

Nombre del género.

Pantallas: Panel de administración, Catálogo (filtro de género), Ficha del juego (géneros).

games/models.py, línea 131

Clase Platform

```
def __str__(self)
```

Nombre completo de la plataforma.

Pantallas: Panel de administración, Catálogo (filtro de plataforma), Nuevo reto (plataformas).

games/models.py, línea 155

```
@property
def short_name(self)
```

Nombre corto para las etiquetas (p. ej. «PS5»); el completo si no tiene abreviatura.

Pantallas: Ficha del juego (plataformas de la cabecera), Perfil (plataforma de cada juego), Página de la reseña (plataforma de la partida), Juegos importados.

games/models.py, línea 163

Clase Game

```
def __str__(self)
```

Nombre del juego.

Pantallas: Panel de administración, Escribir reseña (el juego elegido en los formularios).

games/models.py, línea 264

```
def get_absolute_url(self)
```

URL de la ficha del juego (/juegos/<slug>/).

Pantallas: Inicio, Catálogo, Ficha del juego, Listas del juego, Perfil, Lista de deseos, Escribir reseña, Página de la reseña, Página de la publicación, Página de la lista, Editar lista, Página del reto, Juegos importados (los enlaces a la ficha de cada juego).

games/models.py, línea 271

```
def review_platforms(self)
```

Plataformas que se pueden elegir en una reseña del juego: las suyas y, si sale en PC, la Steam Deck.

Pantallas: Escribir reseña (plataformas que se pueden elegir).

games/models.py, línea 280

```
@property
def cover_url(self)
```

URL de la portada en el CDN de IGDB (la europea si IGDB la tiene), o cadena vacía si no hay portada.

Pantallas: Inicio, Catálogo, Ficha del juego, Listas del juego, Perfil, Lista de deseos, Escribir reseña, Página de la reseña, Página de la publicación, Página de la lista, Editar lista, Nueva tier list, Página del reto, Avisos, Juegos importados, Editar perfil (las portadas de los juegos).

games/models.py, línea 291

```
def merged_languages(self)
```

Idiomas de todas las fuentes juntos: {código: {"audio", "text"}}. Si una fuente dice que tiene voces, las tiene.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/models.py, línea 300

```
def refresh_spanish(self)
```

Recalcula spanish_audio y spanish_text con los idiomas de languages (sin guardar el juego).

Castellano si está el español de España, latino si solo está el de México y «sin español» si hay idiomas pero ninguno es español; vacío si ninguna fuente da idiomas. Las voces cuentan también como textos.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/models.py, línea 312

```
@property
def spain_name(self)
```

Nombre con el que se vende en España si es distinto del original («Los Sims 4»); vacío si no.

Pantallas: Ficha del juego (nombre en España de la cabecera).

games/models.py, línea 336

```
@property
def language_sources(self)
```

Fuentes que han dado idiomas del juego, para citarlas en la ficha: p. ej. ["IGDB", "Steam"].

Pantallas: Ficha del juego (de dónde salen los idiomas).

games/models.py, línea 344

```
@property
def language_rows(self)
```

Idiomas para la tabla de la ficha: [(nombre en español, voces, textos)], con los españoles y el inglés primero.

Pantallas: Ficha del juego (tabla de idiomas).

games/models.py, línea 353

```
@property
def release_year(self)
```

Año de lanzamiento, o None si no se conoce la fecha.

Pantallas: Catálogo, Inicio, Ficha del juego, Perfil, Página de la reseña, Página de la lista, Editar lista, Página del reto, Juegos importados (el año de cada juego).

games/models.py, línea 371

```
@property
def price_lookups(self)
```

Grupos ([(tienda, ID en la tienda), ...], nombre de edición) con los que buscar precios, por preferencia.

Primero los IDs del propio juego y después los de sus ediciones y remasters. Los IDs de un mismo grupo son el mismo producto, así que sus ofertas se juntan. Si hay un ID de Steam manual, solo se usa ese.

Pantallas: Ficha del juego (Dónde comprarlo: los IDs de tienda con los que se buscan), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/models.py, línea 380

```
@property
def xbox_lookup_ids(self)
```

IDs de producto de Xbox con los que buscar el precio: el manual si lo hay, si no los de IGDB.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/models.py, línea 401

```
@property
def on_current_xbox(self)
```

Indica si el juego está en Xbox One o Xbox Series X|S.

Pantallas: Ficha del juego (Dónde comprarlo: Xbox).

games/models.py, línea 410

```
@property
def nintendo_lookup_nsuid(self)
```

NSUID con el que pedir el precio de la eShop: el manual si lo hay, si no el encontrado por el nombre.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/models.py, línea 418

```
@property
def best_price(self)
```

Oferta más barata de todas las tiendas (PC, Xbox y Nintendo), o None si no hay precios.

Pantallas: Lista de deseos (mejor precio de hoy), Página de la lista (lo que cuesta lo que te falta).

games/models.py, línea 427

```
def cheapest(self, shops=None)
```

Oferta más barata de esas tiendas (ids; None = todas), o None si ninguna lo vende.

Pantallas: Lista de deseos (mejor precio de hoy en tus tiendas), Página de la lista (lo que cuesta lo que te falta), Avisos (bajadas de precio).

games/models.py, línea 434

```
@property
def is_free(self)
```

Indica si alguna tienda lo da gratis (precio y precio normal a 0), es decir, si es free-to-play.

Pantallas: Ficha del juego (Dónde comprarlo: «Gratis»).

games/models.py, línea 447

```
def price_groups(self, shops=None)
```

Reparte los precios de esas tiendas (ids; None = todas) para la ficha en (destacados, listados, plegados, mejor).

destacados: las tiendas más conocidas (Steam, Epic, Microsoft, GOG, Xbox, Nintendo) en un orden fijo. listados: las demás más baratas, solo si el juego no está en ninguna destacada. plegados: el resto por precio, dentro de «Ver N tiendas más». mejor: la oferta más barata cuando ha quedado plegada, para poder señalarla igualmente.

Pantallas: Ficha del juego (Dónde comprarlo: destacadas, el resto y las plegadas).

games/models.py, línea 454

```
def store_searches(self)
```

(tienda, plantilla del logo, URL de búsqueda) de las consolas en las que está el juego pero sin precio.

Son PlayStation Store y, si no se encontró su precio, Nintendo eShop y Xbox Store.

Pantallas: Ficha del juego (Dónde comprarlo: enlaces a buscarlo en las tiendas de consola sin precio).

games/models.py, línea 473

```
def community_rating(self)
```

Media de las valoraciones de los usuarios de NextPlay: {"average": de 0,5 a 5 estrellas o None, "count": n}, y la misma solo con quienes tienen horas importadas de Steam, PlayStation o Xbox («verified_average» y «verified_count»), que no pueden valorarlo sin haberlo jugado.

Las notas se guardan en medias estrellas (de 1 a 10), así que las medias se dividen entre dos.

Pantallas: Ficha del juego (valoración de la comunidad).

games/models.py, línea 488

```
@property
def critic_score(self)
```

Nota de la crítica redondeada (0-100), o None si no tiene.

Pantallas: Ficha del juego (nota de la crítica).

games/models.py, línea 513

```
@property
def critic_tier(self)
```

Franja de la nota para colorearla: «high» (75 o más), «mid» (50-74), «low» o vacío sin nota.

Pantallas: Ficha del juego (color de la nota de la crítica).

games/models.py, línea 521

```
@property
def historical_low_symbol(self)
```

Símbolo de la moneda del mínimo histórico (€, $, £...).

Pantallas: Ficha del juego (mínimo histórico), Lista de deseos (mínimo histórico).

games/models.py, línea 532

```
def currency_symbol(code)
```

Símbolo de una moneda a partir de su código ISO («EUR» → «€»); el propio código si no lo conoce.

Pantallas: Ficha del juego (Dónde comprarlo), Lista de deseos, Página de la lista (los símbolos de moneda).

games/models.py, línea 584

```
def known_shops()
```

Tiendas con algún precio en el catálogo, [(id, nombre)]: las destacadas primero y después por nombre.

Pantallas: Editar perfil (Tiendas en las que compras).

games/models.py, línea 592

```
def format_price(amount, symbol='€')
```

Precio escrito como en España para los textos que se generan fuera de las plantillas: «2,99 €».

Pantallas: Ficha del juego (historial de precios), Página de la lista (lo que cuesta lo que te falta), Avisos (bajadas de precio), Editar perfil (mensaje al traer la lista de deseos de Steam).

games/models.py, línea 607

Clase GamePrice

Precio actual de un juego en una tienda.

```
def __str__(self)
```

Texto del precio en el admin: «juego · tienda: precio moneda».

Pantallas: Panel de administración.

games/models.py, línea 641

```
@property
def symbol(self)
```

Símbolo de la moneda del precio.

Pantallas: Ficha del juego (Dónde comprarlo), Lista de deseos (precios).

games/models.py, línea 649

```
@property
def is_free(self)
```

Indica si el juego es gratis en esta tienda (no solo rebajado a 0 por una oferta).

Pantallas: Ficha del juego (Dónde comprarlo: «Gratis»).

games/models.py, línea 657

```
@property
def is_featured(self)
```

Indica si la tienda es una de las destacadas (Steam, Epic, Microsoft, GOG, Xbox, Nintendo).

Pantallas: Ficha del juego (Dónde comprarlo: tiendas destacadas).

games/models.py, línea 665

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la tienda; solo para tiendas destacadas.

Pantallas: Ficha del juego (Dónde comprarlo: logos de las tiendas).

games/models.py, línea 673

games/nintendo.py

Precios de la eShop de Nintendo (Switch y Switch 2) en la tienda europea (sin documentar, sin clave).

IGDB no tiene el NSUID de los juegos (su ID en la eShop), así que se busca por el nombre en el buscador de Nintendo Europa, el de su web, y se guarda en el juego. El precio sale de la API de precios de la eShop, la misma que usa la web de Nintendo. Los NSUID de la tienda europea valen en todos sus países.
Pantallas de este archivo: Ficha del juego, Lista de deseos, Página de la lista

Clase NintendoError

Fallo al consultar la eShop de Nintendo (red o respuesta no válida).

```
def _country()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»); tiene que ser de la tienda europea.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 36

```
def _amount(value)
```

Importe de la API de precios («14.99») como Decimal con dos decimales.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 45

```
def fetch_prices(nsuids)
```

Devuelve {NSUID: (precio, precio normal, moneda)} de los juegos que se pueden comprar.

Pide los precios de 50 en 50; los que no están a la venta (sin salir o retirados) no aparecen.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 54

```
def _offer(item)
```

(precio, precio normal, moneda) de un juego de la respuesta de precios, o None si no se vende.

El precio es el rebajado si tiene una oferta en marcha.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 82

```
def _normalize(title)
```

Título para comparar: en minúsculas, sin tildes ni signos (™, dos puntos, guiones...) y con los espacios simplificados, porque la tienda los pone a su manera: «South Park™ : La vara de la verdad™», «ABZÛ».

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 99

```
def search_game(names, systems)
```

NSUID del juego de esas consolas («HAC», «BEE») que se llama exactamente como uno de names, o None.

Busca con cada nombre hasta encontrarlo y compara con el título de la tienda española, el original en inglés y sus traducciones. Si dos juegos de una consola se llaman igual no elige ninguno; si es el mismo título en Switch y en Switch 2 se queda con el de Switch, que también funciona en Switch 2.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 112

```
def store_url(nsuid)
```

Enlace al juego en la eShop, que lleva a su página en nintendo.com.

Pantallas: Ficha del juego (Dónde comprarlo: enlace a la eShop).

games/nintendo.py, línea 156

```
def _systems(game)
```

Códigos del buscador de Nintendo de las consolas del juego: «HAC» (Switch) y «BEE» (Switch 2).

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 164

```
@transaction.atomic
def _store(game, nsuid, offer)
```

Sustituye el precio de Nintendo del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 174

```
def update_nintendo_prices(games, sleep=time.sleep)
```

Guarda el precio de la eShop de cada juego y devuelve cuántos acabaron con precio.

Los juegos de Switch o Switch 2 sin NSUID (ni manual ni encontrado antes) lo buscan primero por su nombre y su nombre en España, salvo que otro juego del catálogo se llame igual; el que se encuentra se guarda para no volver a buscarlo.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Nintendo), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/nintendo.py, línea 196

games/prices.py

Cliente de IsThereAnyDeal y la sincronización que guarda los precios actuales de PC de cada juego.

También está aquí la búsqueda al momento, en todas las tiendas, de los precios de los juegos que nunca se han buscado (fetch_missing), para no esperar a la tarea de cada noche.
Pantallas de este archivo: Ficha del juego, Perfil, Lista de deseos, Página de la lista, Editar perfil

Clase ITADError

Fallo al hablar con IsThereAnyDeal (clave, red o respuesta no válida).

```
def is_configured()
```

Indica si está puesta la clave de IsThereAnyDeal.

Pantallas: Ficha del juego (Dónde comprarlo: sin clave de IsThereAnyDeal no hay precios de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 33

Clase ITADClient

```
def __init__(self, api_key=None, country=None, retries=MAX_RETRIES)
```

Prepara el cliente con la clave y el país dados o los de settings; falla si no hay clave.

retries son las veces que se repite una petición cuando la API pide calma (429). Con 0 no se espera nada, que es lo que conviene cuando alguien está esperando la página.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC; historial de precios), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 43

```
def _post(self, path, body, params=None)
```

POST a la API con la clave y devuelve el JSON.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 58

```
def _get(self, path, params)
```

GET a la API con la clave y devuelve el JSON.

Pantallas: Ficha del juego (historial de precios).

games/prices.py, línea 66

```
def _send(self, method, path, **kwargs)
```

Llama a la API con method (requests.get o requests.post) y la clave, y devuelve el JSON.

Si la API responde 429 (demasiadas peticiones) espera lo que diga Retry-After (máximo 5 minutos) y reintenta hasta self.retries veces (3, si no se dice otra cosa).

Pantallas: Ficha del juego (Dónde comprarlo: los de PC; historial de precios), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 73

```
def lookup(self, store, store_ids)
```

Traduce IDs de una tienda (de Steam o de GOG) a los UUID de juego de ITAD (None si no lo conoce).

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 96

```
def prices(self, itad_ids)
```

Ofertas actuales y mínimo histórico de cada juego de ITAD en el país configurado.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 106

```
def history(self, itad_id, since)
```

Cambios de precio de un juego de ITAD desde la fecha since, en todas las tiendas del país.

Cada cambio es {"timestamp", "shop": {"id", "name"}, "deal": {"price", "regular", "cut"}}; «deal» es None cuando la tienda deja de venderlo. Vienen del más nuevo al más viejo.

Pantallas: Ficha del juego (historial de precios).

games/prices.py, línea 114

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 126

```
def _batches(items)
```

Parte la lista en trozos de BATCH_SIZE, lo máximo que acepta la API por petición.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 135

```
def _cheapest_per_shop(deals)
```

Se queda con la oferta más barata de cada tienda.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 145

```
@transaction.atomic
def _store_prices(game, data, edition='', itad_ids=())
```

Sustituye los precios de PC del juego por las ofertas de data y guarda su mínimo histórico.

Con data vacío deja el juego sin precios de PC. edition es el nombre de la edición de la que salen los precios (vacío si son del propio juego) e itad_ids, los juegos de ITAD de los que salen; si cambian, se borra el historial guardado, que era de otros.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 161

```
def _lookup(client, keys)
```

Traduce un conjunto de (tienda, ID) a UUID de ITAD, tienda por tienda y por lotes.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 200

```
def _fetch_prices(client, itad_ids)
```

Precios de todos los UUID de ITAD pedidos por lotes: {uuid: datos de precios}.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 214

```
def _merge(items)
```

Junta los precios de varios juegos de ITAD que son el mismo producto (p. ej. un relanzamiento).

Suma todas sus ofertas y se queda con el mínimo histórico más bajo. Devuelve None si no hay datos.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 226

```
def update_prices(games, client=None)
```

Actualiza los precios de PC de los juegos y devuelve cuántos acabaron con precios.

Va por rondas: en la primera busca con los IDs del propio juego; los juegos que se quedan sin ofertas pasan a la siguiente ronda con su primera edición o remaster, y así sucesivamente.

Todos quedan con la fecha de la búsqueda (prices_updated_at), también los que no tienen IDs de Steam ni de GOG con los que buscarlos en ITAD, que se quedan sin precios de PC: así consta que ya se han mirado.

Pantallas: Ficha del juego (Dónde comprarlo: los de PC), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/prices.py, línea 244

```
def fetch_missing(games, consoles=False)
```

Busca ya, sin esperar a la tarea de cada noche, los precios de los juegos que nunca se han buscado (prices_updated_at vacío) y devuelve cuántos han quedado buscados.

Los de PC se piden a IsThereAnyDeal de todos a la vez y sin esperas si la API pide calma, y con eso quedan como buscados. Después van los de Xbox y Nintendo: con consoles (la ficha de un juego), de todos; si no (una importación), solo de los que se han quedado sin ningún precio, porque esas tiendas se buscan por el nombre juego a juego y con muchos la importación tardaría demasiado; a los demás se los trae update_prices por la noche.

Si una tienda falla se apunta en el log y se sigue. Si falla IsThereAnyDeal, los juegos siguen sin buscar y se volverán a pedir la próxima vez; en una importación tampoco se buscan entonces sus precios de consola. Sin ITAD_API_KEY no hace nada.

Pantallas: Ficha del juego (Dónde comprarlo de un juego cuyos precios nunca se han buscado), Lista de deseos, Página de la lista, Editar perfil (Cuentas vinculadas: al importar), Perfil (modal «Vincular cuentas» de su dueño).

games/prices.py, línea 282

```
def lookup_ids(game, client=None)
```

UUID de ITAD de los IDs de tienda con los que se encontraron los precios actuales del juego.

Sirve para los juegos cuyos precios se guardaron antes de que se apuntaran sus itad_ids.

Pantallas: Ficha del juego (historial de precios).

games/prices.py, línea 322

games/spanish.py

Datos en castellano de los juegos: idiomas (voces y textos), nombre en España, portada europea y sinopsis en español.

Los idiomas, los nombres y la portada europea vienen de IGDB en la misma consulta de la importación (SPANISH_FIELDS, que se añade a GAME_FIELDS). La sinopsis en español y los idiomas de la tienda vienen de Steam (appdetails en español) y, si el juego no está en Steam o no tiene sinopsis, de la descripción de la tienda de Microsoft. Steam admite pocas peticiones (unas 200 cada 5 minutos), así que esa parte la hace el comando localize_games con pausas, nunca al abrir una ficha.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Listas del juego, Perfil, Página de la publicación, Escribir reseña, Página de la reseña, Lista de deseos, Página de la lista, Nueva tier list, Editar lista, Página del reto, Avisos, Editar perfil, Juegos importados

Clase StoreError

Fallo al pedir a Steam los datos de un juego (red, límite de peticiones o respuesta no válida).

```
def igdb_languages(data)
```

Idiomas de un juego de IGDB: {código: ["audio", "text"]}, con «text» si tiene interfaz o subtítulos.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/spanish.py, línea 56

```
def spanish_names(data)
```

(nombre en España, otros nombres para el buscador) de un juego de IGDB.

El nombre en España es el primer nombre alternativo que IGDB marca como «Spanish title». Para el buscador se juntan, en minúsculas y sin repetir, los demás nombres alternativos (siglas, títulos de otros países...) y los de las versiones regionales, salvo los de los ejecutables.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca; modal «Vincular cuentas» de su dueño), Lista de deseos (filtro de idioma), Editar perfil (Cuentas vinculadas: nombres con los que se emparejan los juegos importados).

games/spanish.py, línea 74

```
def europe_cover(data)
```

ID de imagen de la portada europea de un juego de IGDB (versión regional de Europa), o vacío si no tiene.

Pantallas: Inicio (la portada europea), Catálogo, Ficha del juego, Listas del juego, Perfil, Lista de deseos, Escribir reseña, Página de la reseña, Página de la publicación, Página de la lista, Editar lista, Nueva tier list, Página del reto, Avisos, Juegos importados, Editar perfil (las portadas de los juegos).

games/spanish.py, línea 100

```
def apply_igdb(game, data)
```

Copia en game (sin guardarlo) los idiomas, el nombre en España, los otros nombres y la portada europea de IGDB.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma), Inicio (la portada europea), Listas del juego, Escribir reseña, Página de la reseña, Página de la publicación, Página de la lista, Editar lista, Nueva tier list, Página del reto, Avisos, Juegos importados, Editar perfil (las portadas de los juegos).

games/spanish.py, línea 113

```
def steam_languages(text)
```

Idiomas de la cadena supported_languages de Steam (pedida en español): {código: ["text"] o ["audio", "text"]}.

Steam marca con un asterisco los idiomas con voces; lo que va después del primer
 es la nota que explica el asterisco. Los idiomas que no están en STEAM_LANGUAGES se ignoran.

Pantallas: Ficha del juego (idiomas de Steam), Catálogo (filtro de idioma), Perfil (filtro de idioma), Lista de deseos (filtro de idioma).

games/spanish.py, línea 128

```
def clean_text(value)
```

Texto plano a partir del HTML de una tienda: sin etiquetas, con las entidades traducidas y sin espacios de más.

Pantallas: Ficha del juego (sinopsis en español).

games/spanish.py, línea 146

```
def steam_details(app_id, sleep=time.sleep)
```

Idiomas y sinopsis en español de una app de Steam: (idiomas, sinopsis); ({}, "") si Steam no la conoce.

Los cortes de conexión y los errores de Steam (también el 429 por pedir demasiado) se reintentan tras esperar RETRY_WAITS; si sigue fallando, lanza StoreError.

Pantallas: Ficha del juego (sinopsis en español e idiomas de Steam).

games/spanish.py, línea 156

```
def localize_games(games, pause=STEAM_PAUSE, sleep=time.sleep)
```

Pide a las tiendas la sinopsis en español y los idiomas de Steam de esos juegos y los guarda.

Steam va de uno en uno, con pause segundos entre petición y petición. Los que se quedan sin sinopsis y están en la tienda de Microsoft se piden después, en lotes. Cada juego queda marcado con localized_at para no volver a pedirlo hasta dentro de un tiempo. Si Steam deja de responder (StoreError) se para ahí: lo ya pedido queda guardado, se completa con Microsoft y después se vuelve a lanzar el error.

Devuelve {"steam": con sinopsis de Steam, "xbox": con sinopsis de Microsoft, "none": sin sinopsis}.

Pantallas: Ficha del juego (nombre en España, sinopsis en español e idiomas), Catálogo (búsqueda por nombre en España y filtro de idioma), Perfil (filtro de idioma de la biblioteca), Lista de deseos (filtro de idioma).

games/spanish.py, línea 186

games/views.py

Vistas de los juegos: Inicio (la portada), Catálogo y Ficha del juego, con los trozos de la ficha que se cargan con HTMX («Dónde comprarlo», el historial de precios y el carrusel de imágenes y vídeos) y el botón del Catálogo para traer juegos de IGDB.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego

```
def trending_games(limit=5)
```

Los juegos que más gente ha añadido a su biblioteca en los últimos TRENDING_DAYS días.

Cada juego trae players (cuántos lo tienen en total) y recent (cuántos lo añadieron en ese tiempo). Si no llegan a limit, se completa con los más populares de IGDB.

Pantallas: Inicio (juegos en tendencia).

games/views.py, línea 33

```
def home(request)
```

Portada: tu resumen (con tus seguidores y seguidos) y tu menú a la izquierda; en el centro la caja para publicar (social.views.composer_context, con los juegos que estás jugando para elegir de cuál hablas) y el feed, y los juegos en tendencia a la derecha.

El feed tiene dos pestañas (?ver=, ver social.views.feed_context): «siguiendo», lo tuyo y lo de la gente que sigues, con sugerencias de gente a la que seguir si no sigues a nadie, y «descubrir», lo de toda la comunidad; ?antes= es el cursor de «Cargar más» para quien no tiene JavaScript. Sin sesión, en el centro van los juegos populares y lo último de la comunidad.

Pantallas: Inicio.

games/views.py, línea 52

```
def catalog(request)
```

Catálogo de juegos.

Filtra por nombre (?q=, que también busca el nombre en España y los otros nombres del juego, como sus siglas), género (?genero=), plataforma (?plataforma=) e idioma (?idioma=, ver LANGUAGE_FILTERS) y pagina de 24 en 24. Si hay sesión, marca en cada tarjeta el estado del juego en la biblioteca del usuario (library_status, su nombre, y library_status_key, su valor, que le da el color). Ofrece importar de IGDB cuando hay búsqueda, sesión y credenciales de IGDB.

Pantallas: Catálogo.

games/views.py, línea 92

```
def detail(request, slug)
```

Ficha de un juego: cabecera con su arte, datos, nota de la crítica, valoración de la comunidad, widget de biblioteca, botones de favoritos del perfil y de «Añadir a una lista», las listas en las que está (lists.views.game_page_context), carrusel de imágenes y vídeos, precios y reseñas.

La cabecera y el carrusel salen de games.media: si sus imágenes de IGDB están al día se pintan ya; si no, la ficha carga el carrusel después con HTMX (game_media). «Dónde comprarlo» (_prices_context) se pinta ya si los precios del juego se han buscado alguna vez; si no, la ficha lo carga después con HTMX (game_prices), que antes los busca. Las reseñas de perfiles privados solo las ve su autor.

Pantallas: Ficha del juego.

games/views.py, línea 153

```
def game_media(request, slug)
```

Carrusel de imágenes y vídeos de la ficha (fragmento HTML que la ficha carga con HTMX).

Si nunca se pidieron a IGDB o tienen más de media.MAX_AGE, los pide antes; si IGDB falla, enseña los que hubiera. Si con ellos cambia la cabecera (un juego que no está en Steam y ahora tiene arte), manda también la nueva para que HTMX la cambie en la página.

Pantallas: Ficha del juego (carrusel de imágenes y vídeos que se carga con HTMX).

games/views.py, línea 196

```
@require_POST
@login_required
def import_search(request)
```

Importa de IGDB los resultados de una búsqueda que no estaban en el catálogo local.

Después vuelve al catálogo con la misma búsqueda y avisa de cuántos juegos se importaron, o de que IGDB no respondió.

Pantallas: Catálogo (botón para traer de IGDB lo que buscas).

games/views.py, línea 229

```
def _user_shops(request)
```

Tiendas en las que compra el usuario con sesión (conjunto de ids), o vacío si no ha elegido o no hay sesión.

Pantallas: Ficha del juego (Dónde comprarlo e historial: primero tus tiendas).

games/views.py, línea 248

```
def _prices_context(request, game)
```

Variables de «Dónde comprarlo» (games/_prices.html).

Los precios llegan ya repartidos por Game.price_groups(); store_searches() da los enlaces de búsqueda a las tiendas de consola sin precio. Si el usuario ha elegido sus tiendas solo se enseñan esas, salvo con ?tiendas=todas. El historial de precios de PC va ya pintado si está al día; si hay que pedirlo a ITAD, se carga después con HTMX (price_history), y su gráfica elige tienda con ?tienda=.

Pantallas: Ficha del juego (Dónde comprarlo).

games/views.py, línea 258

```
def game_prices(request, slug)
```

«Dónde comprarlo» de la ficha (fragmento HTML que la ficha carga con HTMX si los precios del juego nunca se han buscado).

Si siguen sin buscarse, antes los pide a todas las tiendas: las de PC a IsThereAnyDeal y también Xbox y Nintendo (prices.fetch_missing con consoles), y la ficha dice que se acaban de buscar (just_searched). Si IsThereAnyDeal falla, enseña los que haya y avisa de que no se han podido buscar (prices_failed); se volverá a intentar la próxima vez que se abra la ficha.

Pantallas: Ficha del juego (Dónde comprarlo, cuando se carga con HTMX).

games/views.py, línea 288

```
def _history_context(request, game, user_shops)
```

Variables de la sección del historial: el resumen de las tiendas elegidas con ?tienda= (por defecto, las del usuario), las opciones del selector y la elegida.

Pantallas: Ficha del juego (historial de precios).

games/views.py, línea 321

```
def price_history(request, slug)
```

Historial de precios de PC del juego con el veredicto «¿compro ahora o espero?» (fragmento HTML).

Si el guardado tiene más de un día, lo pide antes a IsThereAnyDeal; si ITAD falla, enseña el que hubiera. La ficha lo carga con HTMX para no esperar a ITAD, y el selector de tienda lo vuelve a pedir con ?tienda=.

Pantallas: Ficha del juego (historial de precios: «¿compro ahora o espero?»).

games/views.py, línea 336

games/xbox.py

Precios de consola Xbox sacados del catálogo público de la tienda de Microsoft (sin documentar, sin clave).
Pantallas de este archivo: Ficha del juego, Lista de deseos, Página de la lista

Clase XboxError

Fallo al consultar la tienda de Xbox (red o respuesta no válida).

```
def _market()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»).

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox; sinopsis de Microsoft), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 24

```
def _locale()
```

Idioma y país de la tienda, p. ej. «es-ES».

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox; sinopsis de Microsoft), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 33

```
def fetch_products(product_ids)
```

Devuelve {ID de producto: (precio, precio normal, moneda)} de los productos que se pueden comprar en consola Xbox.

Pide los productos de 20 en 20; los que no se venden en consola no aparecen en el resultado.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 42

```
def fetch_descriptions(product_ids)
```

Descripción en español de los productos de la tienda de Microsoft: {ID de producto: texto}.

Es la descripción corta o, si no tiene, el primer párrafo de la larga. Pide los productos de 20 en 20 en el idioma de la tienda (es-ES); los que no tienen descripción no aparecen.

Pantallas: Ficha del juego (sinopsis en español de la tienda de Microsoft).

games/xbox.py, línea 70

```
def _console_offer(product)
```

Oferta de compra más barata de un producto que funciona en Xbox, o None si no hay.

Ignora las de 0,00, que son Game Pass, pruebas o copias incluidas en packs.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 101

```
def _normalize(title)
```

Título para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 130

```
def search_games(name)
```

IDs de producto de los juegos de Xbox que se llaman exactamente como name (títulos en inglés, como en IGDB).

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 139

```
def store_url(product_id)
```

Enlace a la página del producto en xbox.com.

Pantallas: Ficha del juego (Dónde comprarlo: enlace a la tienda de Xbox).

games/xbox.py, línea 170

```
@transaction.atomic
def _store(game, product_id, offer)
```

Sustituye el precio de Xbox del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 179

```
def update_xbox_prices(games, sleep=time.sleep)
```

Guarda el precio de Xbox de cada juego y devuelve cuántos acabaron con precio.

Usa el primer ID de producto (el suyo y después sus ediciones) que se venda en consola. Como IGDB a menudo solo tiene productos que ya no se venden (p. ej. la versión original de un relanzamiento), los juegos de Xbox One/Series sin precio se buscan también por su título exacto, salvo que otro juego del catálogo se llame igual o que la búsqueda dé más de un producto.

Pantallas: Ficha del juego (Dónde comprarlo: el precio de Xbox), Lista de deseos (mejor precio de hoy y mínimo histórico), Página de la lista (lo que cuesta lo que te falta).

games/xbox.py, línea 201

App «library»

Biblioteca de cada usuario: qué juegos tiene, en qué estado, su valoración y su reseña.

library/admin.py

Panel de administración de la biblioteca: entradas de cada usuario, lista de deseos, reseñas (con sus comentarios, imágenes y denuncias pendientes) y las denuncias de reseñas, comentarios, publicaciones y listas.
Pantallas de este archivo: Panel de administración

Clase ReviewCommentInline

Comentarios de una reseña, para moderarlos desde ella.

Clase ReviewImageInline

Imágenes de una reseña, con su miniatura, para quitar las que no deban estar.

```
@admin.display(description='imagen')
def preview(self, image)
```

Miniatura de la imagen, que lleva a la grande.

Pantallas: Panel de administración (miniaturas de las imágenes de una reseña).

library/admin.py, línea 49

Clase ReportInline

Denuncias de una reseña o un comentario.

Clase ReviewAdmin

```
def get_queryset(self, request)
```

Las reseñas con sus denuncias pendientes contadas.

Pantallas: Panel de administración (reseñas con sus denuncias pendientes).

library/admin.py, línea 77

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, review)
```

Cuántas denuncias de la reseña quedan por revisar.

Pantallas: Panel de administración (columna de denuncias pendientes de las reseñas).

library/admin.py, línea 87

Clase PendingFilter

Filtro de las denuncias por si ya se han revisado.

```
def lookups(self, request, model_admin)
```

Las dos opciones del filtro.

Pantallas: Panel de administración (filtro de denuncias revisadas o sin revisar).

library/admin.py, línea 109

```
def queryset(self, request, queryset)
```

Las denuncias sin revisar o las ya revisadas, según la opción elegida.

Pantallas: Panel de administración (filtro de denuncias revisadas o sin revisar).

library/admin.py, línea 116

Clase ReportAdmin

Denuncias de reseñas y comentarios. Desde cada una se va a lo denunciado (en la web o en el admin) para borrarlo si hace falta, y después se marca como revisada.

```
def has_add_permission(self, request)
```

Las denuncias solo las crean los usuarios desde la web.

Pantallas: Panel de administración (denuncias: no se crean a mano).

library/admin.py, línea 141

```
@admin.display(description='qué es')
def kind(self, report)
```

Qué se ha denunciado: «reseña» o «comentario».

Pantallas: Panel de administración (columna de qué se ha denunciado).

library/admin.py, línea 149

```
@admin.display(description='denunciado')
def target_link(self, report)
```

Enlaces a lo denunciado: su página en la web y su ficha en el admin; «borrado» si ya no existe.

Pantallas: Panel de administración (enlaces a lo denunciado).

library/admin.py, línea 157

```
@admin.action(description='Marcar como revisadas')
def mark_resolved(self, request, queryset)
```

Marca las denuncias elegidas como revisadas ahora.

Pantallas: Panel de administración (acción de marcar denuncias como revisadas).

library/admin.py, línea 173

```
@admin.action(description='Volver a dejar pendientes')
def mark_pending(self, request, queryset)
```

Vuelve a dejar pendientes las denuncias elegidas.

Pantallas: Panel de administración (acción de volver a dejarlas pendientes).

library/admin.py, línea 182

library/apps.py

Configuración de la app library (biblioteca, reseñas y lista de deseos): al arrancar Django conecta sus señales (library/signals.py).

Clase LibraryConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (borran del disco las imágenes de las reseñas que se quitan).

Pantallas: ninguna (se ejecuta al arrancar Django y conecta las señales de library/signals.py).

library/apps.py, línea 11

library/autofill.py

Plataforma y fechas de una partida sacadas de las importaciones, para que una reseña nueva se rellene sola.

Cada título importado (accounts.ImportedTitle) guarda sus minutos, la última partida, la primera (solo la da PlayStation) y las plataformas en las que se jugó (PS4 o PS5 en PlayStation, los dispositivos de Xbox y el PC en Steam, que además dice cuántos minutos fueron en una Steam Deck). Con eso el editor elige la plataforma en la que

más ha jugado el usuario y propone las fechas, diciendo de dónde sale cada dato para que se pueda cambiar.
Pantallas de este archivo: Escribir reseña

```
def _platforms(row)
```

Plataformas (nombres de IGDB) en las que se jugó un título importado: las guardadas o, si no hay, las que se deducen.

Pantallas: Escribir reseña (la plataforma de la partida, rellenada con lo importado).

library/autofill.py, línea 23

```
def _played_on(row)
```

Dónde se jugó un título importado y cuántos minutos en cada sitio: [(nombre de plataforma, minutos)].

Lo jugado en una Steam Deck se separa de lo del PC. En lo demás no se sabe cómo se reparten las horas entre sus plataformas (una consola y su sucesora, por ejemplo) y cada una cuenta con todos los minutos.

Pantallas: Escribir reseña (la plataforma y las horas de la partida, rellenadas con lo importado).

library/autofill.py, línea 37

```
def _duration(minutes)
```

Tiempo jugado corto para las notas: «12 h» o «40 min».

Pantallas: Escribir reseña (tiempo jugado de las notas de la partida).

library/autofill.py, línea 53

```
def play_details(user, game, entry)
```

Plataforma y fechas de la partida de user en game según sus importaciones, para el editor de reseñas.

Devuelve {"initial": {campo: valor}, "notes": {campo: de dónde sale}}: la plataforma del juego en la que más minutos lleva (la Steam Deck cuenta como una más en los juegos de PC); «Lo empecé el» con su primera partida (si alguna importación la da) y, si tiene el juego como Completado, «Lo terminé el» con su última partida. Vacíos si el juego no viene de ninguna importación o sus plataformas no son las del juego.

Pantallas: Escribir reseña (plataforma y fechas de la partida, rellenadas con lo importado).

library/autofill.py, línea 61

library/backlog.py

El repaso de la biblioteca (Repasar la biblioteca) y lo que cuesta el pendiente.

Las importaciones de Steam, PlayStation y Xbox nunca dan un juego por terminado: lo que se jugó hace tiempo entra como «Jugado», sin saber si se terminó, y lo que se jugó hace poco, como «Jugando», aunque luego se deje. El repaso los va sacando de uno en uno (queue) con lo que NextPlay ya sabe de cada uno (hint: horas, última partida y logros) para que se contesten con un botón (answer). Lo repasado no se vuelve a preguntar (LibraryEntry.reviewed_at).

value es lo que cuesta hoy el pendiente de alguien, contando solo lo que tiene precio.

Pantallas: Repasar la biblioteca, Perfil (aviso de lo que queda por repasar; lo que cuesta su pendiente).
Pantallas de este archivo: Inicio, Perfil, Repasar la biblioteca, Retos, Página del reto, Avisos

```
def last_played(entry)
```

La última partida de entry en cualquier plataforma (Steam, PlayStation o Xbox), o None si no se sabe.

Pantallas: Repasar la biblioteca (cuándo lo jugaste por última vez y la fecha con la que se apunta terminado).

library/backlog.py, línea 43

```
def is_stale(entry, now=None)
```

Indica si entry está en «Jugando» y no se toca desde hace STALE_DAYS días: ni se juega (las importaciones lo cambiarían) ni se cambia a mano ni se repasa (todo eso mueve updated_at).

Pantallas: Repasar la biblioteca (los de «Jugando» que se preguntan), Perfil (aviso de lo que queda por repasar).

library/backlog.py, línea 53

```
def pending(entries)
```

Las de entries que pregunta el repaso: (las de «Jugando» que no se tocan desde hace STALE_DAYS días, las de «Jugado» que nunca se han repasado), sin ordenar. No consulta nada: sirve con la biblioteca ya cargada.

Pantallas: Repasar la biblioteca, Perfil (aviso de lo que queda por repasar).

library/backlog.py, línea 63

```
def queue(user)
```

Lo que le queda por repasar a user, en el orden en que se pregunta: primero los de «Jugando» que llevan más tiempo sin tocarse; después los de «Jugado», primero los que tienen todos sus logros o el platino (los que mejor se contestan) y luego los de más horas.

Pantallas: Repasar la biblioteca (el siguiente juego y cuántos quedan).

library/backlog.py, línea 75

```
def _complete_games(user, game_ids)
```

Ids de los juegos de game_ids de los que user tiene todos los logros o el platino en alguna plataforma.

Pantallas: Repasar la biblioteca (los que se preguntan primero).

library/backlog.py, línea 92

```
def _number(value)
```

Un número como se escribe en España, sin decimales de sobra: 12,5 o 3.

Pantallas: Repasar la biblioteca (las horas de cada juego).

library/backlog.py, línea 103

```
def hint(entry)
```

Lo que NextPlay sabe de entry para contestarlo en el repaso: {"lines": frases con sus horas, su última partida y sus logros, "suggest": la respuesta que propone (clave de ANSWERS) o None, "finished_on": la fecha que se apuntará si se marca terminado, o None si sería hoy, "finished_from": de dónde sale esa fecha}.

Con todos los logros o el platino propone «terminado», con la fecha de su último logro si se conoce (los de Steam se guardan uno a uno) o, si no, la de su última partida.

Pantallas: Repasar la biblioteca (la tarjeta de cada juego).

library/backlog.py, línea 111

```
def answer(entry, choice)
```

Apunta la respuesta del repaso (clave de ANSWERS) para entry: le pone el estado que toca y lo da por repasado.

Si pasa a «Completado», la fecha es la de hint (su último logro o su última partida; hoy si no se sabe ninguna), para que no cuente en un reto que no le toca, y luego se mira si con eso consigue alguno. Lo que termina o deja va a su actividad, todo junto (social.activity.library_tidied), y no como una entrada por juego. Devuelve el estado con el que queda.

Pantallas: Repasar la biblioteca (botones de cada tarjeta), Perfil (biblioteca y pestaña Actividad), Inicio (feed: «ordenó su biblioteca»), Retos, Página del reto (progreso), Avisos (reto conseguido).

library/backlog.py, línea 157

```
def today_summary(user)
```

Lo que user ha repasado hoy, según cómo ha quedado cada juego: {"total", "completed", "dropped", "playing", "backlog", "played"} (en «played», los que ha saltado y siguen en «Jugado»).

Pantallas: Repasar la biblioteca (lo que llevas hoy y el resumen al acabar).

library/backlog.py, línea 184

```
def value(user)
```

Lo que cuesta hoy el pendiente de user en sus tiendas (todas si no ha elegido): {"count": juegos en «Pendiente», "priced": cuántos tienen precio, "total": lo que suman, "total_text": eso escrito}. Los que no vende nadie no cuentan: no se inventa lo que falta.

Pantallas: Perfil (pestaña Pendiente: lo que cuesta).

library/backlog.py, línea 198

library/forms.py

Formularios de la biblioteca: el de Escribir reseña (ReviewForm, con «Lo mejor» y «Lo peor», y ReviewImagesForm, sus imágenes), el de comentar una reseña (CommentForm, en la Página de la reseña) y el de denunciar (ReportForm, en los formularios de denuncia de reseñas, comentarios, publicaciones y listas).
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Página de la publicación, Escribir reseña, Página de la reseña, Página de la lista, Página del reto, Panel de administración

```
def stars_label(value)
```

Texto de una nota de 1 a 10 en estrellas, como se lee en España: «3,5 estrellas», «1 estrella».

Pantallas: Catálogo, Inicio, Ficha del juego, Perfil, Página de la reseña, Página de la lista, Página del reto, Escribir reseña (la nota en estrellas).

library/forms.py, línea 18

Clase PointsWidget

MAX_POINTS casillas de texto para los puntos de «Lo mejor» o «Lo peor» (se llaman <campo>_0, <campo>_1...).

```
def __init__(self, attrs=None)
```

Una casilla por punto, con el largo máximo de un punto.

Pantallas: Escribir reseña («Lo mejor» y «Lo peor»).

library/forms.py, línea 32

```
def decompress(self, value)
```

La lista guardada repartida en las casillas; las que sobran, vacías.

Pantallas: Escribir reseña («Lo mejor» y «Lo peor»).

library/forms.py, línea 39

Clase PointsField

Lista de puntos de «Lo mejor» o «Lo peor»: hasta MAX_POINTS frases de hasta POINT_MAX_LENGTH caracteres.

```
def __init__(self, **kwargs)
```

Un CharField opcional por casilla; la lista entera también es opcional.

Pantallas: Escribir reseña («Lo mejor» y «Lo peor»).

library/forms.py, línea 53

```
def compress(self, data_list)
```

Los puntos escritos, en orden, sin espacios de sobra, vacíos ni repetidos.

Pantallas: Escribir reseña («Lo mejor» y «Lo peor»).

library/forms.py, línea 61

Clase ReviewForm

Reseña de un juego: nota en medias estrellas, texto con formato, lo mejor y lo peor, spoilers, plataforma, fechas de la partida y si es una rejugada. La plataforma solo puede ser una de las del juego.

```
def __init__(self, *args, game, **kwargs)
```

Limita la plataforma a las del juego (y la Steam Deck si sale en PC, ver Game.review_platforms).

Pantallas: Escribir reseña (plataformas que se pueden elegir).

library/forms.py, línea 118

```
def clean_text(self)
```

Quita los espacios del principio y del final: una reseña en blanco no vale.

Pantallas: Escribir reseña (el texto).

library/forms.py, línea 127

```
def point_slots(self, name)
```

Casillas de «Lo mejor» (name="pros") o «Lo peor» ("cons") para pintarlas una a una: [(nombre, id, valor)].

Llevan lo que se acaba de enviar o, si no, lo guardado.

Pantallas: Escribir reseña (casillas de «Lo mejor» y «Lo peor»).

library/forms.py, línea 137

```
@property
def pro_slots(self)
```

Casillas de «Lo mejor» (ver point_slots).

Pantallas: Escribir reseña (casillas de «Lo mejor»).

library/forms.py, línea 149

```
@property
def con_slots(self)
```

Casillas de «Lo peor» (ver point_slots).

Pantallas: Escribir reseña (casillas de «Lo peor»).

library/forms.py, línea 157

```
def clean(self)
```

Comprueba que las fechas no sean del futuro y que no se termine antes de empezar.

Pantallas: Escribir reseña (fechas de la partida).

library/forms.py, línea 164

Clase MultipleImageInput

Selector de archivos que deja elegir varias imágenes a la vez.

Clase MultipleImageField

Varias imágenes subidas en el mismo campo: devuelve la lista de archivos (vacía si no se sube ninguna).

```
def __init__(self, **kwargs)
```

Con el selector múltiple y opcional.

Pantallas: Escribir reseña (imágenes).

library/forms.py, línea 189

```
def clean(self, data, initial=None)
```

Comprueba cada archivo como lo haría un FileField y los devuelve en una lista.

Pantallas: Escribir reseña (imágenes).

library/forms.py, línea 197

Clase ReviewImagesForm

Imágenes de una reseña, como mucho MAX_IMAGES: las que ya tiene (se pueden marcar como spoiler o quitar), las que se suben (pasadas a WebP y sin EXIF por accounts.images) y las capturas y artes oficiales del juego, que se eligen sin subir nada. Las nuevas van detrás de las que había: primero las subidas y después las oficiales.

```
def __init__(self, data=None, files=None, *, review, official)
```

review es la reseña que se edita y official, las imágenes oficiales de su juego (games.media.official_images).

En el elector solo salen las oficiales que la reseña aún no tiene.

Pantallas: Escribir reseña (imágenes subidas y de la ficha).

library/forms.py, línea 220

```
@property
def free_slots(self)
```

Cuántas imágenes más caben en la reseña sin quitar ninguna.

Pantallas: Escribir reseña (cuántas imágenes más caben).

library/forms.py, línea 237

```
def clean(self)
```

Comprueba que no pasen de MAX_IMAGES entre las que se quedan y las nuevas y prepara las subidas.

Cada subida se convierte a WebP en dos tamaños (la grande para el visor y la miniatura para la reseña); las que no son una imagen válida, pesan demasiado o tienen demasiados píxeles dan un error con su nombre.

Pantallas: Escribir reseña (imágenes).

library/forms.py, línea 244

```
def save(self, review)
```

Aplica los cambios a las imágenes de review, que ya debe estar guardada.

Quita las marcadas (sus archivos se borran con ellas, ver library.signals), pone o quita el spoiler de las demás y añade las nuevas detrás.

Pantallas: Escribir reseña (imágenes), Página de la reseña (imágenes), Ficha del juego (imágenes de las reseñas).

library/forms.py, línea 267

Clase ReportForm

Denuncia de una reseña o un comentario: el motivo y, si se quiere, algo más para el administrador.

```
def clean_detail(self)
```

Los detalles sin espacios de sobra al principio ni al final.

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página de la publicación, Página de la lista (formularios de denuncia), Panel de administración (denuncias).

library/forms.py, línea 308

Clase CommentForm

Comentario en una reseña, con el mismo formato sencillo que las reseñas.

El campo quita solo los espacios del principio y del final, así que uno en blanco da el error de «required».

library/markup.py

Formato sencillo y seguro de las reseñas y los comentarios.

Se escribe como en muchos foros: **negrita**, *cursiva*, ||spoiler|| y [texto](https://enlace); una línea en blanco separa párrafos. Primero se escapa todo el HTML y después se cambian solo esas marcas, así que no se puede colar ninguna etiqueta ni script. Los enlaces solo pueden ser http o https y llevan rel="nofollow ugc noopener" para que no sirvan de spam.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Página de la publicación, Escribir reseña, Página de la reseña, Página de la lista, Página del reto

```
def _inline(text)
```

Un párrafo ya escapado con sus marcas cambiadas por HTML.

Pantallas: Escribir reseña (vista previa), Página de la reseña, Ficha del juego, Perfil, Inicio, Página de la publicación, Página de la lista, Página del reto (el texto con formato de reseñas, comentarios, publicaciones, listas y retos).

library/markup.py, línea 23

```
def render(text)
```

HTML seguro de un texto con formato: un <p> por párrafo,
 en cada salto de línea y las marcas cambiadas.

Pantallas: Escribir reseña (vista previa), Página de la reseña, Ficha del juego, Perfil, Inicio, Página de la publicación, Página de la lista, Página del reto (el texto con formato de reseñas, comentarios, publicaciones, listas y retos).

library/markup.py, línea 53

```
def render_inline(text)
```

HTML seguro de un texto corto de una sola línea (un punto de «Lo mejor» o «Lo peor»), sin párrafo alrededor.

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página de la lista («Lo mejor», «Lo peor» y otros textos cortos con formato).

library/markup.py, línea 65

```
def plain(text)
```

El texto sin las marcas de formato y con los spoilers tapados («[spoiler]»), en una sola línea.

Sirve para resúmenes, la portada y la descripción de la página.

Pantallas: Ficha del juego, Perfil, Inicio, Página de la reseña, Página de la publicación, Página de la lista (resúmenes sin formato y con los spoilers tapados).

library/markup.py, línea 75

library/models.py

Modelos de la biblioteca: cada juego de la biblioteca de un usuario (estado, nota y horas de cada plataforma), sus sesiones de juego (para los retos de horas), las reseñas con sus «me gusta», comentarios e imágenes, las denuncias y la lista de deseos. Se ven en la Ficha del juego, el Perfil, la Página de la reseña, la Lista de deseos y, con sus portadas y estados, en casi todas las pantallas que enseñan juegos.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Publicar, Página de la publicación, Escribir reseña, Página de la reseña, Lista de deseos, Página de la lista, Nueva tier list, Retos, Página del reto, Avisos, Editar perfil, Juegos importados, Panel de administración

Clase LibraryEntry

```
def __str__(self)
```

Texto de la entrada en el admin: «usuario · juego · estado».

Pantallas: Panel de administración.

library/models.py, línea 89

```
@classmethod
def imported_status(cls, minutes, last_played)
```

Estado de un juego que llega de una importación (Steam, PlayStation o Xbox) según cuánto y cuándo se jugó.

Sin minutos ni partida: «Pendiente». Jugado en los últimos IMPORT_RECENT_DAYS días: «Jugando». Si no: «Jugado».

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas: el estado con el que llega cada juego importado), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

library/models.py, línea 97

```
def update_import(self, platform, minutes, last_played)
```

Guarda los minutos y la última partida de una plataforma ("steam", "psn" o "xbox") en una entrada existente.

Conserva el estado que puso el usuario, salvo «Pendiente» cuando ya hay horas jugadas: entonces lo cambia a «Jugando» o «Jugado». Devuelve True si ha cambiado algo.

Pantallas: Editar perfil (Cuentas vinculadas), Perfil (modal «Vincular cuentas» de su dueño; biblioteca y horas), Ficha del juego (tus horas), Juegos importados (los que no se encuentran), Inicio (feed: la importación), Retos (progreso), Página del reto (progreso).

library/models.py, línea 113

```
@property
def playtime_minutes(self)
```

Minutos jugados sumando Steam, PlayStation y Xbox, o None si no viene de ninguna.

Pantallas: Perfil (horas de la biblioteca), Ficha del juego (tus horas), Escribir reseña (horas de la partida), Catálogo, Inicio, Página de la lista, Página del reto (horas en las portadas).

library/models.py, línea 138

```
@property
def playtime_hours(self)
```

Horas jugadas (Steam + PlayStation + Xbox) redondeadas a una décima, o None si no hay datos.

Pantallas: Perfil (horas de la biblioteca), Ficha del juego (tus horas), Escribir reseña (horas de la partida), Catálogo, Inicio, Página de la lista, Página del reto (horas en las portadas).

library/models.py, línea 150

```
@property
def hours_by_platform(self)
```

Horas de cada plataforma de la que vienen, de la que tiene más a la que menos: [{"key", "name", "hours", "last_played"}]. key es la clave de HOURS_SOURCES («steam», «psn» o «xbox»), que da su logo (library/_platform_mark.html). Solo salen las que llegan a una décima de hora: abrir un juego no es jugarlo.

Pantallas: Perfil (horas de cada plataforma), Ficha del juego (tus horas en cada plataforma), Catálogo, Inicio, Página de la lista, Página del reto (horas en las portadas).

library/models.py, línea 160

```
@property
def playtime_sources(self)
```

Plataformas de las que vienen las horas jugadas, como claves de HOURS_SOURCES: p. ej. ["steam", "psn"].

Pantallas: Escribir reseña (de dónde salen las horas al publicarla), Página de la reseña (sello de horas verificadas).

library/models.py, línea 176

```
@cached_property
def achievements(self)
```

Cómo va con los logros de este juego en cada plataforma de la que se han importado (lo que da accounts.achievements.progress_by_game), o una lista vacía si no se conocen.

Pantallas: Ficha del juego (tus logros en cada plataforma).

library/models.py, línea 185

```
@property
def stars(self)
```

Nota en estrellas (de 0,5 a 5), o None sin nota.

Pantallas: Perfil (nota media de las cifras de la cabecera).

library/models.py, línea 194

Clase PlaySession

Lo que ha jugado un usuario a un juego en una plataforma un día: lo que ha subido su tiempo entre dos sincronizaciones, apuntado el día de su última partida (varias el mismo día se suman).

Lo apunta la importación (accounts.matching.update_library) solo si ya se conocían las horas de esa plataforma: lo que trae la primera importación es de antes y no se sabe cuándo se jugó. Es lo que miden los retos de horas y, más adelante, el diario de partidas.

```
def __str__(self)
```

Texto en el admin: «usuario · juego · día · minutos».

Pantallas: Panel de administración.

library/models.py, línea 233

```
@classmethod
def add(cls, user, game_id, platform, day, minutes)
```

Suma minutes a lo que user ha jugado a game_id en platform el día day (la crea si es la primera de ese día).

Pantallas: Editar perfil (Cuentas vinculadas: al importar horas), Perfil (modal «Vincular cuentas» de su dueño), Retos (progreso de los de horas), Página del reto (progreso), Nueva tier list (plantillas por horas jugadas).

library/models.py, línea 241

Clase ReviewQuerySet

```
def published(self)
```

Reseñas publicadas, sin los borradores.

Pantallas: Ficha del juego (reseñas: sin los borradores), Perfil (pestaña Reseñas y reseña destacada), Página de la reseña, Inicio (feed).

library/models.py, línea 264

```
def visible_to(self, viewer)
```

Reseñas publicadas que viewer puede leer, de la más reciente a la más antigua, con sus «me gusta» (like_count) y comentarios (comment_count) contados y sus imágenes ya cargadas.

Las de perfiles privados solo las ve su autor, igual que su biblioteca (Profile.visible_filter).

Pantallas: Ficha del juego (reseñas: solo las que puede leer quien mira), Perfil (pestaña Reseñas y reseña destacada), Página de la reseña, Inicio (feed).

library/models.py, línea 272

Clase Review

Reseña de un juego. Cada usuario puede escribir varias del mismo juego (una por partida, por ejemplo al rejugarlo); se ven en la ficha, en su perfil, en la portada y en su propia página.

Al publicarla se guardan las horas que llevaba jugadas (de las importaciones de Steam, PlayStation y Xbox) y el estado del juego en su biblioteca, para enseñar «escrita a las 12 h», filtrar las de quien lo ha terminado y plegar las que van más adelante que quien las lee. La nota va de 1 a 10 medias estrellas. Los borradores solo los ve su autor.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

Pantallas: Panel de administración.

library/models.py, línea 340

```
def get_absolute_url(self)
```

URL de la página de la reseña: /u/<usuario>/resenas/<id>/.

Pantallas: Ficha del juego (reseñas), Perfil (pestaña Reseñas y reseña destacada), Página de la reseña, Inicio (feed), Avisos (los enlaces a cada reseña).

library/models.py, línea 347

```
@property
def stars(self)
```

Nota en estrellas (de 0,5 a 5), o None sin nota.

Pantallas: ninguna (ahora no la usa nadie: las estrellas se pintan desde rating con library/_stars.html).

library/models.py, línea 356

```
@property
def finished(self)
```

Indica si el autor lo tenía como «Completado» al publicarla.

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página del reto (si lo terminó al publicarla).

library/models.py, línea 364

```
@property
def status_note(self)
```

Cómo iba el autor con el juego al publicarla, para la reseña: «Lo terminó», «Aún jugándolo»...; vacía si no se sabe.

Pantallas: Ficha del juego, Perfil, Página de la reseña (cómo iba el autor con el juego al publicarla).

library/models.py, línea 372

```
@property
def hours_source_names(self)
```

Nombres de las plataformas de las que salen las horas: «Steam» o «Steam y PlayStation».

Pantallas: Ficha del juego, Perfil, Página de la reseña (de dónde salen las horas).

library/models.py, línea 380

```
@property
def hours_source_logos(self)
```

Plataformas de las que salen las horas como [(nombre, plantilla de su logo)], para el sello de horas verificadas.

Pantallas: Ficha del juego, Perfil, Página de la reseña (sello de horas verificadas).

library/models.py, línea 389

```
def hours_since(self, entry)
```

Horas que ha jugado el autor desde que la publicó, si son bastantes para proponerle actualizarla; si no, None.

entry es su entrada de biblioteca del juego. Son bastantes si lleva al menos el doble que al escribirla y como mínimo UPDATE_MIN_EXTRA_HOURS más («Llevas 40 h más: ¿la actualizas?»). Los borradores y las reseñas escritas sin horas importadas no cuentan.

Pantallas: Página de la reseña (propuesta de actualizarla), Avisos (reseñas que se han quedado atrás).

library/models.py, línea 396

```
def snapshot(self, entry)
```

Apunta el estado y las horas que tiene ahora la entrada de biblioteca entry (o nada si no hay).

Pantallas: Escribir reseña (al publicar se guardan el estado y las horas), Página de la reseña, Ficha del juego, Perfil (datos de la partida).

library/models.py, línea 411

Clase ReviewLike

«Me gusta» de un usuario en una reseña (uno por usuario y reseña).

```
def __str__(self)
```

Texto en el admin: «usuario → reseña».

Pantallas: Panel de administración.

library/models.py, línea 435

Clase ReviewComment

Comentario en una reseña. Lo pueden borrar quien lo escribió y el autor de la reseña.

```
def __str__(self)
```

Texto en el admin: «usuario en reseña: comienzo del comentario».

Pantallas: Panel de administración.

library/models.py, línea 458

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su reseña.

Pantallas: Avisos (enlace a un comentario de una reseña), Inicio (feed).

library/models.py, línea 465

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el autor de la reseña.

Pantallas: Página de la reseña (botón «Borrar» de los comentarios).

library/models.py, línea 472

```
def attached_image_upload_to(instance, filename)
```

Ruta de una imagen adjunta: la carpeta la decide cada modelo (upload_folder) y el nombre aleatorio, accounts.images.

Pantallas: Escribir reseña, Publicar, Inicio (dónde se guardan las imágenes subidas a reseñas y publicaciones).

library/models.py, línea 480

Clase AttachedImage

Imagen adjunta a algo que escribe un usuario: de momento, las reseñas; más adelante, también las publicaciones.

O la ha subido él (image, pasada a WebP y sin EXIF por accounts.images, con su miniatura en thumb) o es una captura o un arte oficial del juego de IGDB (igdb_image_id), que no se sube. Cada una se puede marcar como spoiler: entonces va borrosa hasta que se pulsa.

```
def upload_folder(self)
```

Carpeta de MEDIA_ROOT en la que se guardan las imágenes subidas; la pone cada modelo.

Pantallas: Escribir reseña, Publicar, Inicio (carpeta de las imágenes subidas).

library/models.py, línea 507

```
@property
def is_official(self)
```

Indica si es una imagen oficial del juego (de IGDB) y no una subida por el usuario.

Pantallas: Escribir reseña, Página de la reseña, Ficha del juego, Perfil, Inicio, Página de la publicación (imágenes de la ficha frente a las subidas).

library/models.py, línea 515

```
@property
def url(self)
```

URL de la imagen grande, para el visor a pantalla completa.

Pantallas: Página de la reseña, Página de la publicación, Ficha del juego, Perfil, Inicio (el visor a pantalla completa de las imágenes).

library/models.py, línea 524

```
@property
def thumb_url(self)
```

URL de la miniatura: la de IGDB o la reducida al subirla (la grande si no la hay).

Pantallas: Escribir reseña, Página de la reseña, Ficha del juego, Perfil, Inicio, Página de la publicación (miniaturas de las imágenes).

library/models.py, línea 533

```
def delete_files(self)
```

Borra del disco la imagen subida y su miniatura (las oficiales no tienen archivo).

Pantallas: Escribir reseña, Página de la publicación (al quitar una imagen o borrar lo que la lleva), Panel de administración.

library/models.py, línea 543

Clase ReviewImage

Imagen de una reseña (como mucho MAX_IMAGES por reseña). Las subidas se guardan en resenas/<id del autor>/.

```
def __str__(self)
```

Texto en el admin: «reseña · imagen n».

Pantallas: Panel de administración.

library/models.py, línea 563

```
def upload_folder(self)
```

Las imágenes de las reseñas van en resenas/<id del autor>.

Pantallas: Escribir reseña (carpeta de las imágenes de las reseñas).

library/models.py, línea 570

Clase Report

Denuncia de un usuario sobre algo que ha escrito otro (una reseña o un comentario) para que la revise un administrador.

Va por contenttypes para que sirva también para lo que se añada después (publicaciones, grupos...). Hay una por usuario y cosa denunciada: volver a denunciarla cambia el motivo y la deja otra vez pendiente.

```
def __str__(self)
```

Texto en el admin: «denunciante → lo denunciado: motivo».

Pantallas: Panel de administración.

library/models.py, línea 612

```
@property
def target_kind(self)
```

Qué se ha denunciado, en palabras: «reseña» o «comentario» (el nombre del modelo de lo denunciado).

Pantallas: Panel de administración (columna de qué se ha denunciado).

library/models.py, línea 620

Clase WishlistItem

Juego que el usuario quiere («Lo quiero»), con un precio por debajo del cual avisarle (opcional).

Los avisos los crea library.wishlist.check_alerts después de actualizar los precios; también avisa cuando el juego está rebajado a su mínimo histórico, aunque no haya precio objetivo.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

Pantallas: Panel de administración.

library/models.py, línea 658

```
@property
def on_steam(self)
```

Indica si el juego está también en la lista de deseos de Steam del usuario (según la última importación).

Pantallas: Lista de deseos (sello de la lista de Steam).

library/models.py, línea 666

library/points.py

«Lo mejor» y «Lo peor» de las reseñas de un juego, sumados: las sugerencias del editor y lo que más se repite.

El editor sugiere los puntos que ya han escrito otros en el mismo juego. Si la gente los reutiliza, salen las mismas frases y la ficha puede enseñar «Lo que más destacan» con cuántas reseñas lo dicen. Dos puntos cuentan como el mismo si solo cambian las mayúsculas, los espacios o la puntuación del principio y del final.
Pantallas de este archivo: Ficha del juego, Escribir reseña

```
def key(point)
```

Forma de comparar un punto: sin formato, en minúsculas, con los espacios simplificados y sin puntuación en los extremos.

Pantallas: Ficha del juego (lo que más destacan y lo que menos gusta), Escribir reseña (sugerencias de «Lo mejor» y «Lo peor»).

library/points.py, línea 24

```
def _tally(rows)
```

Cuenta los puntos de rows ([(pros, cons)]): {"pros": [(punto, veces)], "cons": [...]}, de más a menos repetido.

Cada reseña cuenta una vez por punto; de cada punto se enseña cómo lo escribió el primero.

Pantallas: Ficha del juego (lo que más destacan y lo que menos gusta), Escribir reseña (sugerencias de «Lo mejor» y «Lo peor»).

library/points.py, línea 33

```
def _visible(game, viewer)
```

Reseñas publicadas del juego que viewer puede leer (las de perfiles públicos y las suyas), de la más antigua a la más nueva: así cada punto se enseña como lo escribió el primero.

Pantallas: Ficha del juego (lo que más destacan y lo que menos gusta), Escribir reseña (sugerencias de «Lo mejor» y «Lo peor»).

library/points.py, línea 51

```
def suggestions(game, viewer)
```

Puntos que más han escrito otros en las reseñas del juego, para sugerirlos en el editor: {"pros": [...], "cons": [...]}.

Pantallas: Escribir reseña (sugerencias de «Lo mejor» y «Lo peor»).

library/points.py, línea 61

```
def consensus(game, viewer)
```

Lo que más destacan y lo que menos gusta del juego: {"pros": [(punto, reseñas)], "cons": [...]}.

Solo los puntos que dicen al menos CONSENSUS_MINIMUM reseñas, como mucho CONSENSUS_SHOWN de cada lista.

Pantallas: Ficha del juego (lo que más destacan y lo que menos gusta).

library/points.py, línea 73

library/reminders.py

Avisos a los autores de reseñas que se han quedado atrás: «Llevas 40 h más en Hades: ¿actualizas tu reseña?».

Cada reseña guarda las horas que llevaba su autor al publicarla (Review.hours). Cuando una importación de Steam, PlayStation o Xbox le sube mucho las horas de ese juego (ver Review.hours_since), se le avisa en la campana para que la actualice. Solo cuenta la última reseña publicada de cada juego: las anteriores son de otras partidas y se quedan como estaban.
Pantallas de este archivo: Perfil, Página de la reseña, Avisos, Editar perfil

```
def latest_reviews(user, games=None)
```

Última reseña publicada de user de cada juego (de esos juegos, si se dan), como {id del juego: reseña}.

Pantallas: Página de la reseña (propuesta de actualizarla), Avisos (reseñas que se han quedado atrás).

library/reminders.py, línea 17

```
def remind_outdated_reviews(user)
```

Avisa a user en la campana de sus reseñas que se han quedado atrás y devuelve cuántos avisos ha creado.

Un aviso por reseña hasta que la vuelva a guardar: si sigue jugando sin tocarla, no se repite. Si ha apagado estos avisos en Editar perfil, no se crea ninguno (accounts.notices).

Pantallas: Avisos (reseñas que se han quedado atrás), todas (el número de la campana), Editar perfil (Cuentas vinculadas: se mira al importar horas), Perfil (modal «Vincular cuentas» de su dueño).

library/reminders.py, línea 31

library/signals.py

Señales de la biblioteca: borran del disco los archivos de las imágenes que se quitan de una reseña (desde Escribir reseña o al borrarla).
Pantallas de este archivo: Escribir reseña, Panel de administración

```
@receiver(post_delete, sender=ReviewImage)
def delete_review_image_files(sender, instance, **kwargs)
```

Borra del disco la imagen subida y su miniatura al quitarla de una reseña o al borrar la reseña o la cuenta.

Pantallas: Escribir reseña (al quitar una imagen o borrar la reseña), Panel de administración.

library/signals.py, línea 12

library/templatetags/reviews.py

Filtros de plantilla de las reseñas: el formato del texto, las notas en estrellas, los permisos de los comentarios y los motivos de las denuncias.
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Página de la publicación, Escribir reseña, Página de la reseña, Página de la lista, Página del reto, Panel de administración

```
@register.simple_tag
def report_reasons()
```

Motivos que se pueden elegir al denunciar una reseña o un comentario: [(valor, texto)].

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página de la publicación, Página de la lista (formularios de denuncia), Panel de administración (denuncias).

library/templatetags/reviews.py, línea 14

```
@register.filter
def review_html(text)
```

HTML seguro de una reseña o un comentario con su formato (ver library.markup).

Pantallas: Escribir reseña (vista previa), Página de la reseña, Ficha del juego, Perfil, Inicio, Página de la publicación, Página de la lista, Página del reto (el texto con formato de reseñas, comentarios, publicaciones, listas y retos).

library/templatetags/reviews.py, línea 24

```
@register.filter
def review_inline(text)
```

HTML seguro de un punto de «Lo mejor» o «Lo peor» con su formato, sin párrafo.

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página de la lista («Lo mejor», «Lo peor» y otros textos cortos con formato).

library/templatetags/reviews.py, línea 35

```
@register.filter
def review_plain(text)
```

Una reseña sin formato y con los spoilers tapados, para resúmenes.

Pantallas: Ficha del juego, Perfil, Inicio, Página de la reseña, Página de la publicación, Página de la lista (resúmenes sin formato y con los spoilers tapados).

library/templatetags/reviews.py, línea 45

```
@register.filter
def stars_label(rating)
```

Nota de 1 a 10 escrita en estrellas: «3,5 estrellas»; vacía sin nota.

Pantallas: Catálogo, Inicio, Ficha del juego, Perfil, Página de la reseña, Página de la lista, Página del reto, Escribir reseña (la nota en estrellas).

library/templatetags/reviews.py, línea 55

```
@register.filter
def can_delete(comment, user)
```

Indica si user puede borrar el comentario (es suyo o la reseña es suya).

Pantallas: Página de la reseña (botón «Borrar» de cada comentario), Página de la publicación, Página de la lista (comentarios).

library/templatetags/reviews.py, línea 65

library/views/comentarios_y_denuncias.py

Comentarios y denuncias, que comparten las reseñas, las publicaciones (social) y las listas (lists): las variables de library/_comments.html y guardar una denuncia avisando a los administradores.
Pantallas de este archivo: Ficha del juego, Perfil, Página de la publicación, Página de la reseña, Página de la lista, Avisos, Panel de administración

```
def comment_thread(parent, comments, viewer, *, comment_url, delete_url_name,
report_url_name, count_id)
```

Variables de library/_comments.html para los comentarios de parent: una reseña o una publicación (social.Post).

comments son sus comentarios (con su parent ya cargado, para saber quién puede borrarlos); salen con sus autores, en orden, y en cada uno si viewer ya lo ha denunciado (reported). comment_url es la dirección del formulario para comentar; delete_url_name y report_url_name, los nombres de las URL para borrar y denunciar un comentario; y count_id, el id del contador de comentarios que HTMX cambia al comentar o borrar.

Pantallas: Página de la reseña, Página de la publicación, Página de la lista (comentarios).

library/views/comentarios_y_denuncias.py, línea 26

```
def _report(request, target, kind, game, back_url)
```

Guarda la denuncia del usuario sobre target (una reseña, un comentario, una publicación o una lista, según kind) y avisa a los administradores.

Si ya lo había denunciado, cambia el motivo y la vuelve a dejar pendiente. Con HTMX devuelve el botón de denunciar (library/_report.html) ya como «Gracias» o, si falta el motivo, abierto con el error; si no, vuelve a back_url con un mensaje.

Pantallas: Ficha del juego, Perfil, Página de la reseña, Página de la publicación, Página de la lista (formularios de denuncia), Panel de administración (denuncias), Avisos (aviso a los administradores).

library/views/comentarios_y_denuncias.py, línea 58

```
def _notify_staff(report, what, game)
```

Avisa en la campana a los administradores de la denuncia de what («una reseña de ana») en game, sin repetirlo mientras tengan uno igual sin leer.

game puede ser None (una publicación que no habla de ningún juego). El aviso lleva a la denuncia en el admin, desde donde se puede ir a lo denunciado y borrarlo.

Pantallas: Avisos (de los administradores), Panel de administración (denuncias).

library/views/comentarios_y_denuncias.py, línea 86

library/views/escribir_resena.py

Vistas de Escribir reseña (<juego>/resenas/nueva/ y resenas/<id>/editar/): el editor con su vista previa, publicar o guardar el borrador y borrar la reseña.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Escribir reseña, Página de la reseña

```
@login_required
def review_new(request, game_id)
```

Editor de una reseña nueva de un juego de la biblioteca del usuario.

Empieza con la nota que ya le había puesto y, si ya tiene alguna reseña publicada del juego, marcada como rejugada. Si el juego no está en su biblioteca, vuelve a la ficha avisando.

Pantallas: Escribir reseña.

library/views/escribir_resena.py, línea 23

```
@login_required
def review_edit(request, review_id)
```

Editor de una reseña ya escrita (o de un borrador) del usuario; las de otros dan 404.

Pantallas: Escribir reseña.

library/views/escribir_resena.py, línea 41

```
def _review_editor(request, review, entry)
```

Muestra y guarda el editor de review. «Publicar» la publica y «Guardar borrador» la guarda sin que la vea nadie más.

Al publicarla se apuntan las horas y el estado que tiene ahora en la biblioteca (Review.snapshot) y, si lleva nota, pasa a ser también la nota del juego en su biblioteca, que es la que cuenta en la media de la comunidad. La fecha de publicación es la de la primera vez. En una reseña nueva, la plataforma y las fechas vienen ya rellenas con lo de las importaciones (library.autofill) y «Lo mejor» y «Lo peor» sugieren lo que han escrito otros (library.points). Las imágenes (subidas o capturas oficiales del juego) van en un formulario aparte, ReviewImagesForm, que se guarda a la vez. La primera vez que se publica se apunta en la actividad del autor (social.activity); al editarla, no.

Pantallas: Escribir reseña (publicar o guardar el borrador), Ficha del juego (reseñas), Perfil (pestaña Reseñas y reseña destacada), Página de la reseña, Inicio (feed).

library/views/escribir_resena.py, línea 51

```
@require_POST
@login_required
def review_delete(request, review_id)
```

Borra una reseña del usuario (con sus «me gusta» y comentarios) y vuelve a las reseñas del juego.

Pantallas: Escribir reseña (botón «Borrar»), Ficha del juego (reseñas), Perfil (pestaña Reseñas).

library/views/escribir_resena.py, línea 112

```
@require_POST
@login_required
def review_preview(request)
```

Vista previa del texto de una reseña o un comentario con su formato (fragmento HTML para el editor).

Pantallas: Escribir reseña (vista previa del texto con formato).

library/views/escribir_resena.py, línea 126

library/views/ficha_del_juego.py

«Tu biblioteca» en la Ficha del juego: los botones de estado, nota y quitar, y la sección de reseñas del juego, con los «me gusta» marcados (mark_liked, que usan también el Perfil y el feed de Inicio).
Pantallas de este archivo: Inicio, Catálogo, Ficha del juego, Perfil, Página de la lista, Retos, Página del reto

```
def mark_liked(reviews, user)
```

Apunta en cada reseña de reviews (liked) si user le ha dado «me gusta»; devuelve la misma lista.

Pantallas: Ficha del juego (reseñas), Perfil (pestaña Reseñas), Inicio (feed: tus «me gusta» marcados).

library/views/ficha_del_juego.py, línea 31

```
def review_context(request, game, entry)
```

Variables de la sección de reseñas de la ficha de game para quien la ve (entry: su juego en la biblioteca).

Las reseñas que puede leer, ordenadas con ?resenas= (recientes o más gustadas) y filtradas con ?filtro= (las de quien lo ha terminado o las escritas con más de LONG_PLAY_HOURS horas); las suyas del juego, borradores incluidos, con las horas que lleva de más desde la última (extra_hours, ver Review.hours_since) para proponerle actualizarla; lo mejor y lo peor que más se repite (consensus, ver library.points); y en cada reseña si le ha dado «me gusta» (liked) y si va más adelante que él en el juego (ahead, ver AHEAD_HOURS) para plegarla.

Pantallas: Ficha del juego (reseñas).

library/views/ficha_del_juego.py, línea 44

```
def _respond(request, game, entry)
```

Respuesta común de las acciones de biblioteca.

Si la petición viene de HTMX devuelve el widget de biblioteca ya actualizado junto con el recuadro de valoración de la comunidad y la sección de reseñas (que HTMX cambia en su sitio); si no, redirige a la ficha del juego.

Pantallas: Ficha del juego (Tu biblioteca: lo que cambia tras cada botón).

library/views/ficha_del_juego.py, línea 107

```
@require_POST
@login_required
def set_status(request, game_id)
```

Añade el juego a la biblioteca del usuario con el estado enviado, o cambia el que tenía.

Si el estado es nuevo, lo apunta en su actividad (social.activity) y, al pasarlo a «Completado», apunta cuándo (completed_at, que se vacía si después cambia de estado) y mira si con eso ha conseguido algún reto (challenges.progress.check). Responde 400 si el estado no es uno de los permitidos.

Pantallas: Ficha del juego (Tu biblioteca: estado), Perfil (biblioteca), Catálogo (estado de cada tarjeta), Página de la lista (estado de cada juego), Inicio (feed y tu resumen), Retos (progreso), Página del reto (progreso).

library/views/ficha_del_juego.py, línea 136

```
@require_POST
@login_required
def remove(request, game_id)
```

Quita el juego de la biblioteca del usuario (no da error si no estaba) y olvida su actividad con él, salvo las reseñas.

Pantallas: Ficha del juego (Tu biblioteca: quitar el juego), Perfil (biblioteca), Catálogo (estado de cada tarjeta), Página de la lista (estado de cada juego), Inicio (feed y tu resumen), Retos (progreso), Página del reto (progreso).

library/views/ficha_del_juego.py, línea 165

```
@require_POST
@login_required
def rate(request, game_id)
```

Guarda la valoración del usuario (de 1 a 10 medias estrellas) para un juego de su biblioteca.

Con rating=0 borra la valoración. Si cambia, lo apunta en su actividad (social.activity). Responde 404 si el juego no está en su biblioteca y 400 si la valoración no es un número del 0 al 10.

Pantallas: Ficha del juego (tu nota y la valoración de la comunidad), Perfil (notas de la biblioteca), Inicio (feed).

library/views/ficha_del_juego.py, línea 181

library/views/lista_de_deseos.py

Vistas de la Lista de deseos (lista-de-deseos/) y del botón «Lo quiero» de la Ficha del juego, con el precio por debajo del cual avisar.
Pantallas de este archivo: Ficha del juego, Lista de deseos, Avisos

```
def _wish_response(request, game, wish, error='')
```

Respuesta de las acciones de la lista de deseos: el botón «Lo quiero» actualizado si es HTMX; si no, vuelve a la página de la que vino (la ficha por defecto) avisando del error si lo hay.

Pantallas: Ficha del juego (botón «Lo quiero»), Lista de deseos.

library/views/lista_de_deseos.py, línea 20

```
@require_POST
@login_required
def wish_toggle(request, game_id)
```

Añade el juego a la lista de deseos del usuario o lo quita si ya estaba.

Si lo quita estando en su lista de deseos de Steam, se apunta para que la importación de Steam no lo vuelva a traer; si lo vuelve a añadir él, se olvida.

Pantallas: Ficha del juego (botón «Lo quiero»), Lista de deseos.

library/views/lista_de_deseos.py, línea 38

```
@require_POST
@login_required
def wish_target(request, game_id)
```

Guarda el precio por debajo del cual avisar (vacío = solo avisar del mínimo histórico).

Al cambiarlo se olvida el último aviso, para volver a comparar con el precio nuevo. Responde 404 si el juego no está en su lista.

Pantallas: Ficha del juego (precio para avisar), Lista de deseos (tu precio límite), Avisos (bajadas de precio).

library/views/lista_de_deseos.py, línea 63

```
def _steam_order(row)
```

Clave de orden «Como en Steam»: primero los que ha ordenado allí, por su puesto; después los demás de su lista de Steam, del añadido más recientemente al más antiguo; al final, los que no están en Steam.

Pantallas: Lista de deseos (orden «Como en Steam»).

library/views/lista_de_deseos.py, línea 86

```
@login_required
def wishlist(request)
```

Lista de deseos del usuario con el mejor precio de hoy de cada juego (en sus tiendas, si las ha elegido), su mínimo histórico, si está por debajo de su límite y si está en su lista de Steam. Se ordena con ?orden= (recientes por defecto, descuento, precio, nombre o como en Steam) y se filtra por idioma con ?idioma= (ver LANGUAGE_FILTERS).

Pantallas: Lista de deseos.

library/views/lista_de_deseos.py, línea 110

library/views/pagina_de_la_resena.py

Vista de la Página de la reseña (u/<usuario>/resenas/<id>/) y los botones de cada reseña, que se pulsan también desde la Ficha del juego, el Perfil y el feed de Inicio: «me gusta», comentar, borrar un comentario y denunciar.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Página de la reseña, Avisos, Panel de administración

```
def _readable_review(request, review_id)
```

Reseña publicada review_id que el usuario puede leer; 404 si es un borrador o de un perfil privado ajeno.

Pantallas: Página de la reseña, Ficha del juego, Perfil, Inicio (404 al pulsar algo de una reseña que no se puede leer).

library/views/pagina_de_la_resena.py, línea 21

```
def review_detail(request, username, review_id)
```

Página de una reseña: el juego, la nota, los datos de la partida, sus imágenes, sus «me gusta» y sus comentarios.

Los borradores y las reseñas de perfiles privados solo los ve su autor; a los demás les da 404. A su autor se le propone actualizarla si ha jugado mucho más desde que la escribió (solo en la última que ha publicado del juego); los demás pueden denunciarla (con ?denunciar, el formulario llega abierto).

Pantallas: Página de la reseña.

library/views/pagina_de_la_resena.py, línea 33

```
def _comments_context(review, viewer)
```

Variables de library/_comments.html para los comentarios de la reseña (ver comment_thread).

Pantallas: Página de la reseña (comentarios).

library/views/pagina_de_la_resena.py, línea 68

```
def _notify(review, actor, kind)
```

Avisa al autor de la reseña de que actor le ha dado «me gusta» o la ha comentado.

No se repite (accounts.notices): un «me gusta» de la misma persona solo avisa una vez aunque lo quite y lo vuelva a dar, y un comentario no avisa si ya hay otro suyo sin leer en esa reseña.

Pantallas: Avisos («me gusta» y comentarios en tus reseñas), todas (el número de la campana).

library/views/pagina_de_la_resena.py, línea 81

```
@require_POST
@login_required
def review_like(request, review_id)
```

Da o quita el «me gusta» del usuario a una reseña de otro (avisando a su autor la primera vez).

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la página de la reseña. Las propias no se pueden marcar (400).

Pantallas: Ficha del juego, Perfil, Inicio, Página de la reseña (botón «me gusta» de las reseñas), Avisos (aviso a su autor).

library/views/pagina_de_la_resena.py, línea 99

```
@require_POST
@login_required
def review_comment(request, review_id)
```

Publica un comentario en una reseña (avisando a su autor si no es él) y vuelve a sus comentarios.

Pantallas: Página de la reseña (comentarios), Avisos (aviso a su autor).

library/views/pagina_de_la_resena.py, línea 123

```
@require_POST
@login_required
def comment_delete(request, comment_id)
```

Borra un comentario: solo quien lo escribió o el autor de la reseña (si no, 403).

Pantallas: Página de la reseña (botón «Borrar» de un comentario).

library/views/pagina_de_la_resena.py, línea 148

```
@require_POST
@login_required
def review_report(request, review_id)
```

Denuncia una reseña de otro para que la revise un administrador; las propias no se pueden denunciar (400).

Pantallas: Página de la reseña, Ficha del juego, Perfil (botón «Denunciar» de las reseñas), Panel de administración (denuncias).

library/views/pagina_de_la_resena.py, línea 167

```
@require_POST
@login_required
def comment_report(request, comment_id)
```

Denuncia un comentario de otro para que lo revise un administrador; los propios no se pueden denunciar (400).

Como con la reseña, da 404 si la reseña del comentario es un borrador o de un perfil privado ajeno.

Pantallas: Página de la reseña (denunciar un comentario), Panel de administración (denuncias).

library/views/pagina_de_la_resena.py, línea 181

library/views/repasar_la_biblioteca.py

Vistas de Repasar la biblioteca (biblioteca/repasar/): los juegos que las importaciones no saben si terminaste, de uno en uno, para contestarlos con un botón (library.backlog).
Pantallas de este archivo: Inicio, Perfil, Repasar la biblioteca, Retos, Página del reto

```
def _card_context(user)
```

Variables de la tarjeta del repaso (library/_tidy_card.html) para user: el siguiente juego (entry, o None si no queda ninguno), su pista (hint), cuántos quedan (remaining) y lo que lleva repasado hoy (summary).

Pantallas: Repasar la biblioteca.

library/views/repasar_la_biblioteca.py, línea 14

```
@login_required
def tidy(request)
```

Repasar la biblioteca: el primer juego por repasar con sus pistas y sus botones, o el resumen si no queda ninguno.

Pantallas: Repasar la biblioteca.

library/views/repasar_la_biblioteca.py, línea 31

```
@require_POST
@login_required
def tidy_answer(request, entry_id)
```

Contesta un juego del repaso (respuesta=, una clave de library.backlog.ANSWERS) y pasa al siguiente: con HTMX devuelve la tarjeta siguiente; sin HTMX, vuelve a la página. Las entradas de otros dan 404 y una respuesta que no existe, 400.

Pantallas: Repasar la biblioteca (botones de cada tarjeta), Perfil (biblioteca), Inicio (feed: «ordenó su biblioteca»), Retos, Página del reto (progreso).

library/views/repasar_la_biblioteca.py, línea 41

library/wishlist.py

Avisos de la lista de deseos: comparan los precios de hoy con lo que quiere cada usuario.

Se lanzan al terminar update_prices. Un juego avisa cuando su mejor precio (PC, Xbox o Nintendo, solo en las tiendas del usuario si las ha elegido) baja del precio que puso el usuario o cuando está rebajado a su mínimo histórico. No se repite el aviso hasta que baje más, o hasta que vuelva a subir y baje otra vez. Los juegos que ya están en la biblioteca del usuario no avisan (ya lo tiene).

También está aquí el resumen que se enseña después de traer la lista de deseos de Steam (con los precios de los juegos nuevos ya pedidos, ver games.prices.fetch_missing): cuántos están hoy en su mínimo histórico y cuántos son más baratos fuera de Steam.
Pantallas de este archivo: Perfil, Lista de deseos, Avisos, Editar perfil

```
def at_historical_low(game, best)
```

Indica si best (la oferta más barata del juego) está rebajada a su mínimo histórico o por debajo.

Solo cuenta si está rebajada: un juego que nunca baja siempre está en su mínimo.

Pantallas: Lista de deseos (en su mínimo histórico), Avisos (bajadas de precio), Editar perfil (mensaje al traer la lista de deseos de Steam).

library/wishlist.py, línea 30

```
def steam_deals(user)
```

De los juegos de «Lo quiero» de user que están en su lista de deseos de Steam: (cuántos son, cuántos están hoy en su mínimo histórico, cuántos son más baratos en otra tienda de PC que en Steam).

Cuentan solo las tiendas del usuario si las ha elegido; el precio de Steam con el que se compara es siempre el de Steam.

Pantallas: Editar perfil, Perfil (mensaje al traer la lista de deseos de Steam desde Cuentas vinculadas).

library/wishlist.py, línea 42

```
def alert_reason(item, best)
```

Por qué avisar del precio best (un GamePrice) de un juego de la lista: «objetivo», «minimo» o None.

«minimo» solo cuenta si está rebajado: un juego que nunca baja siempre está en su mínimo.

Pantallas: Avisos (bajadas de precio de tu lista de deseos).

library/wishlist.py, línea 70

```
def alert_text(item, best, reason)
```

Texto del aviso: el juego, el precio, la tienda y el motivo.

Pantallas: Avisos (texto de cada aviso de bajada de precio, también en el correo).

library/wishlist.py, línea 86

```
def check_alerts(games=None)
```

Avisa de las bajadas de precio de las listas de deseos (de esos juegos, o de todos): en la campana y, a quien también las quiere por correo, en un solo correo con todas las suyas. Devuelve de cuántas bajadas ha avisado.

Lo que recibe cada uno lo eligió en Editar perfil (ver accounts.notices). Si alguien las tiene apagadas en la campana y en el correo, la bajada no cuenta como avisada: si las vuelve a encender, se le avisa del precio de entonces.

Pantallas: Avisos (bajadas de precio de tu lista de deseos, también por correo), todas (el número de la campana).

library/wishlist.py, línea 97

```
def _send_email(user, alerts)
```

Manda al usuario un correo con sus avisos de precio (pares de texto y enlace); si falla, lo apunta en el log y sigue.

Pantallas: ninguna (manda por correo los avisos de precio a quien los tiene encendidos en Editar perfil › Avisos).

library/wishlist.py, línea 144

App «social»

Lo social: seguir a otros jugadores, las listas de seguidores y seguidos y la actividad de cada jugador.

social/activity.py

La actividad de cada jugador: lo que hace con los juegos de su biblioteca, lo que importa, lo que publica, sus listas y sus retos.

Se apunta desde lo que hace el jugador (cambiar el estado de un juego, valorarlo, publicar una reseña, quitarlo de la biblioteca, crear una lista o añadirle juegos) y desde las importaciones de Steam, PlayStation y Xbox, nunca con señales: la sincronización de cada noche también guarda juegos y les cambia el estado, y eso no lo ha hecho el jugador.

Lo que hace con un mismo juego en poco rato se junta en una sola actividad («Terminó Hades □□□□½» con su reseña debajo), y cada importación es una sola aunque traiga cientos de juegos. La lista de deseos no sale: es privada, y no se enseña en ningún otro sitio.
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Publicar, Escribir reseña, Repasar la biblioteca, Página de la lista, Nueva lista, Nueva tier list, Editar lista, Página del reto, Nuevo reto, Editar perfil

```
def _recent(user)
```

Lo último que ha hecho user, si fue hace menos de MERGE_WINDOW (para juntar con ello lo siguiente); si no, None.

Pantallas: Inicio (feed: junta lo que haces seguido en una sola actividad), Perfil (pestaña Actividad).

social/activity.py, línea 28

```
def _game_activity(user, game, merge_only=False, **changes)
```

Junta changes (status, rating o review) con lo último que hizo user si fue con game hace poco; si no, crea otra.

Con merge_only no crea ninguna: sirve para deshacer algo recién hecho (quitar la nota que acaba de poner). Si se queda vacía, se borra. Devuelve la actividad, o None si no hay ninguna.

Pantallas: Inicio (feed: lo que haces con un juego), Perfil (pestaña Actividad).

social/activity.py, línea 39

```
def status_changed(user, game, status)
```

Apunta que user ha pasado game a status (o que lo ha añadido a su biblioteca con ese estado).

Pantallas: Ficha del juego (al cambiar el estado), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 64

```
def rated(user, game, rating)
```

Apunta la nota que user le ha puesto a game. Si la ha quitado (None), solo se deshace la que acababa de poner.

Pantallas: Ficha del juego (al poner o quitar la nota), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 72

```
def review_published(review)
```

Apunta que su autor ha publicado review, con su nota si la lleva.

Solo la primera vez: si la volvió a dejar en borrador y la publica otra vez, ya tenía su actividad.

Pantallas: Escribir reseña (al publicar), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 82

```
def game_removed(user, game)
```

user ha quitado game de su biblioteca: se olvida lo que hizo con él, salvo sus reseñas, que siguen publicadas.

Pantallas: Ficha del juego (al quitar un juego de la biblioteca), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 97

```
def post_published(post)
```

Apunta la publicación post en la actividad de su autor. Cada una va sola: no se junta con nada.

Pantallas: Inicio (caja para publicar y feed), Publicar, Perfil (pestaña Actividad).

social/activity.py, línea 107

```
def games_imported(user, platform, entries)
```

Apunta como una sola actividad los juegos que acaban de entrar en la biblioteca de user con una importación.

platform es la clave de la plataforma ("steam", "psn" o "xbox") y entries, las entradas nuevas de la biblioteca; se guardan cuántas son y los ids de los IMPORT_COVERS juegos más jugados. Una importación de la misma plataforma justo después (al volver a sincronizar, por ejemplo) se suma a la anterior.

Pantallas: Editar perfil (Cuentas vinculadas: al importar), Perfil (modal «Vincular cuentas» de su dueño; pestaña Actividad), Inicio (feed).

social/activity.py, línea 118

```
def _list_activity(game_list, user)
```

Lo último que ha hecho user si fue con game_list hace poco (MERGE_WINDOW); si no, None.

Pantallas: Inicio (feed: lo que haces con una lista), Perfil (pestaña Actividad).

social/activity.py, línea 151

```
def list_created(game_list)
```

Apunta que su dueño ha creado game_list. Los juegos que le añada enseguida se juntan con esto (list_games_added).

Si la lista es privada (o lo es después), la actividad solo la ve su dueño; mientras no tenga juegos no se enseña.

Pantallas: Nueva lista, Nueva tier list (al crearla), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 162

```
def list_games_added(game_list, user, count=1)
```

Apunta que user (su dueño o quien colabora en ella) ha añadido count juegos a game_list: se suman a lo último que hizo si fue con esta lista hace poco (crearla o añadirle otros); si no, es una actividad nueva («añadió 3 juegos a la lista…»).

Pantallas: Editar lista (los juegos: al añadir juegos), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 176

```
def list_game_removed(game_list, user)
```

user ha quitado un juego de game_list: si él le había añadido juegos hace poco, se descuenta de esa actividad (y si solo era eso, se borra), como quitar una nota recién puesta. Lo de antes no se toca.

Pantallas: Editar lista (los juegos: al quitar un juego), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 194

```
def _challenge_activity(user, challenge, flags)
```

Junta flags (en data) con lo último que hizo user si fue con challenge hace poco (MERGE_WINDOW); si no, crea otra actividad del reto. La devuelve.

Pantallas: Inicio (feed: lo que haces con un reto), Perfil (pestaña Actividad).

social/activity.py, línea 212

```
def challenge_joined(user, challenge, created=False)
```

Apunta que user se ha unido a challenge o, con created, que lo ha creado (quien crea un reto queda dentro).

Si el reto es privado, solo la ve quien lo creó (social.views._batch).

Pantallas: Página del reto (botón para unirse), Nuevo reto (al crearlo), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 228

```
def challenge_won(user, challenge)
```

Apunta que user ha conseguido challenge; si se unió a él o lo creó hace poco, va en la misma actividad.

Pantallas: Inicio (feed: «consiguió el reto»), Perfil (pestaña Actividad).

social/activity.py, línea 238

```
def library_tidied(user, game, status)
```

Apunta en una sola actividad lo que user va terminando o dejando al repasar su biblioteca (status es el estado al que ha pasado game: «Completado» o «Abandonado»): cuántos de cada y las portadas de los primeros terminados. Se junta con lo último que hizo si fue otro repaso hace poco (MERGE_WINDOW), para no llenar el feed con un juego tras otro.

Pantallas: Repasar la biblioteca (al contestar), Inicio (feed: «ordenó su biblioteca»), Perfil (pestaña Actividad).

social/activity.py, línea 246

```
def challenge_left(user, challenge)
```

user ha dejado challenge: se olvida lo que hizo con él (unirse, crearlo o conseguirlo).

Pantallas: Página del reto (botón para dejarlo), Inicio (feed), Perfil (pestaña Actividad).

social/activity.py, línea 268

social/admin.py

Panel de administración de lo social: quién sigue a quién, las publicaciones (con sus imágenes, comentarios y denuncias pendientes), sus comentarios y la actividad de cada jugador.
Pantallas de este archivo: Panel de administración

Clase PostImageInline

Imágenes de una publicación, para quitar las que no deban estar.

Clase PostCommentInline

Comentarios de una publicación, para moderarlos desde ella.

Clase PostAdmin

```
def get_queryset(self, request)
```

Las publicaciones con sus denuncias pendientes contadas.

Pantallas: Panel de administración (publicaciones con sus denuncias pendientes).

social/admin.py, línea 45

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, post)
```

Cuántas denuncias de la publicación quedan por revisar.

Pantallas: Panel de administración (columna de denuncias pendientes de las publicaciones).

social/admin.py, línea 55

social/apps.py

Configuración de la app social (seguir, publicaciones y actividad): al arrancar Django conecta sus señales (social/signals.py).

Clase SocialConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django: borran la actividad que se queda vacía al borrar una reseña y los archivos de las imágenes de las publicaciones que se borran.

Pantallas: ninguna (se ejecuta al arrancar Django y conecta las señales de social/signals.py).

social/apps.py, línea 11

social/forms.py

Formularios de lo social: el de publicar (PostForm, en la caja de Inicio y en Publicar) y el de comentar una publicación (PostCommentForm, en la Página de la publicación).
Pantallas de este archivo: Inicio, Perfil, Publicar, Página de la publicación

Clase PostCommentForm

Comentario en una publicación: el mismo que el de las reseñas (texto con su formato sencillo, sin espacios de sobra).

Clase PostForm

Publicación nueva: el texto (con el formato sencillo de las reseñas), el juego del que habla si quiere, si es un spoiler y hasta POST_MAX_IMAGES imágenes, que se pasan a WebP y sin EXIF con su miniatura (accounts.images). Tiene que llevar texto o alguna imagen.

```
def clean_text(self)
```

Quita los espacios del principio y del final.

Pantallas: Inicio (caja para publicar), Publicar (el texto).

social/forms.py, línea 39

```
def clean(self)
```

Comprueba que lleve texto o imágenes y que no pasen de POST_MAX_IMAGES, y prepara las subidas.

Las que no son una imagen válida, pesan demasiado o tienen demasiados píxeles dan un error con su nombre.

Pantallas: Inicio (caja para publicar), Publicar (texto e imágenes).

social/forms.py, línea 46

```
def save(self, user)
```

Guarda la publicación de user con sus imágenes en el orden en que llegaron y la devuelve.

Pantallas: Inicio (caja para publicar y feed), Publicar, Página de la publicación, Perfil (pestaña Actividad).

social/forms.py, línea 67

social/models.py

Modelos de lo social: quién sigue a quién, las publicaciones con sus imágenes, «me gusta» y comentarios, y la actividad de cada jugador (lo que hace con sus juegos, sus listas, sus importaciones y sus publicaciones), que forma el feed de Inicio y la pestaña Actividad del Perfil.
Pantallas de este archivo: Inicio, Perfil, Seguidores y seguidos, Publicar, Página de la publicación, Avisos, Panel de administración

Clase FollowManager

```
def follow(self, follower, followed)
```

Hace que follower siga a followed y avisa a followed en la campana; devuelve True si no lo seguía ya.

El aviso llega solo la primera vez (accounts.notices): si follower deja de seguir a followed y vuelve, no se repite, y no llega si followed ha apagado los avisos de nuevos seguidores en Editar perfil. Lleva a la página de follower o, si followed no puede verla (perfil privado), a la lista de seguidores de followed, donde sale sin enlace. No comprueba si followed se puede seguir (eso lo mira social.views.follow).

Pantallas: Perfil, Seguidores y seguidos, Inicio (botón «Seguir» de las sugerencias y feed «Siguiendo»), Avisos (aviso al seguido).

social/models.py, línea 23

```
def unfollow(self, follower, followed)
```

Hace que follower deje de seguir a followed; devuelve True si lo seguía.

Pantallas: Perfil, Seguidores y seguidos, Inicio (botón «Seguir» y feed «Siguiendo»).

social/models.py, línea 46

Clase Follow

Un usuario (follower) sigue a otro (followed) para ver lo que hace.

Va en un solo sentido y sin solicitudes: followed no tiene que aceptar ni seguir de vuelta. No se puede seguir dos veces a la misma persona ni seguirse a uno mismo (también lo impide la base de datos). Los perfiles privados no se pueden seguir, pero sus dueños sí pueden seguir a otros.

```
def __str__(self)
```

Texto en el admin: «seguidor → seguido».

Pantallas: Panel de administración.

social/models.py, línea 82

Clase Post

Publicación de un jugador: lo que quiera contar, con hasta POST_MAX_IMAGES imágenes y, si quiere, sobre un juego.

Sale en el feed de la portada y en la actividad de su perfil (social.activity.post_published) y tiene su propia página. La ve quien puede ver su perfil (Profile.visible_filter). El texto lleva el mismo formato sencillo que las reseñas (library.markup) y, si es un spoiler, el texto y las imágenes van tapados hasta que se pulsan.

```
def __str__(self)
```

Texto en el admin: «usuario: comienzo del texto».

Pantallas: Panel de administración.

social/models.py, línea 114

```
def get_absolute_url(self)
```

URL de la página de la publicación: /u/<usuario>/publicaciones/<id>/.

Pantallas: Inicio (feed), Perfil (pestaña Actividad), Página de la publicación, Avisos (los enlaces a cada publicación).

social/models.py, línea 121

Clase PostImage

Imagen de una publicación (como mucho POST_MAX_IMAGES). Se guardan en publicaciones/<id del autor>/.

```
def __str__(self)
```

Texto en el admin: «publicación · imagen n».

Pantallas: Panel de administración.

social/models.py, línea 139

```
def upload_folder(self)
```

Las imágenes de las publicaciones van en publicaciones/<id del autor>.

Pantallas: Inicio (caja para publicar), Publicar (carpeta de las imágenes de las publicaciones).

social/models.py, línea 146

Clase PostLike

«Me gusta» de un usuario en una publicación (uno por usuario y publicación).

```
def __str__(self)
```

Texto en el admin: «usuario → publicación».

Pantallas: Panel de administración.

social/models.py, línea 166

Clase PostComment

Comentario en una publicación, como los de las reseñas. Lo pueden borrar quien lo escribió y el autor de la publicación.

```
def __str__(self)
```

Texto en el admin: «usuario en publicación: comienzo del comentario».

Pantallas: Panel de administración.

social/models.py, línea 189

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su publicación.

Pantallas: Avisos (enlace a un comentario de una publicación).

social/models.py, línea 196

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el autor de la publicación.

Pantallas: Página de la publicación (botón «Borrar» de los comentarios).

social/models.py, línea 203

Clase Activity

Algo que ha hecho un jugador: sale en la pestaña Actividad de su perfil y en el feed de la portada.

La apunta social.activity desde lo que hace el jugador, nunca con señales: las importaciones también guardan juegos y les cambian el estado, y eso no lo ha hecho él. Hay tres tipos:

- GAME: lo que hace con un juego de su biblioteca. Lo que hace con el mismo juego en poco rato se junta en una sola (ver social.activity.MERGE_WINDOW): el último estado al que lo ha pasado (status), la nota que le ha puesto (rating) y la reseña que ha publicado (review), las tres o solo alguna. - IMPORT: los juegos nuevos de una importación de Steam, PlayStation o Xbox, todos juntos. En data van la plataforma («platform», clave de HOURS_SOURCES), cuántos son («count») y los ids de los más jugados («games»), para enseñar sus portadas. - POST: una publicación (post), con su juego si lo tiene. Se borra con ella. - LIST: una lista (game_list): que la ha creado («created» en data) y cuántos juegos le ha añadido («added»), juntos si van seguidos (MERGE_WINDOW). Se borra con ella; si es privada, solo la ve su dueño. - CHALLENGE: un reto (challenge): que se ha unido a él, que lo ha creado («created» en data) o que lo ha conseguido («won»), juntos si van seguidos (MERGE_WINDOW). Se borra con el reto o si lo deja; si es privado, solo la ve su creador. - TIDY: lo que ha ordenado al repasar su biblioteca (library.backlog): cuántos juegos ha marcado como terminados («completed» en data) y como abandonados («dropped») y los ids de los primeros terminados («games»), para sus portadas; todo junto si va seguido (MERGE_WINDOW), y no una entrada por juego.

updated_at es la hora de lo último que se ha juntado: la que se enseña y por la que se ordena.

```
def __str__(self)
```

Texto en el admin: «usuario · lo que hizo».

Pantallas: Panel de administración.

social/models.py, línea 290

```
@property
def is_empty(self)
```

Indica si una actividad de un juego se ha quedado sin nada (sin estado, nota ni reseña) y sobra.

Pantallas: Inicio (feed: se borran las actividades que se quedan vacías), Perfil (pestaña Actividad).

social/models.py, línea 298

```
@property
def has_content(self)
```

Indica si hay algo que enseñar de ella a quien la mira: los juegos de una importación, la publicación, la lista (si puede verla y tiene juegos) o, en las de un juego, el estado, la nota o la reseña (si su reseña ya no se puede leer, puede que nada).

Pantallas: Inicio (feed: lo que se enseña de cada actividad), Perfil (pestaña Actividad).

social/models.py, línea 306

```
@property
def list_created(self)
```

Indica si una actividad de una lista cuenta que la ha creado (si no, solo que le ha añadido juegos).

Pantallas: Inicio (feed: actividades de listas), Perfil (pestaña Actividad).

social/models.py, línea 330

```
@property
def list_added(self)
```

Cuántos juegos ha añadido a la lista en esta actividad.

Pantallas: Inicio (feed: actividades de listas), Perfil (pestaña Actividad).

social/models.py, línea 338

```
@property
def challenge_verb(self)
```

Qué hizo con el reto, en minúscula y con su artículo: «consiguió el reto», «creó el reto» o «se unió al reto». Conseguirlo manda sobre lo demás si van juntos.

Pantallas: Inicio (feed: actividades de retos), Perfil (pestaña Actividad).

social/models.py, línea 346

```
@property
def verb(self)
```

Qué hizo con el juego, en minúscula: (lo que va antes del nombre del juego, lo que va después).

Con un estado, lo de STATUS_VERBS; si no, «reseñó» si publicó una reseña que se puede leer, o «valoró».

Pantallas: Inicio (feed: qué hizo con el juego), Perfil (pestaña Actividad).

social/models.py, línea 359

```
@property
def shown_rating(self)
```

Nota que se enseña: la que le puso o, si no, la de su reseña (si la reseña se puede leer); o None.

Pantallas: Inicio (feed: la nota que se enseña), Perfil (pestaña Actividad).

social/models.py, línea 371

```
@property
def platform_name(self)
```

Nombre de la plataforma de una importación («Steam», «PlayStation» o «Xbox»).

Pantallas: Inicio (feed: actividades de importación), Perfil (pestaña Actividad).

social/models.py, línea 382

```
@property
def import_count(self)
```

Cuántos juegos nuevos trajo una importación.

Pantallas: Inicio (feed: actividades de importación), Perfil (pestaña Actividad).

social/models.py, línea 390

```
@property
def summary(self)
```

Lo que hizo, en una línea y sin formato, para el admin: «terminó Hades», «importó 12 juegos de Steam», «publicó» o «creó la lista Indies».

Pantallas: Panel de administración (lo que hizo cada actividad, en una línea).

social/models.py, línea 398

social/signals.py

Señales de lo social: al borrar una reseña quitan la actividad que solo era esa reseña, y al borrar una publicación borran del disco sus imágenes.
Pantallas de este archivo: Inicio, Perfil, Página de la publicación, Escribir reseña, Panel de administración

```
@receiver(post_delete, sender=Review)
def forget_empty_activity(sender, instance, **kwargs)
```

Al borrar una reseña, borra la actividad que solo era esa reseña; la que tenía también estado o nota se queda sin ella.

Pantallas: Inicio (feed: al borrar una reseña), Perfil (pestaña Actividad), Escribir reseña (botón «Borrar»).

social/signals.py, línea 14

```
@receiver(post_delete, sender=PostImage)
def delete_post_image_files(sender, instance, **kwargs)
```

Borra del disco la imagen subida y su miniatura al borrar la publicación o la cuenta de su autor.

Pantallas: Página de la publicación (al borrarla), Panel de administración.

social/signals.py, línea 26

social/views.py

Vistas de lo social: el botón de seguir y Seguidores y seguidos, el feed de Inicio y la pestaña Actividad del Perfil (con su «Cargar más»), la caja para publicar de Inicio y Publicar, y la Página de la publicación con sus «me gusta», comentarios y denuncias.
Pantallas de este archivo: Inicio, Perfil, Seguidores y seguidos, Publicar, Página de la publicación, Avisos, Panel de administración

```
def follow_context(viewer, user)
```

Variables del botón de seguir y de los contadores de user para quien mira (viewer).

Cuántos seguidores tiene user (follower_count) y a cuántos sigue (following_count), si viewer ya sigue a user (is_following) y si user sigue a viewer (follows_you). Las dos últimas son falsas sin sesión y en el propio perfil.

Pantallas: Perfil (botón «Seguir» y contadores), Seguidores y seguidos, Inicio (sugerencias de gente a la que seguir).

social/views.py, línea 53

```
@require_POST
@login_required
def follow(request, username)
```

Sigue a username (seguir=1) o deja de seguirlo (seguir=0).

Seguir a quien ya se sigue, o dejar a quien no se sigue, no hace nada: pulsar dos veces no deshace lo hecho. Seguirse a uno mismo da 400. Un perfil privado ajeno da 404, como su página, salvo para dejar de seguirlo (pudo hacerse privado después). Al seguir se avisa en la campana (Follow.objects.follow). Con HTMX devuelve el botón nuevo, del tamaño de donde estaba (desde: «perfil» o «lista»), y en el perfil también el número de seguidores de la cabecera; sin HTMX vuelve a la página de «next» o al perfil.

Pantallas: Perfil, Seguidores y seguidos, Inicio (botón «Seguir»), Avisos (aviso al seguido).

social/views.py, línea 79

```
def followers(request, username)
```

Seguidores de username, del más reciente al más antiguo (ver _people).

Pantallas: Seguidores y seguidos.

social/views.py, línea 114

```
def following(request, username)
```

Gente a la que sigue username, de la más reciente a la más antigua (ver _people).

Pantallas: Seguidores y seguidos.

social/views.py, línea 122

```
def _people(request, username, view)
```

Página con los seguidores de username (view «seguidores») o con la gente a la que sigue («siguiendo»).

Va de PEOPLE_PER_PAGE en PEOPLE_PER_PAGE (?page=). Un perfil privado da 404 a cualquiera que no sea su dueño, como su página. En las listas salen también los perfiles privados (a quién sigue cada uno se ve, como lo que comenta), pero sin enlace a su página (profile_visible) ni botón para seguirlos, solo para dejar de seguirlos. En cada persona se apunta si quien mira la sigue (viewer_follows) y si ella sigue a quien mira (follows_viewer).

Pantallas: Seguidores y seguidos.

social/views.py, línea 130

```
def _batch(activities, viewer, cursor)
```

Una tanda de ACTIVITY_PAGE actividades de activities (ya filtradas) tal como las ve viewer, de lo más reciente a lo más antiguo.

Devuelve (actividades, cursor de la tanda siguiente o "" si no hay más). cursor dice desde dónde seguir: lo da la tanda anterior, y vacío (o roto) es desde el principio. Cada actividad trae su reseña solo si viewer puede leerla (si su autor la volvió a dejar en borrador, no), con sus imágenes y sus «me gusta» y comentarios contados; su lista, solo si viewer puede verla, con su mosaico, sus «me gusta» y cuánto ha jugado de ella viewer; su reto, solo si viewer puede verlo (los privados, solo quien los crea), con cuánta gente está; y las importaciones y los repasos de la biblioteca traen los juegos de sus portadas (cover_games). Las que se quedan sin nada que enseñar no salen. No mira de quién son: eso lo decide quien la llama.

Pantallas: Inicio (feed: cada tanda, también la de «Cargar más»), Perfil (pestaña Actividad).

social/views.py, línea 174

```
def activity_context(user, viewer, cursor='')
```

Variables de la pestaña Actividad del perfil de user tal como la ve viewer, desde cursor (ver _batch).

La tanda que toca (activities), el cursor de la siguiente (next_cursor) y, si quedan más, las direcciones de «Cargar más»: la que pide HTMX (more_url) y la de la pestaña desde ahí, para quien no tenga JavaScript (more_href). No mira si viewer puede ver el perfil: eso lo hace quien la llama.

Pantallas: Perfil (pestaña Actividad).

social/views.py, línea 213

```
def feed_context(viewer, tab='', cursor='')
```

Variables del feed de la portada para viewer en la pestaña tab, desde cursor (ver _batch).

«siguiendo» (la de por defecto) es lo de viewer y la gente que sigue; «descubrir», lo de toda la comunidad; sin sesión solo hay «descubrir». En las dos sale solo lo de perfiles que viewer puede ver (Profile.visible_filter): si alguien a quien sigue se hace privado, deja de ver lo suyo. Además de la tanda y de «Cargar más» (como activity_context), lleva la pestaña (feed_tab) y las que hay (feed_tabs); si viewer no sigue a nadie (follows_nobody), en la primera tanda va también gente a la que seguir (suggestions).

Pantallas: Inicio (feed: pestañas «Siguiendo» y «Descubrir»).

social/views.py, línea 232

```
def suggested_people(viewer, limit=SUGGESTIONS)
```

Gente a la que viewer podría seguir: perfiles públicos a los que aún no sigue, primero los que tienen más juegos en común con viewer y después los que más han hecho en los últimos SUGGESTION_DAYS días.

Solo salen quienes tienen algún juego en común o actividad reciente. Cada uno trae cuántos juegos tiene en común (shared) y si sigue a viewer (follows_viewer), y lo que necesitan su foto y el botón de seguir. Se cuenta con subconsultas para no cruzar la biblioteca con la actividad de cada uno.

Pantallas: Inicio (gente a la que podrías seguir).

social/views.py, línea 274

```
def _cursor(activity)
```

Cursor de «Cargar más» para seguir después de activity: «microsegundos desde 1970 (UTC)-id».

Se cuenta en microsegundos enteros, sin decimales, para que la fecha vuelva exacta al leerlo.

Pantallas: Inicio (feed: «Cargar más»), Perfil (pestaña Actividad).

social/views.py, línea 314

```
def _parse_cursor(cursor)
```

Lo contrario de _cursor: (fecha, id), o None si cursor está vacío o no tiene esa forma.

Pantallas: Inicio (feed: «Cargar más»), Perfil (pestaña Actividad).

social/views.py, línea 324

```
def activity(request, username)
```

Tanda siguiente de la actividad de username (?antes=, el cursor de la anterior) para «Cargar más».

Con HTMX devuelve solo las actividades nuevas y el botón para la siguiente; sin HTMX lleva a la pestaña Actividad del perfil desde ese punto. Un perfil privado da 404 a cualquiera que no sea su dueño.

Pantallas: Perfil (pestaña Actividad: «Cargar más»).

social/views.py, línea 336

```
def feed(request)
```

Tanda siguiente del feed de la portada (?ver=, la pestaña, y ?antes=, el cursor de la anterior) para «Cargar más».

Con HTMX devuelve solo las actividades nuevas y el botón para la siguiente; sin HTMX lleva a la portada desde ese punto.

Pantallas: Inicio (feed: «Cargar más»).

social/views.py, línea 353

```
def _posts()
```

Publicaciones con lo que enseñan sus tarjetas ya cargado: autor (con su foto), juego, imágenes, «me gusta» (like_count) y comentarios (comment_count).

Pantallas: Inicio (feed), Perfil (pestaña Actividad), Página de la publicación (las tarjetas de las publicaciones).

social/views.py, línea 367

```
def mark_liked_posts(posts, user)
```

Apunta en cada publicación de posts (liked) si user le ha dado «me gusta»; devuelve la misma lista.

Pantallas: Inicio, Perfil, Página de la publicación (tus «me gusta» marcados).

social/views.py, línea 381

```
def composer_context(user, game=None, form=None)
```

Variables de la caja de publicar de user (social/_composer.html).

El formulario (post_form: form o uno vacío), los juegos que ofrece para elegir de cuál va la publicación (composer_games: game, si se da, y los últimos COMPOSER_GAMES que está jugando), el elegido (composer_game, su id o None) y las medidas con las que el navegador reduce las imágenes muy grandes.

Pantallas: Inicio (caja para publicar), Publicar.

social/views.py, línea 394

```
@login_required
def post_new(request)
```

Publica lo que escriba el usuario (POST) o enseña la página para hacerlo (GET).

Con ?juego= (desde la ficha de un juego) la publicación ya va sobre ese juego. Al publicar se apunta en su actividad (social.activity.post_published) y vuelve a la portada, donde sale la primera de Siguiendo; si falta algo, enseña esta página con los errores y lo que había escrito.

Pantallas: Publicar, Inicio (caja para publicar).

social/views.py, línea 417

```
def _readable_post(request, post_id)
```

Publicación post_id con sus tarjetas cargadas, si el usuario puede verla; 404 si es de un perfil privado ajeno.

Pantallas: Página de la publicación, Inicio, Perfil (404 al pulsar algo de una publicación que no se puede ver).

social/views.py, línea 439

```
def post_detail(request, username, post_id)
```

Página de una publicación: su autor, el juego del que habla, el texto entero, sus imágenes, sus «me gusta» y sus comentarios.

La ve quien puede ver el perfil de su autor (si no, 404). Su autor puede borrarla; los demás, denunciarla (con ?denunciar llega abierto el formulario).

Pantallas: Página de la publicación.

social/views.py, línea 450

```
def _post_comments(post, viewer)
```

Variables de library/_comments.html para los comentarios de post (ver library.views.comment_thread).

Pantallas: Página de la publicación (comentarios).

social/views.py, línea 474

```
@require_POST
@login_required
def post_comment(request, post_id)
```

Publica un comentario en una publicación (avisando a su autor si no es él) y vuelve a sus comentarios.

Con HTMX devuelve los comentarios ya con el nuevo (o el formulario con el error) y el contador de la página.

Pantallas: Página de la publicación (comentarios), Avisos (aviso a su autor).

social/views.py, línea 489

```
def _notify_post_comment(post, actor)
```

Avisa al autor de post de que actor la ha comentado, salvo si ya tiene otro suyo sin leer (accounts.notices).

Pantallas: Avisos (comentarios en tus publicaciones), todas (el número de la campana).

social/views.py, línea 514

```
@require_POST
@login_required
def post_comment_delete(request, comment_id)
```

Borra un comentario de una publicación: solo quien lo escribió o el autor de la publicación (si no, 403).

Pantallas: Página de la publicación (botón «Borrar» de un comentario).

social/views.py, línea 527

```
@require_POST
@login_required
def post_comment_report(request, comment_id)
```

Denuncia un comentario de otro en una publicación; los propios no (400) y los de publicaciones que no se pueden ver dan 404.

Pantallas: Página de la publicación (denunciar un comentario), Panel de administración (denuncias).

social/views.py, línea 546

```
@require_POST
@login_required
def post_delete(request, post_id)
```

Borra una publicación del usuario, con sus imágenes, sus «me gusta» y su actividad, y vuelve a la portada; las de otros dan 404.

Pantallas: Página de la publicación (botón «Borrar»), Inicio (feed), Perfil (pestaña Actividad).

social/views.py, línea 560

```
@require_POST
@login_required
def post_like(request, post_id)
```

Da o quita el «me gusta» del usuario a una publicación de otro, avisando a su autor la primera vez.

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la página de la publicación. Las propias no se pueden marcar (400).

Pantallas: Inicio, Perfil, Página de la publicación (botón «me gusta»), Avisos (aviso a su autor).

social/views.py, línea 572

```
def _notify_post_like(post, actor)
```

Avisa al autor de post de que a actor le ha gustado, solo la primera vez aunque lo quite y lo vuelva a dar (accounts.notices).

Pantallas: Avisos (a quien le gusta tu publicación), todas (el número de la campana).

social/views.py, línea 593

```
@require_POST
@login_required
def post_report(request, post_id)
```

Denuncia una publicación de otro para que la revise un administrador (library.views._report); las propias no (400).

Pantallas: Página de la publicación (botón «Denunciar»), Panel de administración (denuncias).

social/views.py, línea 607

App «lists»

Listas de juegos de cada usuario: crearlas, ordenarlas y comentarlas, y verlas en su página, en el perfil y en la ficha de cada juego.

lists/admin.py

Panel de administración de las listas: cada lista con sus juegos, colaboradores, comentarios y denuncias pendientes, y los comentarios de las listas.
Pantallas de este archivo: Panel de administración

Clase ListItemInline

Juegos de una lista, en su orden, para corregirlos o quitarlos desde ella.

Clase ListCollaboratorInline

Gente invitada a editar la lista y si ha aceptado.

Clase ListCommentInline

Comentarios de una lista, para moderarlos desde ella.

Clase GameListAdmin

```
def get_queryset(self, request)
```

Las listas con cuántos juegos tiene cada una y cuántas denuncias suyas quedan por revisar.

Pantallas: Panel de administración (listas con cuántos juegos y denuncias pendientes tienen).

lists/admin.py, línea 48

```
@admin.display(description='Juegos', ordering='game_count')
def game_count(self, obj)
```

Cuántos juegos tiene la lista.

Pantallas: Panel de administración (columna de cuántos juegos tiene cada lista).

lists/admin.py, línea 59

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, obj)
```

Cuántas denuncias de la lista quedan por revisar.

Pantallas: Panel de administración (columna de denuncias pendientes de las listas).

lists/admin.py, línea 67

lists/community.py

La tier list de la comunidad: la que sale de juntar una tier list abierta (GameList.is_open) con las versiones que hace la gente con «Haz tu versión», con cada juego en el nivel del medio de sus votos. Se ve en la Página de la lista de las tier lists abiertas.
Pantallas de este archivo: Página de la lista

```
def counted_lists(game_list)
```

Las tier lists que cuentan en la de la comunidad de game_list: ella misma y sus versiones, solo las que puede ver cualquiera (públicas y de perfiles públicos), para que no se pueda adivinar lo que ha votado una privada.

Pantallas: Página de la lista (la tier list de la comunidad).

lists/community.py, línea 16

```
def middle_tier(positions)
```

El nivel que le da la comunidad a un juego, con positions el puesto en los niveles (0 es el mejor) de cada voto: el del medio. Con un número par de votos y los dos del medio distintos, el que queda entre ellos o, si no queda ninguno, el de los dos más cercano a la media de todos (el mejor, si están igual de cerca).

Pantallas: Página de la lista (el nivel de cada juego en la tier list de la comunidad).

lists/community.py, línea 25

```
def community_board(game_list, items=None)
```

La tier list de la comunidad de game_list, una tier list abierta, con sus juegos items (sus ListItem con su juego, en orden; si no se dan, se cargan).

Devuelve voters (cuánta gente la ha hecho: las de counted_lists con algún juego suyo colocado en uno de sus niveles), missing (cuánta falta para MIN_VOTERS; 0 si ya se enseña) y rows, como GameList.tier_rows: [(nivel, juegos)] con los niveles de game_list y, al final, los juegos que nadie ha colocado (nivel None), si hay alguno. Cada juego lleva votes (cuánta gente lo ha colocado) y spread ([(nivel, cuántos)] de los niveles en los que lo han puesto, de mejor a peor, para su barra de colores). En cada nivel van de los de mejor media a los de peor y, si empatan, de los más votados a los que menos; si no, en el orden de game_list.

Solo cuentan los votos a sus juegos y en sus niveles (por su id): los de un nivel que ella no tiene, no.

Pantallas: Página de la lista (la tier list de la comunidad).

lists/community.py, línea 40

lists/forms.py

Formularios de las listas: el de sus datos (GameListForm, en Editar lista), el de Nueva lista (NewListForm), el de Nueva tier list (TierListForm) y el de comentar una lista (ListCommentForm, en la Página de la lista).
Pantallas de este archivo: Inicio, Ficha del juego, Listas del juego, Perfil, Página de la lista, Nueva lista, Nueva tier list, Editar lista

Clase ListCommentForm

Comentario en una lista: el mismo que el de las reseñas y las publicaciones (texto con su formato sencillo).

Clase GameListForm

Datos de una lista: el título, la descripción (con el formato sencillo de las reseñas), si la ven los demás y su tipo: lista, numerada o tier list. Los juegos se añaden aparte, uno a uno (lists.views.add).

Para las tier lists lleva también sus niveles: MAX_TIERS filas con nombre (nivel_<n>) y color (color_<n>), de mejor a peor. La fila n es el nivel con id n, así que renombrar uno no cambia los juegos que tiene; los que se dejan sin nombre se quitan y sus juegos quedan sin clasificar.

Y si está abierta a la comunidad (is_open, solo si es pública; una versión no se puede abrir). Si ya tiene versiones, o es una, sus niveles quedan fijos (tiers_lock): se pueden renombrar y cambiar de color, pero no quitar ni añadir, y no puede dejar de ser tier list.

```
def __init__(self, *args, **kwargs)
```

Añade las filas de los niveles, rellenas con los de la lista (o los de una tier list nueva).

Sin tipo en lo enviado, la lista se queda con el que tenía (una nueva, lista sin más). Con los niveles fijos (tiers_lock), los que tiene no se pueden dejar sin nombre, las filas sin usar no se pueden rellenar y el tipo no se puede cambiar. «Abierta a la comunidad» solo sale donde se hacen tier lists, y no en las versiones.

Pantallas: Nueva lista (con los niveles de las tier lists), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 71

```
def _tiers_lock(self)
```

Por qué no se le pueden quitar ni añadir niveles («» si se puede): es una versión de otra tier list, o ya hay versiones de esta. Así lo que ha colocado cada uno sigue contando igual en la de la comunidad.

Pantallas: Editar lista (los niveles de una tier list abierta con versiones y los de una versión).

lists/forms.py, línea 105

```
@property
def switches(self)
```

Los campos de sí o no (visible para los demás y, donde se hacen tier lists, abierta a la comunidad), que la plantilla pinta como interruptores.

Pantallas: Nueva lista (interruptor de si la ven los demás), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 121

```
@property
def tier_fields(self)
```

Las filas de los niveles para la plantilla: [(nombre, color)] de cada uno, de mejor a peor.

Pantallas: Nueva lista (filas de los niveles), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 131

```
def clean_title(self)
```

Deja un solo espacio entre palabras (el campo ya quita los del principio y el final, y sin título da error).

Pantallas: Nueva lista (el título), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 138

```
def clean_description(self)
```

Quita los espacios del principio y del final.

Pantallas: Nueva lista (la descripción), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 145

```
def clean(self)
```

Arma los niveles con las filas que tienen nombre (cleaned_data["tiers"]); un color que no sea «#rrggbb» da error. Una tier list necesita al menos un nivel; las demás guardan los niveles por si vuelven a serlo. Abrirla a la comunidad pide que sea visible para los demás, y la que deja de ser tier list deja de estar abierta.

Pantallas: Nueva lista (los niveles y sus colores), Nueva tier list, Editar lista (los datos de la lista).

lists/forms.py, línea 152

```
def save(self, commit=True)
```

Guarda la lista con sus niveles.

> Pantallas: Nueva lista (al guardar), Nueva tier list, Editar lista (los datos de la lista), Página de la lista, Perfil (pestañas Listas y Tier lists y escaparate), Ficha del juego (listas en las que sale), Listas del juego, Inicio (feed).

lists/forms.py, línea 180

Clase NewListForm

GameListForm de «Nueva lista»: una lista sin más o numerada. Las tier lists se crean en su página (TierListForm), aunque una lista ya creada puede pasar a serlo en su editor.

```
def __init__(self, *args, **kwargs)
```

Quita «Por niveles» de los tipos.

> Pantallas: Nueva lista (los tipos que se pueden elegir).

lists/forms.py, línea 200

Clase TierListForm

GameListForm de «Nueva tier list»: el tipo va fijo y oculto, y sus niveles se ven siempre.

```
def __init__(self, *args, **kwargs)
```

Empieza con una tier list nueva (con los niveles de serie), oculta el tipo y pone ejemplos de tier list.

> Pantallas: Nueva tier list (niveles de serie y ejemplos).

lists/forms.py, línea 213

```
def clean_mode(self)
```

Siempre tier list, aunque se mande otro tipo.

> Pantallas: Nueva tier list (siempre tier list).

lists/forms.py, línea 224

lists/models.py

Modelos de las listas: cada lista o tier list (con sus niveles y, si está abierta a la comunidad, las versiones que hace la gente), sus juegos en orden, quién colabora en ella, sus «me gusta» y sus comentarios. Se ven en la Página de la lista, en Editar lista, en las pestañas Listas y Tier lists del Perfil, en la pestaña Tier lists, en la Ficha del juego y en Listas del juego.
Pantallas de este archivo: Inicio, Ficha del juego, Listas del juego, Perfil, Página de la lista, Nueva lista, Nueva tier list, Editar lista, Tier lists, Página del reto, Nuevo reto, Avisos, Panel de administración

```
def default_tiers()
```

Niveles con los que empieza una tier list: [{"id", "label", "color"}] de DEFAULT_TIERS.

> Pantallas: Nueva tier list (niveles de serie), Editar lista (niveles al convertir una lista en tier list).

lists/models.py, línea 34

Clase GameListQuerySet

```
def visible_to(self, viewer)
```

Listas que viewer puede ver: las suyas, las que le han invitado a editar y las públicas de perfiles públicos.

Es la regla de GameList.visible_to como condición de consulta: tienen que decir lo mismo.

> Pantallas: Perfil (pestañas Listas y Tier lists y escaparate), Ficha del juego (listas en las que sale), Listas del juego, Inicio (feed), Página de la lista (solo las que puede ver quien mira).

lists/models.py, línea 43

```
def editable_by(self, user)
```

Listas cuyos juegos puede cambiar user: las suyas y en las que colabora (ha aceptado la invitación).

Pantallas: Ficha del juego (desplegable «Añadir a una lista»), Nuevo reto (listas que se pueden convertir en reto).

lists/models.py, línea 57

```
def with_preview(self)
```

Listas con lo que enseñan sus tarjetas ya cargado: su autor (con su foto), cuántos juegos tienen (game_count), sus «me gusta» (like_count) y comentarios (comment_count), los MOSAIC_COVERS primeros juegos para el mosaico (preview_items) y, en las versiones, la tier list de la que salen con su autor (parent).

Van de la más reciente a la más antigua: con los recuentos, Django ya no aplica el orden de Meta.

Pantallas: Perfil (pestañas Listas y Tier lists y escaparate: lo que enseña cada tarjeta), Tier lists, Ficha del juego (listas en las que sale), Listas del juego, Inicio (feed).

lists/models.py, línea 66

Clase GameList

Lista de juegos de un usuario («Mejores metroidvanias», «Para jugar en pareja»), con su descripción.

Los juegos van en el orden que elija (ListItem.position), cada uno con su comentario si quiere. Según su tipo (mode) es una lista sin más, una numerada (cada juego lleva su puesto, para rankings) o una tier list (cada juego va en uno de sus niveles: S, A, B...). Las tier lists tienen su propia sección (su página para crearlas y su pestaña del perfil), pero son listas como las demás: una puede cambiar de tipo en su editor y conserva sus niveles. Su dueño puede invitar a otros a editar sus juegos (ListCollaborator). La ven los demás si es pública y su perfil también (visible_to); las privadas solo las ven su dueño y los invitados. Una tier list pública se puede abrir a la comunidad (is_open): los demás hacen su versión (make_version, que apunta de cuál es en parent) y su página enseña la tier list que sale de juntarlas todas (lists.community). Los demás pueden darle «me gusta», comentarla y denunciarla, y sale en la actividad de su dueño (social.activity.list_created).

```
def __str__(self)
```

Texto en el admin: «título (usuario)».

Pantallas: Panel de administración, Nuevo reto (las listas que se pueden elegir).

lists/models.py, línea 145

```
def get_absolute_url(self)
```

URL de la página de la lista: /u/<usuario>/listas/<id>/.

Pantallas: Página de la lista, Perfil (pestañas Listas y Tier lists y escaparate), Ficha del juego (listas en las que sale), Listas del juego, Inicio (feed), Editar lista, Página del reto, Avisos (los enlaces a cada lista).

lists/models.py, línea 152

```
def visible_to(self, viewer)
```

Indica si viewer puede ver la lista: siempre su dueño y quien está invitado a editarla (aunque aún no haya aceptado: tiene que verla para hacerlo); los demás, si es pública y pueden ver el perfil de su dueño.

En las consultas la aplica GameListQuerySet.visible_to, que tiene que decir lo mismo.

Pantallas: Página de la lista (404 si no puedes verla), Editar lista.

lists/models.py, línea 161

```
def can_edit_games(self, user)
```

Indica si user puede añadir, quitar, ordenar y comentar los juegos: su dueño y quien colabora en ella.

Pantallas: Página de la lista (botones para editar sus juegos), Editar lista, Ficha del juego (desplegable «Añadir a una lista»).

lists/models.py, línea 173

```
@property
def is_ranked(self)
```

Indica si es numerada: cada juego lleva su puesto.

Pantallas: Página de la lista, Editar lista, Perfil, Inicio (los puestos de las listas numeradas).

lists/models.py, línea 184

```
@property
def is_tiered(self)
```

Indica si es una tier list: cada juego va en uno de sus niveles.

Pantallas: Página de la lista, Editar lista, Perfil (pestaña Tier lists), Ficha del juego, Nueva lista, Nueva tier list.

lists/models.py, línea 192

```
@property
def kind_note(self)
```

Lo que se dice de su tipo junto a cuántos juegos tiene: «numerada», «tier list» o nada.

Pantallas: Perfil, Listas del juego, Inicio (el tipo de cada lista en su tarjeta).

lists/models.py, línea 201

```
@property
def noun(self)
```

Cómo se la llama en los textos y los avisos: «tier list» o «lista».

Pantallas: Página de la lista, Editar lista, Perfil, Ficha del juego, Inicio, Avisos («tier list» o «lista» en los textos).

lists/models.py, línea 209

```
@property
def profile_tab(self)
```

Pestaña del perfil de su dueño en la que sale (?vista=): las tier lists tienen la suya, aparte de las demás.

Pantallas: Página de la lista, Editar lista (a qué pestaña del perfil vuelven).

lists/models.py, línea 218

```
@property
def accepts_versions(self)
```

Indica si los demás pueden hacer su versión: es una tier list pública y abierta a la comunidad.

Pantallas: Página de la lista (botón «Haz tu versión» y la tier list de la comunidad), Tier lists (las abiertas), Perfil (pestaña Tier lists: la marca «Abierta»).

lists/models.py, línea 226

```
@property
def public_parent(self)
```

La tier list de la que es versión, si la puede ver cualquiera (pública y de un perfil público); si no, None, para no enseñar el título de una que ya no lo es.

Pantallas: Página de la lista (de cuál es versión), Perfil (pestaña Tier lists), Tier lists, Inicio (feed: «hizo su versión de…»).

lists/models.py, línea 235

```
def make_version(self, user)
```

La versión de user de esta tier list abierta: la que ya tiene o, si no, una nueva, suya y pública, con el mismo título, los mismos niveles y los mismos juegos en el mismo orden, todos sin clasificar. Devuelve (versión, si es nueva). No mira si se puede: eso lo hace quien la llama (accepts_versions, y que no sea de user).

Pantallas: Página de la lista (botón «Haz tu versión»), Perfil (pestaña Tier lists), Tier lists, Inicio (feed).

lists/models.py, línea 247

```
@property
def tier_list(self)
```

Sus niveles, de mejor a peor, cada uno con el color de su texto (text, ver accounts.models.text_on).

Pantallas: Página de la lista (tablero), Editar lista (niveles), Perfil (tablero en miniatura), Nueva tier list.

lists/models.py, línea 273

```
def tier_rows(self, items, keep_loose=False)
```

Los juegos items (ListItem, en su orden) repartidos por niveles: [(nivel, juegos)] en el orden de los niveles y, al final, los que no están en ninguno con el nivel None, si hay alguno (con keep_loose, aunque no haya ninguno: en el tablero que se puede editar es donde se sueltan los que se sacan de su nivel).

Pantallas: Página de la lista (tablero de la tier list), Editar lista (juegos por niveles).

lists/models.py, línea 281

```
@property
def mosaic_gaps(self)
```

Huecos vacíos del mosaico cuando tiene menos de MOSAIC_COVERS juegos (para recorrerlos en la plantilla).

Pantallas: Perfil, Ficha del juego, Listas del juego, Página de la lista, Inicio, Nueva tier list (huecos del mosaico de portadas).

lists/models.py, línea 295

```
def touch(self)
```

Da la lista por actualizada ahora (sin tocar nada más).

Pantallas: Editar lista (los juegos), Ficha del juego (desplegable «Añadir a una lista»; listas en las que sale), Página de la lista (tablero de las tier lists), Perfil (pestañas Listas y Tier lists y escaparate: la fecha de la última vez que cambió), Listas del juego, Inicio (feed).

lists/models.py, línea 303

```
def add(self, game, user=None)
```

Añade game al final de la lista, apuntando quién lo añade (user); devuelve True si no estaba ya. En una tier list abierta, también se lo añade sin clasificar a las versiones que ya ha hecho la gente.

No mira LIST_MAX_GAMES ni si user puede editarla: eso lo hace quien la llama.

Pantallas: Editar lista (los juegos: añadir un juego), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists y las versiones de las abiertas).

lists/models.py, línea 312

```
def remove(self, game)
```

Quita game de la lista y corre los de detrás para no dejar hueco; devuelve True si estaba.

Pantallas: Editar lista (los juegos: quitar un juego), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists).

lists/models.py, línea 332

```
def move(self, game, position)
```

Lleva game al puesto position (desde 0; si se pasa, al principio o al final) y corre los demás.

Devuelve True si game está en la lista.

Pantallas: Editar lista (cambiar de puesto un juego), Página de la lista.

lists/models.py, línea 344

```
def place(self, game, level, row)
```

Pone game en el nivel level (su id; None, sin clasificar) y lo deja en ese nivel donde lo han soltado al arrastrarlo en el tablero: row son los ids de los juegos de esa fila en su nuevo orden, con el suyo.

Queda justo detrás del juego que tiene delante en la fila o, si es el primero, justo delante del que le sigue; solo en la fila, sin nadie más, conserva su puesto. Los demás juegos no cambian de orden entre sí. De row solo cuentan los juegos que están de verdad en ese nivel (quien arrastra puede tener la página sin actualizar). Devuelve True si game está en la lista; no mira si level es uno de sus niveles: eso lo hace quien la llama.

Pantallas: Página de la lista (arrastrar por el tablero de una tier list), Editar lista (nivel de cada juego).

lists/models.py, línea 361

```
@staticmethod
def _save_order(items)
```

Guarda como puestos seguidos (0, 1, 2...) el orden de items, solo en los que cambian.

Pantallas: Editar lista, Página de la lista (el orden de los juegos).

lists/models.py, línea 394

Clase ListItem

Un juego de una lista, en su puesto y con el comentario de quien la hace («por qué está aquí»), si lo tiene.

```
def __str__(self)
```

Texto en el admin: «lista · juego».

Pantallas: Panel de administración.

lists/models.py, línea 429

Clase ListCollaborator

Alguien a quien el dueño de una lista invita a editar sus juegos: añadirlos, quitarlos, ordenarlos y comentarlos.

Hasta que acepta (accepted_at) solo puede verla, aunque sea privada, para aceptar o rechazar la invitación. Los datos de la lista, quién colabora y borrarla siguen siendo cosa de su dueño. Quien colabora puede irse cuando quiera; lo que añadió se queda.

```
def __str__(self)
```

Texto en el admin: «usuario en lista (pendiente)».

Pantallas: Panel de administración.

lists/models.py, línea 458

Clase ListLike

«Me gusta» de un usuario en una lista de otro (uno por usuario y lista).

```
def __str__(self)
```

Texto en el admin: «usuario → lista».

Pantallas: Panel de administración.

lists/models.py, línea 478

Clase ListComment

Comentario en una lista, como los de las reseñas y las publicaciones. Lo pueden borrar quien lo escribió y el dueño de la lista.

```
def __str__(self)
```

Texto en el admin: «usuario en lista: comienzo del comentario».

Pantallas: Panel de administración.

lists/models.py, línea 502

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su lista.

Pantallas: Avisos (enlace a un comentario de una lista).

lists/models.py, línea 509

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el dueño de la lista.

Pantallas: Página de la lista (botón «Borrar» de los comentarios).

lists/models.py, línea 516

lists/presets.py

Plantillas de tier list que se rellenan solas con la biblioteca del usuario: «Jugados en 2026», sus más jugados, su pendiente…

Cada plantilla (PRESETS) elige juegos de la biblioteca en un orden y da el título y los niveles de la tier list; lists.views.from_library la crea con esos juegos en «Sin clasificar», o ya colocados por la nota que les ha puesto (RATING_TIERS), para que solo haya que arrastrarlos. Las de un año son del año en curso: Steam solo da la última partida y el historial de partidas (library.models.PlaySession) es reciente, así que en un año pasado faltarían los juegos que se han vuelto a jugar después.
Pantallas de este archivo: Página de la lista, Nueva tier list

```
def library(user)
```

La biblioteca de user (LibraryEntry, con su juego) con los minutos jugados en todas las plataformas (minutes): de más a menos horas y, a igualdad, de lo añadido hace menos a lo que más.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»).

lists/presets.py, línea 34

```
def year_bounds(year)
```

Principio de year y del año siguiente, en la zona horaria del sitio.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: las del año).

lists/presets.py, línea 47

```
def played_in(user, year)
```

Lo jugado en year: lo que tiene ese año su última partida de Steam, PlayStation o Xbox, partidas apuntadas (PlaySession) o una reseña empezada o terminada; lo completado ese año; y, si es el año en curso, lo que está jugando.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: lo jugado este año).

lists/presets.py, línea 55

```
def completed_in(user, year)
```

Lo terminado en year: lo pasado a «Completado» ese año o con una reseña terminada ese año.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: lo terminado este año).

lists/presets.py, línea 74

```
def most_played(user, year)
```

Los MOST_PLAYED juegos con más horas, sumando las de todas las plataformas.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: lo más jugado).

lists/presets.py, línea 84

```
def full_achievements(user, year)
```

Lo que tiene con el platino de PlayStation o con todos los logros en alguna plataforma (accounts.achievements).

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: lo que tiene con todos los logros).

lists/presets.py, línea 92

```
def played_on(platform)
```

Cómo elige sus juegos la plantilla de una plataforma: lo que tiene horas en platform (clave de HOURS_SOURCES), de más a menos horas en ella.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: las de cada plataforma).

lists/presets.py, línea 105

```
def backlog(user, year)
```

Lo pendiente, de lo añadido hace menos a lo que más.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: lo pendiente).

lists/presets.py, línea 120

Clase Preset

Una plantilla: su clave (la que manda el formulario), su nombre y su explicación en «Nueva lista» (label y help), el título de la tier list que crea (label y title pueden llevar {year}), cómo elige los juegos (entries: del usuario y el año a sus

LibraryEntry, en orden), sus niveles ([(nombre, color)], de mejor a peor) y si coloca de entrada los juegos que tienen nota (by_rating, solo con los niveles de serie).

```
def label_for(self, year)
```

Su nombre en «Nueva lista» en year.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: nombre de cada plantilla).

lists/presets.py, línea 143

```
def title_for(self, year)
```

Título de la tier list que crea en year.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: título de la tier list que crea), Página de la lista.

lists/presets.py, línea 150

```
def choose(self, user, year)
```

Los juegos de la biblioteca de user que entran en year, en su orden (LibraryEntry), como mucho LIST_MAX_GAMES.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: los juegos que entran).

lists/presets.py, línea 158

```
def tier_list(self)
```

Sus niveles como los guarda GameList.tiers: [{"id", "label", "color"}].

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: sus niveles), Página de la lista.

lists/presets.py, línea 165

```
def tier_of(self, entry)
```

Id del nivel en el que empieza el juego de entry: con by_rating, el de su nota (RATING_TIERS); si no, o sin nota, ninguno (sin clasificar).

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»: el nivel en el que empieza cada juego), Página de la lista (tablero).

lists/presets.py, línea 172

```
def choices(user)
```

Las plantillas del bloque «Desde tu biblioteca» de «Nueva tier list», en el año en curso, con lo que enseña cada una: su clave, su nombre y su explicación (key, label y help), cuántos juegos metería (count), las portadas de los primeros (preview_items y mosaic_gaps, como el mosaico de una lista), si se puede crear (ready: al menos MIN_GAMES) y, si user ya tiene una lista con el título que le pondría, esa lista (existing). Las que no tienen ningún juego no salen.

Pantallas: Nueva tier list (plantillas «Desde tu biblioteca»).

lists/presets.py, línea 212

lists/views/colaboradores.py

Quién puede editar una lista: invitar y quitar a quien colabora (en Editar lista, sección «Quién puede editarla») y aceptar o dejar la invitación (en la Página de la lista).
Pantallas de este archivo: Página de la lista, Editar lista, Avisos

```
def collaborators_context(game_list, owner)
```

Variables de la sección «Quién puede editarla» del editor (lists/_collaborators.html): la gente invitada, con si ha aceptado; cuántos caben; y, para escribir su nombre, la gente a la que sigue owner que aún no está invitada (suggested, como mucho SUGGESTED_COLLABORATORS).

Pantallas: Editar lista («Quién puede editarla»).

lists/views/colaboradores.py, línea 24

```
def _collaborators_response(request, game_list, error='')
```

Respuesta de invitar o quitar a alguien de una lista: con HTMX, la sección «Quién puede editarla» del editor (con el error, si lo hay); sin HTMX, vuelve a ella avisando del error.

Pantallas: Editar lista («Quién puede editarla»).

lists/views/colaboradores.py, línea 43

```
@require_POST
@login_required
def invite(request, list_id)
```

Invita a alguien (usuario=, su nombre de usuario, con o sin @) a editar los juegos de una lista del usuario y se lo avisa en la campana. Caben MAX_COLLABORATORS; no puede invitarse a sí mismo ni a quien ya está invitado.

Pantallas: Editar lista (invitar a alguien), Avisos (aviso al invitado), Página de la lista (le pregunta si acepta).

lists/views/colaboradores.py, línea 60

```
@require_POST
@login_required
def collaborator_remove(request, list_id, user_id)
```

Quita a alguien de los que pueden editar una lista del usuario, o retira su invitación; lo que añadió se queda.

Pantallas: Editar lista (quitar a alguien o retirar su invitación).

lists/views/colaboradores.py, línea 91

```
@require_POST
@login_required
def accept(request, list_id)
```

Acepta la invitación del usuario a editar una lista y lleva a su editor; 404 si no le han invitado.

Pantallas: Página de la lista (aceptar la invitación), Editar lista.

lists/views/colaboradores.py, línea 103

```
@require_POST
@login_required
def leave(request, list_id)
```

Rechaza la invitación del usuario a editar una lista o deja de colaborar en ella (lo que añadió se queda).

Vuelve a la lista si aún puede verla y, si no (era privada), a sus listas (o a sus tier lists, si lo era); 404 si no le habían invitado.

Pantallas: Página de la lista (rechazar la invitación), Editar lista (dejar de colaborar).

lists/views/colaboradores.py, línea 118

lists/views/comun.py

Lo que comparten las vistas que cambian una lista: la del usuario, o una en la que colabora, o un 404.

Pantallas de este archivo: Ficha del juego, Página de la lista, Editar lista

```
def _own_list(request, list_id)
```

Lista list_id del usuario; las de otros dan 404.

Pantallas: Editar lista (404 si no es tuya).

lists/views/comun.py, línea 9

```
def _editable_list(request, list_id)
```

Lista list_id si el usuario puede editar sus juegos (su dueño o quien colabora en ella); si no, 404.

Pantallas: Editar lista (los juegos: 404 si no puedes editarla), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists).

lists/views/comun.py, línea 17

lists/views/editar_lista.py

Vistas de Editar lista (listas/<id>/editar/): los datos de la lista, su buscador y los botones de cada juego (añadir, quitar, mover, comentar y poner en un nivel), que también se pulsan desde «Añadir a una lista» de la Ficha del juego y desde el tablero de la Página de la lista. Quién puede editarla va en colaboradores.py.
Pantallas de este archivo: Ficha del juego, Perfil, Página de la lista, Editar lista

```
def editor_context(game_list)
```

Variables de los juegos del editor (lists/_editor_items.html): la lista, sus juegos en orden (con quién añadió cada uno), si es una tier list sus niveles (tiers) y el tablero en el que se arrastran sus portadas (tier_rows, siempre con la fila «Sin clasificar»), y los máximos.

Pantallas: Editar lista (los juegos del editor), Ficha del juego (al añadir desde «Añadir a una lista»), Página de la lista.

lists/views/editar_lista.py, línea 30

```
def board_context(game_list, viewer)
```

Variables del tablero de la página de una tier list (lists/_tier_board.html) para quien puede editarla, que es lo que se le devuelve al soltar una portada en él: sus juegos por niveles, con la fila «Sin clasificar» aunque esté vacía, y el estado de cada uno en la biblioteca de viewer (viewer_status), como en detail.

Pantallas: Página de la lista (tablero de la tier list al soltar una portada).

lists/views/editar_lista.py, línea 49

```
def search_context(game_list, query)
```

Variables del buscador del editor: la búsqueda (search_query; no query, que es la del buscador de la cabecera) y, si la hay, los SEARCH_RESULTS juegos más populares que la cumplen (results, como el buscador del catálogo), cada uno marcado si ya está en la lista (in_list).

Pantallas: Editar lista (buscador de juegos para añadir).

lists/views/editar_lista.py, línea 64

```
@login_required
def edit(request, list_id)
```

Editor de una lista: sus datos (título, descripción, si la ven los demás, su tipo y sus niveles), que guarda su dueño con el formulario; sus juegos, que se buscan (?q=, ver search), se añaden, se ordenan, se ponen en un nivel, se comentan y se quitan uno a uno (add, move, tier, note y remove); y quién puede editarla (invite y collaborator_remove). Quien colabora solo ve los juegos (y puede irse, leave); a los demás les da 404.

El formulario trabaja con una copia de la lista para que, si no es válido, la página siga con lo guardado.

Pantallas: Editar lista.

lists/views/editar_lista.py, línea 81

```
@require_POST
@login_required
def delete(request, list_id)
```

Borra una lista del usuario (los juegos siguen en su biblioteca y en el catálogo) y vuelve a sus listas, o a sus tier lists si lo era.

Pantallas: Editar lista (botón «Borrar»), Perfil (pestañas Listas y Tier lists).

lists/views/editar_lista.py, línea 113

```
@login_required
def search(request, list_id)
```

Busca en el catálogo juegos para añadir a una lista que el usuario puede editar (?q=, ver search_context).

Con HTMX devuelve solo los resultados; sin HTMX lleva al editor con la búsqueda hecha.

Pantallas: Editar lista (resultados del buscador).

lists/views/editar_lista.py, línea 127

```
@require_POST
@login_required
def add(request, list_id, game_id)
```

Añade un juego al final de una lista que el usuario puede editar (si ya estaba, no hace nada); caben LIST_MAX_GAMES.

Se apunta quién lo añade y su actividad (social.activity.list_games_added), y responde según desde dónde se pulsa (ver _item_response).

Pantallas: Editar lista (los juegos: añadir un juego), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists).

lists/views/editar_lista.py, línea 143

```
@require_POST
@login_required
def remove(request, list_id, game_id)
```

Quita un juego de una lista que el usuario puede editar (si no estaba, no hace nada) y responde como add. Si lo acababa de añadir él, también se descuenta de su actividad (social.activity.list_game_removed).

Pantallas: Editar lista (los juegos: quitar un juego), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists).

lists/views/editar_lista.py, línea 164

```
@require_POST
@login_required
def move(request, list_id, game_id)
```

Cambia de puesto un juego de una lista que el usuario puede editar: uno arriba o abajo (a=arriba o a=abajo) o al puesto que se escriba (puesto=, desde 1; si se pasa, al principio o al final). Responde como add.

Da 404 si el juego no está en la lista y 400 si el puesto no es un número.

Pantallas: Editar lista (subir, bajar o arrastrar un juego), Página de la lista.

lists/views/editar_lista.py, línea 180

```
@require_POST
@login_required
def note(request, list_id, game_id)
```

Guarda el comentario de quien hace la lista sobre uno de sus juegos (vacío lo quita); caben NOTE_MAX_LENGTH caracteres.

Lo pueden escribir su dueño y quien colabora. Con HTMX devuelve solo su formulario, marcado como guardado o con el error, para no rehacer los demás juegos del editor; sin HTMX vuelve al editor, a ese juego.

Pantallas: Editar lista (comentario de cada juego), Página de la lista (los comentarios de los juegos).

lists/views/editar_lista.py, línea 207

```
@require_POST
@login_required
def tier(request, list_id, game_id)
```

Pone un juego de una tier list en uno de sus niveles (nivel=, su id; vacío lo deja sin clasificar). Con orden= (los ids de los juegos de esa fila separados por comas, como han quedado al soltarlo en el tablero) además lo deja en su sitio dentro del nivel (GameList.place). Lo pueden hacer su dueño y quien colabora. Responde como add; 404 si el juego no está en la lista y 400 si el nivel no existe.

Pantallas: Página de la lista (arrastrar por el tablero), Editar lista (nivel de cada juego).

lists/views/editar_lista.py, línea 237

```
def _item_response(request, game_list, game, error='')
```

Respuesta de añadir, quitar, mover o poner en un nivel un juego de una lista, según desde dónde se pulsa (desde=).

Con HTMX: desde la ficha («ficha»), el desplegable «Añadir a una lista» abierto; desde el buscador del editor («buscador»), los juegos del editor y, fuera de su sitio, el resultado ya marcado como añadido; desde el tablero de la página de la lista («tablero»), el tablero; desde el editor, sus juegos. Sin HTMX vuelve a next (si es de esta web) o al editor, avisando del error si lo hay.

Pantallas: Editar lista (los juegos: lo que cambia tras añadir, quitar, mover o poner en un nivel), Ficha del juego (desplegable «Añadir a una lista»), Página de la lista (tablero de las tier lists).

lists/views/editar_lista.py, línea 261

lists/views/listas_del_juego.py

Las listas de un juego: la pantalla Listas del juego (juegos/<juego>/listas/) y lo que enseña de ellas la Ficha del juego (el desplegable «Añadir a una lista» y «En N listas»).
Pantallas de este archivo: Ficha del juego, Listas del juego

```
def _lists_with(game, viewer)
```

Listas que viewer puede ver en las que está game, de la más reciente a la más antigua, con sus tarjetas cargadas.

Pantallas: Ficha del juego (listas en las que sale), Listas del juego.

lists/views/listas_del_juego.py, línea 20

```
def picker_context(user, game)
```

Variables del desplegable «Añadir a una lista» de la ficha de game (lists/_picker.html).

Las listas que user puede editar (las suyas y en las que colabora), de la más reciente a la más antigua, cada una con su dueño, cuántos juegos tiene (game_count) y si game ya está en ella (has_game), y en cuántas está (in_lists).

Pantallas: Ficha del juego (desplegable «Añadir a una lista»).

lists/views/listas_del_juego.py, línea 32

```
def game_page_context(request, game)
```

Variables de las listas en la ficha de game: con sesión, las del desplegable «Añadir a una lista» (picker_context); y para todos, «En N listas»: las GAME_PAGE_LISTS más recientes que puede ver quien mira (game_lists) y cuántas son (game_lists_count).

Pantallas: Ficha del juego (desplegable «Añadir a una lista» y «En N listas»).

lists/views/listas_del_juego.py, línea 49

```
def game_lists(request, slug)
```

Todas las listas en las que está un juego que puede ver quien mira, de la más reciente a la más antigua, de LISTS_PER_PAGE en LISTS_PER_PAGE (?page=).

Pantallas: Listas del juego.

lists/views/listas_del_juego.py, línea 63

lists/views/nueva_lista.py

Vistas de Nueva lista (listas/nueva/) y Nueva tier list (tier-lists/nueva/), en blanco o desde una de las plantillas que la rellenan con tu biblioteca.
Pantallas de este archivo: Inicio, Perfil, Página de la lista, Nueva lista, Nueva tier list

```
def _create(request, form, game=None)
```

Guarda como del usuario la lista nueva de form (ya validado), que empieza con game si se da; la apunta en su actividad (social.activity.list_created) y lleva a su editor, para buscar y añadir sus juegos.

Pantallas: Nueva lista, Nueva tier list (al crearla), Inicio (feed), Perfil (pestañas Listas y Tier lists).

lists/views/nueva_lista.py, línea 23

```
@login_required
def new(request)
```

Crea una lista del usuario (POST) o enseña el formulario para hacerlo (GET): una lista sin más o numerada; las tier lists tienen su propia página (new_tier_list).

Con juego= (desde «Nueva lista con este juego» en la ficha) la lista empieza con ese juego. Se apunta en su actividad y lleva a su editor, para buscar y añadir más (_create).

Pantallas: Nueva lista.

lists/views/nueva_lista.py, línea 44

```
@login_required
def new_tier_list(request)
```

Página «Nueva tier list»: las plantillas que la crean ya rellena con la biblioteca del usuario (lists.presets; se crean con from_library) y el formulario para empezar una en blanco con sus niveles (POST), que la crea como new.

Pantallas: Nueva tier list.

lists/views/nueva_lista.py, línea 62

```
@require_POST
@login_required
def from_library(request)
```

Crea una tier list del usuario con una de las plantillas de su biblioteca (plantilla=, ver lists.presets) y lleva a su página, para arrastrar sus juegos en el tablero.

Los juegos entran en el orden de la plantilla, sin clasificar o ya colocados por su nota, y la lista se crea privada: recién hecha está sin colocar, y su dueño la hace visible cuando quiera. Se apunta en su actividad como cualquier lista (social.activity). Si ya tiene una lista con ese título, lleva a ella en vez de repetirla; con menos de presets.MIN_GAMES juegos, vuelve a «Nueva tier list» con un aviso.

Pantallas: Nueva tier list (botón de cada plantilla), Página de la lista.

lists/views/nueva_lista.py, línea 78

lists/views/pagina_de_la_lista.py

Vistas de la Página de la lista (u/<usuario>/listas/<id>/): la lista con sus juegos, cuánto llevas jugado y lo que cuesta lo que te falta, en las tier lists abiertas la de la comunidad, y sus botones: «me gusta», comentarios, denuncias, «Lo quiero» de lo que te falta y «Haz tu versión».
Pantallas de este archivo: Inicio, Perfil, Lista de deseos, Página de la lista, Tier lists, Avisos, Panel de administración

```
def detail(request, username, list_id)
```

Página de una lista: su mosaico, el título, el autor (y quien colabora), la descripción y los juegos, en portadas (en las tier lists, por niveles) o con el comentario de cada uno (?vista=; sin ella, con los comentarios si alguno tiene), y debajo sus comentarios. En el tablero de una tier list, quien puede editarla arrastra las portadas de un nivel a otro (lists/_tier_board.html y tier).

En las tier lists abiertas a la comunidad enseña también cuánta gente la ha hecho, el botón «Haz tu versión» (make_version) o la tuya si ya la tienes, y la tier list de la comunidad (lists.community, ?vista=comunidad), que es la que sale primero en cuanto la ha hecho bastante gente. En las versiones dice de cuál son.

La ve quien puede verla (GameList.visible_to); si no, 404. A quien está invitado a editarla le pregunta si acepta. Con sesión, enseña cuánto ha jugado quien mira de la lista, el estado de cada juego en su biblioteca y, de lo que no tiene, cuánto cuesta (list_progress). Los demás pueden darle «me gusta» y denunciarla (con ?denunciar, el formulario llega abierto).

Pantallas: Página de la lista.

lists/views/pagina_de_la_lista.py, línea 34

```
def _readable_list(request, list_id)
```

Lista list_id con su dueño cargado, si el usuario puede verla; si no, 404.

Pantallas: Página de la lista, Perfil, Inicio (404 al pulsar algo de una lista que no se puede ver).

lists/views/pagina_de_la_lista.py, línea 112

```
@require_POST
@login_required
def make_version(request, list_id)
```

«Haz tu versión» de una tier list abierta a la comunidad: crea la del usuario (GameList.make_version), la apunta en su actividad y lleva a su página, para colocar los juegos en su tablero; si ya la tiene, lleva a ella. Las que no están abiertas dan 404, y la propia 400: su tablero ya cuenta en la de la comunidad.

Pantallas: Página de la lista (botón «Haz tu versión»), Perfil (pestaña Tier lists), Tier lists, Inicio (feed: «hizo su versión de…»).

lists/views/pagina_de_la_lista.py, línea 125

```
@require_POST
@login_required
def like(request, list_id)
```

Da o quita el «me gusta» del usuario a una lista de otro, avisando a su dueño la primera vez.

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la lista. Las propias no se pueden marcar (400).

Pantallas: Página de la lista, Perfil, Inicio (botón «me gusta» de las listas), Avisos (aviso a su dueño).

lists/views/pagina_de_la_lista.py, línea 149

```
def _notify_like(game_list, actor)
```

Avisa al dueño de game_list de que a actor le ha gustado, solo la primera vez aunque lo quite y lo vuelva a dar (accounts.notices).

Pantallas: Avisos (a quien le gusta tu lista), todas (el número de la campana).

lists/views/pagina_de_la_lista.py, línea 171

```
def _list_comments(game_list, viewer)
```

Variables de library/_comments.html para los comentarios de game_list (ver library.views.comment_thread).

Pantallas: Página de la lista (comentarios).

lists/views/pagina_de_la_lista.py, línea 184

```
@require_POST
@login_required
def comment(request, list_id)
```

Publica un comentario en una lista (avisando a su dueño si no es él) y vuelve a sus comentarios.

Con HTMX devuelve los comentarios ya con el nuevo (o el formulario con el error) y el contador de la página.

Pantallas: Página de la lista (comentarios), Avisos (aviso a su dueño).

lists/views/pagina_de_la_lista.py, línea 199

```
def _notify_comment(game_list, actor)
```

Avisa al dueño de game_list de que actor la ha comentado, salvo si ya tiene otro suyo sin leer (accounts.notices).

Pantallas: Avisos (comentarios en tus listas), todas (el número de la campana).

lists/views/pagina_de_la_lista.py, línea 224

```
@require_POST
@login_required
def comment_delete(request, comment_id)
```

Borra un comentario de una lista: solo quien lo escribió o el dueño de la lista (si no, 403).

Pantallas: Página de la lista (botón «Borrar» de un comentario).

lists/views/pagina_de_la_lista.py, línea 239

```
@require_POST
@login_required
def comment_report(request, comment_id)
```

Denuncia un comentario de otro en una lista; los propios no (400) y los de listas que no se pueden ver dan 404.

Pantallas: Página de la lista (denunciar un comentario), Panel de administración (denuncias).

lists/views/pagina_de_la_lista.py, línea 258

```
@require_POST
@login_required
def report(request, list_id)
```

Denuncia una lista de otro para que la revise un administrador (library.views._report); las propias no (400).

Pantallas: Página de la lista (botón «Denunciar»), Panel de administración (denuncias).

lists/views/pagina_de_la_lista.py, línea 272

```
def list_progress(games, viewer)
```

Cómo va viewer con los juegos games de una lista (con sus precios ya cargados).

Cuántos son (total), cuántos tiene en su biblioteca (owned), cuántos ha jugado (played: cualquier estado menos Pendiente; percent, en porcentaje), terminado (completed) y tiene en Pendiente (backlog), las horas que lleva en ellos (hours: las de Steam, PlayStation y Xbox), el estado de cada uno (statuses: id del juego → nombre del estado; status_keys: id del juego → valor del estado, para su color) y lo que le falta (ver _missing).

Pantallas: Página de la lista (cuánto llevas jugado y lo que cuesta lo que te falta).

lists/views/pagina_de_la_lista.py, línea 283

```
def _missing(games, viewer)
```

Lo que le falta a viewer de una lista: games, los juegos que no están en su biblioteca, y lo que cuestan.

Cuántos son (missing); cuántos tienen precio en sus tiendas (priced) y lo que suman hoy (today) y si espera a que cada uno baje a su mínimo histórico de PC (at_lows; el de hoy si no lo tiene o si hoy está más barato), con saving si así sale más barato; cuántos no tienen precio (unpriced: los de consolas sin tienda); y cuántos están ya en su lista de deseos (wished) y cuántos no (to_wish), para el botón de pasarlos a «Lo quiero».

Pantallas: Página de la lista (lo que te falta y lo que cuesta).

lists/views/pagina_de_la_lista.py, línea 310

```
@require_POST
@login_required
def wish_missing(request, list_id)
```

Pasa a la lista de deseos del usuario los juegos de una lista que ni tiene en su biblioteca ni quiere ya, y vuelve a la lista. Como «Lo quiero» en la ficha: si alguno lo había quitado de su lista de deseos de Steam, se olvida, para que la importación de Steam no lo vuelva a quitar.

Pantallas: Página de la lista (botón «Lo quiero» de lo que te falta), Lista de deseos.

lists/views/pagina_de_la_lista.py, línea 346

lists/views/tarjetas.py

Lo que enseñan de las listas otras pantallas: las del Perfil (pestañas Listas y Tier lists y escaparate, con el tablero en miniatura de las tier lists), las de la pestaña Tier lists y las del feed de Inicio (cuánto has jugado de cada una y tus «me gusta»).
Pantallas de este archivo: Inicio, Listas del juego, Perfil, Página de la lista, Tier lists

```
def mark_progress(lists, viewer)
```

Apunta en cada lista de lists cuántos de sus juegos ha jugado viewer (viewer_played; None sin sesión).

Jugado es cualquier estado de la biblioteca menos Pendiente (PLAYED_STATUSES). Devuelve la misma lista.

Pantallas: Perfil (pestaña Listas), Tier lists, Listas del juego, Inicio (feed: cuántos juegos de cada lista has jugado).

lists/views/tarjetas.py, línea 24

```
def mark_liked_lists(lists, viewer)
```

Apunta en cada lista de lists (liked) si viewer le ha dado «me gusta»; devuelve la misma lista.

Pantallas: Página de la lista, Perfil, Inicio (tus «me gusta» marcados).

lists/views/tarjetas.py, línea 45

```
def profile_lists(owner, viewer, public_only=False)
```

Listas de owner y en las que colabora que puede ver viewer, de la más reciente a la más antigua, con lo que enseñan sus tarjetas (GameListQuerySet.with_preview) y cuántos de sus juegos ha jugado viewer (mark_progress). Las que son de otro llevan from_other, para decir de quién son.

Con public_only salen solo las públicas, como las vería cualquiera (la vista previa de Editar perfil).

Pantallas: Perfil (pestañas Listas y Tier lists y escaparate), Tier lists (las tuyas).

lists/views/tarjetas.py, línea 58

```
def mark_boards(lists)
```

Apunta en cada tier list de lists su tablero en miniatura (lists/_tier_card.html): board, con [(nivel, juegos, más)] de cada nivel en su orden (sus BOARD_COVERS primeros juegos y cuántos más tiene), y loose_count, cuántos están sin clasificar (también los de un nivel que ya no existe, como en GameList.tier_rows).

Son dos consultas para todas: los primeros juegos de cada nivel y cuántos tiene cada uno. Devuelve la misma lista.

Pantallas: Perfil (tablero en miniatura de cada tier list), Tier lists (el de cada tarjeta).

lists/views/tarjetas.py, línea 79

lists/views/tier_lists.py

Vista de la pestaña Tier lists (tier-lists/): las tuyas, las de la gente que sigues y las abiertas a la comunidad, de las que tienen más versiones a las últimas.
Pantallas de este archivo: Tier lists

```
def explore(request)
```

Pestaña Tier lists: con sesión, las tuyas (también tus versiones y en las que colaboras) y las últimas de la gente que sigues; para todos, las abiertas a la comunidad con más versiones y las últimas que han cambiado. Cada una con su tablero en miniatura (mark_boards) y, las abiertas, cuántas versiones públicas tiene (version_count). Solo salen las que puede ver quien mira y, con sesión, las suyas solo arriba.

Pantallas: Tier lists.

lists/views/tier_lists.py, línea 15

App «challenges»

Retos: crearlos, unirse y contar el progreso de cada uno con su biblioteca (juegos completados y horas jugadas).

challenges/admin.py

Panel de administración de los retos, con quienes se han unido a cada uno.

Clase ParticipationInline

Quién se ha unido al reto y si lo ha conseguido.

challenges/closing.py

El repaso de cada noche de los retos (comando close_challenges, que lanza nightly).

Apunta a quien ha llegado al objetivo sin que se hubiera mirado (lo normal es que lo apunten al momento la biblioteca y las importaciones: challenges.progress.check), avisa a quien le quedan pocos días en uno que aún no ha conseguido y, pasado el margen tras su fecha de fin (progress.GRACE_DAYS), avisa a cada uno de cómo ha quedado y lo cierra.

Pantallas: Avisos (retos que se acaban y cómo has quedado), Retos, Página del reto (quién lo ha conseguido), Perfil (insignias de la sección «Retos»), Inicio (feed: quién consigue un reto).
Pantallas de este archivo: Inicio, Perfil, Retos, Página del reto, Avisos

```
def close_all(today=None)
```

Repasa los retos que no están cerrados y ya han empezado, y devuelve lo que ha hecho: {"won": a cuántos se les ha apuntado un reto conseguido, "ending": cuántos avisos de que se acaba ha mandado, "closed": cuántos ha cerrado}.

En cada uno apunta a quien ha llegado al objetivo. Si le quedan ENDING_DAYS días o menos, avisa a quien aún no lo ha conseguido (una vez por reto). Si terminó hace más de progress.GRACE_DAYS días, avisa a cada uno de cómo ha quedado y lo marca como cerrado (Challenge.closed_at): desde entonces ya no cuenta nada nuevo.

Pantallas: Avisos (retos conseguidos, que se acaban y cómo has quedado), Retos, Página del reto (quién lo ha conseguido), Perfil (insignias de la sección «Retos»), Inicio (feed: «consiguió el reto»).

challenges/closing.py, línea 26

```
def _amount(mine)
```

Lo que lleva alguien, en palabras, con lo de progress.summary: «2 de 5 juegos» o «12,5 de 50 horas».

Pantallas: Avisos (retos que se acaban y cómo has quedado).

challenges/closing.py, línea 62

```
def _remind(challenge, participations, won, done, days_left)
```

Avisa a los de participations que aún no han conseguido challenge (won son los que sí) de que le quedan days_left días y de cómo van. Devuelve cuántos avisos ha creado: uno por persona y reto (accounts.notices).

Pantallas: Avisos (retos que se acaban), todas (el número de la campana).

challenges/closing.py, línea 73

```
def _announce(challenge, participations, won, done)
```

Avisa a cada uno de participations de cómo ha quedado en challenge, que ya ha terminado: si lo consiguió (won) o hasta dónde llegó, y su puesto, de quien más lleva a quien menos (a igualdad, primero quien se unió antes, como en la clasificación de la página del reto). Con una sola persona no hay puesto que dar.

Pantallas: Avisos (cómo has quedado en un reto), todas (el número de la campana).

challenges/closing.py, línea 92

challenges/forms.py

Formularios de los retos: el de Nuevo reto (ChallengeForm, con su tipo, sus juegos o sus filtros y sus fechas) y el de Editar reto (ChallengeEditForm: título, descripción y si es público).
Pantallas de este archivo: Retos, Página del reto, Nuevo reto, Editar reto

Clase ChallengeEditForm

Lo que se puede cambiar de un reto ya creado: el título, la descripción (con el formato sencillo de las reseñas) y si es público. Sus reglas no, para que sea justo con quien ya se ha unido.

```
def clean_title(self)
```

Deja un solo espacio entre palabras (el campo ya quita los del principio y el final, y sin título da error).

Pantallas: Nuevo reto, Editar reto (el título).

challenges/forms.py, línea 35

```
def clean_description(self)
```

Quita los espacios del principio y del final.

Pantallas: Nuevo reto, Editar reto (la descripción).

challenges/forms.py, línea 42

Clase ChallengeForm

Un reto nuevo: lo de ChallengeEditForm y sus reglas, según su tipo.

De juegos concretos: los de una lista (source_list: una de las que puede editar quien lo crea o, desde «Convertir en reto», la que estaba mirando) y cuántos de ellos hay que terminar (target; vacío, todos). De número de juegos o de horas: cuántos (target) y, si se quiere, de qué género, plataforma y años. Para todos, las fechas, que son opcionales.

```
def __init__(self, *args, user, game_list=None, **kwargs)
```

Prepara las listas que se pueden elegir (las que puede editar user y, si se da, game_list, desde «Convertir en reto») y los géneros y plataformas del catálogo.

Pantallas: Nuevo reto (las listas, los géneros y las plataformas que se pueden elegir).

challenges/forms.py, línea 86

```
def clean(self)
```

Comprueba lo que pide cada tipo y que las fechas y los años vayan en orden.

Pantallas: Nuevo reto (lo que pide cada tipo de reto y el orden de las fechas y los años).

challenges/forms.py, línea 108

```
def save(self, user)
```

Guarda el reto de user. Uno de juegos concretos se queda con los que tiene su lista ahora (aunque la lista cambie después) y sin filtros; los demás, sin lista.

Pantallas: Nuevo reto (al crearlo), Retos, Página del reto.

challenges/forms.py, línea 139

challenges/management/commands/close_challenges.py

Comando «manage.py close_challenges»: el repaso de cada noche de los retos (challenges.closing.close_all). Lo lanza nightly; lo que hace se ve en Avisos, Retos, la Página del reto, el Perfil y el feed de Inicio.
Pantallas de este archivo: Inicio, Perfil, Retos, Página del reto, Avisos

Clase Command

```
def handle(self, *args, **options)
```

Lanza el repaso y dice cuántos retos se han conseguido, cuántos avisos de que se acaban ha mandado y cuántos retos ha cerrado.

Pantallas: Avisos (retos conseguidos, que se acaban y cómo has quedado), Retos, Página del reto (quién lo ha conseguido), Perfil (insignias de la sección «Retos»), Inicio (feed: «consiguió el reto»).

challenges/management/commands/close_challenges.py, línea 16

challenges/models.py

Modelos de los retos: cada reto (de número de juegos, de horas o de juegos concretos, con sus fechas y filtros) y quién se ha unido a él. Se ven en Retos, en la Página del reto y en Editar reto.
Pantallas de este archivo: Retos, Página del reto, Editar reto, Panel de administración

```
def long_date(day)
```

Una fecha como se escribe en España: «1 de octubre de 2026».

Pantallas: Retos, Página del reto, Editar reto (las fechas en palabras).

challenges/models.py, línea 19

Clase ChallengeQuerySet

```
def visible_to(self, viewer)
```

Retos que viewer puede ver: los públicos y los suyos. Es la regla de Challenge.visible_to en una consulta.

Pantallas: Retos, Página del reto (solo los públicos y los tuyos).

challenges/models.py, línea 28

```
def with_counts(self)
```

Retos con su creador cargado y cuánta gente se ha unido (participant_count).

Pantallas: Retos, Página del reto (cuánta gente se ha unido).

challenges/models.py, línea 38

Clase Challenge

Un reto que crea un jugador y al que se puede unir cualquiera: terminar unos juegos concretos, terminar un número de juegos (de un género, una plataforma o unos años, si se quiere) o jugar un número de horas, entre unas fechas o sin ellas.

El progreso de cada uno se cuenta solo con su biblioteca (challenges.progress): los juegos que pasa a «Completado» entre las fechas (LibraryEntry.completed_at) o lo que juega en ellas (library.models.PlaySession). Sus reglas no cambian después de crearlo, para que sea justo con quien ya se ha unido. Los públicos los ve cualquiera; los

privados, solo quien los crea (un objetivo propio, como «20 juegos en 2027»).

Cuando termina, aún cuenta lo que llegue en los días de margen (challenges.progress.GRACE_DAYS): las sincronizaciones de PlayStation y Xbox son a mano y pueden llegar tarde. Pasado el margen, el repaso de cada noche (challenges.closing) avisa a cada uno de cómo ha quedado y apunta que está cerrado (closed_at).

```
def __str__(self)
```

Texto en el admin: su título.

Pantallas: Panel de administración.

challenges/models.py, línea 105

```
def get_absolute_url(self)
```

URL de la página del reto: /retos/<id>/.

Pantallas: Retos, Página del reto, Editar reto (los enlaces a cada reto).

challenges/models.py, línea 112

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el reto: los públicos, cualquiera; los privados, solo quien los crea.

Pantallas: Página del reto (los privados solo los ve quien los crea).

challenges/models.py, línea 119

```
@property
def goal(self)
```

Lo que hay que llegar a tener, en la unidad de challenges.progress: juegos terminados o minutos jugados.

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación).

challenges/models.py, línea 127

```
@property
def is_upcoming(self)
```

Indica si aún no ha empezado.

Pantallas: Retos, Página del reto (si aún no ha empezado).

challenges/models.py, línea 139

```
@property
def is_over(self)
```

Indica si ya ha terminado: desde entonces no se puede unir nadie.

Pantallas: Retos, Página del reto (si ya ha terminado y ya no se puede unir nadie).

challenges/models.py, línea 147

```
@property
def days_left(self)
```

Días que le quedan si está en marcha y tiene fecha de fin (hoy incluido); si no, None.

Pantallas: Retos, Página del reto (los días que le quedan).

challenges/models.py, línea 155

```
@property
def filters_text(self)
```

De qué juegos va, en palabras: «de Rol de PC lanzados entre 2000 y 2010»; vacío si de cualquiera.

Pantallas: Retos, Página del reto, Editar reto (de qué juegos va).

challenges/models.py, línea 165

```
@property
def rules_text(self)
```

Lo que hay que hacer, en una frase: «Termina 5 juegos de Terror», «Juega 50 horas», «Termina los 6 juegos».

Pantallas: Retos, Página del reto, Editar reto (lo que hay que hacer).

challenges/models.py, línea 185

```
@property
def dates_text(self)
```

Cuándo cuenta, en palabras: «del 1 de octubre al 31 de octubre de 2026», «desde…», «hasta…» o «sin fechas».

Pantallas: Retos, Página del reto, Editar reto (cuándo cuenta).

challenges/models.py, línea 202

Clase Participation

Un jugador que se ha unido a un reto. Su progreso se calcula con su biblioteca (challenges.progress); cuando llega al objetivo se apunta cuándo (completed_at), una vez.

```
def __str__(self)
```

Texto en el admin: «usuario en reto».

Pantallas: Panel de administración.

challenges/models.py, línea 231

challenges/progress.py

El progreso de cada jugador en los retos, contado con su biblioteca.

En los de juegos (concretos o un número) cuentan los que ha pasado a «Completado» entre las fechas del reto (LibraryEntry.completed_at); sin fecha de inicio cuentan también los de antes, aunque no se sepa cuándo los terminó. En los de horas se suma lo jugado entre las fechas (library.models.PlaySession), que se apunta desde que existen las sesiones. done_by lo cuenta de golpe para muchos jugadores, para la clasificación de la página del reto.

Quien llega al objetivo queda apuntado (mark_completed) y se le cuenta: un aviso en la campana y «consiguió el reto» en su actividad. Aquí está también lo que enseña de los retos de cada uno la sección «Retos» de su perfil (showcase).
Pantallas de este archivo: Inicio, Ficha del juego, Perfil, Retos, Página del reto, Nuevo reto, Avisos, Editar perfil

```
def _start(day)
```

El principio del día day en la hora de España, para compararlo con fechas y horas.

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación), Ficha del juego (al pasar un juego a «Completado» se mira si consigues un reto).

challenges/progress.py, línea 32

```
def _game_filters(challenge, prefix='game__')
```

Condiciones sobre el juego de un reto de número de juegos o de horas: su género, su plataforma y sus años.

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación), Ficha del juego (al pasar un juego a «Completado» se mira si consigues un reto).

challenges/progress.py, línea 41

```
def _completed(challenge, users)
```

Juegos de la biblioteca de users que cuentan en un reto de juegos: los completados entre sus fechas que son de los suyos (juegos concretos) o que cumplen sus filtros (número de juegos).

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación), Ficha del juego (al pasar un juego a «Completado» se mira si consigues un reto).

challenges/progress.py, línea 59

```
def done_by(challenge, users)
```

Cuánto lleva cada uno de users en challenge: {id del usuario: juegos terminados o minutos jugados}. Los que no llevan nada no salen.

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación), Nuevo reto (al crearlo ya cuenta lo que llevas), Ficha del juego (al pasar un juego a «Completado» se mira si consigues un reto).

challenges/progress.py, línea 78

```
def summary(challenge, done)
```

Cómo va alguien que lleva done (lo de done_by) en challenge, para enseñarlo: lo hecho y el objetivo en juegos o en horas (value y goal_value), el porcentaje (percent, como mucho 100) y si ha llegado (reached).

Pantallas: Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación).

challenges/progress.py, línea 95

```
def finished_games(challenge, user)
```

Ids de los juegos de un reto de juegos concretos que user ha terminado y cuentan.

Pantallas: Página del reto (los juegos que llevas de uno de juegos concretos).

challenges/progress.py, línea 112

```
def mark_completed(challenge, done)
```

Apunta que lo han conseguido (Participation.completed_at, una sola vez) los participantes de challenge que, según done (lo que da done_by), han llegado al objetivo, y se lo cuenta a cada uno (_award). Devuelve los ids de los usuarios a los que se les ha apuntado ahora.

Cada uno se apunta solo si aún no lo estaba en ese momento, para que dos repasos a la vez no lo cuenten dos veces.

Pantallas: Retos (cómo vas en cada uno: quién lo ha conseguido), Página del reto (tu progreso y la clasificación), Perfil (insignias de la sección «Retos»), Avisos (reto conseguido), Inicio (feed: «consiguió el reto»).

challenges/progress.py, línea 120

```
def _award(challenge, user)
```

Le cuenta a user que ha conseguido challenge: un aviso en la campana y «consiguió el reto» en su actividad.

Pantallas: Avisos (reto conseguido), todas (el número de la campana), Inicio (feed), Perfil (pestaña Actividad).

challenges/progress.py, línea 146

```
def check(user)
```

Mira si user ha conseguido alguno de los retos en los que está y que aún no tenía: los que siguen en marcha y los que terminaron hace menos de GRACE_DAYS días, porque una sincronización a mano puede llegar tarde. Los cerrados ya no se miran.

Lo llaman la biblioteca al pasar un juego a «Completado» y la importación al apuntar sesiones. Devuelve los retos conseguidos ahora.

Pantallas: Ficha del juego (al pasar un juego a «Completado»), Editar perfil (Cuentas vinculadas: al importar horas), Perfil (modal «Vincular cuentas» de su dueño; insignias de la sección «Retos»), Retos (cómo vas en cada uno), Página del reto (tu progreso y la clasificación), Avisos (reto conseguido).

challenges/progress.py, línea 158

```
def showcase(owner, viewer)
```

Lo que enseña a viewer la sección «Retos» del escaparate de owner, solo con los retos que viewer puede ver (los privados, solo su creador): sus insignias (won, los que ha conseguido, del último al primero, cada uno con won_at) y los que lleva en marcha (active, los que no han terminado, primero los que antes acaban, cada uno con cómo va en mine), y cuántos hay de cada (won_count y active_count). None si no hay nada que enseñar.

Pantallas: Perfil (sección «Retos» del escaparate), Editar perfil (vista previa).

challenges/progress.py, línea 177

challenges/views.py

Vistas de los retos: Retos, Página del reto, Nuevo reto y Editar reto, y los botones de borrar, unirse y dejar un reto.
Pantallas de este archivo: Inicio, Perfil, Repasar la biblioteca, Retos, Página del reto, Nuevo reto, Editar reto, Avisos

```
def _open(challenges)
```

Los retos de challenges a los que aún se puede unir alguien: los que no han terminado.

Pantallas: Retos (los que siguen abiertos).

challenges/views.py, línea 30

```
def _with_progress(challenges, user)
```

Pone en cada reto de challenges cómo va user en él (mine, ver challenges.progress.summary) y si lo ha conseguido (won); devuelve la misma lista.

Pantallas: Retos (cómo vas en cada uno).

challenges/views.py, línea 38

```
def explore(request)
```

Página de retos: con sesión, los tuyos (con cómo vas en cada uno) y los de la gente que sigues; para todos, los más populares que siguen abiertos (con más gente) y los últimos que se han creado. Solo salen los públicos, salvo los tuyos, y con sesión, abajo solo aquellos en los que aún no estás: los tuyos ya salen arriba.

Pantallas: Retos.

challenges/views.py, línea 54

```
def _visible(request, challenge_id)
```

Reto challenge_id si el usuario puede verlo; si no, 404.

Pantallas: Página del reto (404 si no puedes verlo).

challenges/views.py, línea 76

```
def detail(request, challenge_id)
```

Página de un reto: qué hay que hacer y cuándo, quién lo ha creado, cómo vas tú (en los de juegos concretos, cuáles llevas) y la clasificación de quienes se han unido, de quien más lleva a quien menos.

En la clasificación solo salen quienes tienen el perfil visible para quien mira. Quien ha llegado al objetivo sale como que lo ha conseguido aunque aún no esté apuntado: lo apuntan al momento la biblioteca y las importaciones (challenges.progress.check) y, si algo se escapa, el repaso de cada noche (challenges.closing). La página no guarda nada.

Pantallas: Página del reto.

challenges/views.py, línea 87

```
@login_required
def new(request)
```

Crea un reto (POST) o enseña el formulario (GET). Quien lo crea queda unido a él, y sale en su actividad que lo ha creado.

Con lista= (desde «Convertir en reto» de una lista que puede ver) empieza como uno de juegos concretos con los de esa lista; con anual= (desde el final de Repasar la biblioteca), como el reto del año: terminar 12 juegos del 1 de enero al 31 de diciembre, solo para ti; si no, como uno de número de juegos.

Pantallas: Nuevo reto, Repasar la biblioteca (botón «Ponte un reto para este año»), Inicio (feed: «creó el reto»), Perfil (pestaña Actividad).

challenges/views.py, línea 134

```
@login_required
def edit(request, challenge_id)
```

Cambia el título, la descripción o si es público un reto del usuario (sus reglas no se cambian); los de otros dan 404.

Pantallas: Editar reto.

challenges/views.py, línea 172

```
@require_POST
@login_required
def delete(request, challenge_id)
```

Borra un reto del usuario, con quienes se habían unido, y vuelve a la página de retos.

Pantallas: Editar reto (botón «Borrar»), Retos.

challenges/views.py, línea 188

```
@require_POST
@login_required
def join(request, challenge_id)
```

Une al usuario a un reto que puede ver (si ya estaba, no hace nada) y mira si ya lo ha conseguido con lo que lleva. Lo apunta en su actividad y se lo cuenta a quien lo creó, una vez por persona (accounts.notices). A uno que ya ha terminado no se puede unir nadie (400).

Pantallas: Página del reto (botón para unirse), Retos, Avisos (a quien lo creó), Inicio (feed: «se unió al reto»), Perfil (pestaña Actividad).

challenges/views.py, línea 201

```
@require_POST
@login_required
def leave(request, challenge_id)
```

Saca al usuario de un reto en el que estaba (si no estaba, no hace nada), olvida lo que hizo con él en su actividad y vuelve a él si aún puede verlo.

Pantallas: Página del reto (botón para dejarlo), Retos, Inicio (feed), Perfil (pestaña Actividad).

challenges/views.py, línea 227

Parte 3 · JavaScript de las plantillas

El JavaScript va dentro de las plantillas, en bloques <script>. Cada bloque empieza con un comentario que dice qué hace y en qué pantallas («// Pantallas: …»), y cada función con nombre lleva encima una línea que dice qué hace. HTMX (las peticiones que cambian un trozo de la página) y SortableJS (arrastrar) se cargan de fuera.

accounts/edit_profile.html

Bloque de la línea 380

Editar perfil: contador de la biografía, emojis del estado, paleta de colores, imágenes elegidas y su encuadre, campos que dependen de otra opción, orden de favoritos y secciones, oscurecido del fondo y vista previa en directo.

Pantallas: Editar perfil.

updateCount(): Escribe cuántos caracteres lleva la biografía de los que caben.

markSwatch(): Marca la muestra de la paleta que coincide con el color elegido.

shrink(): Si la imagen es más grande de lo que se guarda (o pesa más de lo que se admite), el navegador la reduce antes de subirla, como haría el servidor: así se sube antes y no se rechaza. Llama a done con el archivo que se subirá y, si se ha reducido, un aviso para el usuario.

render(): Pinta la imagen en sus marcos con el encuadre de los campos (posición, zoom y «entera»).

place(): Guarda en los campos la posición del encuadre, en % y entre 0 y 100.

moved(): Avisa al formulario (y con él a la vista previa): mientras se arrastra, «input»; al terminar, «change».

measure(): Apunta las medidas de la imagen, para pasar lo arrastrado a %.

stop(): Al soltar, termina el arrastre y avisa del encuadre nuevo.

frameSource(): La imagen de cada marco, como la elige el servidor (Profile.banner_url y background_url): la recién elegida, la subida (si no se ha marcado para quitarla) o, en la cabecera, la imagen del juego. null si aún no se sabe (llegan las imágenes del juego): el marco se queda como está.

syncFramers(): Pone en cada editor de encuadre la imagen que toca según el formulario, y en el de la foto, su forma.

checked(): Campos que solo tienen sentido con una opción concreta (p. ej. el juego, con la cabecera «de un juego»). checked da el valor de la opción marcada de un campo (vacío si no hay ninguna).

syncConditional(): Enseña u oculta los campos que dependen de otra opción ([data-show-when]).

previewData(): Lo que se manda para pintar la vista previa: el formulario, con las imágenes nuevas como direcciones blob:.

setState(): Cambia el estado de la vista previa: «En directo», «Actualizando…» o «Sin actualizar».

paint(): Pone en un marco la última página recibida, si no la tiene ya.

paintVisible(): Pinta la página en los marcos que se ven ahora (el del ordenador, el del móvil o los dos).

refresh(): Pide al servidor (profile_preview) la página del perfil con lo que hay en el formulario y la pinta.

schedule(): Pide la vista previa dentro de delay milisegundos, cancelando la que estuviera pendiente.

accounts/linking/_dialog.html

Bloque de la línea 40

Abre y cierra el modal; también se cierra al pulsar fuera de él o con Esc (lo hace <dialog>).

Pantallas: Perfil (modal «Vincular cuentas» de su dueño).

accounts/linking/_platforms.html

Bloque de la línea 40

Cambia la etiqueta y la ayuda del campo según la plataforma elegida.

> Pantallas: Editar perfil (Cuentas vinculadas › Otras plataformas), Perfil (modal «Vincular cuentas»).

base.html

Bloque de la línea 84

Reseñas y comentarios: los ||spoilers|| se destapan al pulsarlos (o con Enter o Espacio; si llevan un enlace, el primer toque solo destapa) y «Copiar enlace» copia la dirección de una reseña.

> Pantallas: todas (se nota donde hay reseñas, comentarios o publicaciones con spoilers y el botón «Copiar enlace»).

reveal(): Destapa un spoiler y le quita lo que lo hacía un botón.

Bloque de la línea 118

En el móvil las pestañas se deslizan de lado: al entrar, la activa se centra en su barra para que no quede cortada o fuera de la vista (solo se mueve la barra, no la página).

> Pantallas: todas (las que tienen pestañas: Inicio, Perfil, Seguidores y seguidos y Juegos importados).

Bloque de la línea 145

Imágenes de las reseñas (library/_review_images.html): las marcadas como spoiler se destapan al pulsarlas y las demás se abren en el visor, donde se pasa de una a otra con las flechas, el teclado o deslizando el dedo. Va por delegación, así que también vale con las reseñas que llegan después por HTMX.

> Pantallas: todas (se nota donde hay imágenes de reseñas o de publicaciones: Ficha del juego, Perfil, Inicio, Página de la reseña y Página de la publicación).

uncover(): Destapa la miniatura de un spoiler (ya no va borrosa y se puede ampliar).

show(): Enseña en el visor la imagen i de la reseña (da la vuelta al llegar a un extremo). Un spoiler sin destapar sale borroso, con un botón para verlo.

Bloque de la línea 232

Si una petición de HTMX falla, lo dice: un aviso abajo que se va solo o, en lo que se carga al abrir la página (imágenes, historial de precios), un texto en su sitio en vez de «Cargando…» para siempre.

> Pantallas: todas (lo que se carga o se pulsa con HTMX).

toast(): Enseña text en un aviso que desaparece a los seis segundos.

failed(): Cambia lo que estaba cargando por un aviso con un enlace para recargar la página.

Bloque de la línea 292

Cierra el menú de la cuenta y el desplegable «Añadir a una lista» de la ficha al pulsar fuera de ellos.

> Pantallas: todas (el menú de la cuenta de la barra de arriba), Ficha del juego (desplegable «Añadir a una lista»).

games/_media.html

Bloque de la línea 59

Carrusel de la ficha: filtros, flechas, visor a pantalla completa (con teclado y deslizando el dedo) y «Poner de cabecera». Vale tanto si la ficha lo pinta directamente como si llega después por HTMX (que ejecuta este script).

> Pantallas: Ficha del juego (carrusel de imágenes y vídeos).

visible(): Miniaturas que deja ver el filtro elegido, en orden.

updateArrows(): Apaga cada flecha cuando la tira ya está en ese extremo (o cuando cabe entera).

videoPoster(): Portada de un vídeo con su botón de reproducir: hasta que se pulsa no se carga nada de YouTube más que la imagen.

show(): Enseña en el visor la miniatura i de las visibles (da la vuelta al llegar a un extremo).

games/_price_history.html

Bloque de la línea 44

Etiqueta de la gráfica: fecha, precio, tienda y rebaja del tramo bajo el cursor (o el elegido con las flechas). Vale tanto si la ficha la pinta directamente como si llega después por HTMX (que ejecuta este script).

Pantallas: Ficha del juego (historial de precios).

day(): Fecha de ms escrita como en España («3 nov 2026»).

show(): Muestra el tramo i en la posición x (en unidades del viewBox) con la fecha ms.

hide(): Esconde la guía, el punto y la etiqueta.

follow(): Sigue al cursor: busca el tramo que hay debajo y lo muestra.

games/detail.html

Bloque de la línea 7

Cabecera: si la imagen de Steam no existe, pasa a la de repuesto de IGDB (y cambia el texto de procedencia) o la quita y queda la carátula desenfocada; si falla el logo, pone el nombre en texto. Vale también para la cabecera que llega por HTMX.

Pantallas: Ficha del juego (cabecera).

library/review_form.html

Bloque de la línea 181

Editor de reseñas: botones de formato (también Ctrl+B y Ctrl+I), contador de caracteres y el nombre de la nota elegida.

Pantallas: Escribir reseña.

updateCount(): Escribe cuántos caracteres lleva el texto de los que caben.

changed(): Avisa de que el texto ha cambiado, para el contador y la vista previa.

wrap(): Rodea lo seleccionado con la marca (o la pone a los dos lados del cursor) y deja seleccionado lo de dentro.

room(): Cuántas imágenes nuevas caben ahora.

warn(): Enseña el aviso de las imágenes, o lo quita si message va vacío.

sync(): Pone en el selector los archivos de items (el servidor recibe esos) y pinta sus miniaturas.

shrink(): Devuelve (en una promesa) el archivo que se subirá: el mismo o, si es más grande de lo que se guarda o pesa demasiado, una copia reducida en WebP (en PNG si el navegador no sabe hacer WebP).

lists/_drag.html

Bloque de la línea 6

Arrastrar portadas y filas con SortableJS (ver el comentario de arriba).

Pantallas: Página de la lista (tablero de las tier lists), Editar lista (orden de los juegos y tablero de las tier lists).

within(): Lo que hay en root que cumple selector, root incluido (HTMX puede cambiar el tablero entero).

putBack(): Vuelve a poner donde estaba lo que se ha soltado (from y oldIndex son de dónde salió, según Sortable).

save(): Guarda lo que se acaba de soltar: manda values a url y cambia target con la respuesta. sortables son las listas que no se pueden tocar mientras tanto; si la petición falla, se desbloquean y lo soltado vuelve a su sitio.

setUpBoard(): Tablero: cada fila es una lista y las portadas pasan de una a otra; al soltar se manda el nivel y el orden de su fila.

setUpRows(): Filas del editor: se cogen por el asa y al soltar se manda su puesto nuevo (desde 1).

social/_composer.html

Bloque de la línea 37

Caja de publicar: enseña las imágenes elegidas (como mucho data-max, que se pueden elegir en varias veces y quitar) y reduce las que no caben o pesan demasiado antes de subirlas, como hace el editor de reseñas.

Pantallas: Inicio (caja para publicar), Publicar.

shrink(): Devuelve (en una promesa) el archivo que se subirá: el mismo o, si es más grande de lo que se guarda o pesa demasiado, una copia reducida en WebP (en PNG si el navegador no sabe hacer WebP).

sync(): Pone en el selector los archivos elegidos (el servidor recibe esos) y pinta sus miniaturas con un botón para quitarlas.